

XCPC Template Library

Morning Flower
and Evening Oath

Tongji University

双码风算法模板与数学速查

建设稿 · 2026-09-12

状态说明

本稿包含首批已进行本地验证的模块与完整来源覆盖表的入口。尚未完成 kuangbin、WIDA 全部条目的双码风迁移，尚未完成近三年中国赛站逐题覆盖审计，最大流、平衡树、双连通分量、可持久化等部分接口已有在线 AC 记录，具体题号、源码摘要及验证范围见 [verification/oj.json](#)。其余接口须分别核验，不能由同文件中某算法通过推断全部通过。本地对拍通过不代表对所有输入的绝对正确性证明；适用范围、数值上限与退化约定属于接口的一部分。

目录

使用与验证	xv
第一部分 动态容器风格	1
第一章 Tarjan 与缩点	2
1.1 Tarjan 强连通分量与缩点	2
第二章 函数图	4
2.1 函数图分解、跳转与到达	4
第三章 双连通分量与圆方树	7
3.1 Tarjan 点双、边双与圆方树	7
第四章 奇环顶点判定	10
4.1 属于简单奇环的全部顶点	10
第五章 删点连通性	12
5.1 由点双分解计算删点连通块数	12
第六章 边双缩点与边定向	13
6.1 边双缩点的大小与内部边数	13
6.2 无向边定向与强连通定向	14
第七章 桥树最少加边	15
7.1 连通无向图最少加边方案	15
第八章 数据结构	16
8.1 并查集	16
8.2 可撤销并查集	17
8.3 树状数组与第 k 小	18
8.4 区间加与区间和	18
8.5 64 位异或线性基	21
第九章 带位置线性基	24
9.1 带位置限制的异或线性基	24
第十章 线性空间求交	26
10.1 异或线性空间求交	26
10.2 Zassenhaus: 同时求和空间与交空间	27
第十一章 线段树常用修改	28
11.1 线段树: 区间乘、加、赋值与模意义求和	28

第十二章 网络流	31
12.1 最大流与方案	31
12.2 势能最小费用最大流	33
12.3 上下界可行循环流	35
第十三章 最大权闭合子图	37
13.1 最大权闭合子图与选点方案	37
第十四章 图论	38
14.1 非负权最短路	38
14.2 强连通分量	39
14.3 2-SAT 与任意可行方案	40
14.4 Hopcroft-Karp 最大匹配与最小点覆盖	41
14.5 割点与桥	43
第十五章 字典序最小 2-SAT	45
15.1 递归染色与回滚求最小赋值	45
第十六章 有向欧拉路	47
16.1 有向图字典序最小欧拉路	47
第十七章 字典序单词链	49
17.1 字典序最小单词链	49
第十八章 无向欧拉路	50
18.1 无向图欧拉路与边方向	50
第十九章 混合图欧拉定向	52
19.1 欧拉回路与指定端点定向	52
19.2 任意起终点的混合欧拉路径	54
第二十章 图上异或行走	55
20.1 无向图最大异或行走	55
第二十一章 树上算法	57
21.1 重链剖分与 LCA	57
第二十二章 离线最近公共祖先	59
22.1 离线 Tarjan LCA	59
第二十三章 在线最近公共祖先	61
23.1 欧拉序与 ST 表 LCA	61
第二十四章 树上点集摘要	64
24.1 树上点集直径摘要	64
第二十五章 倍增祖先与路径最值	66
25.1 倍增 LCA、祖先与最大边权	66
第二十六章 树上路径交	69
26.1 两条树路径的交集	69

第二十七章 点分治	70
27.1 点分治：树上距离点对计数	70
第二十八章 树上启发式合并	73
28.1 树上启发式合并：子树众数颜色和	73
第二十九章 虚树	76
29.1 虚树：关键点压缩与路径信息	76
第三十章 字符串	79
30.1 前缀函数	79
30.2 KMP 匹配	79
30.3 Z 函数	80
30.4 Manacher 回文半径	80
30.5 最小循环表示	81
30.6 AC 自动机出现次数	81
30.7 倍增后缀数组与 LCP	84
30.8 后缀自动机	86
第三十一章 后缀查询与重复子串	88
31.1 后缀 LCP 与子串比较	88
31.2 两个前缀的最长公共后缀	89
31.3 重复子串 AA 的起止位置计数	90
第三十二章 回文树	91
32.1 回文树 PAM：回文后缀与出现次数	91
第三十三章 数论	93
33.1 模运算、素性、CRT 与类欧几里德	93
33.2 Pollard-Rho 因数分解	95
33.3 线性筛、欧拉函数与 Möbius 函数	97
33.4 静态模整数	98
33.5 阶乘组合数	99
第三十四章 二元一次不定方程	100
34.1 二元一次不定方程	100
第三十五章 线性同余方程	101
35.1 线性同余方程	101
第三十六章 区间筛素数	102
36.1 区间筛素数	102
第三十七章 批量模逆元	103
37.1 任意数组的批量逆元	103
第三十八章 连续整数逆元表	104
38.1 连续整数的线性逆元表	104
第三十九章 混合进制中国剩余定理	105
39.1 Garner：大周期 CRT 与混合进制	105

第四十章 乘法阶与原根	107
40.1 乘法阶与原根	107
第四十一章 矩形 GCD 计数	110
41.1 矩形 GCD 计数	110
第四十二章 带权类欧几里德	111
42.1 整除和、带权和与平方和	111
第四十三章 模几何级数与约数和	112
43.1 模幂与几何级数同时求值	112
43.2 幂的约数和	113
第四十四章 单个数的欧拉函数	114
44.1 单个数的欧拉函数	114
第四十五章 Carmichael 函数	115
45.1 Carmichael: 模乘法群的公共指数	115
第四十六章 整数分拆	116
46.1 整数分拆与重数上限	116
第四十七章 素数模数组合数	117
47.1 Lucas: 小素数模的大下标组合数	117
第四十八章 合数模数组合数	119
48.1 合数模数组合数 (扩展 Lucas)	119
第四十九章 二次剩余	122
49.1 素数模平方根 (Cipolla)	122
第五十章 素数模高次剩余	124
50.1 素数模高次剩余: 全部根的紧凑表示	124
第五十一章 素数幂模高次剩余	125
51.1 素数幂模高次剩余: 含非互素被开方数	125
第五十二章 合数求根的准备	128
52.1 合数求根准备: 分解模数与素数幂原根	128
第五十三章 合数模高次剩余	129
53.1 一般合数模高次剩余: CRT 组合与按索引取根	129
第五十四章 多项式插值	131
54.1 拉格朗日插值与连续点优化	131
第五十五章 参数化 NTT 卷积	133
55.1 可指定素数与原根的 NTT 卷积	133
第五十六章 多项式平移	135
56.1 多项式系数平移	135
第五十七章 等比点求值	136
57.1 Chirp Z: 等比点求值	136

第五十八章 斯特林数整行计算	138
58.1 第二类斯特林数：同一行前缀	138
58.2 无符号第一类斯特林数：同一行	139
第五十九章 集合变换与卷积	140
59.1 按位 OR、AND、XOR 卷积	140
第六十章 不相交子集卷积	142
60.1 不相交子集卷积	142
第六十一章 多项式	143
61.1 NTT、形式幂级数、FWT 与递推	143
第六十二章 代数与数论进阶	148
62.1 模高斯消元、行列式、矩阵树定理	148
62.2 杜教筛	151
62.3 扩展 BSGS	153
第六十三章 任意模数行列式	155
63.1 任意正模数行列式：欧几里得消元	155
第六十四章 带权生成树计数	156
64.1 带权矩阵树：无向树与内外向树形图	156
第六十五章 合数模生成树计数	157
65.1 任意正模数带权矩阵树	157
第六十六章 计算几何	158
66.1 整数精确二维几何	158
66.2 浮点点线圆构造	161
第六十七章 优化与可持久化	165
67.1 Max-plus 矩阵幂	165
67.2 离散李超树与最优直线编号	165
67.3 主席树：静态区间第 k 小	167
第六十八章 可持久化数组	170
68.1 可持久化数组：历史版本分叉与单点修改	170
第六十九章 可持久化区间操作	173
69.1 可持久化区间加与版本拼接	173
第七十章 树上路径第 k 小	176
70.1 树上路径点权第 k 小	176
第七十一章 动态区间顺序统计	179
71.1 动态区间第 k 小（树状数组套权值线段树）	179
第七十二章 区间不同数统计	182
72.1 静态区间不同数计数（前驱位置主席树）	182

第七十三章 图论进阶	184
73.1 矩形二分图最小权匹配	184
73.2 有向最小树形图	186
73.3 无向全局最小割	188
第七十四章 最小割树	190
74.1 Gomory–Hu 最小割树	190
第七十五章 最小割树查询	191
75.1 最小割树全点对查询	191
第七十六章 带权二分图匹配	192
76.1 二分图最大权匹配（缺边与可选匹配）	192
第七十七章 精确有界半平面交	194
77.1 整数半平面的有界多边形交	194
第七十八章 圆与多边形面积	197
78.1 圆与简单多边形相交面积	197
第七十九章 最小覆盖圆	199
79.1 随机增量最小覆盖圆	199
第八十章 圆的切线	201
80.1 两圆公切线与过点切线	201
第八十一章 浮点最近点对	203
81.1 浮点平面最近点对	203
第八十二章 精确几何进阶	205
82.1 整数最近点对与 Minkowski 和	205
82.2 整数三维共线、共面与方向	208
第八十三章 凸包支撑点查询	209
83.1 支撑点查询恢复下凸包	209
第八十四章 动态树	211
84.1 Link-cut tree: 动态森林与路径异或	211
第八十五章 随机平衡树	215
85.1 FHQ Treap: 可重集合与顺序统计	215
85.2 隐式 Treap: 序列翻转、加值与区间和	217
第八十六章 伸展树	222
86.1 Splay: 可重集合与顺序统计	222
第八十七章 状态序列维护	227
87.1 状态序列 Treap: 分组区间 GCD	227
第八十八章 一般图匹配	231
88.1 一般图最大基数匹配: 带花树	231

第八十九章 线性递推	234
89.1 Berlekamp–Massey: 最短线性递推	234
89.2 特征多项式求线性递推第 k 项	236
第九十章 快速线性递推	238
90.1 快速递推与有理生成函数单项	238
第二部分 传统接口风格	240
第九十一章 Tarjan 与缩点	241
91.1 Tarjan 强连通分量与缩点	241
第九十二章 函数图	243
92.1 函数图分解、跳转与到达	243
第九十三章 双连通分量与圆方树	246
93.1 Tarjan 点双、边双与圆方树	246
第九十四章 奇环顶点判定	249
94.1 属于简单奇环的全部顶点	249
第九十五章 删点连通性	251
95.1 由点双分解计算删点连通块数	251
第九十六章 边双缩点与边定向	252
96.1 边双缩点的大小与内部边数	252
96.2 无向边定向与强连通定向	253
第九十七章 桥树最少加边	254
97.1 连通无向图最少加边方案	254
第九十八章 数据结构	255
98.1 并查集	255
98.2 可撤销并查集	256
98.3 树状数组与第 k 小	257
98.4 区间加与区间和	258
98.5 64 位异或线性基	260
第九十九章 带位置线性基	262
99.1 带位置限制的异或线性基	262
第一百章 线性空间求交	264
100.1 异或线性空间求交	264
100.2 Zassenhaus: 同时求和空间与交空间	265
第一百零一章 线段树常用修改	266
101.1 线段树: 区间乘、加、赋值与模意义求和	266
第一百零二章 网络流	269
102.1 最大流与方案	269
102.2 势能最小费用最大流	271
102.3 上下界可行循环流	274

第一百零三章 最大权闭合子图	275
103.1 最大权闭合子图与选点方案	275
第一百零四章 图论	276
104.1 非负权最短路	276
104.2 强连通分量	277
104.3 2-SAT 与任意可行方案	278
104.4 Hopcroft–Karp 最大匹配与最小点覆盖	279
104.5 割点与桥	281
第一百零五章 字典序最小 2-SAT	283
105.1 递归染色与回滚求最小赋值	283
第一百零六章 有向欧拉路	285
106.1 有向图字典序最小欧拉路	285
第一百零七章 字典序单词链	288
107.1 字典序最小单词链	288
第一百零八章 无向欧拉路	289
108.1 无向图欧拉路与边方向	289
第一百零九章 混合图欧拉定向	292
109.1 欧拉回路与指定端点定向	292
109.2 任意起终点的混合欧拉路径	294
第一百一十章 图上异或行走	295
110.1 无向图最大异或行走	295
第一百一十一章 树上算法	297
111.1 重链剖分与 LCA	297
第一百一十二章 离线最近公共祖先	299
112.1 离线 Tarjan LCA	299
第一百一十三章 在线最近公共祖先	301
113.1 欧拉序与 ST 表 LCA	301
第一百一十四章 树上点集摘要	304
114.1 树上点集直径摘要	304
第一百一十五章 倍增祖先与路径最值	305
115.1 倍增 LCA、祖先与最大边权	305
第一百一十六章 树上路径交	308
116.1 两条树路径的交集	308
第一百一十七章 点分治	309
117.1 点分治：树上距离点对计数	309
第一百一十八章 树上启发式合并	312
118.1 树上启发式合并：子树众数颜色和	312

第一百一十九章 虚树	315
119.1 虚树：关键点压缩与路径信息	315
第一百二十章 字符串	318
120.1 前缀函数	318
120.2 KMP 匹配	318
120.3 Z 函数	319
120.4 Manacher 回文半径	319
120.5 最小循环表示	320
120.6 AC 自动机出现次数	320
120.7 倍增后缀数组与 LCP	323
120.8 后缀自动机	325
第一百二十一章 后缀查询与重复子串	327
121.1 后缀 LCP 与子串比较	327
121.2 两个前缀的最长公共后缀	328
121.3 重复子串 AA 的起止位置计数	329
第一百二十二章 回文树	330
122.1 回文树 PAM：回文后缀与出现次数	330
第一百二十三章 数论	332
123.1 模运算、素性、CRT 与类欧几里德	332
123.2 Pollard-Rho 因数分解	334
123.3 线性筛、欧拉函数与 Möbius 函数	336
123.4 静态模整数	337
123.5 阶乘组合数	338
第一百二十四章 二元一次不定方程	339
124.1 二元一次不定方程	339
第一百二十五章 线性同余方程	340
125.1 线性同余方程	340
第一百二十六章 区间筛素数	341
126.1 区间筛素数	341
第一百二十七章 批量模逆元	342
127.1 任意数组的批量逆元	342
第一百二十八章 连续整数逆元表	343
128.1 连续整数的线性逆元表	343
第一百二十九章 混合进制中国剩余定理	344
129.1 Garner：大周期 CRT 与混合进制	344
第一百三十章 乘法阶与原根	345
130.1 乘法阶与原根	345
第一百三十一章 矩形 GCD 计数	348
131.1 矩形 GCD 计数	348

第一百三十二章 带权类欧几里德	349
132.1 整除和、带权和与平方和	349
第一百三十三章 模几何级数与约数和	350
133.1 模幂与几何级数同时求值	350
133.2 幂的约数和	350
第一百三十四章 单个数的欧拉函数	351
134.1 单个数的欧拉函数	351
第一百三十五章 Carmichael 函数	352
135.1 Carmichael: 模乘法群的公共指数	352
第一百三十六章 整数分拆	353
136.1 整数分拆与重数上限	353
第一百三十七章 素数模数组合数	354
137.1 Lucas: 小素数模的大下标组合数	354
第一百三十八章 合数模数组合数	355
138.1 合数模数组合数 (扩展 Lucas)	355
第一百三十九章 二次剩余	358
139.1 素数模平方根 (Cipolla)	358
第一百四十章 素数模高次剩余	360
140.1 素数模高次剩余: 全部根的紧凑表示	360
第一百四十一章 素数幂模高次剩余	361
141.1 素数幂模高次剩余: 含非互素被开方数	361
第一百四十二章 合数求根的准备	364
142.1 合数求根准备: 分解模数与素数幂原根	364
第一百四十三章 合数模高次剩余	365
143.1 一般合数模高次剩余: CRT 组合与按索引取根	365
第一百四十四章 多项式插值	367
144.1 拉格朗日插值与连续点优化	367
第一百四十五章 参数化 NTT 卷积	369
145.1 可指定素数与原根的 NTT 卷积	369
第一百四十六章 多项式平移	371
146.1 多项式系数平移	371
第一百四十七章 等比点求值	372
147.1 Chirp Z: 等比点求值	372
第一百四十八章 斯特林数整行计算	374
148.1 第二类斯特林数: 同一行前缀	374
148.2 无符号第一类斯特林数: 同一行	375

第一百四十九章 集合变换与卷积	376
149.1 按位 OR、AND、XOR 卷积	376
第一百五十章 不相交子集卷积	378
150.1 不相交子集卷积	378
第一百五十一章 多项式	379
151.1 NTT、形式幂级数、FWT 与递推	379
第一百五十二章 代数与数论进阶	384
152.1 模高斯消元、行列式、矩阵树定理	384
152.2 杜教筛	387
152.3 扩展 BSGS	388
第一百五十三章 任意模数行列式	389
153.1 任意正模数行列式：欧几里得消元	389
第一百五十四章 带权生成树计数	390
154.1 带权矩阵树：无向树与内外向树形图	390
第一百五十五章 合数模生成树计数	391
155.1 任意正模数带权矩阵树	391
第一百五十六章 计算几何	392
156.1 整数精确二维几何	392
156.2 浮点点线圆构造	395
第一百五十七章 优化与可持久化	399
157.1 Max-plus 矩阵幂	399
157.2 离散李超树与最优直线编号	399
157.3 主席树：静态区间第 k 小	401
第一百五十八章 可持久化数组	404
158.1 可持久化数组：历史版本分叉与单点修改	404
第一百五十九章 可持久化区间操作	407
159.1 可持久化区间加与版本拼接	407
第一百六十章 树上路径第 k 小	410
160.1 树上路径点权第 k 小	410
第一百六十一章 动态区间顺序统计	413
161.1 动态区间第 k 小（树状数组套权值线段树）	413
第一百六十二章 区间不同数统计	416
162.1 静态区间不同数计数（前驱位置主席树）	416
第一百六十三章 图论进阶	418
163.1 矩形二分图最小权匹配	418
163.2 有向最小树形图	420
163.3 无向全局最小割	422

第一百六十四章 最小割树	424
164.1 Gomory–Hu 最小割树	424
第一百六十五章 最小割树查询	425
165.1 最小割树全点对查询	425
第一百六十六章 带权二分图匹配	426
166.1 二分图最大权匹配（缺边与可选匹配）	426
第一百六十七章 精确有界半平面交	428
167.1 整数半平面的有界多边形交	428
第一百六十八章 圆与多边形面积	431
168.1 圆与简单多边形相交面积	431
第一百六十九章 最小覆盖圆	433
169.1 随机增量最小覆盖圆	433
第一百七十章 圆的切线	435
170.1 两圆公切线与过点切线	435
第一百七十一章 浮点最近点对	437
171.1 浮点平面最近点对	437
第一百七十二章 精确几何进阶	439
172.1 整数最近点对与 Minkowski 和	439
172.2 整数三维共线、共面与方向	442
第一百七十三章 凸包支撑点查询	443
173.1 支撑点查询恢复下凸包	443
第一百七十四章 动态树	445
174.1 Link-cut tree: 动态森林与路径异或	445
第一百七十五章 随机平衡树	449
175.1 FHQ Treap: 可重集合与顺序统计	449
175.2 隐式 Treap: 序列翻转、加值与区间和	452
第一百七十六章 伸展树	456
176.1 Splay: 可重集合与顺序统计	456
第一百七十七章 状态序列维护	461
177.1 状态序列 Treap: 分组区间 GCD	461
第一百七十八章 一般图匹配	465
178.1 一般图最大基数匹配: 带花树	465
第一百七十九章 线性递推	468
179.1 Berlekamp–Massey: 最短线性递推	468
179.2 特征多项式求线性递推第 k 项	469
第一百八十章 快速线性递推	470
180.1 快速递推与有理生成函数单项	470

第三部分 数学速查与维护

472

第一百八十一章 数学速查：条件先于公式	473
181.1 整除、卷积与筛法	473
181.2 杜教筛：预筛与缓存都不可省略	473
181.3 64 位判素与质因数分解	473
181.4 单个数的欧拉函数	474
181.5 线性筛：三张表的约定	474
181.6 区间筛：只保存目标区间	475
181.7 矩形 GCD 计数：缩放后容斥	475
181.8 整数分拆：重数限制的差一问题	475
181.9 幂的约数和：不除以 $p - 1$	476
181.10 类欧几里德：有符号整除求和	476
181.11 类欧几里德的三个求和量	477
181.12 模运算的适用条件	477
181.13 任意数组的批量逆元	477
181.14 连续整数的线性逆元表	478
181.15 一次不定方程：特解与正整数解范围	478
181.16 线性同余：最小解与最小周期	479
181.17 扩展 CRT：逐式合并	479
181.18 Garner：总周期可以超过整数范围	479
181.19 素数模平方根	480
181.20 乘法阶与原根	480
181.21 Carmichael：最小公共指数	480
181.22 Lucas：按素数进制逐位计算	481
181.23 合数模数组合数与插值	481
181.24 阶乘表：组合数与排列数	481
181.25 组合计数	482
181.26 群作用计数	482
181.27 生成函数与递推	482
181.28 线性代数与生成树	482
181.29 模线性方程组：工厂工期	483
181.30 概率、期望与博弈	483
181.31 数值算法与几何	483
181.32 线性递推：系数方向与外推条件	483
181.33 素数模高次剩余与单位根	484
181.34 素数幂模高次剩余与根的提升	485
181.35 一般合数模高次剩余：CRT 组合	486
181.36 自动准备模数分解与原根	487
181.37 带权矩阵树定理与方向约定	488
181.38 任意正模数行列式	489
181.39 任意正模数的矩阵树计数	490
181.40 斯特林数的快速整行计算	491
181.41 参数化 NTT 与模数适配	492
181.42 Bostan–Mori 与快速递推单项	493
181.43 集合变换与不相交子集卷积	494
181.44 Taylor 平移与等比点求值	495
181.45 可持久化区间拼接的懒标记	496

181.46	杭州 2023 K：跨版本拼接应用	497
181.47	非负权树的点集直径摘要	498
181.48	函数图的入环链与到达查询	499
181.49	杭州 2023 H：随机事件顺序与函数图	500
第一百八十二章 计算几何：精确判定与适用范围		501
182.1	整数系数的有界半平面交	501
182.2	最近点对、Minkowski 和与三维整数判定	502
182.3	浮点最近点对	503
182.4	公切线与过点切线	504
182.5	随机增量最小覆盖圆	505
182.6	圆与简单多边形相交面积	506
第一百八十三章 来源与覆盖维护		507
183.1	参考基线	507
183.2	后续覆盖项	507

使用与验证

英文署名已于 QOJ 队伍账号 `ucup-team7502` 核对。两套风格依据训练提交 2932575、2930805、2931279 提取，不擅自将共享队号的提交归属到个人。

代码采用 C++20。以赛时快速抄写为先，一行一条语句，函数体展开。无状态的简单算法使用独立函数，有状态算法使用轻量 `struct`。动态版默认需要 `bits/stdc++.h` 和 `cassert`；传统版各头文件列明依赖。抄写单个函数或结构体时需同时抄写其依赖结构。同名的不同风格实现通常用于不同提交，不建议混在同一编译单元中。点号除特别注明外为 1-based，字符串和多项式为 0-based；所有区间约定应以条目说明为准。

测试源码位于 `tests/`，固定种子便于复现。验证包含独立暴力参考、边界案例、静态与动态实现、AddressSanitizer 和 UndefinedBehaviorSanitizer。`verification/` 保存实际运行记录与源码摘要；OJ 状态只在获得可查提交结果后登记。最大流记录为 <https://www.luogu.com.cn/record/297501480> 与 <https://www.luogu.com.cn/record/297502364>。

原始来源目录共登记 713 项，见 `docs/coverage.csv`。它包含不同版本、例题及公式，不应将其数目视为独立算法数，也不能将“已登记”视为“已完成”。

第一部分

动态容器风格

第一章 Tarjan 与缩点

1.1 Tarjan 强连通分量与缩点

TarjanSCC(n), 点号 1..n; add(u,v) 加有向边, run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号, 跨分量边总从大编号指向小编号, 可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表, 保留重边; 若 DP 需要对不同邻点去重, 调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储 $O(n+m)$, 递归 DFS 另需 $O(n)$ 调用栈; 本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
1 struct TarjanSCC
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<int>> g;
5     vector<int> dfn, low, bel, st;
6
7     TarjanSCC(int n) : n(n), g(n + 1)
8     {
9     }
10
11     void add(int u, int v)
12     {
13         assert(1 <= u && u <= n && 1 <= v && v <= n);
14         g[u].push_back(v);
15     }
16
17     void dfs(int u)
18     {
19         dfn[u] = low[u] = ++timer;
20         st.push_back(u);
21         for (int v : g[u])
22         {
23             if (!dfn[v])
24             {
25                 dfs(v);
26                 low[u] = min(low[u], low[v]);
27             }
28             else if (!bel[v])
29                 low[u] = min(low[u], dfn[v]);
30         }
31         if (low[u] == dfn[u])
32         {
33             ++cnt;
34             while (true)
35             {
36                 int v = st.back();
37                 st.pop_back();
38                 bel[v] = cnt;
39                 if (v == u)
40                     break;
41             }
42         }
43     }
```

```
44
45 // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
46 void run()
47 {
48     timer = 0;
49     cnt = 0;
50     dfn.assign(n + 1, 0);
51     low = dfn;
52     bel = dfn;
53     st.clear();
54     for (int u = 1; u <= n; u++)
55     {
56         if (!dfn[u])
57             dfs(u);
58     }
59 }
60
61 // Retains parallel edges. Use after run().
62 vector<vector<int>> dag() const
63 {
64     vector<vector<int>> result(cnt + 1);
65     for (int u = 1; u <= n; u++)
66     {
67         for (int v : g[u])
68         {
69             if (bel[u] != bel[v])
70                 result[bel[u]].push_back(bel[v]);
71         }
72     }
73     return result;
74 }
75 };
```

第二章 函数图

2.1 函数图分解、跳转与到达

FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。 $O(n \log n)$ 预处理及空间，入环前查询 $O(\log n)$ ，入环后取模 $O(1)$ ，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。 $n \leq \text{INT_MAX}$ ，实际规模须满足内存。传统接口 Functional_Graph::Build / Advance / Steps。

```
1 struct FunctionalGraph
2 {
3     using U = unsigned long long;
4     int n;
5     vector<int> component, position, entry, depth, order;
6     vector<vector<int>> cycles, up;
7
8     explicit FunctionalGraph(vector<int> to = {})
9     {
10         build(move(to));
11     }
12
13     void build(vector<int> to)
14     {
15         assert(to.size() <= INT_MAX);
16         n = to.size();
17         vector<int> degree(n);
18         for (int v : to)
19         {
20             assert(0 <= v && v < n);
21             degree[v]++;
22         }
23         order.clear();
24         for (int u = 0; u < n; u++)
25             if (degree[u] == 0)
26                 order.push_back(u);
27         for (int i = 0; i < int(order.size()); i++)
28         {
29             int v = to[order[i]];
30             if (--degree[v] == 0)
31                 order.push_back(v);
32         }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```

33     component.assign(n, -1);
34     position.assign(n, -1);
35     entry.resize(n);
36     depth.assign(n, 0);
37     cycles.clear();
38     for (int u = 0; u < n; u++)
39         if (degree[u] && component[u] == -1)
40         {
41             vector<int> cycle;
42             int v = u, id = cycles.size();
43             do
44             {
45                 component[v] = id;
46                 position[v] = cycle.size();
47                 entry[v] = v;
48                 cycle.push_back(v);
49                 v = to[v];
50             } while (v != u);
51             cycles.push_back(move(cycle));
52         }
53     for (int i = int(order.size()) - 1; i >= 0; i--)
54     {
55         int u = order[i], v = to[u];
56         component[u] = component[v];
57         entry[u] = entry[v];
58         depth[u] = depth[v] + 1;
59     }
60     up.assign(1, to);
61     for (int k = 1; (1ULL << k) <= U(n); k++)
62     {
63         vector<int> row(n);
64         for (int u = 0; u < n; u++)
65             row[u] = up[k - 1][up[k - 1][u]];
66         up.push_back(move(row));
67     }
68 }
```

跳转与最少步数接口（接上页同一结构体）：

```

70     int advance(int u, U k) const
71     {
72         assert(0 <= u && u < n);
73         if (k >= U(depth[u]))
74         {
75             k -= depth[u];
76             u = entry[u];
77             const auto &cycle = cycles[component[u]];
78             return cycle[(U(position[u] + k % cycle.size()) % cycle.size())];
79         }
80         for (int bit = 0; k; bit++, k >>= 1)
81             if (k & 1)
82                 u = up[bit][u];
83         return u;
84     }
85
86     int steps(int u, int v) const
87     {
88         assert(0 <= u && u < n && 0 <= v && v < n);
89         if (component[u] != component[v] || depth[u] < depth[v])
90             return -1;
91         if (depth[v] > 0)
92         {
93             int d = depth[u] - depth[v];
94             return advance(u, d) == v ? d : -1;
95         }
96         int offset = position[v] - position[entry[u]];
97         if (offset < 0)
98             offset += int(cycles[component[u]].size());
99         return depth[u] + offset;
100     }
101 };

```

第三章 双连通分量与圆方树

3.1 Tarjan 点双、边双与圆方树

Biconnected(n), 点号 1..n; add(u,v) 返回原边的 0-based 编号, run() 从头计算。blocks 为点双的顶点列表: 桥的两端构成一个块, 孤立点独立成块; 同一个割点可出现在多个块。cut[u] 标记割点, bridge[id] 标记桥; bel[u] 是 1..cnt 的边双编号。block_forest() 返回圆方森林, 原点 1..n, blocks[i] 的方点编号 n+i+1; 孤立点也连接自己的单点方块。bridge_forest() 返回边双缩点森林, 邻接项为 (相邻分量, 原桥编号)。重边按边号处理; 自环保留原边编号但不参与 DFS, 不是桥, 不额外产生点双。全程 $O(n+m)$, 递归 DFS, 分量栈不替代递归调用。两种森林必须在 run 后获取。

```
1 struct Biconnected
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<vector<int>> blocks;
7     vector<int> dfn, low, cut, bridge, bel, st;
8
9     Biconnected(int n) : n(n), g(n + 1)
10    {
11    }
12
13    int add(int u, int v)
14    {
15        assert(1 <= u && u <= n && 1 <= v && v <= n);
16        int id = edges.size();
17        edges.push_back({u, v});
18        if (u != v)
19        {
20            g[u].push_back({v, id});
21            g[v].push_back({u, id});
22        }
23        return id;
24    }
25
26    void dfs(int u, int parent_edge)
27    {
28        dfn[u] = low[u] = ++timer;
29        st.push_back(u);
30        int children = 0;
31        for (auto [v, id] : g[u])
32        {
33            if (id == parent_edge)
34                continue;
35            if (!dfn[v])
36            {
37                children++;
38                dfs(v, id);
39                low[u] = min(low[u], low[v]);
40                if (low[v] > dfn[u])
41                    bridge[id] = 1;
```

```

42         if (low[v] >= dfn[u])
43         {
44             if (parent_edge != -1 || children > 1)
45                 cut[u] = 1;
46             blocks.push_back({u});
47             while (true)
48             {
49                 int x = st.back();
50                 st.pop_back();
51                 blocks.back().push_back(x);
52                 if (x == v)
53                     break;
54             }
55         }
56     }
57     else
58         low[u] = min(low[u], dfn[v]);
59 }
60 if (parent_edge == -1 && children == 0)
61     blocks.push_back({u});
62 }
63
64 void paint(int u)
65 {
66     bel[u] = cnt;
67     for (auto [v, id] : g[u])
68     {
69         if (!bridge[id] && !bel[v])
70             paint(v);
71     }
72 }
73
74 void run()
75 {
76     timer = 0;
77     cnt = 0;
78     dfn.assign(n + 1, 0);
79     low = dfn;
80     cut = dfn;
81     bel = dfn;
82     bridge.assign(edges.size(), 0);
83     blocks.clear();
84     st.clear();
85     for (int u = 1; u <= n; u++)
86     {
87         if (!dfn[u])
88         {
89             dfs(u, -1);
90             st.pop_back();
91         }
92     }
93     for (int u = 1; u <= n; u++)
94     {
95         if (bel[u])
96             continue;
97         ++cnt;

```



```

98         paint(u);
99     }
100 }
101
102 // Original vertices 1..n; block i has node n+i+1. Includes isolated vertices.
103 vector<vector<int>> block_forest() const
104 {
105     vector<vector<int>> tree(n + blocks.size() + 1);
106     for (int i = 0; i < (int)blocks.size(); i++)
107     {
108         int b = n + i + 1;
109         for (int u : blocks[i])
110         {
111             tree[u].push_back(b);
112             tree[b].push_back(u);
113         }
114     }
115     return tree;
116 }
117
118 // Edge-connected component IDs 1..cnt; forest edges carry original edge IDs.
119 vector<vector<pair<int, int>>> bridge_forest() const
120 {
121     vector<vector<pair<int, int>>> tree(cnt + 1);
122     for (int id = 0; id < (int)edges.size(); id++)
123     {
124         if (!bridge[id])
125             continue;
126         auto [u, v] = edges[id];
127         u = bel[u];
128         v = bel[v];
129         tree[u].push_back({v, id});
130         tree[v].push_back({u, id});
131     }
132     return tree;
133 }
134 };

```

第四章 奇环顶点判定

4.1 属于简单奇环的全部顶点

先对 Biconnected 调用 run, 再调用 odd_cycle_vertices(graph), 返回 answer[1..n], 值为 1 表示该点属于某个长度至少为 3 的简单奇环, answer[0]=0。允许重边、断开图及空图; 自环不计入这里的奇环。某点双非二分当且仅当其中存在奇环, 此时该点双每个顶点都属于某个奇环; 答案为所有非二分点双的顶点并集。通过圆方森林把每条非自环边分到唯一点双: 两端中森林深度较大者的父方点即该块, 同深时二者父方点相同。随后只对块内边染色, 避免在每个块中重复扫描割点的全部邻边。两次 DFS 均递归, 分解后额外 $O(n+m)$ 时间和空间。可重排 blocks, 不能改变分解内容。六点以内所有简单图枚举奇环对拍, 另有重边/自环、长链、星形和大奇环测试; POJ2942 补图驱动本地验证, 在线待补。依赖 biconnected.hpp, 传统版函数为 Odd_Cycle_Vertices。

```
1 // After run(). Simple odd cycles have length >= 3; self-loops are ignored.
2 inline vector<int> odd_cycle_vertices(const Biconnected &graph)
3 {
4     int n = graph.n;
5     auto forest = graph.block_forest();
6     vector<int> parent(forest.size(), depth(forest.size()));
7     auto root_tree = [&](auto &&self, int u, int p) -> void
8     {
9         parent[u] = p;
10        for (int v : forest[u])
11            if (v != p)
12            {
13                depth[v] = depth[u] + 1;
14                self(self, v, u);
15            }
16    };
17    for (int u = 1; u <= n; u++)
18        if (!depth[u])
19        {
20            depth[u] = 1;
21            root_tree(root_tree, u, 0);
22        }
```

```

23     vector<vector<pair<int, int>>> edges(graph.blocks.size());
24     for (auto [u, v] : graph.edges)
25         if (u != v)
26         {
27             int block = parent[depth[u] >= depth[v] ? u : v] - n - 1;
28             assert(0 <= block && block < (int)edges.size());
29             edges[block].push_back({u, v});
30         }
31     vector<vector<int>> g(n + 1);
32     vector<int> color(n + 1), answer(n + 1);
33     for (int block = 0; block < (int)edges.size(); block++)
34     {
35         for (int u : graph.blocks[block])
36         {
37             g[u].clear();
38             color[u] = 0;
39         }
40         for (auto [u, v] : edges[block])
41         {
42             g[u].push_back(v);
43             g[v].push_back(u);
44         }
45         bool bipartite = true;
46         auto paint = [&](auto &&self, int u) -> void
47         {
48             for (int v : g[u])
49                 if (!color[v])
50                 {
51                     color[v] = -color[u];
52                     self(self, v);
53                 }
54             else if (color[v] == color[u])
55                 bipartite = false;
56         };
57         int root = graph.blocks[block][0];
58         color[root] = 1;
59         paint(paint, root);
60         if (!bipartite)
61             for (int u : graph.blocks[block])
62                 answer[u] = 1;
63     }
64     return answer;
65 }

```

第五章 删点连通性

5.1 由点双分解计算删点连通块数

在 `Biconnected.run()` 后调用 `removal_components(graph)`, 返回 (原连通块数 `before`, 删点结果 `after`)。 `after[u]` 为独立删除原顶点 `u` 后剩余连通块数, 点号 `1..n`; `after[0]=0` 不参与计算。传统版 `Removal_Components(graph)` 自动推导 `Biconnected_Graph<N>` 的 `N`。 `before` 等于边双数量减去桥数量; `after[u]` 等于 `before-1` 加包含 `u` 的非单点点双数量。单点块必须跳过, 因此孤立点删去后会少一个连通块。空图返回 `(0,{0})`。不修改原分解, $O(n+m)$ 时间、 $O(n)$ 额外空间。若只需要割点、桥和删点增量, 直接用更短的 `Lowlink/Low_Link`; 已经需要点双分解时可用本函数, 避免再次 DFS。

```
1 // Call after run(). Each vertex is deleted independently from the original graph.
2 inline pair<int, vector<int>> removal_components(const Biconnected &graph)
3 {
4     int before = graph.cnt - count(graph.bridge.begin(), graph.bridge.end(), 1);
5     vector<int> after(graph.n + 1, before - 1);
6     after[0] = 0;
7     for (const auto &block : graph.blocks)
8         if (block.size() > 1)
9             for (int u : block)
10                 after[u]++;
11     return {before, after};
12 }
```

第六章 边双缩点与边定向

6.1 边双缩点的大小与内部边数

在 `Biconnected.run()` 后构造 `EdgeCompression(graph)`, 传统版 `Edge_Compression(graph)` 自动推导 `Biconnected_Graph<N>` 的 `N`。 `n` 为边双数量; `size[c]` 为第 `c` 块的原顶点数, `inside[c]` 为块内逻辑无向边数, 分量号 `1..n`, `0` 号槽忽略。 `edges` 每项为 {块 `u`, 块 `v`, 原边 `id`} , 只含跨块的桥, 每条桥出现一次, 按原边号递增; 是无向森林, 不承诺端点为父子方向。 重边逐条计数, 自环在所属块内部计一次, 不能用邻接表度数直接计边而漏掉这些约定。 空图返回 `n=0`、长度 `1` 的零计数数组和空边表。 时间 $O(N+M)$, 输出空间 $O(N+M)$, N/M 指原图规模。

```
1 struct EdgeCompression
2 {
3     int n;
4     vector<int> size, inside;
5     vector<array<int, 3>> edges;
6
7     EdgeCompression(const Biconnected &graph) : n(graph.cnt), size(n + 1), inside(n + 1)
8     {
9         for (int u = 1; u <= graph.n; u++)
10             size[graph.bel[u]]++;
11         for (int id = 0; id < (int)graph.edges.size(); id++)
12         {
13             auto [u, v] = graph.edges[id];
14             u = graph.bel[u];
15             v = graph.bel[v];
16             if (u == v)
17                 inside[u]++;
18             else
19                 edges.push_back({u, v, id});
20         }
21     }
22 };
```

6.2 无向边定向与强连通定向

`orient_edges(graph)` 接受 Lowlink 对象，传统版 `Orient_Edges` 接受 `Low_Link`。返回长度等于逻辑边数的方向数组，第 `id` 项为该边定向后的 {起点, 终点}，顶点 1-based、边 `id` 为 0-based。每条无向边仅处理一次：DFS 树边向下，非树边指向祖先，自环保留 {u,u}，重边不合并。支持非连通图；每个边双块内定向后强连通，桥只给出某个方向，不保证跨桥互相可达。若要求整个非空图强连通，需另外 `run/Run` 检查 `components==1` 且无桥；不满足则不存在这样的方向。该函数只读取邻接表与边数，不依赖已有 DFS 状态，不修改原图，调用时无须先 `run`。复杂度 $O(N+M)$ 时间、 $O(N+M)$ 额外空间，DFS 保持递归。

```

1 // Tree edges go down; other edges go to ancestors. Handles each logical edge once.
2 inline vector<pair<int, int>> orient_edges(const Lowlink &graph)
3 {
4     vector<pair<int, int>> direction(graph.edges);
5     vector<int> seen(graph.n + 1);
6     auto dfs = [&](auto &&self, int u) -> void
7     {
8         seen[u] = 1;
9         for (auto [v, id] : graph.g[u])
10         {
11             if (direction[id].first)
12                 continue;
13             direction[id] = {u, v};
14             if (!seen[v])
15                 self(self, v);
16         }
17     };
18     for (int u = 1; u <= graph.n; u++)
19         if (!seen[u])
20             dfs(dfs, u);
21     return direction;
22 }
```

第七章 桥树最少加边

7.1 连通无向图最少加边方案

先对非空连通无向图调用 `Biconnected.run()`，再调用 `bridge_augmentation(graph)`。返回原图点号构成的新增边列表，不修改原图。允许新增重边，不产生自环；不适用于非连通图。已无桥时返回空列表。桥树有 L 个叶子时最少加 $(L+1)/2$ 条边（整数除法）。递归 DFS 顺序收集叶子，奇数时复制首叶，前后半段配对。每条桥一侧的叶子是循环连续区间，半周配对必跨过该割，故所有桥被消除；每个叶子至少需要一个新边端点，得到同样的下界。分解后本函数 $O(n+m)$ 时间与空间。依赖 `biconnected.hpp`。P2860 双风格完整程序本地验证；新增方案以逐边删除连通性独立验证，尚无本库在线 AC。

```
1 // Connected, nonempty graph after run(); new parallel edges are allowed.
2 inline vector<pair<int, int>> bridge_augmentation(const Biconnected &graph)
3 {
4     assert(graph.n > 0 && graph.cnt > 0);
5     int bridges = count(graph.bridge.begin(), graph.bridge.end(), 1);
6     assert(graph.cnt - bridges == 1);
7     if (graph.cnt == 1)
8         return {};
9     auto tree = graph.bridge_forest();
10    vector<int> representative(graph.cnt + 1);
11    for (int u = 1; u <= graph.n; u++)
12        representative[graph.bel[u]] = u;
13    vector<int> leaves;
14    auto dfs = [&](auto &&self, int u, int parent) -> void
15    {
16        if (tree[u].size() == 1)
17            leaves.push_back(u);
18        for (auto [v, id] : tree[u])
19            if (v != parent)
20                self(self, v, u);
21    };
22    dfs(dfs, 1, 0);
23    if (leaves.size() % 2)
24        leaves.push_back(leaves[0]);
25    int half = leaves.size() / 2;
26    vector<pair<int, int>> added;
27    for (int i = 0; i < half; i++)
28        added.push_back({representative[leaves[i]], representative[leaves[i + half]]});
29    return added;
30 }
```

第八章 数据结构

8.1 并查集

`init(n)`、`find(x)`、`merge(x,y)`，点号 $1..n$ ；`siz[find(x)]` 为集合大小。路径压缩与按大小合并，均摊 $O(\alpha(n))$ 。

```
1 struct DSU
2 {
3     vector<int> fa, siz;
4
5     DSU(int n = 0)
6     {
7         init(n);
8     }
9
10    void init(int n)
11    {
12        fa.resize(n + 1);
13        iota(fa.begin(), fa.end(), 0);
14        siz.assign(n + 1, 1);
15    }
16
17    int find(int x)
18    {
19        while (x != fa[x])
20        {
21            fa[x] = fa[fa[x]];
22            x = fa[x];
23        }
24        return x;
25    }
26
27    bool merge(int x, int y)
28    {
29        x = find(x);
30        y = find(y);
31        if (x == y)
32            return false;
33        if (siz[x] < siz[y])
34            swap(x, y);
35        fa[y] = x;
36        siz[x] += siz[y];
37        return true;
38    }
39 };
```


8.2 可撤销并查集

构造后 `snapshot()` 保存历史长度，`rollback(t)` 回退。禁止路径压缩；`find/merge` 为 $O(\log n)$ ，回退每次成功合并 $O(1)$ 。

```

1 struct RollbackDSU
2 {
3     vector<int> fa, siz;
4     vector<pair<int, int>> his;
5
6     RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1)
7     {
8         iota(fa.begin(), fa.end(), 0);
9     }
10
11     int find(int x) const
12     {
13         while (x != fa[x])
14             x = fa[x];
15         return x;
16     }
17
18     int snapshot() const
19     {
20         return (int)his.size();
21     }
22
23     bool merge(int x, int y)
24     {
25         x = find(x);
26         y = find(y);
27         if (x == y)
28             return false;
29         if (siz[x] < siz[y])
30             swap(x, y);
31         his.push_back({y, siz[x]});
32         fa[y] = x;
33         siz[x] += siz[y];
34         return true;
35     }
36
37     void rollback(int t)
38     {
39         assert(0 <= t && t <= snapshot());
40         while (snapshot() > t)
41         {
42             auto [y, s] = his.back();
43             his.pop_back();
44             siz[fa[y]] = s;
45             fa[y] = y;
46         }
47     }
48 };

```

8.3 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。

```

1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1)
7     {
8     }
9
10    void add(int x, T v)
11    {
12        assert(x >= 1);
13        for (; x <= n; x += x & -x)
14            a[x] += v;
15    }
16
17    T sum(int x) const
18    {
19        T ans = 0;
20        for (; x; x -= x & -x)
21            ans += a[x];
22        return ans;
23    }
24
25    T query(int l, int r) const
26    {
27        return sum(r) - sum(l - 1);
28    }
29
30    // Nonnegative frequencies; returns n+1 if k exceeds total.
31    int kth(T k) const
32    {
33        assert(k > 0);
34        int x = 0, b = 1;
35        while (b <= n / 2)
36            b *= 2;
37        for (; b; b >>= 1)
38            if (x + b <= n && a[x + b] < k)
39            {
40                k -= a[x + b];
41                x += b;
42            }
43        return x + 1;
44    }
45 };

```

8.4 区间加与区间和

初值全零； $\text{add}(l,r,v)$ 、 $\text{query}(l,r)$ ， $1 \leq l \leq r \leq n$ 。维护 sum 与加法 tag ；所有中间和与长度乘积须在 long long 范围内。单次 $O(\log n)$ 。

```

1 struct LazySeg
2 {
3     int n;
4     vector<long long> sum, tag;
5
6     LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4)
7     {
8         assert(n > 0);
9     }
10
11     void apply(int p, int l, int r, long long v)
12     {
13         sum[p] += (r - l + 1) * v;
14         tag[p] += v;
15     }
16
17     void push(int p, int l, int r)
18     {
19         int m = (l + r) / 2;
20         apply(p * 2, l, m, tag[p]);
21         apply(p * 2 + 1, m + 1, r, tag[p]);
22         tag[p] = 0;
23     }
24
25     void add(int l, int r, long long v)
26     {
27         add(1, 1, n, l, r, v);
28     }
29
30     void add(int p, int l, int r, int ql, int qr, long long v)
31     {
32         if (ql <= l && r <= qr)
33         {
34             apply(p, l, r, v);
35             return;
36         }
37         push(p, l, r);
38         int m = (l + r) / 2;
39         if (ql <= m)
40             add(p * 2, l, m, ql, qr, v);
41         if (qr > m)
42             add(p * 2 + 1, m + 1, r, ql, qr, v);
43         sum[p] = sum[p * 2] + sum[p * 2 + 1];
44     }
45
46     long long query(int l, int r)
47     {
48         return query(1, 1, n, l, r);
49     }
50
51     long long query(int p, int l, int r, int ql, int qr)
52     {
53         if (ql <= l && r <= qr)
54             return sum[p];
55         push(p, l, r);

```

```
56     int m = (l + r) / 2;
57     long long ans = 0;
58     if (ql <= m)
59         ans += query(p * 2, l, m, ql, qr);
60     if (qr > m)
61         ans += query(p * 2 + 1, m + 1, r, ql, qr);
62     return ans;
63 }
64 };
```

8.5 64 位异或线性基

值类型 U 为 `unsigned long long`，完整支持 64 位。默认空基；`init/Init` 清空。`insert/Insert` 返回是否增秩；`contains/Contains` 判断可表示性（允许空子集，所以 0 总可表示）。`query/Query(x=0)` 返回 x 与所张成空间异或后的最大无符号值。`kth/Kth(k,nonempty=true)` 返回第 k 小的不同异或值，1-based，参数 k 为 `__uint128_t`，非法序号返回 `nullopt`。默认只允许非空输入子集，0 仅在 `dependent` 为真时出现；传 `false` 则包括空子集所得的 0。`minimum/Minimum` 同此约定，空输入的非空最小值不存在。`merge/Merge` 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。`rank` 是维数，`dependent` 表示存在非空零异或子集。插入、成员判断、最大值为 $O(64)$ ；合并 $O(64^2)$ 。`kth` 会在需要时 `rebuild/Rebuild` 规范化缓存，首次有效查询 $O(64^2)$ ，后续 $O(64)$ ；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```

1 struct XorBasis
2 {
3     using U = unsigned long long;
4     using Wide = __uint128_t;
5     U a[64]{};
6     vector<U> p;
7     int rank = 0;
8     bool dependent = false, dirty = false;
9
10    void init()
11    {
12        fill(a, a + 64, 0);
13        p.clear();
14        rank = 0;
15        dependent = false;
16        dirty = false;
17    }
18
19    bool insert(U x)
20    {
21        for (int i = 63; i >= 0; i--)
22            if (x >> i & 1)
23            {
24                if (!a[i])
25                {
26                    a[i] = x;
27                    rank++;
28                    dirty = true;
29                    return true;
30                }
31                x ^= a[i];
32            }
33        dependent = true;
34        return false;
35    }

```

```
37     bool contains(U x) const
38     {
39         for (int i = 63; i >= 0; i--)
40             if (x >> i & 1)
41                 x ^= a[i];
42         return x == 0;
43     }
44
45     U query(U x = 0) const
46     {
47         for (int i = 63; i >= 0; i--)
48             x = max(x, x ^ a[i]);
49         return x;
50     }
51
52     void rebuild()
53     {
54         vector<U> b(a, a + 64);
55         for (int i = 0; i < 64; i++)
56             if (b[i])
57                 for (int j = i + 1; j < 64; j++)
58                     if (b[j] >> i & 1)
59                         b[j] ^= b[i];
60         p.clear();
61         for (U x : b)
62             if (x)
63                 p.push_back(x);
64         dirty = false;
65     }
```

```

67 // k is 1-based; nonempty excludes the empty input subset, not value zero.
68 optional<U> kth(Wide k, bool nonempty = true)
69 {
70     if (!k)
71         return nullopt;
72     Wide index = k;
73     if (!nonempty || dependent)
74         index--;
75     if (index >= (Wide(1) << rank))
76         return nullopt;
77     if (dirty)
78         rebuild();
79     U answer = 0;
80     for (int i = 0; i < rank; i++)
81         if (index >> i & 1)
82             answer ^= p[i];
83     return answer;
84 }
85
86 optional<U> minimum(bool nonempty = true)
87 {
88     return kth(1, nonempty);
89 }
90
91 void merge(const XorBasis &other)
92 {
93     dependent |= other.dependent;
94     for (U x : other.a)
95         if (x)
96             insert(x);
97 }
98 };

```

第九章 带位置线性基

9.1 带位置限制的异或线性基

默认构造为空，init/Init 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作 $O(64)$ ，空间 $O(64)$ 。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为 $O(64n)$ 。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```
1 struct PositionBasis
2 {
3     using U = unsigned long long;
4     U a[64];
5     int pos[64];
6
7     PositionBasis()
8     {
9         init();
10    }
11
12    void init()
13    {
14        fill(a, a + 64, 0);
15        fill(pos, pos + 64, -1);
16    }
```



```

18 void insert(U x, int at)
19 {
20     assert(at >= 0);
21     for (int i = 63; i >= 0; i--)
22         if (x >> i & 1)
23             {
24                 if (!a[i])
25                     {
26                         a[i] = x;
27                         pos[i] = at;
28                         return;
29                     }
30                 if (at > pos[i])
31                     {
32                         swap(a[i], x);
33                         swap(pos[i], at);
34                     }
35                 x ^= a[i];
36             }
37 }
38
39 bool contains(U x, long long lower = 0) const
40 {
41     assert(lower >= 0);
42     for (int i = 63; i >= 0; i--)
43         if ((x >> i & 1) && pos[i] >= lower)
44             x ^= a[i];
45     return x == 0;
46 }
47
48 U query(long long lower = 0, U x = 0) const
49 {
50     assert(lower >= 0);
51     for (int i = 63; i >= 0; i--)
52         if (pos[i] >= lower)
53             x = max(x, x ^ a[i]);
54     return x;
55 }
56
57 int rank(long long lower = 0) const
58 {
59     assert(lower >= 0);
60     int answer = 0;
61     for (int i = 0; i < 64; i++)
62         answer += pos[i] >= lower;
63     return answer;
64 }
65 };

```

第十章 线性空间求交

10.1 异或线性空间求交

basis_intersection(a,b) 返回 span(a) 与 span(b) 的交空间的一组独立生成元，结果类型 XorBasis；传统版 Basis_Intersection 接受并返回 Xor_Basis。支持完整 64 位无符号值，不修改输入，也不要求先 rebuild。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间 $O(64^2)$ ，额外空间 $O(64)$ 。输出按独立生成元解释：dependent 为 false，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 result.kth(k,false)，包含零；零空间返回空基，rank=0、contains(0)=true。

```
1 inline XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
2 {
3     using U = unsigned long long;
4     U pivot[64], from_b[64]{};
5     copy(a.a, a.a + 64, pivot);
6     XorBasis result;
7     for (U value : b.a)
8     {
9         U x = value, part = value;
10        for (int i = 63; i >= 0 && x; i--)
11            if (x >> i & 1)
12            {
13                if (!pivot[i])
14                {
15                    pivot[i] = x;
16                    from_b[i] = part;
17                    break;
18                }
19                x ^= pivot[i];
20                part ^= from_b[i];
21            }
22        if (!x && part)
23            result.insert(part);
24    }
25    return result;
26 }
```

10.2 Zassenhaus：同时求和空间与交空间

`basis_sum_intersection(a,b)` 返回和空间基、交空间基这一对结果。传统接口名为 `Basis_Sum_Intersection`。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 `dependent` 均为 `false`。将 `a` 的向量 `x` 编成 `(x,x)`，`b` 的向量 `y` 编成 `(y,0)`，用 `__uint128_t` 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间 $O(128^2)$ ，额外空间 $O(128)$ 。结果满足 $\dim(A+B)+\dim(A \cap B)=\dim(A)+\dim(B)$ ，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```

1 // Return independent generators of (sum space, intersection space).
2 inline pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a,
3                                                         const XorBasis &b)
4 {
5     using W = __uint128_t;
6     W pivot[128]{};
7     auto insert = [&](W x)
8     {
9         for (int i = 127; i >= 0 && x; i--)
10             if (x >> i & 1)
11             {
12                 if (!pivot[i])
13                 {
14                     pivot[i] = x;
15                     return;
16                 }
17                 x ^= pivot[i];
18             }
19     };
20     for (auto x : a.a)
21         insert((W(x) << 64) | x);
22     for (auto x : b.a)
23         insert(W(x) << 64);
24     XorBasis sum, intersection;
25     for (int i = 0; i < 128; i++)
26         if (pivot[i])
27         {
28             if (i < 64)
29                 intersection.insert((unsigned long long)pivot[i]);
30             else
31                 sum.insert((unsigned long long)(pivot[i] >> 64));
32         }
33     return {sum, intersection};
34 }

```

第十一章 线段树常用修改

11.1 线段树：区间乘、加、赋值与模意义求和

`AffineSegTree(a,mod)` 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。`update(l,r,mul,add)` 将区间元素 x 改为 $mul \cdot x + add$ ；乘法用 $(mul,0)$ ，加法用 $(1,add)$ ，赋值用 $(0,value)$ 。`query(l,r)` 返回区间和。 $1 \leq mod \leq INT_MAX$ ，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 (m,b) 后应用新标记 (a,c) ，复合为 $(m \cdot a, b \cdot a + c)$ ，乘法取模后再相加。建树 $O(n)$ ，单次 $O(\log n)$ ，空间 $O(n)$ 。内部 `build`、`apply` 等函数不作为公共接口；动态版重建新对象，传统版可重复 `Init`。

```
1 struct AffineSegTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    vector<Node> t;
12
13    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
14    {
15        assert(n > 0 && mod > 0);
16        build(1, 1, n, a);
17    }
18
19    ll norm(ll x) const
20    {
21        x %= mod;
22        if (x < 0)
23            x += mod;
24        return x;
25    }
26
27    void pull(int p)
28    {
29        t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod;
30    }
31
32    void build(int p, int l, int r, const vector<ll> &a)
33    {
34        t[p] = Node{};
35        if (l == r)
36        {
37            t[p].sum = norm(a[l - 1]);
38            return;
39        }
40        int m = (l + r) / 2;
41        build(p * 2, l, m, a);
42        build(p * 2 + 1, m + 1, r, a);
```

```

43     pull(p);
44 }
45
46 // New transform follows the old pending transform: x -> x*a+b.
47 void apply(int p, int len, ll a, ll b)
48 {
49     t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
50     t[p].mul = t[p].mul * a % mod;
51     t[p].add = (t[p].add * a % mod + b) % mod;
52 }
53
54 void push(int p, int l, int r)
55 {
56     if (t[p].mul == 1 && t[p].add == 0)
57         return;
58     int m = (l + r) / 2;
59     apply(p * 2, m - l + 1, t[p].mul, t[p].add);
60     apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
61     t[p].mul = 1;
62     t[p].add = 0;
63 }
64
65 void update(int l, int r, ll a, ll b)
66 {
67     assert(1 <= l && l <= r && r <= n);
68     update(1, 1, n, l, r, norm(a), norm(b));
69 }
70
71 void update(int p, int l, int r, int ql, int qr, ll a, ll b)
72 {
73     if (ql <= l && r <= qr)
74     {
75         apply(p, r - l + 1, a, b);
76         return;
77     }
78     push(p, l, r);
79     int m = (l + r) / 2;
80     if (ql <= m)
81         update(p * 2, l, m, ql, qr, a, b);
82     if (qr > m)
83         update(p * 2 + 1, m + 1, r, ql, qr, a, b);
84     pull(p);
85 }
86
87 ll query(int l, int r)
88 {
89     assert(1 <= l && l <= r && r <= n);
90     return query(1, 1, n, l, r);
91 }
92
93 ll query(int p, int l, int r, int ql, int qr)
94 {
95     if (ql <= l && r <= qr)
96         return t[p].sum;

```

```
97     push(p, l, r);
98     int m = (l + r) / 2;
99     ll answer = 0;
100    if (ql <= m)
101        answer += query(p * 2, l, m, ql, qr);
102    if (qr > m)
103        answer += query(p * 2 + 1, m + 1, r, ql, qr);
104    return answer % mod;
105 }
106 };
```

第十二章 网络流

12.1 最大流与方案

Dinic(n), add(u,v,c) 返回偶数原边编号; flow(s,t,limit) 返回本次新增流; used(id) 查询原边净流, cut(s) 返回残量可达点集。允许重边、自环, 容量非负, $s \neq t$; 重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$, 递归增广深度 $O(V)$ 。总流不得超过 long long 上限。

```
1 struct Dinic
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll cap, initial;
9     };
10
11     int n;
12     vector<Edge> e;
13     vector<vector<int>> g;
14     vector<int> dep, cur;
15
16     Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1)
17     {
18     }
19
20     int add(int u, int v, ll c)
21     {
22         assert(c >= 0);
23         int id = (int)e.size();
24         e.push_back({u, v, c, c});
25         e.push_back({v, u, 0, 0});
26         g[u].push_back(id);
27         g[v].push_back(id + 1);
28         return id;
29     }
30
31     bool bfs(int s, int t)
32     {
33         fill(dep.begin(), dep.end(), -1);
34         queue<int> q;
35         dep[s] = 0;
36         q.push(s);
37         while (!q.empty())
38         {
39             int u = q.front();
40             q.pop();
41             for (int id : g[u])
42                 if (e[id].cap && dep[e[id].to] < 0)
43                 {
44                     dep[e[id].to] = dep[u] + 1;
```

```

45         q.push(e[id].to);
46     }
47 }
48 return dep[t] >= 0;
49 }
50
51 ll dfs(int u, int t, ll f)
52 {
53     if (u == t)
54         return f;
55     for (int &i = cur[u]; i < (int)g[u].size(); i++)
56     {
57         int id = g[u][i], v = e[id].to;
58         if (!e[id].cap || dep[v] != dep[u] + 1)
59             continue;
60         ll d = dfs(v, t, min(f, e[id].cap));
61         if (d)
62         {
63             e[id].cap -= d;
64             e[id ^ 1].cap += d;
65             return d;
66         }
67     }
68     return 0;
69 }
70
71 // Returns additional flow; reuse only with the same source/sink.
72 ll flow(int s, int t, ll limit = LLONG_MAX)
73 {
74     assert(s != t && limit >= 0);
75     ll ans = 0;
76     while (ans < limit && bfs(s, t))
77     {
78         fill(cur.begin(), cur.end(), 0);
79         while (ans < limit)
80         {
81             ll d = dfs(s, t, limit - ans);
82             if (!d)
83                 break;
84             ans += d;
85         }
86     }
87     return ans;
88 }
89
90 ll used(int id) const
91 {
92     return e[id].initial - e[id].cap;
93 }
94
95 vector<int> cut(int s)

```



```

96     {
97         bfs(s, s);
98         vector<int> ans;
99         for (int u = 1; u <= n; u++)
100             if (dep[u] >= 0)
101                 ans.push_back(u);
102         return ans;
103     }
104 };

```

12.2 势能最小费用最大流

$\text{add}(u, v, \text{cap}, \text{cost})$ 返回边编号, $\text{flow}(s, t, \text{limit})$ 返回新增流量及 128 位费用, $\text{used}(id)$ 返回原边流量。允许负费用, 初始残量网络必须没有负环; 检测到负环抛异常。Bellman-Ford 初始化 $O(VE)$, 每次增广 $O(E \log V)$; 增广次数不能简单假设为 $O(V)$ 。另要求 $\sum(\text{cap} \times |\text{cost}|) < 2^{120}$, 防止总费用超过 int128 。

```

1 struct MinCostFlow
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5
6     struct Edge
7     {
8         int from, to;
9         ll cap, cost, initial;
10    };
11
12    int n;
13    vector<Edge> e;
14    vector<vector<int>> g;
15
16    MinCostFlow(int n) : n(n), g(n + 1)
17    {
18    }
19
20    int add(int u, int v, ll cap, ll cost)
21    {
22        assert(cap >= 0 && cost != LLONG_MIN);
23        int id = (int)e.size();
24        e.push_back({u, v, cap, cost, cap});
25        e.push_back({v, u, 0, -cost, 0});
26        g[u].push_back(id);
27        g[v].push_back(id + 1);
28        return id;
29    }
30
31    ll used(int id) const
32    {
33        return e[id].initial - e[id].cap;
34    }
35
36    // Negative costs allowed; initial residual graph must have no negative cycle.
37    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
38    {
39        assert(s != t && limit >= 0);

```

```

40  const i128 inf = i128(1) << 120;
41  vector<i128> h(n + 1, 0), d(n + 1);
42  vector<int> pre(n + 1);
43  // Super-source Bellman-Ford: feasible potentials for every vertex.
44  for (int it = 0; it < n; it++)
45  {
46      bool changed = false;
47      for (auto &a : e)
48          if (a.cap && h[a.to] > h[a.from] + a.cost)
49          {
50              h[a.to] = h[a.from] + a.cost;
51              changed = true;
52          }
53      if (!changed)
54          break;
55      if (it == n - 1)
56          throw invalid_argument("negative cycle");
57  }
58  ll f = 0;
59  i128 cost = 0;
60  while (f < limit)
61  {
62      fill(d.begin(), d.end(), inf);
63      d[s] = 0;
64      priority_queue<pair<i128, int>,
65                    vector<pair<i128, int>>,
66                    greater<pair<i128, int>>>
67          q;
68      q.push({0, s});
69      while (!q.empty())
70      {
71          auto [du, u] = q.top();
72          q.pop();
73          if (du != d[u])
74              continue;
75          for (int id : g[u])
76          {
77              auto &a = e[id];
78              i128 nd = du + a.cost + h[u] - h[a.to];
79              if (a.cap && nd < d[a.to])
80              {
81                  d[a.to] = nd;
82                  pre[a.to] = id;
83                  q.push({nd, a.to});
84              }
85          }
86      }
87      if (d[t] == inf)
88          break;
89      for (int u = 1; u <= n; u++)
90          if (d[u] != inf)
91              h[u] += d[u];
92      ll take = limit - f;
93      for (int u = t; u != s; u = e[pre[u]].from)
94          take = min(take, e[pre[u]].cap);
95      for (int u = t; u != s; u = e[pre[u]].from)

```

```

96         {
97             int id = pre[u];
98             e[id].cap -= take;
99             e[id ^ 1].cap += take;
100             cost += i128(take) * e[id].cost;
101         }
102         f += take;
103     }
104     return {f, cost};
105 }
106 };

```

12.3 上下界可行循环流

$\text{add}(u, v, lo, hi)$, $0 \leq lo \leq hi$; $\text{solve}()$ 仅调用一次, 返回可行性, 仅成功后读取 $\text{used}(id)$ 。约束为每点流量守恒; 不是带源汇最大流。所有上下界及其总和须在 long long 范围内。依赖 Dinic。

```

1 struct BoundedCirculation
2 {
3     using ll = long long;
4     int n;
5     Dinic g;
6     vector<ll> balance, lower;
7     vector<int> ids;
8     bool solved = false, feasible = false;
9
10    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1)
11    {
12    }
13
14    int add(int u, int v, ll lo, ll hi)
15    {
16        assert(!solved && 0 <= lo && lo <= hi);
17        int id = ids.size();
18        ids.push_back(g.add(u, v, hi - lo));
19        lower.push_back(lo);
20        balance[u] -= lo;
21        balance[v] += lo;
22        return id;
23    }
24
25    bool solve()
26    {
27        assert(!solved);
28        solved = true;
29        ll need = 0;
30        for (int u = 1; u <= n; u++)
31            if (balance[u] > 0)
32            {
33                g.add(n + 1, u, balance[u]);
34                need += balance[u];
35            }
36        else if (balance[u] < 0)

```

```
37         g.add(u, n + 2, -balance[u]);
38     return feasible = (g.flow(n + 1, n + 2) == need);
39 }
40
41 ll used(int id) const
42 {
43     assert(feasible);
44     return lower[id] + g.used(ids[id]);
45 }
46 };
```

第十三章 最大权闭合子图

13.1 最大权闭合子图与选点方案

`maximum_closure(weight, dependencies)` 返回 (最大权值, 选中点列表), `weight` 为 0-based `vector<long long>`, 图中点号 `1..n`。依赖边 (u, v) 表示选 u 必须选 v , 允许环、自环与重边; 允许空选集, 因此答案不小于 0。依赖 `flow.hpp` 中的 `Dinic`。正权连源、负权连汇, 依赖边容量取正权总和加 1; 要求所有正权之和严格小于 `LLONG_MAX`。负权先提升到 `int128` 再取负, 并截断到依赖边容量, 因此 `LLONG_MIN` 也可处理。返回残量图源点可达的原顶点, 权值为正权总和减最小割; 超大负权边不会出现在最优割中, 截断不改变原问题最优值。每次调用重建网络。复杂度沿用 `Dinic`: 一般 $O(V^2E)$, 空间 $O(V+E)$, 增广保持递归。返回一种最优闭合集, 不承诺最大或最小选点个数。

```
1 inline pair<long long, vector<int>>
2 maximum_closure(const vector<long long> &weight,
3                 const vector<pair<int, int>> &dependencies)
4 {
5     using I = __int128_t;
6     int n = weight.size(), s = n + 1, t = n + 2;
7     I positive = 0;
8     for (long long w : weight)
9         if (w > 0)
10             positive += w;
11     assert(positive < LLONG_MAX);
12     long long inf = (long long)positive + 1;
13     Dinic graph(n + 2);
14     for (int u = 1; u <= n; u++)
15     {
16         long long w = weight[u - 1];
17         if (w > 0)
18             graph.add(s, u, w);
19         else if (w < 0)
20             graph.add(u, t, (long long)min(-I(w), I(inf)));
21     }
22     for (auto [u, v] : dependencies)
23     {
24         assert(1 <= u && u <= n && 1 <= v && v <= n);
25         graph.add(u, v, inf);
26     }
27     long long answer = (long long)positive - graph.flow(s, t);
28     vector<int> selected;
29     for (int u : graph.cut(s))
30         if (u <= n)
31             selected.push_back(u);
32     return {answer, selected};
33 }
```

第十四章 图论

14.1 非负权最短路

`add(u,v,w)`、`run(s)`，`dis` 为距离，`pre` 为前驱，`path(t)` 返回点序列。不可达用 `LLONG_MAX`；要求所有有限最短距离严格小于该哨兵， $w \geq 0$ 。 $O((V+E)\log V)$ 。

```
1 struct Dijkstra
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9
10    Dijkstra(int n) : n(n), g(n + 1)
11    {
12    }
13
14    void add(int u, int v, ll w)
15    {
16        assert(w >= 0);
17        g[u].push_back({v, w});
18    }
19
20    void run(int s)
21    {
22        dis.assign(n + 1, inf);
23        pre.assign(n + 1, -1);
24        dis[s] = 0;
25        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
26        q.push({0, s});
27        while (!q.empty())
28        {
29            auto [d, u] = q.top();
30            q.pop();
31            if (d != dis[u])
32                continue;
33            for (auto [v, w] : g[u])
34                if (w < inf - d && d + w < dis[v])
35                {
36                    dis[v] = d + w;
37                    pre[v] = u;
38                    q.push({dis[v], v});
39                }
40        }
41    }
42
43    vector<int> path(int t) const
44    {
```

```

45     if (dis[t] == inf)
46         return {};
47     vector<int> ans;
48     for (; t != -1; t = pre[t])
49         ans.push_back(t);
50     reverse(ans.begin(), ans.end());
51     return ans;
52 }
53 };

```

14.2 强连通分量

add(u,v)、run(), bel[1..n] 为分量编号, cnt 为数量; 分量编号按缩点图拓扑序递增。递归 Kosaraju, 按用户要求保留 DFS 写法; $O(V+E)$, 同时存正反图。

```

1 struct SCC
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
6
7     SCC(int n) : n(n), g(n + 1), rg(n + 1)
8     {
9     }
10
11     void add(int u, int v)
12     {
13         g[u].push_back(v);
14         rg[v].push_back(u);
15     }
16
17     void dfs1(int u, vector<int> &vis, vector<int> &order)
18     {
19         vis[u] = 1;
20         for (int v : g[u])
21         {
22             if (!vis[v])
23                 dfs1(v, vis, order);
24         }
25         order.push_back(u);
26     }
27
28     void dfs2(int u)
29     {
30         bel[u] = cnt;
31         for (int v : rg[u])
32         {
33             if (!bel[v])
34                 dfs2(v);
35         }
36     }
37
38     // Recursive Kosaraju; component IDs are in topological order, 1..cnt.
39     void run()
40     {

```

```

41     vector<int> vis(n + 1), order;
42     cnt = 0;
43     bel.assign(n + 1, 0);
44     for (int s = 1; s <= n; s++)
45     {
46         if (!vis[s])
47             dfs1(s, vis, order);
48     }
49     reverse(order.begin(), order.end());
50     for (int s : order)
51     {
52         if (!bel[s])
53         {
54             ++cnt;
55             dfs2(s);
56         }
57     }
58 }
59 };

```

14.3 2-SAT 与任意可行方案

add(x,a,y,b) 添加 (x=a) 或 (y=b), 变量 1..n; solve() 返回可满足性, 仅成功时读取 ans。依赖同文件 SCC; $O(V+E)$, 不保证字典序最小。

```

1 struct TwoSAT
2 {
3     int n;
4     SCC g;
5     vector<int> ans;
6
7     TwoSAT(int n) : n(n), g(2 * n)
8     {
9     }
10
11     int id(int x, bool value) const
12     {
13         return 2 * x - 1 + value;
14     }
15
16     void add(int x, bool a, int y, bool b)
17     {
18         g.add(id(x, !a), id(y, b));
19         g.add(id(y, !b), id(x, a));
20     }
21
22     bool solve()
23     {
24         g.run();
25         ans.assign(n + 1, 0);

```



```

26     for (int x = 1; x <= n; x++)
27     {
28         if (g.bel[id(x, 0)] == g.bel[id(x, 1)])
29             return false;
30         ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
31     }
32     return true;
33 }
34 };

```

14.4 Hopcroft–Karp 最大匹配与最小点覆盖

左右两侧分别编号 $1..n$ 、 $1..m$ ，可有空侧及重边； $\text{add}(u,v)$ 加边， $\text{solve}()$ 返回最大匹配总数， $l[u]$ 与 $r[v]$ 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve ；不支持删边或手动更改匹配数组。BFS 用 $\text{dep}[0]$ 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。 $V=n+m$ ，单次 solve 最坏 $O((V+E)\sqrt{V})$ ，空间 $O(V+E)$ 。 $\text{cover}()$ 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```

1 struct BipartiteMatching
2 {
3     int n, m;
4     vector<vector<int>> g;
5     vector<int> l, r, dep, cur;
6
7     BipartiteMatching(int n, int m)
8         : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
9     {
10    }
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= m);
15        g[u].push_back(v);
16    }
17
18    bool bfs()
19    {
20        queue<int> q;
21        fill(dep.begin(), dep.end(), n + 1);
22        fill(cur.begin(), cur.end(), 0);
23        for (int u = 1; u <= n; u++)
24        {
25            if (!l[u])
26            {
27                dep[u] = 0;
28                q.push(u);
29            }
30        }
31        while (!q.empty())
32        {
33            int u = q.front();
34            q.pop();
35            if (dep[u] >= dep[0])
36                continue;

```

```

37     for (int v : g[u])
38     {
39         int next = r[v];
40         if (dep[next] != n + 1)
41             continue;
42         dep[next] = dep[u] + 1;
43         if (next)
44             q.push(next);
45     }
46 }
47 return dep[0] != n + 1;
48 }
49
50 bool dfs(int u)
51 {
52     if (!u)
53         return true;
54     for (int &i = cur[u]; i < (int)g[u].size(); i++)
55     {
56         int v = g[u][i];
57         if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
58         {
59             l[u] = v;
60             r[v] = u;
61             return true;
62         }
63     }
64     dep[u] = n + 1;
65     return false;
66 }
67
68 // Retains an existing matching; edges may be added between calls.
69 int solve()
70 {
71     int answer = 0;
72     for (int u = 1; u <= n; u++)
73         answer += (l[u] != 0);
74     while (bfs())
75     {
76         for (int u = 1; u <= n; u++)
77         {
78             if (!l[u] && dfs(u))
79                 answer++;
80         }
81     }
82     return answer;
83 }
84
85 // Call after solve(), before adding more edges. Returns left/right vertex IDs.
86 pair<vector<int>, vector<int>> cover() const
87 {
88     vector<int> left, right;
89     for (int u = 1; u <= n; u++)
90     {
91         if (dep[u] == n + 1)
92             left.push_back(u);

```

```

93     }
94     for (int v = 1; v <= m; v++)
95     {
96         if (r[v] && dep[r[v]] != n + 1)
97             right.push_back(v);
98     }
99     return {left, right};
100 }
101 };

```

14.5 割点与桥

无向多重图，点号 1..n；add/Insert(u,v) 返回逻辑无向边的 0-based 编号，自环和重边均可输入。run/Run 后 cut[u] 为割点标记，bridge[id] 为桥标记，components 为原图连通块数。delta[u] 是独立删除顶点 u 及其关联边后增加的连通块数，剩余块数为 components+delta[u]；孤立点 delta=-1，普通非割点为 0，割点为正。根的增量为 DFS 子树数减 1；非根统计满足 low[child]>=dfn[u] 的子树数。n=1 的点删除后剩余 0 块。DFS 按父边号跳过，支持重边、非连通图和重复运行。复杂度 $O(n+m)$ ，递归 DFS；新增边后需重新 run。每个删点结果都针对原图，不表示逐次删除。

```

1 struct Lowlink
2 {
3     int n, timer = 0, components = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge, delta;
6     int edges = 0;
7
8     Lowlink(int n) : n(n), g(n + 1)
9     {
10    }
11
12     int add(int u, int v)
13     {
14         g[u].push_back({v, edges});
15         g[v].push_back({u, edges});
16         return edges++;
17    }

```

```

19 void dfs(int u, int pe)
20 {
21     dfn[u] = low[u] = ++timer;
22     int children = 0;
23     for (auto [v, id] : g[u])
24     {
25         if (id == pe)
26             continue;
27         if (!dfn[v])
28         {
29             ++children;
30             dfs(v, id);
31             low[u] = min(low[u], low[v]);
32             if (low[v] > dfn[u])
33                 bridge[id] = 1;
34             if (pe != -1 && low[v] >= dfn[u])
35             {
36                 cut[u] = 1;
37                 delta[u]++;
38             }
39         }
40         else
41             low[u] = min(low[u], dfn[v]);
42     }
43     if (pe == -1 && children > 1)
44         cut[u] = 1;
45     if (pe == -1)
46         delta[u] = children - 1;
47 }
48
49 void run()
50 {
51     timer = 0;
52     components = 0;
53     dfn.assign(n + 1, 0);
54     low = dfn;
55     cut = dfn;
56     delta = dfn;
57     bridge.assign(edges, 0);
58     for (int u = 1; u <= n; u++)
59         if (!dfn[u])
60         {
61             components++;
62             dfs(u, -1);
63         }
64 }
65 };

```

第十五章 字典序最小 2-SAT

15.1 递归染色与回滚求最小赋值

构造 LexTwoSAT(n), 变量号 $1..n$ 。add(x,a,y,b) 加入子句 $(x=a) \text{ OR } (y=b)$, force(x,v) 强制变量值; solve() 成功后 answer[$1..n$] 是按变量编号、0 优先于 1 的字典序最小赋值。失败时 answer 为空, $n=0$ 时可满足, answer[0] 始终仅为占位。每个未确定变量先尝试 0, 递归传播所有蕴含; 冲突时仅撤销本次新增标记, 再试 1。history 是撤销记录, 不替代递归调用栈。可重复求解, 追加子句会清空旧答案; marked/history 仅为内部工作区, 失败时不可当作方案。内部 $\text{id}(x,v)=2(x-1)+v$, 从 0 开始, 可用异或 1 取反; 不要混用另一份 SCC 2-SAT 的 1-based 字面量编号。最坏 $O(n(n+m))$ 时间、 $O(n+m)$ 空间, 递归栈 $O(n)$; 100000 变量测试是特定蕴含链, 不代表一般大规模输入的性能保证。HDU1814 完整程序本地对拍, 在线待补。

```
1 struct LexTwoSAT
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> marked, history, answer;
6
7     LexTwoSAT(int n) : n(n), g(2 * n), marked(2 * n)
8     {
9         assert(n >= 0);
10    }
11
12    int id(int x, bool value) const
13    {
14        assert(1 <= x && x <= n);
15        return 2 * (x - 1) + value;
16    }
17
18    // (x == a) OR (y == b). Internal literal IDs are 0-based.
19    void add(int x, bool a, int y, bool b)
20    {
21        int u = id(x, a), v = id(y, b);
22        g[u ^ 1].push_back(v);
23        g[v ^ 1].push_back(u);
24        answer.clear();
25    }
26
27    void force(int x, bool value)
28    {
29        add(x, value, x, value);
30    }
```

```

32  bool paint(int u)
33  {
34      if (marked[u ^ 1])
35          return false;
36      if (marked[u])
37          return true;
38      marked[u] = 1;
39      history.push_back(u);
40      for (int v : g[u])
41          if (!paint(v))
42              return false;
43      return true;
44  }
45
46  bool solve()
47  {
48      fill(marked.begin(), marked.end(), 0);
49      history.clear();
50      answer.clear();
51      for (int x = 1; x <= n; x++)
52      {
53          int u = id(x, false);
54          if (marked[u] || marked[u ^ 1])
55              continue;
56          int checkpoint = history.size();
57          if (!paint(u))
58          {
59              while ((int)history.size() > checkpoint)
60              {
61                  marked[history.back()] = 0;
62                  history.pop_back();
63              }
64              if (!paint(u ^ 1))
65                  return false;
66          }
67      }
68      answer.assign(n + 1, 0);
69      for (int x = 1; x <= n; x++)
70          answer[x] = marked[id(x, true)];
71      return true;
72  }
73  };

```

第十六章 有向欧拉路

16.1 有向图字典序最小欧拉路

构造 `DirectedEuler(n)`, $n \geq 1$, 点号 $1..n$; `add(u,v)` 返回从 0 开始的逻辑边编号, 支持自环与重边。`run()` 返回是否存在欧拉路; 参数 `start` 默认 0, `lexicographic` 默认 `true`。`start=0` 自动选择最小可行起点, 否则固定起点; 孤立点不影响有边部分的连通性。成功后 `vertices` 长度为 $m+1$, `edge_ids` 长度为 m , 第 i 条原边从 `vertices[i]` 指向 `vertices[i+1]`; 失败清空两数组。无边时返回单点路径, 自动起点为 1。可重复 `run`, `add` 会清空旧方案。默认求顶点序列字典序最小方案; 关闭排序后只保证任意方案, 邻接顺序可能保留上次排序。开启排序总时间 $O(n+m \log(m+1))$, 否则 $O(n+m)$, 空间 $O(n+m)$ 。DFS 递归深度可能达到 $m+1$, 不是 n 。度数检查后, 以消费边数判定是否遍历全部非孤立部分, 逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号, P7771 双风格完整程序本地验证, 在线待补。

```
1 struct DirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> in, out, next, vertices, edge_ids;
7
8     DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
9     {
10         assert(n > 0);
11     }
12
13     int add(int u, int v)
14     {
15         assert(1 <= u && u <= n && 1 <= v && v <= n);
16         int id = edges.size();
17         edges.push_back({u, v});
18         g[u].push_back({v, id});
19         out[u]++;
20         in[v]++;
21         vertices.clear();
22         edge_ids.clear();
23         return id;
24     }
25
26     void dfs(int u, int incoming)
27     {
28         while (next[u] < (int)g[u].size())
29         {
30             auto [v, id] = g[u][next[u]++];
31             dfs(v, id);
32         }
33         vertices.push_back(u);
34         if (incoming != -1)
35             edge_ids.push_back(incoming);
36     }
```

```

38 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
39 bool run(int start = 0, bool lexicographic = true)
40 {
41     assert(0 <= start && start <= n);
42     vertices.clear();
43     edge_ids.clear();
44     int source = 0, sink = 0;
45     for (int u = 1; u <= n; u++)
46     {
47         int delta = out[u] - in[u];
48         if (delta == 1 && !source)
49             source = u;
50         else if (delta == -1 && !sink)
51             sink = u;
52         else if (delta != 0)
53             return false;
54     }
55     if (bool(source) != bool(sink))
56         return false;
57     if (source)
58     {
59         if (start && start != source)
60             return false;
61         start = source;
62     }
63     if (!start)
64     {
65         start = 1;
66         for (int u = 1; u <= n; u++)
67             if (out[u])
68             {
69                 start = u;
70                 break;
71             }
72     }
73     if (lexicographic)
74         for (int u = 1; u <= n; u++)
75             sort(g[u].begin(), g[u].end());
76     fill(next.begin(), next.end(), 0);
77     dfs(start, -1);
78     if (edge_ids.size() != edges.size())
79     {
80         vertices.clear();
81         edge_ids.clear();
82         return false;
83     }
84     reverse(vertices.begin(), vertices.end());
85     reverse(edge_ids.begin(), edge_ids.end());
86     return true;
87 }
88 };

```


第十七章 字典序单词链

17.1 字典序最小单词链

`word_chain(words)` 返回可空的编号序列 (optional), 有解时为原输入的 0-based 单词编号顺序, 每个单词恰用一次; 相邻单词首尾字母相接, 单词序列字典序最小。无解返回 `nullopt`, 空输入返回有值的空序列。单词必须非空且仅含小写 a-z, 重复词视为不同编号。先对原编号按单词排序, 按此顺序加边, 再调用 `DirectedEuler.run(0,false)`; 不可让底层重新按终点排序。欧拉回路从最小出现首字母开始, 非回路起点由度数差确定。O(m log m) 次字符串比较及 O(L+m) 其他工作, L 为总字符数; 空间 O(m), 递归栈 O(m)。依赖 `directed_euler.hpp`; 传统版调用 `Word_Chain`, 内部使用 `Directed_Euler<26>`。全排列对拍、重复词/前缀词、20 万相同词及 POJ2337 完整输出本地验证, 在线待补。

```
1 // Lowercase nonempty words. Returns original 0-based IDs; duplicates stay distinct.
2 inline optional<vector<int>> word_chain(const vector<string> &words)
3 {
4     int n = words.size();
5     vector<int> order(n);
6     iota(order.begin(), order.end(), 0);
7     for (const auto &word : words)
8     {
9         assert(!word.empty());
10        for (char c : word)
11            assert('a' <= c && c <= 'z');
12    }
13    sort(order.begin(),
14         order.end(),
15         [&](int a, int b)
16         {
17             if (words[a] != words[b])
18                 return words[a] < words[b];
19             return a < b;
20         });
21    DirectedEuler graph(26);
22    for (int id : order)
23        graph.add(words[id].front() - 'a' + 1, words[id].back() - 'a' + 1);
24    // Preserve word order in each adjacency list; do not sort by destination.
25    if (!graph.run(0, false))
26        return nullopt;
27    vector<int> result;
28    for (int id : graph.edge_ids)
29        result.push_back(order[id]);
30    return result;
31 }
```

第十八章 无向欧拉路

18.1 无向图欧拉路与边方向

构造 `UndirectedEuler(n)`, $n \geq 1$, 点号 $1..n$; `add(u,v)` 返回从 0 开始的逻辑边号, 自环和重边均保留。`run()` 返回是否存在欧拉路, 参数 `start` 默认 0 (自动), `lexicographic` 默认 `true`。自动选最小可行起点; 指定起点须满足奇度条件且位于有边部分。成功得到 $m+1$ 个 `vertices` 和 m 个 `edge_ids`, 失败清空两数组。第 i 条边方向是 `vertices[i]` 到 `vertices[i+1]`, 与 `edges[edge_ids[i]]` 比较可判断是否反向, 自环任选方向。`used` 按逻辑边号标记, 使两条半边共同只消费一次; 自环贡献度数 2。孤立点不影响有边部分; 无边图返回指定点或点 1 的单点序列。默认求顶点序列字典序最小方案; 关闭排序只保证任意方案, 可能保留上次排序。可重复 `run`, 加边清空旧方案。排序开启时 $O(n+m \log(m+1))$, 否则 $O(n+m)$, 空间 $O(n+m)$, DFS 递归深度可能达到 $m+1$ 。全小图走法、原边号与方向独立验证, P2731 和 SGU101 完整程序本地验证; 在线待补。

```
1 struct UndirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> next, used, vertices, edge_ids;
7
8     UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1)
9     {
10         assert(n > 0);
11     }
12
13     int add(int u, int v)
14     {
15         assert(1 <= u && u <= n && 1 <= v && v <= n);
16         int id = edges.size();
17         edges.push_back({u, v});
18         g[u].push_back({v, id});
19         g[v].push_back({u, id});
20         vertices.clear();
21         edge_ids.clear();
22         return id;
23     }
24
25     void dfs(int u, int incoming)
26     {
27         while (next[u] < (int)g[u].size())
28         {
29             auto [v, id] = g[u][next[u]++];
30             if (used[id])
31                 continue;
32             used[id] = 1;
33             dfs(v, id);
34         }
35         vertices.push_back(u);
36         if (incoming != -1)
37             edge_ids.push_back(incoming);
38     }
```

```

40 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
41 bool run(int start = 0, bool lexicographic = true)
42 {
43     assert(0 <= start && start <= n);
44     vertices.clear();
45     edge_ids.clear();
46     vector<int> odd;
47     for (int u = 1; u <= n; u++)
48         if (g[u].size() % 2)
49             odd.push_back(u);
50     if (!odd.empty() && odd.size() != 2)
51         return false;
52     if (!odd.empty())
53     {
54         if (start && start != odd[0] && start != odd[1])
55             return false;
56         if (!start)
57             start = odd[0];
58     }
59     if (!start)
60     {
61         start = 1;
62         for (int u = 1; u <= n; u++)
63             if (!g[u].empty())
64             {
65                 start = u;
66                 break;
67             }
68     }
69     if (lexicographic)
70         for (int u = 1; u <= n; u++)
71             sort(g[u].begin(), g[u].end());
72     fill(next.begin(), next.end(), 0);
73     used.assign(edges.size(), 0);
74     dfs(start, -1);
75     if (edge_ids.size() != edges.size())
76     {
77         vertices.clear();
78         edge_ids.clear();
79         return false;
80     }
81     reverse(vertices.begin(), vertices.end());
82     reverse(edge_ids.begin(), edge_ids.end());
83     return true;
84 }
85 };

```

第十九章 混合图欧拉定向

19.1 欧拉回路与指定端点定向

`mixed_euler_orientation(n, edges, start, finish)` 返回可空的定向边序列。每条输入为 $\{u, v, \text{type}\}$, $\text{type}=1$ 固定 u 到 v , $\text{type}=0$ 可反转; 点号 $1..n$, $n \geq 1$ 。省略两个端点或均传 0 时请求任意起点的欧拉回路; 两个正端点请求指定起终点的欧拉路径, 相等时为指定起点回路。无解返回 `nullopt`, 有解返回与原边顺序对应的端点对; 无边图可返回有值空序列。忽略无关孤立点, 但正端点须在有边部分。先任意定向可反转边, 扣除目标出入度差; 每条反转将两端差值分别改变 -2 和 $+2$, 奇数差无解, 否则用容量 1 的反转边流调整。检查底层无向连通性后, 满流即得到所需方向。最大流图 $n+2$ 点、至多 $m+n$ 条正向边, 总时间可按 Dinic 的 $O((n+2)^2(m+n))$ 上界计, 空间 $O(n+m)$ 。依赖 `flow.hpp`; 返回边交给 `DirectedEuler` 可生成连续走法, 指定端点时保持指定起点。定向不保证字典序最小。

```
1 // Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
2 // start=finish=0 requests a circuit; positive endpoints request a specified trail.
3 inline optional<vector<pair<int, int>>> mixed_euler_orientation(
4     int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
5 {
6     assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
7     assert(bool(start) == bool(finish));
8     vector<int> parent(n + 1), id;
9     vector<long long> degree(n + 1), balance(n + 1);
10    iota(parent.begin(), parent.end(), 0);
11    auto find = [&](auto &&self, int u) -> int
12    {
13        if (parent[u] != u)
14            parent[u] = self(self, parent[u]);
15        return parent[u];
16    };
17    Dinic flow(n + 2);
18    vector<pair<int, int>> direction;
19    for (auto [u, v, type] : edges)
20    {
21        assert(1 <= u && u <= n && 1 <= v && v <= n);
22        assert(type == 0 || type == 1);
23        parent[find(find, u)] = find(find, v);
24        degree[u]++;
25        degree[v]++;
26        balance[u]++;
27        balance[v]--;
28        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
29        direction.push_back({u, v});
30    }
```

```

31     if (!edges.empty())
32     {
33         int root = find(find, edges[0][0]);
34         for (int u = 1; u <= n; u++)
35             if (degree[u] && find(find, u) != root)
36                 return nullopt;
37         if (start && (!degree[start] || !degree[finish]))
38             return nullopt;
39     }
40     if (start)
41     {
42         balance[start]--;
43         balance[finish]++;
44     }
45     int source = n + 1, sink = n + 2;
46     long long need = 0;
47     for (int u = 1; u <= n; u++)
48     {
49         if (balance[u] % 2)
50             return nullopt;
51         if (balance[u] > 0)
52         {
53             flow.add(source, u, balance[u] / 2);
54             need += balance[u] / 2;
55         }
56         else if (balance[u] < 0)
57             flow.add(u, sink, -balance[u] / 2);
58     }
59     if (flow.flow(source, sink) != need)
60         return nullopt;
61     for (int i = 0; i < (int)edges.size(); i++)
62         if (id[i] != -1 && flow.used(id[i]))
63             swap(direction[i].first, direction[i].second);
64     return direction;
65 }

```

19.2 任意起终点的混合欧拉路径

`mixed_euler_trail(n, edges)` 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 `DirectedEuler` 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```

1 // Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
2 // orders.
3 inline optional<vector<pair<int, int>>>
4 mixed_euler_trail(int n, const vector<array<int, 3>> &edges)
5 {
6     assert(n > 0);
7     vector<long long> degree(n + 1);
8     vector<int> odd;
9     for (auto [u, v, type] : edges)
10     {
11         assert(1 <= u && u <= n && 1 <= v && v <= n);
12         degree[u]++;
13         degree[v]++;
14     }
15     for (int u = 1; u <= n; u++)
16         if (degree[u] % 2)
17             odd.push_back(u);
18     if (odd.empty())
19         return mixed_euler_orientation(n, edges);
20     if (odd.size() != 2)
21         return nullopt;
22     auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
23     if (answer)
24         return answer;
25     return mixed_euler_orientation(n, edges, odd[1], odd[0]);
26 }
```

第二十章 图上异或行走

20.1 无向图最大异或行走

XorWalk(n) 建图，点号 1..n，边权为 unsigned long long，支持完整 64 位。add(u,v,w) 加无向边，允许重边、自环和零权；init(n) 清空图。build(root=1) 用递归 DFS 预处理 root 的连通块，query(u,v) 返回该块内 u 到 v 的最大边权异或和；任一点未被本次 build 覆盖则返回 nullopt，即使这两点在另一连通块相连。加入新边后必须重新 build；可更换根重复构建。空图可初始化，但不能 build。这里的行走允许重复点、重复边；不是最大异或简单路径。u=v 也可能通过绕环得到正值。DFS 距离 dist 给出一条树路径的异或值，每条已访问边形成 dist[u] xor dist[v] xor w 的闭合行走值，加入线性基；两点答案为 cycles.query(dist[u] xor dist[v])。build 后 vis、dist 可只读；cycles 表示环异或空间，不保留具体环或行走方案，不能将其非空子集标志解释为行走非空性。建图 O(n+m)，build 为 O(n+64m)，每次 query 为 O(64)，动态空间 O(n+m+64)。

```
1 struct XorWalk
2 {
3     using U = unsigned long long;
4     int n;
5     bool built = false;
6     vector<vector<pair<int, U>>> g;
7     vector<int> vis;
8     vector<U> dist;
9     XorBasis cycles;
10
11     XorWalk(int n)
12     {
13         init(n);
14     }
15
16     void init(int size)
17     {
18         assert(size >= 0);
19         n = size;
20         g.clear();
21         g.resize(size_t(n) + 1);
22         vis.assign(size_t(n) + 1, 0);
23         dist.assign(size_t(n) + 1, 0);
24         cycles.init();
25         built = false;
26     }
```

```

28 void add(int u, int v, U w)
29 {
30     assert(1 <= u && u <= n && 1 <= v && v <= n);
31     g[u].push_back({v, w});
32     g[v].push_back({u, w});
33     built = false;
34 }
35
36 void dfs(int u)
37 {
38     vis[u] = 1;
39     for (auto [v, w] : g[u])
40     {
41         if (!vis[v])
42         {
43             dist[v] = dist[u] ^ w;
44             dfs(v);
45         }
46         else
47             cycles.insert(dist[u] ^ dist[v] ^ w);
48     }
49 }
50
51 void build(int root = 1)
52 {
53     assert(1 <= root && root <= n);
54     fill(vis.begin(), vis.end(), 0);
55     fill(dist.begin(), dist.end(), 0);
56     cycles.init();
57     dfs(root);
58     built = true;
59 }
60
61 optional<U> query(int u, int v) const
62 {
63     assert(built && 1 <= u && u <= n && 1 <= v && v <= n);
64     if (!vis[u] || !vis[v])
65         return nullopt;
66     return cycles.query(dist[u] ^ dist[v]);
67 }
68 };

```


第二十一章 树上算法

21.1 重链剖分与 LCA

add(u,v) 建无向树; build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 $O(\log n)$, path(u,v,work,edge) 分解路径; work(l,r) 仅适用于可交换聚合, 边权模式排除 LCA 位置。输入必须是连通无环树, 建树 $O(n)$, 两遍递归 DFS; 支持改变根后重复 build, dfn 会重新编号。

```
1 struct HLD
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     HLD(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),
11          dep(n + 1),
12          siz(n + 1),
13          son(n + 1),
14          top(n + 1),
15          dfn(n + 1),
16          rk(n + 1)
17     {
18     }
19
20     void add(int u, int v)
21     {
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     void dfs1(int u, int p)
27     {
28         fa[u] = p;
29         siz[u] = 1;
30         son[u] = 0;
31         for (int v : g[u])
32         {
33             if (v == p)
34                 continue;
35             dep[v] = dep[u] + 1;
36             dfs1(v, u);
37             siz[u] += siz[v];
38             if (!son[u] || siz[v] > siz[son[u]])
39                 son[u] = v;
40         }
41     }
42
43     void dfs2(int u, int t)
44     {
```

```

45     top[u] = t;
46     dfn[u] = ++timer;
47     rk[timer] = u;
48     if (son[u])
49         dfs2(son[u], t);
50     for (int v : g[u])
51     {
52         if (v != fa[u] && v != son[u])
53             dfs2(v, v);
54     }
55 }
56
57 // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
58 void build(int root = 1)
59 {
60     assert(1 <= root && root <= n);
61     timer = 0;
62     dep[root] = 0;
63     dfs1(root, root);
64     dfs2(root, root);
65     assert(timer == n);
66 }
67
68 int lca(int u, int v) const
69 {
70     while (top[u] != top[v])
71     {
72         if (dep[top[u]] < dep[top[v]])
73             swap(u, v);
74         u = fa[top[u]];
75     }
76     return dep[u] < dep[v] ? u : v;
77 }
78
79 // Commutative operations only; edge=true excludes the LCA's position.
80 template <class F> void path(int u, int v, F work, bool edge = false) const
81 {
82     while (top[u] != top[v])
83     {
84         if (dep[top[u]] < dep[top[v]])
85             swap(u, v);
86         work(dfn[top[u]], dfn[u]);
87         u = fa[top[u]];
88     }
89     if (dep[u] > dep[v])
90         swap(u, v);
91     if (dfn[u] + edge <= dfn[v])
92         work(dfn[u] + edge, dfn[v]);
93 }
94 };

```

第二十二章 离线最近公共祖先

22.1 离线 Tarjan LCA

构造 `OfflineLCA(n)`, `add(u,v)` 加无向树边, `add_query(u,v)` 登记查询并返回从 0 开始的编号, `run(root=1)` 后读取 `answer[id]`。输入必须是非空树, 点号 $1..n$; 允许同点、重复查询及零个查询。`run` 可重复调用或换根, 不清空查询; 追加查询后需重新 `run`, 旧答案在重算前不可按新根使用。DFS 与并查集查找均递归。并查集按大小合并并压缩路径, `ancestor[find(u)]` 保存已合并部分对应的当前树祖先; 不能直接拿并查集代表当 LCA。每个子树完成后合并到父点并更新 `ancestor`; 查询另一端已完成时, 从其集合读取祖先。总时间 $O((n+q) \alpha(n))$, 空间 $O(n+q)$ 。全部点对和根的父链对拍、50 万点链及 P3379、POJ1470 完整驱动本地验证, 在线待补。

```
1 struct OfflineLCA
2 {
3     int n;
4     vector<vector<int>>> g;
5     vector<vector<pair<int, int>>>> queries;
6     vector<int> parent, size, ancestor, done, answer;
7
8     OfflineLCA(int n)
9         : n(n),
10          g(n + 1),
11          queries(n + 1),
12          parent(n + 1),
13          size(n + 1),
14          ancestor(n + 1),
15          done(n + 1)
16     {
17     }
18
19     void add(int u, int v)
20     {
21         assert(1 <= u && u <= n && 1 <= v && v <= n);
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     int add_query(int u, int v)
27     {
28         assert(1 <= u && u <= n && 1 <= v && v <= n);
29         int id = answer.size();
30         answer.push_back(0);
31         queries[u].push_back({v, id});
32         queries[v].push_back({u, id});
33         return id;
34     }
35 }
```

```

36     int find(int u)
37     {
38         if (parent[u] != u)
39             parent[u] = find(parent[u]);
40         return parent[u];
41     }
42
43     void merge(int u, int v)
44     {
45         u = find(u);
46         v = find(v);
47         if (size[u] < size[v])
48             swap(u, v);
49         parent[v] = u;
50         size[u] += size[v];
51     }
52
53     void dfs(int u, int p)
54     {
55         for (int v : g[u])
56             if (v != p)
57             {
58                 dfs(v, u);
59                 merge(u, v);
60                 ancestor[find(u)] = u;
61             }
62         done[u] = 1;
63         for (auto [v, id] : queries[u])
64             if (done[v])
65                 answer[id] = ancestor[find(v)];
66     }
67
68     // Input must be a nonempty tree. Query IDs are 0-based.
69     void run(int root = 1)
70     {
71         assert(1 <= root && root <= n);
72         iota(parent.begin(), parent.end(), 0);
73         iota(ancestor.begin(), ancestor.end(), 0);
74         fill(size.begin(), size.end(), 1);
75         fill(done.begin(), done.end(), 0);
76         fill(answer.begin(), answer.end(), 0);
77         dfs(root, 0);
78     }
79 };

```

第二十三章 在线最近公共祖先

23.1 欧拉序与 ST 表 LCA

构造 EulerLCA(n), add(u,v,w=1) 加非负权无向树边, build(root=1) 预处理; lca(u,v) 返回最近公共祖先, distance(u,v) 仍返回无权路径边数; weighted_distance(u,v) 返回 long long 带权距离, 传统接口 Weighted_Distance。边权非负, 所有树上路径的权和须 $\leq \text{LLONG_MAX}$ 。输入必须为非空树, 点号 1..n; 允许同点查询和重复 build 换根。只可在 build 后查询, 加边会清除 ready 标志。递归 DFS 在进入顶点及从每个儿子返回时记录顶点, euler 长度为 2n-1, first[u] 是从 0 开始的首次出现位置; depth[root]=0。两点首次出现位置间深度最小的顶点就是 LCA, 用 ST 表保存该顶点。预处理 $O(n \log n)$ 时间和空间, 每次查询 $O(1)$ 。ST 内部下标从 0 开始; first、depth、euler、root_dist 可直接读取; root_dist 为当前根的带权距离。50 万点链、全部小树根与点对以及 P3379、POJ1330 双风格完整程序本地验证; 上述无权题在线待补; QOJ8235 的带权应用已双 AC。

```
1 struct EulerLCA
2 {
3     using ll = long long;
4     int n;
5     bool ready = false;
6     vector<vector<int>> g, st;
7     vector<vector<ll>> cost;
8     vector<ll> root_dist;
9     vector<int> first, depth, euler, lg;
10
11     EulerLCA(int n)
12         : n(n), g(n + 1), cost(n + 1), root_dist(n + 1), first(n + 1), depth(n + 1)
13     {
14     }
15
16     void add(int u, int v, ll w = 1)
17     {
18         assert(1 <= u && u <= n && 1 <= v && v <= n);
19         assert(w >= 0);
20         g[u].push_back(v);
21         g[v].push_back(u);
22         cost[u].push_back(w);
23         cost[v].push_back(w);
24         ready = false;
25     }
26
27     int shallower(int u, int v) const
28     {
29         return depth[u] <= depth[v] ? u : v;
30     }
```

递归欧拉序与 RMQ (接上页同一结构体):

```

32 void dfs(int u, int parent)
33 {
34     first[u] = euler.size();
35     euler.push_back(u);
36     for (int i = 0; i < int(g[u].size()); i++)
37     {
38         int v = g[u][i];
39         if (v != parent)
40         {
41             depth[v] = depth[u] + 1;
42             root_dist[v] = root_dist[u] + cost[u][i];
43             dfs(v, u);
44             euler.push_back(u);
45         }
46     }
47 }
48
49 // Input must be a nonempty tree. Rebuilding may change the root.
50 void build(int root = 1)
51 {
52     assert(1 <= root && root <= n);
53     euler.clear();
54     depth[root] = 0;
55     root_dist[root] = 0;
56     dfs(root, 0);
57     int m = euler.size();
58     lg.assign(m + 1, 0);
59     for (int i = 2; i <= m; i++)
60         lg[i] = lg[i / 2] + 1;
61     st.assign(lg[m] + 1, {});
62     st[0] = euler;
63     for (int k = 1; k <= lg[m]; k++)
64     {
65         st[k].resize(m - (1 << k) + 1);
66         for (int i = 0; i + (1 << k) <= m; i++)
67             st[k][i] = shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
68     }
69     ready = true;
70 }

```

LCA、边数距离与带权距离（接上页同一结构体）：

```
72     int lca(int u, int v) const
73     {
74         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
75         int l = first[u], r = first[v];
76         if (l > r)
77             swap(l, r);
78         int k = lg[r - l + 1];
79         return shallower(st[k][l], st[k][r - (1 << k) + 1]);
80     }
81
82     int distance(int u, int v) const
83     {
84         int z = lca(u, v);
85         return depth[u] + depth[v] - 2 * depth[z];
86     }
87
88     ll weighted_distance(int u, int v) const
89     {
90         int z = lca(u, v);
91         return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
92     }
93 };
```

第二十四章 树上点集摘要

24.1 树上点集直径摘要

默认空点集, $a=b=-1$ 、 $length=0$ 。insert($v,distance$) 插入顶点, 重复插入合法; merge($other,distance$) 合并两个点集摘要, 允许自合并; farthest($v,distance$) 返回 optional<pair<int,long long>> (最远顶点、距离), 空集返回 nullopt, 距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离, 点号为非负整数; 所有距离 $\leq LLONG_MAX$ 。每次插入或最远查询最多两次距离调用, 合并最多四次, 摘要只占 $O(1)$ 空间。保存 a 、 b 为点集中某对直径端点, 不保留全部点; 字段只读, 不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合, 重置用默认构造。传统接口 Tree_Diameter::Insert / Merge / Farthest。

```
1 struct TreeDiameter
2 {
3     using ll = long long;
4     int a = -1, b = -1;
5     ll length = 0;
6
7     // distance is the metric of one fixed tree with nonnegative edge weights.
8     template <class Distance> void insert(int v, const Distance &distance)
9     {
10         assert(v >= 0);
11         if (a == -1)
12         {
13             a = v;
14             b = v;
15             return;
16         }
17         ll x = distance(a, v), y = distance(b, v);
18         if (x >= y && x > length)
19         {
20             b = v;
21             length = x;
22         }
23         else if (y > length)
24         {
25             a = v;
26             length = y;
27         }
28     }
```


合并与最远点查询（接上页同一结构体）：

```
30     template <class Distance> void merge(TreeDiameter other, const Distance &distance)
31     {
32         if (other.a == -1)
33             return;
34         insert(other.a, distance);
35         insert(other.b, distance);
36     }
37
38     template <class Distance>
39     optional<pair<int, ll>> farthest(int v, const Distance &distance) const
40     {
41         if (a == -1)
42             return nullopt;
43         ll x = distance(a, v), y = distance(b, v);
44         return x >= y ? make_pair(a, x) : make_pair(b, y);
45     }
46 };
```

第二十五章 倍增祖先与路径最值

25.1 倍增 LCA、祖先与最大边权

构造 `LiftingLCA(n)`, `add(u,v,w=1)` 加边, `build(root=1)` 预处理非空无向树, 点号 $1..n$ 。 `jump(u,k)` 返回 k 级祖先, k 非负, 超过根返回 0; `lca(u,v)` 求最近公共祖先; `distance` 返回路径边数, 不是权值和; `max_edge` 返回路径最大边权, 同点返回 `nullopt`, 其他情况返回有值的 `optional<long long>`。支持负权和 `LLONG_MIN`, `LOW` 只是最大值合并的单位元, 不表示空路径。DFS 递归, 根深度为 0; `up[k][u]` 与 `mx[k][u]` 是 2 的 k 次方长度向上段的祖先与最大边权, 0 号节点为哨兵。可重复 `build` 换根; 修改图后必须重建。预处理 $O(n \log n)$ 时间与空间, 各查询 $O(\log n)$ 。不需要边权最值时, 可删去 `mx` 和 `max_edge` 相关部分。全小树点对/根、极值权重及 50 万点负权链本地验证; P3379、POJ1330 驱动只验证 LCA, 最大边权仍是本地证据。

```
1 struct LiftingLCA
2 {
3     using ll = long long;
4     static constexpr ll LOW = numeric_limits<ll>::lowest();
5     int n, levels;
6     bool ready = false;
7     vector<vector<pair<int, ll>>> g;
8     vector<vector<int>> up;
9     vector<vector<ll>> mx;
10    vector<int> depth;
11
12    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
13    {
14        assert(n > 0);
15        levels = 1;
16        while ((1LL << levels) <= n)
17            levels++;
18        up.assign(levels, vector<int>(n + 1));
19        mx.assign(levels, vector<ll>(n + 1, LOW));
20    }
21
22    void add(int u, int v, ll w = 1)
23    {
24        assert(1 <= u && u <= n && 1 <= v && v <= n);
25        g[u].push_back({v, w});
26        g[v].push_back({u, w});
27        ready = false;
28    }
```

```

30 void dfs(int u, int parent, ll weight)
31 {
32     up[0][u] = parent;
33     mx[0][u] = weight;
34     for (int k = 1; k < levels; k++)
35     {
36         int p = up[k - 1][u];
37         up[k][u] = up[k - 1][p];
38         mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
39     }
40     for (auto [v, w] : g[u])
41         if (v != parent)
42         {
43             depth[v] = depth[u] + 1;
44             dfs(v, u, w);
45         }
46     }
47
48     // Nonempty tree only. Node 0 means no ancestor.
49     void build(int root = 1)
50     {
51         assert(1 <= root && root <= n);
52         depth[root] = 0;
53         dfs(root, 0, LOW);
54         ready = true;
55     }
56
57     int jump(int u, long long steps) const
58     {
59         assert(ready && 1 <= u && u <= n && steps >= 0);
60         if (steps > depth[u])
61             return 0;
62         for (int k = 0; k < levels; k++)
63             if (steps >> k & 1)
64                 u = up[k][u];
65         return u;
66     }

```

```

68  int lca(int u, int v) const
69  {
70      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
71      if (depth[u] < depth[v])
72          swap(u, v);
73      u = jump(u, depth[u] - depth[v]);
74      if (u == v)
75          return u;
76      for (int k = levels - 1; k >= 0; k--)
77          if (up[k][u] != up[k][v])
78              {
79                  u = up[k][u];
80                  v = up[k][v];
81              }
82      return up[0][u];
83  }
84
85  int distance(int u, int v) const
86  {
87      return depth[u] + depth[v] - 2 * depth[lca(u, v)];
88  }
89
90  optional<ll> max_edge(int u, int v) const
91  {
92      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
93      if (u == v)
94          return nullopt;
95      if (depth[u] < depth[v])
96          swap(u, v);
97      ll answer = LOW;
98      int delta = depth[u] - depth[v];
99      for (int k = 0; k < levels; k++)
100          if (delta >> k & 1)
101              {
102                  answer = max(answer, mx[k][u]);
103                  u = up[k][u];
104              }
105      if (u == v)
106          return answer;
107      for (int k = levels - 1; k >= 0; k--)
108          if (up[k][u] != up[k][v])
109              {
110                  answer = max(answer, max(mx[k][u], mx[k][v]));
111                  u = up[k][u];
112                  v = up[k][v];
113              }
114      return max(answer, max(mx[0][u], mx[0][v]));
115  }
116  };

```

第二十六章 树上路径交

26.1 两条树路径的交集

`path_intersection(a,b,c,d,depth,lca)` 返回路径 `a-b` 与 `c-d` 的公共路径，字段 `u`、`v` 是交集端点，`vertices` 是公共顶点数；空交返回 `{0,0,0}`，单点交的 `u=v`。点号从 1 开始，`depth` 数组与 `lca` 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能传带权根距离。支持数组或 `vector` 深度表，回调接收两个点并返回 LCA。将 `c`、`d` 分别投影到 `a-b` 得 `p`、`q`；投影是 `lca(a,b)`、`lca(a,x)`、`lca(b,x)` 中最深者。若 `p≠q`，`c-d` 必经过 `p-q`；若 `p=q`，检验 `p` 是否在 `c-d` 上即可区分单点交与空交。不依赖顶点编号大小。常数 LCA 调用、 $O(1)$ 额外空间；使用 EulerLCA 时为 $O(1)$ ，HLD/倍增时为 $O(\log n)$ 。公共边数为 $\max(0, \text{vertices}-1)$ 。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
1 struct PathIntersection
2 {
3     int u, v, vertices;
4 };
5
6 // Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
7 template <class Depth, class Lca>
8 PathIntersection
9 path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
10 {
11     int z = lca(a, b);
12     auto project = [&](int x)
13     {
14         int p = z;
15         for (int v : {lca(a, x), lca(b, x)})
16             if (depth[v] > depth[p])
17                 p = v;
18         return p;
19     };
20     auto distance = [&](int u, int v)
21     {
22         return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
23     };
24     int p = project(c), q = project(d);
25     if (p == q && distance(c, p) + distance(p, d) != distance(c, d))
26         return {0, 0, 0};
27     return {p, q, int(distance(p, q) + 1)};
28 }
```

第二十七章 点分治

27.1 点分治：树上距离点对计数

CentroidPairs(n), 点号 $1..n$; add(u,v,w) 建树, 允许森林但禁止环、重边、自环, 边权非负; 所有简单路径长度须能用 long long 表示。build() 预处理, 可重复调用; 改变边后须重建。count_leq(k) 统计距离 $\leq k$ 的无序不同顶点对, count_exact(k) 统计距离恰为 k 的点对; 负 k 返回 0, 零距离只统计不同顶点, 跨连通块不配对。parent[c] 是点分树父节点, 根为 0。all[c] 为该层分治块到重心的距离, 含重心距离 0; part[c] 为去掉重心后各子块的距离。预处理 $O(n \log^2 n)$, 空间 $O(n \log n)$, 每次计数 $O(n \log n)$; 固定树上多次查询可以复用排序结果。DFS 递归, 距离和比较采用减法避免溢出。不是动态点分树更新模板。

```
1 struct CentroidPairs
2 {
3     using ll = long long;
4     int n;
5     vector<vector<pair<int, ll>>> g;
6     vector<int> siz, removed, parent;
7     vector<vector<ll>> all;
8     vector<vector<vector<ll>>> part;
9
10    CentroidPairs(int n)
11        : n(n),
12          g(n + 1),
13          siz(n + 1),
14          removed(n + 1),
15          parent(n + 1),
16          all(n + 1),
17          part(n + 1)
18    {
19    }
20
21    void add(int u, int v, ll w)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26    }
27
28    void size_dfs(int u, int p)
29    {
30        siz[u] = 1;
31        for (auto [v, w] : g[u])
32        {
33            if (v == p || removed[v])
34                continue;
35            size_dfs(v, u);
36            siz[u] += siz[v];
37        }
38    }
39
40    int centroid(int u, int p, int total)
41    {
```

```

42     for (auto [v, w] : g[u])
43     {
44         if (v != p && !removed[v] && siz[v] > total / 2)
45             return centroid(v, u, total);
46     }
47     return u;
48 }
49
50 void collect(int u, int p, ll d, vector<ll> &out)
51 {
52     out.push_back(d);
53     for (auto [v, w] : g[u])
54     {
55         if (v == p || removed[v])
56             continue;
57         assert(d <= numeric_limits<ll>::max() - w);
58         collect(v, u, d + w, out);
59     }
60 }
61
62 void decompose(int entry, int p)
63 {
64     size_dfs(entry, 0);
65     int c = centroid(entry, 0, siz[entry]);
66     removed[c] = 1;
67     parent[c] = p;
68     all[c] = {0};
69     for (auto [v, w] : g[c])
70     {
71         if (removed[v])
72             continue;
73         part[c].push_back({});
74         auto &d = part[c].back();
75         collect(v, c, w, d);
76         sort(d.begin(), d.end());
77         all[c].insert(all[c].end(), d.begin(), d.end());
78     }
79     sort(all[c].begin(), all[c].end());
80     for (auto [v, w] : g[c])
81     {
82         if (!removed[v])
83             decompose(v, c);
84     }
85 }
86
87 // Input must be a forest. Rebuild after changing edges.
88 void build()
89 {
90     fill(removed.begin(), removed.end(), 0);
91     for (int u = 1; u <= n; u++)
92     {
93         all[u].clear();
94         part[u].clear();
95     }
96     for (int u = 1; u <= n; u++)
97     {

```

```

98         if (!removed[u])
99             decompose(u, 0);
100     }
101 }
102
103 ll pairs(const vector<ll> &d, ll k) const
104 {
105     int l = 0, r = (int)d.size() - 1;
106     ll answer = 0;
107     while (l < r)
108     {
109         if (d[l] <= k - d[r])
110         {
111             answer += r - l;
112             l++;
113         }
114         else
115             r--;
116     }
117     return answer;
118 }
119
120 // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
121 ll count_leq(ll k) const
122 {
123     if (k < 0)
124         return 0;
125     ll answer = 0;
126     for (int c = 1; c <= n; c++)
127     {
128         answer += pairs(all[c], k);
129         for (const auto &d : part[c])
130             answer -= pairs(d, k);
131     }
132     return answer;
133 }
134
135 ll count_exact(ll k) const
136 {
137     if (k < 0)
138         return 0;
139     return count_leq(k) - (k ? count_leq(k - 1) : 0);
140 }
141 };

```


第二十八章 树上启发式合并

28.1 树上启发式合并：子树众数颜色和

SubtreeColors(n), 点号 1..n, add(u,v) 建无向树; 要求图连通且无环, n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组, colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和, max_freq[u] 为该频次; 支持负颜色与稀疏编号, 内部离散化, 和用 long long。重复 run 可更换颜色或根。重儿子信息保留, 轻儿子通过 DFS 序区间加入; 清空整棵临时子树后再重置最大值, 不尝试逐次删除时维护众数。O(n log n) 时间, O(n) 空间, 全部 DFS 递归。insert 是内部增量统计, 可按题目修改统计量, 但相应清空规则也必须调整。

```
1 struct SubtreeColors
2 {
3     int n, best = 0;
4     long long sum = 0;
5     vector<vector<int>>> g;
6     vector<int> siz, heavy, tin, tout, order, color, value, freq;
7     vector<int> max_freq;
8     vector<long long> answer;
9
10    SubtreeColors(int n) : n(n), g(n + 1)
11    {
12    }
13
14    void add(int u, int v)
15    {
16        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
17        g[u].push_back(v);
18        g[v].push_back(u);
19    }
20
21    void prepare(int u, int p)
22    {
23        siz[u] = 1;
24        tin[u] = order.size();
25        order.push_back(u);
26        for (int v : g[u])
27        {
28            if (v == p)
29                continue;
30            prepare(v, u);
31            siz[u] += siz[v];
32            if (siz[v] > siz[heavy[u]])
33                heavy[u] = v;
34        }
35        tout[u] = (int)order.size() - 1;
36    }
37
38    void insert(int u)
39    {
40        int c = color[u];
41        freq[c]++;
42        if (freq[c] > best)
```

```

43     {
44         best = freq[c];
45         sum = value[c];
46     }
47     else if (freq[c] == best)
48         sum += value[c];
49 }
50
51 void solve(int u, int p, bool keep)
52 {
53     for (int v : g[u])
54     {
55         if (v != p && v != heavy[u])
56             solve(v, u, false);
57     }
58     if (heavy[u])
59         solve(heavy[u], u, true);
60     for (int v : g[u])
61     {
62         if (v == p || v == heavy[u])
63             continue;
64         for (int i = tin[v]; i <= tout[v]; i++)
65             insert(order[i]);
66     }
67     insert(u);
68     max_freq[u] = best;
69     answer[u] = sum;
70     if (!keep)
71     {
72         for (int i = tin[u]; i <= tout[u]; i++)
73             freq[color[order[i]]]--;
74         best = 0;
75         sum = 0;
76     }
77 }
78
79 // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
80 void run(const vector<int> &colors, int root = 1)
81 {
82     assert((int)colors.size() == n + 1);
83     siz.assign(n + 1, 0);
84     heavy = siz;
85     tin = siz;
86     tout = siz;
87     color = siz;
88     max_freq = siz;
89     answer.assign(n + 1, 0);
90     order.clear();
91     value.assign(colors.begin() + 1, colors.end());
92     sort(value.begin(), value.end());
93     value.erase(unique(value.begin(), value.end()), value.end());
94     freq.assign(value.size(), 0);
95     best = 0;
96     sum = 0;

```

```
97     if (!n)
98         return;
99     assert(1 <= root && root <= n);
100    for (int u = 1; u <= n; u++)
101        color[u] =
102            lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
103    prepare(root, 0);
104    solve(root, 0, false);
105    }
106};
```

第二十九章 虚树

29.1 虚树：关键点压缩与路径信息

`VirtualTree(n)`, $n \geq 1$, 点号 $1..n$; `add(u,v,w=1)` 建非负权连通树, 无环和重边。 `prepare(root=1)` 递归预处理, 可换根重建; 根到任一点距离须在 `long long` 范围内。 `build(keys)` 自动去重并补齐必要 LCA, 返回 `vertices` 和 `edges`; 全部编号沿用原树, `vertices` 按 DFS 序排列, 非空时首项即虚树根。 `edges` 为 `(from,to,distance,minimum)`, 分别是祖先到后代的虚边、压缩路径长度和路径最小边权。 空关键点返回空结果, 单点返回单节点; 原树根不会强制加入, 需要时由调用者加入 `keys`。 边按后代 DFS 序排列, 可逆序做自底向上的 DP。 不分配或清空 n 个查询邻接表; 每次仅返回 $O(k)$ 结果。 预处理 $O(n \log n)$ 时间空间, 单次 $O(k \log k + k \log n)$ 。 `lca`、`ancestor` 也可直接使用, 要求有效点号且已经 `prepare`。 构造用的祖先链栈属于虚树算法, 原树 DFS 保持递归。

```
1 struct VirtualTree
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll distance, minimum;
9     };
10
11     struct Result
12     {
13         vector<int> vertices;
14         vector<Edge> edges;
15     };
16
17     int n, log = 1, timer = 0;
18     vector<vector<pair<int, ll>>> g;
19     vector<vector<int>> up;
20     vector<vector<ll>> mn;
21     vector<int> depth, tin, tout;
22     vector<ll> dist;
23
24     VirtualTree(int n) : n(n), g(n + 1)
25     {
26     }
27
28     void add(int u, int v, ll w = 1)
29     {
30         assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
31         g[u].push_back({v, w});
32         g[v].push_back({u, w});
33     }
34
35     void dfs(int u, int p)
36     {
37         tin[u] = ++timer;
38         up[0][u] = p;
39         for (int j = 1; j < log; j++)
```

```

40     {
41         up[j][u] = up[j - 1][up[j - 1][u]];
42         mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
43     }
44     for (auto [v, w] : g[u])
45     {
46         if (v == p)
47             continue;
48         assert(dist[u] <= numeric_limits<ll>::max() - w);
49         depth[v] = depth[u] + 1;
50         dist[v] = dist[u] + w;
51         mn[0][v] = w;
52         dfs(v, u);
53     }
54     tout[u] = timer;
55 }
56
57 void prepare(int root = 1)
58 {
59     assert(1 <= root && root <= n);
60     log = 1;
61     while ((1LL << log) <= n)
62         log++;
63     up.assign(log, vector<int>(n + 1));
64     mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
65     depth.assign(n + 1, 0);
66     tin = depth;
67     tout = depth;
68     dist.assign(n + 1, 0);
69     timer = 0;
70     dfs(root, root);
71 }
72
73 bool ancestor(int u, int v) const
74 {
75     return tin[u] <= tin[v] && tout[v] <= tout[u];
76 }
77
78 int lca(int u, int v) const
79 {
80     if (ancestor(u, v))
81         return u;
82     if (ancestor(v, u))
83         return v;
84     for (int j = log - 1; j >= 0; j--)
85     {
86         if (!ancestor(up[j][u], v))
87             u = up[j][u];
88     }
89     return up[0][u];
90 }
91
92 Edge path(int u, int v) const
93 {
94     assert(u != v && ancestor(u, v));
95     ll minimum = numeric_limits<ll>::max();

```

```

96     int x = v, steps = depth[v] - depth[u];
97     for (int j = 0; j < log; j++)
98     {
99         if (steps >> j & 1)
100         {
101             minimum = min(minimum, mn[j][x]);
102             x = up[j][x];
103         }
104     }
105     return {u, v, dist[v] - dist[u], minimum};
106 }
107
108 // Uses original vertex IDs. Empty input gives an empty result.
109 Result build(vector<int> keys) const
110 {
111     if (keys.empty())
112         return {};
113     for (int u : keys)
114         assert(1 <= u && u <= n);
115     auto cmp = [&](int u, int v)
116     {
117         return tin[u] < tin[v];
118     };
119     sort(keys.begin(), keys.end(), cmp);
120     keys.erase(unique(keys.begin(), keys.end()), keys.end());
121     int k = keys.size();
122     for (int i = 1; i < k; i++)
123         keys.push_back(lca(keys[i - 1], keys[i]));
124     sort(keys.begin(), keys.end(), cmp);
125     keys.erase(unique(keys.begin(), keys.end()), keys.end());
126     Result result{keys, {}};
127     vector<int> st;
128     for (int u : keys)
129     {
130         while (!st.empty() && !ancestor(st.back(), u))
131             st.pop_back();
132         if (!st.empty())
133             result.edges.push_back(path(st.back(), u));
134         st.push_back(u);
135     }
136     return result;
137 }
138 };

```

第三十章 字符串

30.1 前缀函数

`prefix_function(s)` 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
1 inline vector<int> prefix_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = p[i - 1];
8         while (j && s[i] != s[j])
9             j = p[j - 1];
10        p[i] = j + (s[i] == s[j]);
11    }
12    return p;
13 }
```

30.2 KMP 匹配

`kmp_match(s,t)` 返回非空模式串 `t` 在 `s` 中的所有起点 (0-based)，允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
1 inline vector<int> kmp_match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = prefix_function(t);
5     vector<int> ans;
6     for (int i = 0, j = 0; i < (int)s.size(); i++)
7     {
8         while (j && s[i] != t[j])
9             j = p[j - 1];
10        j += (s[i] == t[j]);
11        if (j == (int)t.size())
12        {
13            ans.push_back(i - j + 1);
14            j = p[j - 1];
15        }
16    }
17    return ans;
18 }
```

30.3 Z 函数

`z_function(s)` 返回后缀 `s[i..]` 与原串的 LCP，约定 `z[0]=n`，空串返回空数组。O(n)。

```

1 inline vector<int> z_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if (n)
6         a[0] = n;
7     for (int i = 1, l = 0, r = 0; i < n; i++)
8     {
9         if (i < r)
10            a[i] = min(r - i, a[i - l]);
11        while (i + a[i] < n && s[a[i]] == s[i + a[i]])
12            ++a[i];
13        if (i + a[i] > r)
14        {
15            l = i;
16            r = i + a[i];
17        }
18    }
19    return a;
20 }
```

30.4 Manacher 回文半径

`manacher(s)` 返回 `odd` 与 `even`；`odd[i]` 的半径包含中心，`even[i]` 的中心在 `i` 前。字符串及下标为 0-based，空串合法，O(n)。

```

1 // odd[i] includes the center; even[i] is centered before i.
2 inline pair<vector<int>, vector<int>> manacher(const string &s)
3 {
4     int n = (int)s.size();
5     vector<int> odd(n), even(n);
6     for (int i = 0, l = 0, r = -1; i < n; i++)
7     {
8         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
9         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k])
10            ++k;
11        odd[i] = k--;
12        if (i + k > r)
13        {
14            l = i - k;
15            r = i + k;
16        }
17    }
18    for (int i = 0, l = 0, r = -1; i < n; i++)
19    {
20        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
21        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k])
22            ++k;

```



```

23     even[i] = k--;
24     if (i + k > r)
25     {
26         l = i - k - 1;
27         r = i + k;
28     }
29 }
30 return {odd, even};
31 }

```

30.5 最小循环表示

`minimum_rotation(s)` 返回字典序最小循环表示的起点 (0-based), 空串返回 0; 按无符号字节比较, 与 `string` 字典序一致。O(n)。

```

1 inline int minimum_rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if (!n)
5         return 0;
6     while (i < n && j < n && k < n)
7     {
8         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
9         if (a == b)
10            {
11                ++k;
12                continue;
13            }
14         if (a > b)
15            {
16                i += k + 1;
17                if (i == j)
18                    ++i;
19            }
20         else
21            {
22                j += k + 1;
23                if (i == j)
24                    ++j;
25            }
26         k = 0;
27     }
28     return min(i, j);
29 }

```

30.6 AC 自动机出现次数

字母表 `a..z`, `add(s)` 返回终止状态, 要求模式非空; 全部插入后 `build()`, 只允许一次构建。`count(text)` 返回每个状态对应模式的出现次数, 可重复查询; 重复模式共享状态。构建 $O(26N)$, 查询 $O(|text|+N)$ 。

```

1 struct AhoCorasick
2 {
3     struct Node
4     {

```

```

5      array<int, 26> go{};
6      int fail = 0;
7  };
8
9  vector<Node> a{Node{}};
10 vector<int> order;
11 bool built = false;
12
13 int add(const string &s)
14 {
15     assert(!built && !s.empty());
16     int u = 0;
17     for (char ch : s)
18     {
19         int c = ch - 'a';
20         assert(0 <= c && c < 26);
21         if (!a[u].go[c])
22         {
23             int v = (int)a.size();
24             a[u].go[c] = v;
25             a.push_back(Node{});
26         }
27         u = a[u].go[c];
28     }
29     return u;
30 }
31
32 void build()
33 {
34     assert(!built);
35     built = true;
36     queue<int> q;
37     for (int v : a[0].go)
38         if (v)
39             q.push(v);
40     while (!q.empty())
41     {
42         int u = q.front();
43         q.pop();
44         order.push_back(u);
45         for (int c = 0; c < 26; c++)
46         {
47             int v = a[u].go[c];
48             if (v)
49             {
50                 a[v].fail = a[a[u].fail].go[c];
51                 q.push(v);
52             }
53             else
54                 a[u].go[c] = a[a[u].fail].go[c];
55         }
56     }
57 }
58
59 vector<long long> count(const string &s) const
60 {

```

```
61     assert(built);
62     vector<long long> ans(a.size());
63     int u = 0;
64     for (char ch : s)
65     {
66         u = a[u].go[ch - 'a'];
67         ++ans[u];
68     }
69     for (int i = (int)order.size() - 1; i >= 0; i--)
70         ans[a[order[i]].fail] += ans[order[i]];
71     return ans;
72 }
73 };
```

30.7 倍增后缀数组与 LCP

构造 `SuffixArray(s)` 支持任意字节串，按 `unsigned char` 的 0..255 数值比较，包含内嵌零字节；也可构造 `SuffixArray(a,alphabet)`，整数序列要求 $0 \leq a[i] < \text{alphabet}$ 、 $1 \leq \text{alphabet} < \text{INT_MAX}$ 。`init(a,alphabet)` 可重复重建，传统版为 `Init`。长度须小于 `INT_MAX`。`sa[k]` 为第 k 个非空后缀起点，`rk[i]` 为排名，全部 0-based；`lcp[k]` 为 `sa[k-1]` 与 `sa[k]` 的最长公共前缀，`lcp[0]=0`。空序列合法，三数组均为空。无需手动追加哨兵，也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序，倍增 $O(n \log n + \text{alphabet})$ ，空间 $O(n + \text{alphabet})$ ；字节串 `alphabet=256`。大整数先自行保序离散化，避免按最大原值开桶。Kasai 求 `lcp` 为 $O(n)$ 。不同起点 x,y 的 LCP 为 `lcp[min(rk[x],rk[y])+1..max(rk[x],rk[y])]` 的最小值；本结构不内建 RMQ，需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 `n-x`。覆盖倍增构造，不替代 DC3/SA-IS。

```

1 struct SuffixArray
2 {
3     vector<int> sa, rk, lcp;
4
5     SuffixArray(const string &s)
6     {
7         vector<int> a;
8         a.reserve(s.size());
9         for (unsigned char c : s)
10             a.push_back(c);
11         init(a, 256);
12     }
13
14     SuffixArray(const vector<int> &s, int alphabet)
15     {
16         init(s, alphabet);
17     }
18
19     void init(const vector<int> &s, int alphabet)
20     {
21         assert(s.size() < INT_MAX);
22         assert(1 <= alphabet && alphabet < INT_MAX);
23         int n = (int)s.size();
24         sa.resize(n);
25         rk.resize(n);
26         lcp.assign(n, 0);
27         if (!n)
28             return;
29         vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
30         int m = alphabet;
31         for (int i = 0; i < n; i++)
32         {
33             assert(0 <= s[i] && s[i] < alphabet);
34             rk[i] = s[i] + 1;
35             cnt[rk[i]]++;
36         }
37         for (int i = 1; i <= m; i++)
38             cnt[i] += cnt[i - 1];
39         for (int i = n - 1; i >= 0; i--)
40             sa[--cnt[rk[i]]] = i;

```

```

41     for (int k = 1; k < n; k *= 2)
42     {
43         int p = 0;
44         for (int i = n - k; i < n; i++)
45             order[p++] = i;
46         for (int i : sa)
47             if (i >= k)
48                 order[p++] = i - k;
49         fill(cnt.begin(), cnt.begin() + m + 1, 0);
50         for (int x : rk)
51             cnt[x]++;
52         for (int i = 1; i <= m; i++)
53             cnt[i] += cnt[i - 1];
54         for (int i = n - 1; i >= 0; i--)
55             sa[--cnt[rk[order[i]]]] = order[i];
56         old.swap(rk);
57         auto second = [&](int i)
58         {
59             return i < n - k ? old[i + k] : 0;
60         };
61         m = 1;
62         rk[sa[0]] = m;
63         for (int i = 1; i < n; i++)
64         {
65             int a = sa[i - 1], b = sa[i];
66             if (old[a] != old[b] || second(a) != second(b))
67                 m++;
68             rk[b] = m;
69         }
70         if (m == n)
71             break;
72     }
73     for (int i = 0; i < n; i++)
74         rk[sa[i]] = i;
75     for (int i = 0, k = 0; i < n; i++)
76     {
77         if (!rk[i])
78         {
79             k = 0;
80             continue;
81         }
82         int j = sa[rk[i] - 1];
83         while (k < n - i && k < n - j && s[i + k] == s[j + k])
84             ++k;
85         lcp[rk[i]] = k;
86         if (k)
87             --k;
88     }
89 }
90 };

```

30.8 后缀自动机

`extend(c)` 接收整数 0..25; `distinct()` 返回不同非空子串数, `counts()` 返回 `endpos` 次数且不修改结构, 可重复调用。状态 `a`, 初始状态 0, `link` 根为 -1; 空间 $O(26n)$, 构建 $O(26n)$ 。

```

1 struct SuffixAutomaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
12
13    void extend(int c)
14    {
15        assert(0 <= c && c < 26);
16        int cur = (int)a.size();
17        a.push_back(Node{});
18        a[cur].len = a[last].len + 1;
19        a[cur].occ = 1;
20        int p = last;
21        while (p != -1 && !a[p].go[c])
22        {
23            a[p].go[c] = cur;
24            p = a[p].link;
25        }
26        if (p == -1)
27            a[cur].link = 0;
28        else
29        {
30            int q = a[p].go[c];
31            if (a[p].len + 1 == a[q].len)
32                a[cur].link = q;
33            else
34            {
35                int clone = (int)a.size();
36                Node copy = a[q];
37                a.push_back(copy);
38                a[clone].len = a[p].len + 1;
39                a[clone].occ = 0;
40                while (p != -1 && a[p].go[c] == q)
41                {
42                    a[p].go[c] = clone;
43                    p = a[p].link;
44                }
45                a[q].link = a[cur].link = clone;
46            }
47        }
48        last = cur;
49    }
50
51    vector<long long> counts() const

```

```
52     {
53         vector<int> bucket(a[last].len + 1), order(a.size());
54         vector<long long> ans(a.size());
55         for (auto &v : a)
56             ++bucket[v.len];
57         partial_sum(bucket.begin(), bucket.end(), bucket.begin());
58         for (int i = 0; i < (int)a.size(); i++)
59             {
60                 order[--bucket[a[i].len]] = i;
61                 ans[i] = a[i].occ;
62             }
63         for (int i = (int)order.size() - 1; i > 0; i--)
64             ans[a[order[i]].link] += ans[order[i]];
65         return ans;
66     }
67
68     long long distinct() const
69     {
70         long long ans = 0;
71         for (int i = 1; i < (int)a.size(); i++)
72             ans += a[i].len - a[a[i].link].len;
73         return ans;
74     }
75 };
```

第三十一章 后缀查询与重复子串

31.1 后缀 LCP 与子串比较

先构造 SuffixArray, 再构造 SuffixLCP(sa); 传统版对应 Suffix_Array 和 Suffix_LCP。init(sa) / Init(sa) 可重新建表。复制所需排名与 LCP 数据, 不保存原字符串或源对象引用, 源对象可销毁或重建。query(x,y) 返回 s[x..n) 与 s[y..n) 的最长公共前缀, $0 \leq x, y \leq n$, 允许空后缀; 同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串, 允许空串, 返回 -1/0/1; 传统接口为 Query/Compare。若公共前缀覆盖较短子串, 按长度比较, 否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表 $O(n \log n)$ 时间、空间, 每次查询 $O(1)$, 无需哈希和原串。n、rk、lg、st 可读, 禁止调用方破坏。

```
1 struct SuffixLCP
2 {
3     int n;
4     vector<int> rk, lg;
5     vector<vector<int>>> st;
6
7     SuffixLCP(const SuffixArray &suffix)
8     {
9         init(suffix);
10    }
11
12    void init(const SuffixArray &suffix)
13    {
14        n = suffix.sa.size();
15        rk = suffix.rk;
16        lg.assign(n + 1, 0);
17        for (int i = 2; i <= n; i++)
18            lg[i] = lg[i / 2] + 1;
19        st.assign(lg[n] + 1, {});
20        st[0] = suffix.lcp;
21        for (int k = 1; k <= lg[n]; k++)
22        {
23            int len = 1 << k;
24            st[k].resize(n - len + 1);
25            for (int i = 0; i <= n - len; i++)
26                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
27        }
28    }
```



```

30 int query(int x, int y) const
31 {
32     assert(0 <= x && x <= n && 0 <= y && y <= n);
33     if (x == y || x == n || y == n)
34         return n - max(x, y);
35     int a = rk[x], b = rk[y];
36     if (a > b)
37         swap(a, b);
38     int k = lg[b - a];
39     return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
40 }
41
42 // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
43 int compare(int l1, int r1, int l2, int r2) const
44 {
45     assert(0 <= l1 && l1 <= r1 && r1 <= n);
46     assert(0 <= l2 && l2 <= r2 && r2 <= n);
47     int a = r1 - l1, b = r2 - l2;
48     if (query(l1, l2) >= min(a, b))
49         return (a > b) - (a < b);
50     return rk[l1] < rk[l2] ? -1 : 1;
51 }
52 };

```

31.2 两个前缀的最长公共后缀

`prefix_lcs(reversed,x,y)` 返回原序列 `s[0..x)` 与 `s[0..y)` 的最长公共后缀，端点满足 $0 \leq x, y \leq n$ 。reversed 必须在原序列的反转序列上构造 SuffixLCP，不能传正向对象；转换为 `reversed.query(n-x,n-y)`，单次 $O(1)$ 。传统版 Prefix_Lcs 接受 Suffix_LCP。只需前向 LCP 时不必构造反向对象；该接口中的 LCS 是最长公共后缀，不是最长公共子序列。对于两个任意子串，以右端点查询后还需截断到两个子串长度的最小值。

```

1 // reversed must be built on the reversed original sequence.
2 inline int prefix_lcs(const SuffixLCP &reversed, int x, int y)
3 {
4     assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
5     return reversed.query(reversed.n - x, reversed.n - y);
6 }

```

31.3 重复子串 AA 的起止位置计数

`square_counts(forward,reversed)` 返回 `(start,finish)` 两个长度 `n` 的 `long long` 数组, `start[i]/finish[i]` 为开始/结束于位置 `i` 的非空 AA 型子串个数; 结束位置是最后一个字符下标, 均 0-based。两对象必须分别对应同一原序列及其反序列, 长度相同, 允许空序列。传统接口为 `Square_Counts`。枚举半长 `len`, 并以其整数倍 `i` 为锚点; 向左匹配截断为 `len-1`, 向右截断为 `len`, 由 LCP/LCS 得出合法起点连续区间, 差分累计。左侧不能放宽到 `len`, 否则相邻锚点会重复计数。预处理查询器后, 此函数 $O(n \log n)$ 时间、 $O(n)$ 额外空间。不同出现位置、不同半长分别计数; 不要求 `A` 为最小周期。P1117 的 AABB 拆分数为所有分界点 `finish[i]*start[i+1]` 之和, 累加类型需按题目范围选择, 函数本身只返回起止计数。

```

1 // Count nonempty squares AA by their starting and ending positions.
2 inline pair<vector<long long>, vector<long long>>
3 square_counts(const SuffixLCP &forward, const SuffixLCP &reversed)
4 {
5     int n = forward.n;
6     assert(reversed.n == n);
7     vector<long long> start(n + 1), finish(n + 1);
8     for (int len = 1; len <= n / 2; len++)
9         for (int i = 0; i < n - len; i += len)
10            {
11                int j = i + len;
12                int left = min(len - 1, prefix_lcs(reversed, i, j));
13                int right = min(len, forward.query(i, j));
14                if (left + right < len)
15                    continue;
16                int l = i - left, r = i + right - len;
17                start[l]++;
18                start[r + 1]--;
19                finish[l + 2 * len - 1]++;
20                finish[r + 2 * len]--;
21            }
22     for (int i = 1; i < n; i++)
23     {
24         start[i] += start[i - 1];
25         finish[i] += finish[i - 1];
26     }
27     start.resize(n);
28     finish.resize(n);
29     return {start, finish};
30 }
```

第三十二章 回文树

32.1 回文树 PAM: 回文后缀与出现次数

`add(ch)` 追加小写英文字母, 返回最长回文后缀节点, 亦保存于 `last`。节点 0 是长度 0 根, 1 是长度 -1 根, 实际回文节点从 2 开始; `distinct()` 为不同非空回文串数。`t[u].len` 为长度, `fail` 为最长真回文后缀链接, `suffixes` 为该节点的非空回文后缀数量; 追加后的 `t[last].suffixes` 就是本位置结尾的回文串数。`total` 为当前全部回文子串的出现次数之和。`end` 为首次出现时的 0-based 右端点, 可恢复原串子段; `hits` 仅统计作为最长达后缀的次数, 不能当出现次数。`occurrences()` 返回按节点编号的完整出现次数副本, 根置零, $O(n)$; 重复调用以及查询后继续追加均安全。`init()` 清空重建。固定 26 字符集下建树 $O(n)$, 空间 $O(26n)$ 。

```
1 struct PalindromicTree
2 {
3     struct Node
4     {
5         int next[26]{};
6         int len = 0, fail = 0, suffixes = 0, end = -1;
7         long long hits = 0;
8     };
9
10    vector<Node> t;
11    vector<int> s;
12    int last = 0;
13    long long total = 0;
14
15    PalindromicTree()
16    {
17        init();
18    }
19
20    void init()
21    {
22        t.assign(2, Node{});
23        t[0].fail = 1;
24        t[1].len = -1;
25        s.assign(1, -1);
26        last = 0;
27        total = 0;
28    }
29
30    int get_fail(int u) const
31    {
32        int pos = s.size() - 1;
33        while (s[pos - t[u].len - 1] != s[pos])
34            u = t[u].fail;
35        return u;
36    }
37
38    // Append a lowercase letter; return the longest palindromic suffix node.
39    int add(char ch)
40    {
41        int c = ch - 'a';
```

```

42     assert(0 <= c && c < 26);
43     s.push_back(c);
44     int u = get_fail(last);
45     if (!t[u].next[c])
46     {
47         int v = t.size();
48         t.push_back(Node{});
49         t[v].len = t[u].len + 2;
50         t[v].fail = t[get_fail(t[u].fail)].next[c];
51         t[v].suffixes = t[t[v].fail].suffixes + 1;
52         t[v].end = (int)s.size() - 2;
53         t[u].next[c] = v;
54     }
55     last = t[u].next[c];
56     t[last].hits++;
57     total += t[last].suffixes;
58     return last;
59 }
60
61 int distinct() const
62 {
63     return (int)t.size() - 2;
64 }
65
66 // Copy-and-propagate: repeated calls and later appends remain valid.
67 vector<long long> occurrences() const
68 {
69     vector<long long> count(t.size());
70     for (int u = 2; u < (int)t.size(); u++)
71         count[u] = t[u].hits;
72     for (int u = (int)t.size() - 1; u >= 2; u--)
73         count[t[u].fail] += count[u];
74     count[0] = 0;
75     count[1] = 0;
76     return count;
77 }
78 };

```

第三十三章 数论

33.1 模运算、素性、CRT 与类欧几里德

mul/power 支持无符号 64 位正模数，乘积用 uint128。prime 使用确定性 64 位底数。inverse(a,m) 无逆元返回 -1。crt(r,m,b,n) 合并 $x \equiv r \pmod{m}$ 与 $x \equiv b \pmod{n}$ ，要求 $m,n > 0$ 。从 $r=0$ 、 $m=1$ 开始逐项合并；成功后 r 为最小非负解、 m 为最小正周期。余数允许完整 long long 范围。无解返回 false；有解但新周期超过 LLONG_MAX 抛 overflow_error。两种失败都会先将 r 归一化， m 不变；失败后不能继续当作完整解使用。递归 exgcd 和乘法中间量用 int128，单次 $O(\log \min(m,n))$ ，抄写 CRT 时只需类型别名、exgcd 和 crt。floor_sum 为 $i=0..n-1$ 的 $\text{floor}((ai+b)/m)$ 之和，支持负 a 、 b ，返回 int128；约定 $n \leq 10^9$ 、 $m \leq 10^9$ ， a 、 b 为有符号 64 位。exgcd 单独调用约定 $0 \leq a, b \leq 2^{63}$ ，输出 int128 系数满足 $ax+by=\text{gcd}(a,b)$ ， $(0,0)$ 返回 0；有符号方程用 linear_equation。

```
1 struct NumberTheory
2 {
3     using ll = long long;
4     using ull = unsigned long long;
5     using i128 = __int128_t;
6     using u128 = __uint128_t;
7
8     static ull mul(ull a, ull b, ull m)
9     {
10         return u128(a) * b % m;
11     }
12
13     static ull power(ull a, ull b, ull m)
14     {
15         assert(m);
16         ull r = 1 % m;
17         a %= m;
18         for (; b; b >>= 1, a = mul(a, a, m))
19             if (b & 1)
20                 r = mul(r, a, m);
21         return r;
22     }
23
24     static bool prime(ull n)
25     {
26         if (n < 2)
27             return false;
28         for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
29             if (n % p == 0)
30                 return n == p;
31         ull d = n - 1;
32         int s = 0;
33         while (!(d & 1))
34         {
35             d >>= 1;
36             ++s;
37         }
38         for (ull a :
39             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
```

```

40     {
41         if (a % n == 0)
42             continue;
43         ull x = power(a, d, n);
44         if (x == 1 || x == n - 1)
45             continue;
46         bool ok = false;
47         for (int j = 1; j < s; j++)
48         {
49             x = mul(x, x, n);
50             if (x == n - 1)
51             {
52                 ok = true;
53                 break;
54             }
55         }
56         if (!ok)
57             return false;
58     }
59     return true;
60 }
61
62 static i128 exgcd(i128 a, i128 b, i128 &x, i128 &y)
63 {
64     if (!b)
65     {
66         x = 1;
67         y = 0;
68         return a;
69     }
70     i128 d = exgcd(b, a % b, y, x);
71     y -= a / b * x;
72     return d;
73 }
74
75 static ll inverse(ll a, ll m)
76 {
77     assert(m > 0);
78     a %= m;
79     if (a < 0)
80         a += m;
81     i128 x, y;
82     if (exgcd(a, m, x, y) != 1)
83         return -1;
84     return (x % m + m) % m;
85 }
86
87 // x = r (mod m); false means inconsistent. Throws if lcm exceeds int64.
88 static bool crt(ll &r, ll &m, ll b, ll n)
89 {
90     assert(m > 0 && n > 0);
91     r %= m;
92     if (r < 0)
93         r += m;
94     b %= n;
95     if (b < 0)

```

```

96         b += n;
97     i128 x, y;
98     ll g = (ll)exgcd(m, n, x, y);
99     i128 diff = i128(b) - r;
100    if (diff % g)
101        return false;
102    i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
103    if (mod > LLONG_MAX)
104        throw overflow_error("CRT modulus");
105    r = (r + i128(m) * k) % mod;
106    m = (ll)mod;
107    return true;
108 }
109
110 // sum_{i=0}^{n-1} floor((a*i+b)/m), signed a,b; 0<=n<=1e9, 1<=m<=1e9.
111 static i128 floor_sum(ll n, ll m, ll a, ll b)
112 {
113     assert(n >= 0 && m > 0);
114     i128 ans = 0;
115     auto norm = [&](ll &v)
116     {
117         ll q = v / m;
118         if (v % m < 0)
119             --q;
120         v = (ll)(i128(v) - i128(q) * m);
121         return q;
122     };
123     ans += i128(norm(a)) * n * (n - 1) / 2;
124     ans += i128(norm(b)) * n;
125     while (true)
126     {
127         ans += i128(n) * (n - 1) / 2 * (a / m);
128         a %= m;
129         ans += i128(n) * (b / m);
130         b %= m;
131         i128 y = i128(a) * n + b;
132         if (y < m)
133             break;
134         n = (ll)(y / m);
135         b = (ll)(y % m);
136         swap(a, m);
137     }
138     return ans;
139 }
140 };

```

33.2 Pollard-Rho 因数分解

`factor(n)` 返回升序质因子（含重数）， $1 \leq n \leq \text{ULLONG_MAX}$ ；传统接口 `Factor`。`n=1` 返回空数组； $n \geq 2$ 时 `size()==1` 等价于 `n` 为素数，否则 `front()/back()` 分别取最小/最大质因子。依赖 `NumberTheory`。公开使用 `factor`；`rho/split` 为内部辅助，`rho` 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 `rho` 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度 $O(\log n)$ 。

```

1 struct PollardRho
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     mt19937_64 rng;
6
7     PollardRho(ull seed = 712367821) : rng(seed)
8     {
9     }
10
11     ull rho(ull n)
12     {
13         if (n % 2 == 0)
14             return 2;
15         for (;;)
16         {
17             ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
18             auto f = [&](ull v)
19             {
20                 return (NumberTheory::mul(v, v, n) + u128(c)) % n;
21             };
22             // Retry bounded attempts; Las Vegas: only exact divisors returned.
23             for (int i = 0; i < 200000 && d == 1; i++)
24             {
25                 x = f(x);
26                 y = f(f(y));
27                 d = gcd(x > y ? x - y : y - x, n);
28             }
29             if (1 < d && d < n)
30                 return d;
31         }
32     }
33
34     void split(ull n, vector<ull> &a)
35     {
36         if (n == 1)
37             return;
38         if (NumberTheory::prime(n))
39         {
40             a.push_back(n);
41             return;
42         }
43         ull d = rho(n);
44         split(d, a);
45         split(n / d, a);
46     }

```



```

47
48     vector<ull> factor(ull n)
49     {
50         assert(n >= 1);
51         vector<ull> a;
52         split(n, a);
53         sort(a.begin(), a.end());
54         return a;
55     }
56 };

```

33.3 线性筛、欧拉函数与 Möbius 函数

构造参数 $0 \leq n < \text{INT_MAX}$ ，且数组须能分配内存。prime（传统版 Prime）为递增质数表；lp/phi/mu 为 $0..n$ 数组。 $x \geq 2$ 时 $\text{lp}[x]$ 是最小质因子， $\text{lp}[x] = x$ 等价于 x 为素数；反复除以 $\text{lp}[x]$ 可分解质因数。 $\text{lp}[0] = \text{lp}[1] = 0$ ，不能直接用 $\text{lp}[x] = x$ 判断 0。 $n \geq 1$ 时 $\text{phi}[1] = \text{mu}[1] = 1$ ；第 0 项均为占位 0。 $O(n)$ 时间和空间，分解单个表内正整数耗时 $O(\log x)$ 。 $n=0$ 不可访问下标 1。

```

1 struct LinearSieve
2 {
3     vector<int> prime, lp, phi, mu;
4
5     LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
6     {
7         if (n)
8             phi[1] = mu[1] = 1;
9         for (int i = 2; i <= n; i++)
10         {
11             if (!lp[i])
12             {
13                 lp[i] = i;
14                 prime.push_back(i);
15                 phi[i] = i - 1;
16                 mu[i] = -1;
17             }
18             for (int p : prime)
19             {
20                 if (p > n / i)
21                     break;
22                 int j = i * p;
23                 lp[j] = p;
24                 if (i % p == 0)
25                 {
26                     phi[j] = phi[i] * p;
27                     mu[j] = 0;
28                     break;
29                 }
30                 phi[j] = phi[i] * (p - 1);
31                 mu[j] = -mu[i];
32             }
33         }
34     }
35 };

```

33.4 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且 ≥ 2 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。
复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。

```

1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod)
6     {
7     }
8
9     ModInt operator+(ModInt b) const
10    {
11        return ModInt((long long)v + b.v);
12    }
13
14    ModInt operator-(ModInt b) const
15    {
16        return ModInt((long long)v - b.v);
17    }
18
19    ModInt operator*(ModInt b) const
20    {
21        return ModInt(1LL * v * b.v);
22    }
23
24    ModInt pow(long long e) const
25    {
26        assert(e >= 0);
27        ModInt a = *this, r = 1;
28        for (; e >>= 1, a = a * a)
29            if (e & 1)
30                r = r * a;
31        return r;
32    }
33
34    ModInt inv() const
35    {
36        assert(v);
37        return pow(mod - 2);
38    } // prime modulus
39
40    ModInt operator/(ModInt b) const
41    {
42        return *this * b.inv();
43    }
44 };

```

33.5 阶乘组合数

`Binomial<prime>(n=0)` 预处理 $0..n$ ，要求 `prime` 为素数且 $0 \leq n < \text{prime}$ 。`init(n)` 只扩大当前上限，较小或相同的 n 不改变表；`choose(n,k)` 返回 $C(n,k)$ ，`permute(n,k)` 返回 $n!/(n-k)!$ 。查询前显式预处理到 n ，查询本身不扩表。 $n < 0$ 、 $k < 0$ 或 $k > n$ 返回 0， $C(0,0)=P(0,0)=1$ 。`fac`、`ifac` 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 $O(1)$ ；从 `old` 扩到 n 新增 $O(n-\text{old}+\log \text{prime})$ 次域运算，重分配还可能复制旧表。首次预处理 $O(n+\log \text{prime})$ ，空间 $O(n)$ 。超过模数不能使用阶乘除法：小素数用 `Lucas`，小合数用 `exLucas`。

```

1  template <int mod> struct Binomial
2  {
3      using Z = ModInt<mod>;
4      vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6      Binomial(int n = 0)
7      {
8          init(n);
9      }
10
11     void init(int n)
12     {
13         assert(0 <= n && n < mod);
14         int old = (int)fac.size() - 1;
15         if (n <= old)
16             return;
17         fac.resize(n + 1);
18         ifac.resize(n + 1);
19         for (int i = old + 1; i <= n; i++)
20             fac[i] = fac[i - 1] * i;
21         ifac[n] = fac[n].inv();
22         for (int i = n; i > old; i--)
23             ifac[i - 1] = ifac[i] * i;
24     }
25
26     Z choose(int n, int k) const
27     {
28         if (k < 0 || k > n)
29             return 0;
30         assert(n < (int)fac.size());
31         return fac[n] * ifac[k] * ifac[n - k];
32     }
33
34     Z permute(int n, int k) const
35     {
36         if (k < 0 || k > n)
37             return 0;
38         assert(n < (int)fac.size());
39         return fac[n] * ifac[n - k];
40     }
41 };

```

第三十四章 二元一次不定方程

34.1 二元一次不定方程

`linear_equation(a,b,c,x,y)` 求 $ax+by=c$ 的任意整数特解；传统接口 `Linear_Equation`。a、b、c 支持完整 long long，x、y 必须为 int128。无解返回 false 且 $x=y=0$ ； $a=b=c=0$ 返回 true、 $x=y=0$ ，此时所有整数对都是解。其他有解情况，令 $g=\gcd(|a|,|b|)$ ，通解为 $(x+(b/g)t, y-(a/g)t)$ 。依赖 `NumberTheory::exgcd`；只需抄递归 `exgcd` 与本函数。时间、递归空间 $O(\log(\max(|a|,|b|)+2))$ ；不承诺特解最小或为正。

```
1 inline bool
2 linear_equation(long long a, long long b, long long c, __int128_t &x, __int128_t &y)
3 {
4     using i128 = __int128_t;
5     x = 0;
6     y = 0;
7     if (!a && !b)
8         return c == 0;
9     i128 aa = a < 0 ? -i128(a) : i128(a);
10    i128 bb = b < 0 ? -i128(b) : i128(b);
11    i128 u, v;
12    i128 g = NumberTheory::exgcd(aa, bb, u, v);
13    if (c % g)
14        return false;
15    x = u * (c / g);
16    y = v * (c / g);
17    if (a < 0)
18        x = -x;
19    if (b < 0)
20        y = -y;
21    return true;
22 }
```

第三十五章 线性同余方程

35.1 线性同余方程

`linear_congruence(a,b,m)` 求 $a*x+b\equiv 0 \pmod m$, 返回 {最小非负解, 最小正周期}, 无解返回 {-1,-1}; 传统接口 `Linear_Congruence`。m 为正 long long, a、b 支持完整 long long。若 $g=\gcd(a,m)$ 整除 b, 周期为 m/g ; 包括 $a=0$ 和 $m=1$, 所有整数均满足时返回 {0,1}。先归一化 a, 并在 int128 中对 b 取相反数和乘法。依赖 `NumberTheory::exgcd`; 递归时间、空间 $O(\log(m+1))$ 。注意本接口的常数项是 +b, 若题目写 $a*x\equiv c$, 应传 -c; c 为 `LLONG_MIN` 时须避免在 64 位中取负。

```
1 inline pair<long long, long long>
2 linear_congruence(long long a, long long b, long long m)
3 {
4     assert(m > 0);
5     using i128 = __int128_t;
6     a %= m;
7     if (a < 0)
8         a += m;
9     i128 x, y;
10    i128 g = NumberTheory::exgcd(a, m, x, y);
11    if (i128(b) % g)
12        return {-1, -1};
13    i128 period = m / g;
14    x = x * (-i128(b) / g) % period;
15    if (x < 0)
16        x += period;
17    return {(long long)x, (long long)period};
18 }
```

第三十六章 区间筛素数

36.1 区间筛素数

`segmented_primes(l,r)` 返回闭区间内递增的全部素数, $0 \leq l \leq r \leq \text{INT_MAX}$; 区间长度须能分配内存。独立函数, 每次重建底筛; 不依赖预存素数表。令 $W=r-l+1$, 时间 $O((\sqrt{r+W})\log \log(r+3))$, 空间 $O(\sqrt{r+W})$ 。倍数起点、递增变量均用 `long long`, 避免 `INT_MAX` 端点溢出; 0 和 1 不进入结果。

```
1 inline vector<int> segmented_primes(int l, int r)
2 {
3     assert(0 <= l && l <= r);
4     int limit = 0;
5     while (1LL * (limit + 1) * (limit + 1) <= r)
6         limit++;
7     vector<char> composite(limit + 1);
8     vector<char> removed(1LL * r - l + 1);
9     for (int p = 2; p <= limit; p++)
10         if (!composite[p])
11         {
12             for (long long j = 1LL * p * p; j <= limit; j += p)
13                 composite[j] = true;
14             long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
15             for (long long j = start; j <= r; j += p)
16                 removed[j - l] = true;
17         }
18     vector<int> primes;
19     for (long long x = max(l, 2); x <= r; x++)
20         if (!removed[x - l])
21             primes.push_back(x);
22     return primes;
23 }
```

第三十七章 批量模逆元

37.1 任意数组的批量逆元

`batch_inverse(a,mod)` 返回与 `a` 同序的逆元向量，传统接口为 `Batch_Inverse`。要求 `mod` 为正 `long long`，每个输入可为完整 `long long`；只要某一项与 `mod` 不互素，整个结果返回 `nullopt`，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 `move(a)` 可避免输入副本。依赖 `NumberTheory` 的 `inverse` 和 `exgcd`，传统版对应 `Inverse`、`Exgcd`。仅求一次逆元，时间 $O(n+\log \text{mod})$ ，空间 $O(n)$ ；128 位中间乘法防止溢出。

```
1 inline optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
2 {
3     assert(mod > 0);
4     int n = a.size();
5     vector<long long> prefix(n + 1, 1 % mod);
6     for (int i = 0; i < n; i++)
7     {
8         a[i] %= mod;
9         if (a[i] < 0)
10             a[i] += mod;
11         prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
12     }
13     long long inverse = NumberTheory::inverse(prefix[n], mod);
14     if (inverse == -1)
15         return nullopt;
16     for (int i = n - 1; i >= 0; i--)
17     {
18         long long value = a[i];
19         a[i] = __int128_t(inverse) * prefix[i] % mod;
20         inverse = __int128_t(inverse) * value % mod;
21     }
22     return a;
23 }
```

第三十八章 连续整数逆元表

38.1 连续整数的线性逆元表

`inverse_table(n,p)` 返回长度 $n+1$ 的 `int` 向量，位置 i 保存 i 模 p 的逆元， $1 \leq i \leq n$ 。位置 0 固定为 0，仅为占位，不表示零有逆元； $n=0$ 返回单个占位项。要求 p 为素数、 $2 \leq p \leq \text{INT_MAX}$ 且 $0 \leq n < p$ ，不检测素性。传统接口为 `Inverse_Table`。时间和空间均 $O(n)$ ，中间乘法用 `long long`；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前 i 与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```
1 inline vector<int> inverse_table(int n, int p)
2 {
3     assert(p >= 2 && 0 <= n && n < p);
4     vector<int> inverse(n + 1);
5     if (n)
6         inverse[1] = 1;
7     for (int i = 2; i <= n; i++)
8         inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
9     return inverse;
10 }
```


第三十九章 混合进制中国剩余定理

39.1 Garner：大周期 CRT 与混合进制

`garner_digits(b,moduli)` 返回按输入顺序排列的混合进制各位；`garner(b,moduli,target)` 返回最小非负 CRT 解对 `target` 的余数。传统接口为 `Garner_Digits`、`Garner`。两个输入向量等长，长度可为 0；模数为正且两两互素，允许模数 1，余数支持完整 `long long` 范围。`target` 为正 `long long`，与输入模数无需互素。输入不互素时抛 `invalid_argument`，即使方程有解也不适用；此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积，中间运算使用 `int128`。 k 个模数耗时 $O(k^2+k \log M)$ ，空间 $O(k)$ ； M 为最大输入模数。依赖 `NumberTheory` 的 `inverse` 和 `exgcd`，传统版对应 `Inverse`、`Exgcd`。

```
1 // Pairwise coprime positive moduli. Digits use the input order.
2 inline vector<long long> garner_digits(const vector<long long> &b,
3                                       const vector<long long> &moduli)
4 {
5     using ll = long long;
6     using i128 = __int128_t;
7     int n = b.size();
8     assert(moduli.size() == b.size());
9     vector<ll> value(n), coefficient(n, 1), digit(n);
10    for (ll m : moduli)
11        assert(m > 0);
12    for (int i = 0; i < n; i++)
13    {
14        ll m = moduli[i];
15        ll inverse = NumberTheory::inverse(coefficient[i], m);
16        if (inverse == -1)
17            throw invalid_argument("Garner moduli are not coprime");
18        i128 delta = (i128(b[i]) - value[i]) % m;
19        if (delta < 0)
20            delta += m;
21        digit[i] = delta * inverse % m;
22        for (int j = i + 1; j < n; j++)
23        {
24            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
25            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
26        }
27    }
28    return digit;
29 }
```

规范解对目标数取模

```
31 // Return the least nonnegative CRT solution modulo target, not all solutions.
32 inline long long
33 garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
34 {
35     assert(target > 0);
36     auto digit = garner_digits(b, moduli);
37     long long answer = 0;
38     for (int i = (int)digit.size() - 1; i >= 0; i--)
39         answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
40     return answer;
41 }
```

第四十章 乘法阶与原根

40.1 乘法阶与原根

构造 `PrimitiveRoot(n)`, 要求 $2 \leq n \leq \text{INT_MAX}$; `init(n)` 可重建。`order(a)` 返回 a 模 n 的乘法阶, a 不可逆时返回 0; `is_root(a)` 判断该剩余类是否为原根, a 可为完整 `long long`。`minimum()` 返回 $1..n-1$ 内最小原根, 无原根返回 -1; `all()` 返回升序的全部原根, 无原根返回空向量。`mod`、`phi`、`exists` 只读; `factors` 为 `phi` 的不同素因子, 辅助 `factor` 为试除分解。依赖 `NumberTheory` 的 `power`。初始化 $O(\sqrt{n})$, 求阶 $O(\log^2 n)$, 原根判定 $O((1+\omega(\phi))\log n)$ 。若最小原根为 G , 顺序枚举最坏检查 G 个候选, 不能假定 G 总是常数。枚举全部另需 $O(\phi \log \phi + c \log c)$ 时间、 $O(c)$ 空间, c 为原根数; 大模数只求最小原根或阶时, 不应调用 `all`。

```
1 struct PrimitiveRoot
2 {
3     int mod, phi;
4     bool exists;
5     vector<int> factors;
6
7     static vector<int> factor(int n)
8     {
9         vector<int> result;
10        for (int p = 2; p <= n / p; p++)
11            if (n % p == 0)
12            {
13                result.push_back(p);
14                while (n % p == 0)
15                    n /= p;
16            }
17        if (n > 1)
18            result.push_back(n);
19        return result;
20    }
21
22    PrimitiveRoot(int n)
23    {
24        init(n);
25    }
```

```
27 void init(int n)
28 {
29     assert(n >= 2);
30     mod = n;
31     phi = n;
32     for (int p : factor(n))
33         phi -= phi / p;
34     int odd = n;
35     if (odd % 2 == 0)
36         odd /= 2;
37     exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
38     factors = factor(phi);
39 }
40
41 int order(long long a) const
42 {
43     a %= mod;
44     if (a < 0)
45         a += mod;
46     if (gcd(a, (long long)mod) != 1)
47         return 0;
48     int result = phi;
49     for (int p : factors)
50         while (result % p == 0 && NumberTheory::power(a, result / p, mod) == 1)
51             result /= p;
52     return result;
53 }
```

```

55     bool is_root(long long a) const
56     {
57         if (!exists)
58             return false;
59         a %= mod;
60         if (a < 0)
61             a += mod;
62         if (gcd(a, (long long)mod) != 1)
63             return false;
64         for (int p : factors)
65             if (NumberTheory::power(a, phi / p, mod) == 1)
66                 return false;
67         return true;
68     }
69
70     int minimum() const
71     {
72         if (!exists)
73             return -1;
74         for (int g = 1; g < mod; g++)
75             if (is_root(g))
76                 return g;
77         return -1;
78     }
79
80     vector<int> all() const
81     {
82         int g = minimum();
83         vector<int> roots;
84         if (g == -1)
85             return roots;
86         long long value = 1;
87         for (int k = 1; k <= phi; k++)
88         {
89             value = value * g % mod;
90             if (gcd(k, phi) == 1)
91                 roots.push_back(value);
92         }
93         sort(roots.begin(), roots.end());
94         return roots;
95     }
96 };

```

第四十一章 矩形 GCD 计数

41.1 矩形 GCD 计数

构造 CoprimePairs(n) 预处理 mu 前缀和, $0 \leq n < \text{INT_MAX}$, $O(n)$ 时间与空间; 依赖 LinearSieve。count(a,b) 求 $[1,a] \times [1,b]$ 的互质数对, a,b 为非负 int, 要求 $\min(a,b) \leq n$, 任一为 0 返回 0。rectangle(a,b,c,d,k) 求 $[a,b] \times [c,d]$ 中 $\gcd(x,y)=k$ 的数对, 要求 $1 \leq a \leq b$, $1 \leq c \leq d$, $k \geq 1$, $\min(b/k, d/k) \leq n$; 闭区间容斥。返回 long long, 中间分块累加用 int128。单次前缀查询 $O(\sqrt{a} + \sqrt{b})$, 额外空间 $O(1)$ 。传统接口 Coprime_Pairs、Count、Rectangle。sum 为 mu 前缀和, 只读。

```
1 struct CoprimePairs
2 {
3     vector<int> sum;
4
5     CoprimePairs(int n) : sum(LinearSieve(n).mu)
6     {
7         for (int i = 1; i <= n; i++)
8             sum[i] += sum[i - 1];
9     }
10
11     long long count(int a, int b) const
12     {
13         assert(a >= 0 && b >= 0 && min(a, b) < (int)sum.size());
14         __int128_t answer = 0;
15         for (int l = 1, r; l <= min(a, b); l = r + 1)
16         {
17             int x = a / l;
18             int y = b / l;
19             r = min(a / x, b / y);
20             answer += __int128_t(sum[r] - sum[l - 1]) * x * y;
21         }
22         return (long long)answer;
23     }
24
25     long long rectangle(int a, int b, int c, int d, int k) const
26     {
27         assert(1 <= a && a <= b && 1 <= c && c <= d && k >= 1);
28         return count(b / k, d / k) - count((a - 1) / k, d / k) -
29             count(b / k, (c - 1) / k) + count((a - 1) / k, (c - 1) / k);
30     }
31 };
```

第四十二章 带权类欧几里德

42.1 整除和、带权和与平方和

`floor_moments(n,m,a,b)` 对 $i=0..n-1$, 令 $y_i=\text{floor}((a*i+b)/m)$, 返回 $\{\sum y_i, \sum i*y_i, \sum y_i^2\}$, 元素为 `ModInt<998244353>`, 取整数用 `.v`。 $0 \leq n \leq 1000000001$, $1 \leq m \leq 10^9$; a, b 支持完整 `long long`。 $n=0$ 返回全零。时间与递归空间 $O(\log(m+1))$; 模数固定, 499122177 与 166374059 分别是 2、6 的逆元。传统接口 `Floor_Moments`, 元素为 `Mod_Int`。P5170 输入上限为闭区间, 驱动传 $n+1$, 输出索引顺序 0、2、1。

```
1 inline array<ModInt<998244353>, 3>
2 floor_moments(long long n, long long m, long long a, long long b)
3 {
4     using Z = ModInt<998244353>;
5     assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
6     if (!n)
7         return {0, 0, 0};
8     auto normalize = [&](long long &v)
9     {
10         long long q = v / m;
11         if (v % m < 0)
12             q--;
13         v = (__int128_t(v) - __int128_t(q) * m);
14         return Z(q);
15     };
16     Z x = normalize(a);
17     Z y = normalize(b);
18     Z count = n;
19     Z s1 = count * Z(n - 1) * Z(499122177);
20     Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
21     Z f = 0, g = 0, h = 0;
22     if (a)
23     {
24         long long height = (a * (n - 1) + b) / m;
25         auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
26         Z t = height;
27         f = count * t - u;
28         g = t * s1 - (w - u) * Z(499122177);
29         h = count * t * t - Z(2) * v - u;
30     }
31     return {f + x * s1 + y * count,
32             g + x * s2 + y * s1,
33             h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
34             Z(2) * y * f};
35 }
```

第四十三章 模几何级数与约数和

43.1 模幂与几何级数同时求值

`power_sum(a,n,mod)` 返回 $\{a^n \bmod \text{mod}, \sum_{i=0..n-1} a^i \bmod \text{mod}\}$ 。a 为完整 unsigned long long, n 为 uint128 项数, mod 为正 unsigned long long。n=0 返回 $\{1 \bmod \text{mod}, 0\}$; a=0 且 n>0 时和为 $1 \bmod \text{mod}$ 。无需逆元, 合数模数也可用。时间与递归空间 $O(\log(n+1))$, 至多 128 次折半。传统接口 Power_Sum。只抄此函数需 utility、cassert。

```
1 inline pair<unsigned long long, unsigned long long>
2 power_sum(unsigned long long a, __uint128_t n, unsigned long long mod)
3 {
4     assert(mod >= 1);
5     if (!n)
6         return {1 % mod, 0};
7     auto [p, s] = power_sum(a, n / 2, mod);
8     unsigned long long square = __uint128_t(p) * p % mod;
9     unsigned long long sum = __uint128_t(s) * (p + 1) % mod;
10    if (n % 2)
11        return {__uint128_t(square) * a % mod, (__uint128_t(sum) + square) % mod};
12    return {square, sum};
13 }
```


43.2 幂的约数和

传入 a 、 b 、 mod 和可复用的 PollardRho 分解器 `solver`，返回 a^b 的正约数和取模。 a 、 b 、 mod 均为 unsigned long long, $a \geq 1$ 、 $\text{mod} \geq 1$ 。 $a=1$ 或 $b=0$ 时结果为 $1 \% \text{mod}$ 。依赖 `power_sum` 与分解器；重数 e 对应 $e*b+1$ 项，长度用 uint128。时间由分解及每个不同质因子的 $O(\log(e*b+2))$ 求和组成。传统接口为 `Divisor_Sum_Power`，分解器为 `Pollard_Rho`。

```
1 inline unsigned long long divisor_sum_power(unsigned long long a,
2                                           unsigned long long b,
3                                           unsigned long long mod,
4                                           PollardRho &solver)
5 {
6     assert(a >= 1 && mod >= 1);
7     if (!b)
8         return 1 % mod;
9     auto factors = solver.factor(a);
10    unsigned long long answer = 1 % mod;
11    for (int i = 0, j; i < (int)factors.size(); i = j)
12    {
13        j = i + 1;
14        while (j < (int)factors.size() && factors[j] == factors[i])
15            j++;
16        __uint128_t length = __uint128_t(j - i) * b + 1;
17        auto sum = power_sum(factors[i], length, mod).second;
18        answer = __uint128_t(answer) * sum % mod;
19    }
20    return answer;
21 }
```

第四十四章 单个数的欧拉函数

44.1 单个数的欧拉函数

euler_phi(n) 使用试除, $O(\sqrt{n})$ 时间、 $O(1)$ 额外空间; euler_phi(n,solver) 复用 PollardRho, 时间由随机分解主导, 分解结果与递归占 $O(\log n)$ 空间。传统接口 Euler_Phi、Pollard_Rho。 $1 \leq n \leq \text{ULLONG_MAX}$, $\text{phi}(1)=1$ 。大素数或平衡大半素数应选分解器入口, 不要用试除穷扫平方根; 批量求 $1..N$ 用 LinearSieve。先除质因子再乘 $p-1$, 结果及中间乘积均不超过原 n 。只抄试除函数时不需要 NumberTheory/PollardRho。

```
1 inline unsigned long long euler_phi(unsigned long long n)
2 {
3     assert(n >= 1);
4     unsigned long long answer = n;
5     for (unsigned long long p = 2; p <= n / p; p++)
6         if (n % p == 0)
7             {
8                 answer = answer / p * (p - 1);
9                 while (n % p == 0)
10                     n /= p;
11             }
12     if (n > 1)
13         answer = answer / n * (n - 1);
14     return answer;
15 }
16
17 inline unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
18 {
19     assert(n >= 1);
20     auto factors = solver.factor(n);
21     unsigned long long answer = n;
22     unsigned long long previous = 0;
23     for (auto p : factors)
24         if (p != previous)
25             {
26                 answer = answer / p * (p - 1);
27                 previous = p;
28             }
29     return answer;
30 }
```

第四十五章 Carmichael 函数

45.1 Carmichael: 模乘法群的公共指数

`carmichael(n)` 返回 $\lambda(n)$, 即使所有与 n 互素的 a 都满足 a 的 $\lambda(n)$ 次幂模 n 为 1 的最小正指数; 约定 $\lambda(1)=1$ 。要求 $1 \leq n \leq \text{INT_MAX}$ 。传统接口为 `Carmichael`。试除分解后将各素数幂的指数取 lcm, 特别处理 2 的三次及以上幂。时间 $O(\sqrt{n})$, 额外空间 $O(1)$; 所有中间值不超过有符号 64 位范围, 结果不超过 n 。该指数只适用于可逆底数, 不能据此直接对非互素底数的幂降指数。函数不依赖原根模块。

```
1 inline int carmichael(int n)
2 {
3     assert(n >= 1);
4     long long answer = 1;
5     for (int p = 2; p <= n / p; p++)
6         if (n % p == 0)
7         {
8             int power = 1;
9             while (n % p == 0)
10             {
11                 n /= p;
12                 power *= p;
13             }
14             int value = power / p * (p - 1);
15             if (p == 2 && power >= 8)
16                 value /= 2;
17             answer = answer / gcd(answer, (long long)value) * value;
18         }
19     if (n > 1)
20         answer = answer / gcd(answer, (long long)(n - 1)) * (n - 1);
21     return answer;
22 }
```

第四十六章 整数分拆

46.1 整数分拆与重数上限

Partitions(N,mod) 求无序正整数分拆, $0 \leq N < \text{INT_MAX}$, $1 \leq \text{mod} \leq \text{INT_MAX}$, $p[n]$ 为不限制重数的分拆数; $p[0]=1 \bmod$ 。预处理 $O(N\sqrt{N})$ 时间、 $O(N)$ 空间; 模数不要求素数。limited(n,k) 限制每个正整数最多出现 k 次, $0 \leq n \leq N$, $0 \leq k \leq \text{INT_MAX}$; $k=0$ 时仅空分拆存在, $k=1$ 为互异分拆。单次 $O(1+\sqrt{(n/(k+1))})$, 额外空间 $O(1)$ 。传统接口 Integer_Partitions、Limited。p/mod 只读; 中间五边形数与累加采用 long long。

```
1 struct Partitions
2 {
3     int mod;
4     vector<int> p;
5
6     Partitions(int n, int mod) : mod(mod), p(n + 1)
7     {
8         assert(n >= 0 && mod >= 1);
9         p[0] = 1 % mod;
10        for (int i = 1; i <= n; i++)
11        {
12            long long value = 0;
13            for (long long j = 1; j * (3 * j - 1) / 2 <= i; j++)
14            {
15                long long a = j * (3 * j - 1) / 2;
16                long long b = j * (3 * j + 1) / 2;
17                int sign = j % 2 ? 1 : -1;
18                value += 1LL * sign * p[i - a];
19                if (b <= i)
20                    value += 1LL * sign * p[i - b];
21            }
22            p[i] = (value % mod + mod) % mod;
23        }
24    }
25
26    int limited(int n, int k) const
27    {
28        assert(0 <= n && n < (int)p.size() && k >= 0);
29        long long step = 1LL + k;
30        long long value = p[n];
31        for (long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++)
32        {
33            long long a = step * (j * (3 * j - 1) / 2);
34            long long b = step * (j * (3 * j + 1) / 2);
35            int sign = j % 2 ? -1 : 1;
36            value += 1LL * sign * p[n - a];
37            if (b <= n)
38                value += 1LL * sign * p[n - b];
39        }
40        return (value % mod + mod) % mod;
41    }
42};
```

第四十七章 素数模数组组合数

47.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p, choose(n,k) 返回 $C(n,k)$ 模 p; n、k 为完整 unsigned long long 范围, $k > n$ 返回 0, $C(0,0)=1$ 。init(p) 可更换模数并重新预处理; 不检测素性, 要求调用者保证 p 为素数且 $p \geq 2$ 。预处理时间 $O(p + \log p)$, 动态版空间 $O(p)$; 单次查询 $O(1 + \log_p(n+1))$ 。适合小素数模数, 通常 $p \leq 10^6$; 大素数不能因查询下标小就忽略这里整段 $O(p)$ 的预处理。fac、ifac 下标为 $0..p-1$, 模数和表只读; 中间乘法用 long long。合数模数请使用 exLucas。

```
1 struct Lucas
2 {
3     int mod;
4     vector<int> fac, ifac;
5
6     Lucas(int p)
7     {
8         init(p);
9     }
10
11     void init(int p)
12     {
13         assert(p >= 2);
14         mod = p;
15         fac.resize(p);
16         ifac.resize(p);
17         fac[0] = 1;
18         for (int i = 1; i < p; i++)
19             fac[i] = 1LL * fac[i - 1] * i % p;
20         long long a = fac[p - 1], r = 1;
21         for (int e = p - 2; e; e >>= 1)
22         {
23             if (e & 1)
24                 r = r * a % p;
25             a = a * a % p;
26         }
27         ifac[p - 1] = r;
28         for (int i = p - 1; i; i--)
29             ifac[i - 1] = 1LL * ifac[i] * i % p;
30     }
```

```
32     int choose(unsigned long long n, unsigned long long k) const
33     {
34         if (k > n)
35             return 0;
36         long long answer = 1;
37         while (n || k)
38         {
39             int a = n % mod, b = k % mod;
40             if (b > a)
41                 return 0;
42             answer = answer * fac[a] % mod;
43             answer = answer * ifac[b] % mod;
44             answer = answer * ifac[a - b] % mod;
45             n /= mod;
46             k /= mod;
47         }
48         return answer;
49     }
50 };
```

第四十八章 合数模数组合数

48.1 合数模数组合数 (扩展 Lucas)

ExLucas(mod) 预处理固定模数, choose(n,k) 返回 $C(n,k)$ 模 mod; n、k 为 unsigned long long, $k > n$ 返回 0, mod=1 返回 0。适合 $\text{mod} \leq 10^6$ 的合数或素数模数, 不能用于 10^9 级素数的整段预处理。分解 mod 为互素素数幂 $q = p^e$, 预处理每段内不含 p 因子的前缀积; unit(n,f) 去除 $n!$ 中全部 p 因子, valuation 统计指数。分子分母指数之差不少于 e 时该项为零, 否则只对互素单位部分求逆, 最后 CRT 合并。预处理 $O(\sqrt{\text{mod}} + \sum q)$, 空间 $O(\sum(q+1))$; 每次查询 $O(\sum(\log_p(n+1) \cdot \log(n+1) + \log q))$, 求和遍历不同素因子。所有乘法用 long long; 同一对象可重复查询, factors、fac 与辅助函数视为内部数据。

```
1 struct ExLucas
2 {
3     using ull = unsigned long long;
4     using ll = long long;
5
6     struct Factor
7     {
8         int p, q, exponent, start, crt;
9     };
10
11     int mod;
12     vector<int> fac;
13     vector<Factor> factors;
14
15     static ll power(ll a, ull b, int m)
16     {
17         ll result = 1;
18         while (b)
19         {
20             if (b & 1)
21                 result = result * a % m;
22             a = a * a % m;
23             b >>= 1;
24         }
25         return result;
26     }
27
28     ExLucas(int m) : mod(m)
29     {
30         assert(m >= 1);
31         int x = m;
32         for (int p = 2; p <= x / p; p++)
33         {
34             if (x % p)
35                 continue;
36             int q = 1, e = 0;
37             do
38             {
39                 x /= p;
40                 q *= p;
41                 e++;
```

```

42         } while (x % p == 0);
43         add_factor(p, q, e);
44     }
45     if (x > 1)
46         add_factor(x, x, 1);
47 }
48
49 void add_factor(int p, int q, int e)
50 {
51     int start = fac.size();
52     fac.push_back(1);
53     for (int i = 1; i < q; i++)
54         fac.push_back(1ll(fac.back()) * (i % p ? i : 1) % q);
55     fac.push_back(fac.back());
56     int phi = q - q / p;
57     int crt = 1ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
58     factors.push_back({p, q, e, start, crt});
59 }
60
61 static ull valuation(ull n, int p)
62 {
63     ull result = 0;
64     while (n)
65     {
66         n /= p;
67         result += n;
68     }
69     return result;
70 }
71
72 ll unit(ull n, const Factor &f) const
73 {
74     ll result = 1;
75     while (n)
76     {
77         result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
78         result = result * fac[f.start + n % f.q] % f.q;
79         n /= f.p;
80     }
81     return result;
82 }

```



```
84     int choose(ull n, ull k) const
85     {
86         if (k > n || mod == 1)
87             return 0;
88         ll answer = 0;
89         for (const auto &f : factors)
90         {
91             ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
92             if (e >= ull(f.exponent))
93                 continue;
94             ll denominator = unit(k, f) * unit(n - k, f) % f.q;
95             ll residue =
96                 unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
97             residue = residue * power(f.p, e, f.q) % f.q;
98             answer = (answer + residue * f.crt) % mod;
99         }
100         return answer;
101     }
102 };
```

第四十九章 二次剩余

49.1 素数模平方根 (Cipolla)

`mod_sqrt(n,p)` 返回 `vector<long long>`, 按升序列出全部不同根; 无解返回空向量, 零或模数 2 返回单根。要求 p 为素数, $2 \leq p \leq \text{INT_MAX}$, n 支持完整 `long long` 范围并在入口归一化。先用 Euler 判别; $p \equiv 3 \pmod{4}$ 时直接快速幂, 其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器; 随机性影响寻找非剩余的尝试次数, 不影响结果, 通常期望 $O(\log p)$ 时间、 $O(1)$ 辅助空间。乘法用 `long long`, 扩域实部的三因子乘积中间先取模; 指数中的 $p+1$ 先转 `long long`, 避免 `INT_MAX` 溢出。此版本不处理合数模数及 64 位大素数, 也不用于浮点开方。

```
1 inline vector<long long> mod_sqrt(long long n, int p)
2 {
3     using ll = long long;
4     assert(p >= 2);
5     n = (n % p + p) % p;
6     if (n == 0 || p == 2)
7         return {n};
8     auto power = [p](ll a, ll b)
9     {
10         ll result = 1;
11         while (b)
12         {
13             if (b & 1)
14                 result = result * a % p;
15             a = a * a % p;
16             b >>= 1;
17         }
18         return result;
19     };
20     if (power(n, (p - 1) / 2) != 1)
21         return {};
22     ll answer;
23     if (p % 4 == 3)
24         answer = power(n, (ll(p) + 1) / 4);
25     else
26     {
27         static mt19937_64 rng(712367);
28         ll a, w;
29         do
30         {
31             a = rng() % p;
32             w = (a * a % p - n + p) % p;
33         } while (power(w, (p - 1) / 2) != p - 1);
34         using P = pair<ll, ll>;
35         auto mul = [p, w](P x, P y)
36         {
37             ll real = (x.first * y.first + x.second * y.second % p * w) % p;
38             ll imag = (x.first * y.second + x.second * y.first) % p;
39             return P{real, imag};
40         };
41         P result{1, 0}, base{a, 1};
```

```
42     ll exponent = (ll(p) + 1) / 2;
43     while (exponent)
44     {
45         if (exponent & 1)
46             result = mul(result, base);
47         base = mul(base, base);
48         exponent >>= 1;
49     }
50     answer = result.first;
51 }
52 ll other = p - answer;
53 if (answer > other)
54     swap(answer, other);
55 return {answer, other};
56 }
```

第五十章 素数模高次剩余

50.1 素数模高次剩余：全部根的紧凑表示

`solve(a,k,p,g)` 求 $x^k \equiv a \pmod{p}$; p 必须是 2 到 $1e12$ 的素数, g 是模 p 原根, k 为正 unsigned long long, a 支持完整 signed long long。无解返回 `nullopt`; 否则得到 `first`、`ratio`、`count`, 全部解为 $\text{first} \cdot \text{ratio}^i \pmod{p}$, $0 \leq i < \text{count}$, 无重复; `first` 不保证最小。 $a \equiv 0$ 时仅有根 0, 返回 $\{0,1,1\}$ 。枚举时每次乘 `ratio`, 乘法先转 `int128`; 无需展开全部根即可读取解数。给定原根后, 期望 $O(\sqrt{p})$ 时间与空间, 依赖 `DiscreteLog`、`linear_congruence` 及模快速幂。不验证 p 的素性或 g 的阶; 合数模数不在此接口范围内。传统接口 `Kth_Residue::Solve`。

```
1 struct KthResidue
2 {
3     using ll = long long;
4     ll first, ratio, count;
5
6     // Prime p <= 1e12; g is a primitive root of p; k > 0.
7     // All roots: first * ratio^i mod p, 0 <= i < count.
8     static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
9     {
10         assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
11         a %= p;
12         if (a < 0)
13             a += p;
14         if (a == 0)
15             return KthResidue{0, 1, 1};
16         ll logarithm = DiscreteLog::solve(g, a, p);
17         if (logarithm < 0)
18             return nullopt;
19         auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
20         if (exponent < 0)
21             return nullopt;
22         ll first = NumberTheory::power(g, exponent, p);
23         ll ratio = NumberTheory::power(g, period, p);
24         return KthResidue{first, ratio, (p - 1) / period};
25     }
26 };
```

第五十一章 素数幂模高次剩余

51.1 素数幂模高次剩余：含非互素被开方数

`solve(a,k,p,e,g)` 求 $x^k \equiv a \pmod{p^e}$ 。p 是素数， $e \geq 1$ ， $p^e \leq 1e12$ ，k 为正 unsigned long long；a 为 signed long long。奇素数须提供模 p^e 的原根 g；p=2 时不使用 g。无解返回 `nullopt`，否则 `size()` 返回解数，`get(i,j,lift)` 返回指定根，范围是 `count[0]`、`count[1]`、`lifts`，全部组合无重复且无遗漏。字段 `first`、`ratio[2]`、`step` 描述乘法根族与加法提升； $step * lifts = p^e$ 。原根与素性由调用者保证。期望 $O(\sqrt{p^e})$ 时间和空间；单次 `get` 为 $O(\log(p^e))$ ，不强制展开全部根。依赖离散对数、线性同余及模幂。传统接口 `Prime_Power_Roots::Solve`、`Size`、`Get`。

```
1 struct PrimePowerRoots
2 {
3     using ll = long long;
4     ll first = 0, step = 1, lifts = 1;
5     array<ll, 2> ratio{1, 1}, count{1, 1};
6
7     ll size() const
8     {
9         return count[0] * count[1] * lifts;
10    }
11
12    ll get(ll i, ll j, ll lift) const
13    {
14        assert(i >= 0 && i < count[0] && j >= 0 && j < count[1]);
15        assert(lift >= 0 && lift < lifts);
16        ll x = (__int128)first * NumberTheory::power(ratio[0], i, step) % step;
17        x = (__int128)x * NumberTheory::power(ratio[1], j, step) % step;
18        return x + lift * step;
19    }
20
21    // p prime, 1 <= e, p^e <= 1e12, k > 0.
22    // For odd p, g must be a primitive root modulo p^e. For p=2, g is unused.
```

接上页同一结构体（第三页继续同一函数）:

```

23  static optional<PrimePowerRoots>
24  solve(ll a, unsigned long long k, ll p, int e, ll g = 0)
25  {
26      assert(p >= 2 && p <= 10000000000000LL && e >= 1 && e <= 39 && k > 0);
27      array<ll, 40> power{1};
28      for (int i = 1; i <= e; i++)
29      {
30          assert(power[i - 1] <= 10000000000000LL / p);
31          power[i] = power[i - 1] * p;
32      }
33      ll mod = power[e];
34      a %= mod;
35      if (a < 0)
36          a += mod;
37      PrimePowerRoots answer;
38      if (a == 0)
39      {
40          answer.step = power[(e - 1) / k + 1];
41          answer.lifts = mod / answer.step;
42          return answer;
43      }
44      int t = 0;
45      while (a % p == 0)
46      {
47          a /= p;
48          ++t;
49      }
50      if (t % k)
51          return nullopt;

```

接上页同一结构体（第三页继续同一函数）：

```

52     ll scale = power[t / k], q = power[e - t];
53     answer.first = 1;
54     answer.step = scale * q;
55     answer.lifts = mod / answer.step;
56     auto cyclic = [&](ll base, ll target, ll order, int slot)
57     {
58         ll logarithm = DiscreteLog::solve(base, target, q);
59         if (logarithm < 0)
60             return false;
61         auto [x, period] = linear_congruence(k % order, -logarithm, order);
62         if (x < 0)
63             return false;
64         answer.first = (__int128)answer.first * NumberTheory::power(base, x, q) % q;
65         answer.ratio[slot] = NumberTheory::power(base, period, q);
66         answer.count[slot] = order / period;
67         return true;
68     };
69     if (p != 2)
70     {
71         assert(g >= 1 && g < mod);
72         if (!cyclic(g % q, a, q / p * (p - 1), 0))
73             return nullopt;
74     }
75     else if (q >= 4)
76     {
77         int sign = a % 4 == 3;
78         auto [x, period] = linear_congruence(k % 2, -sign, 2);
79         if (x < 0)
80             return nullopt;
81         answer.first = x ? q - 1 : 1;
82         answer.ratio[0] = period == 1 ? q - 1 : 1;
83         answer.count[0] = 2 / period;
84         if (q >= 8 && !cyclic(5, sign ? q - a : a, q / 4, 1))
85             return nullopt;
86     }
87     answer.first *= scale;
88     return answer;
89 }
90 };

```

第五十二章 合数求根的准备

52.1 合数求根准备：分解模数与素数幂原根

`root_factors(M,rho)` 为 $1 \leq M \leq 1e12$ 生成完整的 (p,e,g) 列表，可直接传入 `CompositeRoots`。`rho` 是可复用的 `PollardRho` 对象。按素数递增返回； $p=2$ 时 $g=0$ ，其余 g 为模 p^e 的最小正原根。先分解 M ，再分解各个欧拉函数值，逐个检查原根候选的阶。 $M=1$ 返回空列表。分解使用随机重试；原根搜索耗时取决于候选数，不包含 BSGS。传统接口 `Root_Factors(M,rho)`，`rho` 为 `Pollard_Rho`。

```
1 inline vector<tuple<long long, int, long long>> root_factors(long long mod,
2                                     PollardRho &rho)
3 {
4     assert(mod >= 1 && mod <= 1000000000000LL);
5     auto primes = rho.factor(mod);
6     vector<tuple<long long, int, long long>> answer;
7     for (int i = 0; i < (int)primes.size();)
8     {
9         long long p = primes[i], power = 1;
10        int e = 0;
11        while (i < (int)primes.size() && primes[i] == (unsigned long long)p)
12        {
13            power *= p;
14            ++e;
15            ++i;
16        }
17        long long g = 0;
18        if (p != 2)
19        {
20            long long phi = power / p * (p - 1);
21            auto factors = rho.factor(phi);
22            factors.erase(unique(factors.begin(), factors.end()), factors.end());
23            auto primitive = [&](long long candidate)
24            {
25                if (candidate % p == 0)
26                    return false;
27                for (auto q : factors)
28                    if (NumberTheory::power(candidate, phi / q, power) == 1)
29                        return false;
30                return true;
31            };
32            g = 2;
33            while (!primitive(g))
34                ++g;
35        }
36        answer.push_back({p, e, g});
37    }
38    return answer;
39 }
```


第五十三章 合数模高次剩余

53.1 一般合数模高次剩余：CRT 组合与按索引取根

`solve(a,k,factors)` 求 $x^k \equiv a \pmod{M}$, `factors` 是完整且素数互异的 (p,e,g) 列表, M 为所有 p^e 的乘积, $M \leq 1e12$, k 为正 `unsigned long long`, a 为 `signed long long`。奇素数的 g 是模 p^e 原根; $p=2$ 时填 0。无解返回 `nullopt`, 否则 `mod=M`, `total` 为解数, `get(index)` 在 $0 \leq \text{index} < \text{total}$ 上取出每个根恰好一次, 不按数值排序。空因子列表表示模 1, 唯一标准代表为 0。依赖 `PrimePowerRoots` 和模逆元; 列表接口不做分解; 重载 `solve(a,k,M,rho)` 自动分解并寻找原根, `rho` 为 `PollardRho`。重复同模数宜复用因子列表。预处理的主要成本是各素数幂的 `BSGS`; `get` 为 $O(F \log M)$, F 为不同素因子数。无需展开全部根, `parts` 和 `weight` 保存局部根族及 CRT 系数。传统接口 `Composite_Roots::Solve`、`Get`。

```
1 struct CompositeRoots
2 {
3     using ll = long long;
4     ll mod = 1, total = 1;
5     vector<PrimePowerRoots> parts;
6     vector<ll> weight;
7
8     ll get(ll index) const
9     {
10         assert(index >= 0 && index < total);
11         ll answer = 0;
12         for (int t = 0; t < (int)parts.size(); t++)
13         {
14             const auto &p = parts[t];
15             ll local = index % p.size();
16             index /= p.size();
17             ll lift = local % p.lifts;
18             local /= p.lifts;
19             ll j = local % p.count[1], i = local / p.count[1];
20             ll root = p.get(i, j, lift);
21             answer = (answer + (__int128)root * weight[t]) % mod;
22         }
23         return answer;
24     }
25
26     // Factors are distinct primes with positive exponents: (p, e, g).
27     // Odd-prime g is a primitive root modulo p^e; for p=2 use g=0.
```

局部求根与 CRT 系数（接上页同一结构体）：

```

28     static optional<CompositeRoots>
29     solve(ll a, unsigned long long k, const vector<tuple<ll, int, ll>> &factors)
30     {
31         assert(k > 0);
32         CompositeRoots answer;
33         for (auto [p, e, g] : factors)
34         {
35             auto roots = PrimePowerRoots::solve(a, k, p, e, g);
36             if (!roots)
37                 return nullopt;
38             ll m = roots->step * roots->lifts;
39             assert(gcd(answer.mod, m) == 1);
40             assert(answer.mod <= 1000000000000LL / m);
41             answer.mod *= m;
42             answer.total += roots->size();
43             answer.parts.push_back(*roots);
44         }
45         for (const auto &p : answer.parts)
46         {
47             ll m = p.step * p.lifts, rest = answer.mod / m;
48             ll inverse = NumberTheory::inverse(rest % m, m);
49             answer.weight.push_back((__int128)rest * inverse % answer.mod);
50         }
51         return answer;
52     }
53
54     static optional<CompositeRoots>
55     solve(ll a, unsigned long long k, ll mod, PollardRho &rho)
56     {
57         return solve(a, k, root_factors(mod, rho));
58     }
59 };

```

第五十四章 多项式插值

54.1 拉格朗日插值与连续点优化

先声明 `Lagrange.init(xs,ys,p)` 使用任意横坐标, `xs`、`ys` 为等长 0-based `vector<long long>`; `consecutive(ys,p)` 专用于横坐标 $0..n-1$ 。两者均覆盖旧数据。要求 p 为素数, $2 \leq p \leq \text{INT_MAX}$, 点数 $n \leq p$, 任意横坐标取模后两两不同; 不是仅要求原整数不同。`query(k)` 返回次数小于 n 的插值多项式在 k 处的值, 允许 k 、`xs`、`ys` 为完整 `long long` 范围, 内部先归一化; 空数据约定零多项式。通用预处理 $O(n^2+n \log p)$, 连续点预处理 $O(n+\log p)$, 每次查询 $O(n)$, 空间 $O(n)$ 。查询利用前缀积与滚动后缀积, 无需求逆, 因此 k 与样本点模 p 相等时也正确。若求 $1^d + \dots + k^d$, 要求 $0 \leq d \leq p-2$, 先计算 $0..d+1$ 的前缀和作为 `ys`, 再用 `consecutive`; 用快速幂生成这些样本另花 $O(d \log(d+1))$ 。仅处理素数域中的精确插值, 不适用于浮点曲线拟合或重复节点的导数约束。

```
1 struct Lagrange
2 {
3     using ll = long long;
4     int n, mod;
5     vector<ll> x, w;
6
7     ll norm(ll a) const
8     {
9         a %= mod;
10        if (a < 0)
11            a += mod;
12        return a;
13    }
14
15    ll power(ll a, int b) const
16    {
17        ll result = 1;
18        while (b)
19        {
20            if (b & 1)
21                result = result * a % mod;
22            a = a * a % mod;
23            b >>= 1;
24        }
25        return result;
26    }
27
28    void init(const vector<ll> &xs, const vector<ll> &ys, int p)
29    {
30        n = ys.size();
31        mod = p;
32        assert(p >= 2 && n <= p && xs.size() == ys.size());
33        x.resize(n);
34        w.resize(n);
35        for (int i = 0; i < n; i++)
36            x[i] = norm(xs[i]);
37        for (int i = 0; i < n; i++)
38        {
39            ll denominator = 1;
```

```

40     for (int j = 0; j < n; j++)
41         if (i != j)
42             denominator = denominator * norm(x[i] - x[j]) % mod;
43     assert(denominator != 0);
44     w[i] = norm(ys[i]) * power(denominator, mod - 2) % mod;
45 }
46 }
47
48 void consecutive(const vector<ll> &ys, int p)
49 {
50     n = ys.size();
51     mod = p;
52     assert(p >= 2 && n <= p);
53     x.resize(n);
54     w.resize(n);
55     if (!n)
56         return;
57     vector<ll> invfac(n, 1);
58     ll factorial = 1;
59     for (int i = 1; i < n; i++)
60         factorial = factorial * i % mod;
61     invfac[n - 1] = power(factorial, mod - 2);
62     for (int i = n - 1; i > 0; i--)
63         invfac[i - 1] = invfac[i] * i % mod;
64     for (int i = 0; i < n; i++)
65     {
66         x[i] = i;
67         w[i] = norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
68         if ((n - 1 - i) & 1)
69             w[i] = norm(-w[i]);
70     }
71 }
72
73 ll query(ll k) const
74 {
75     k = norm(k);
76     vector<ll> pre(n + 1, 1);
77     for (int i = 0; i < n; i++)
78         pre[i + 1] = pre[i] * norm(k - x[i]) % mod;
79     ll answer = 0, suffix = 1;
80     for (int i = n - 1; i >= 0; i--)
81     {
82         answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
83         suffix = suffix * norm(k - x[i]) % mod;
84     }
85     return answer;
86 }
87 };

```

第五十五章 参数化 NTT 卷积

55.1 可指定素数与原根的 NTT 卷积

声明 `NttConvolution<mod, primitive>`, `primitive` 默认 3。mod 必须为 `int` 范围内的素数, `primitive` 是模 `mod` 原根; 不在接口内验证素性或阶。`max_size` 是 `mod-1` 的最大二次幂因子。`ntt(a,invert)` 原地变换, 长度须为正二次幂且不超过 `max_size`; `invert=true` 为逆变换。`multiply(a,b)` 返回普通卷积, 任一为空则为空; 非空时长度和减一不超过 `max_size`。元素类型 `Z`, 数组类型 `Poly`。 $O(n \log n)$ 时间、 $O(n)$ 空间。998244353 和 167772161 均可用原根 3。传统接口 `Ntt_Convolution::Ntt`、`Multiply`。

```
1 template <int mod, int primitive = 3> struct NttConvolution
2 {
3     static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
4     static constexpr int max_size = (mod - 1) & -(mod - 1);
5     using Z = ModInt<mod>;
6     using Poly = vector<Z>;
7
8     // mod is prime, primitive is a primitive root modulo mod.
9     static void ntt(Poly &a, bool invert = false)
10    {
11        int n = a.size();
12        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
13        for (int i = 1, j = 0; i < n; i++)
14        {
15            int bit = n >> 1;
16            for (; j & bit; bit >>= 1)
17                j ^= bit;
18            j ^= bit;
19            if (i < j)
20                swap(a[i], a[j]);
21        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```

22     for (int half = 1; half < n; half *= 2)
23     {
24         int len = half * 2;
25         Z step = Z(primitive).pow((mod - 1) / len);
26         if (invert)
27             step = step.inv();
28         for (int i = 0; i < n; i += len)
29         {
30             Z w = 1;
31             for (int j = 0; j < half; j++)
32             {
33                 Z u = a[i + j], v = a[i + j + half] * w;
34                 a[i + j] = u + v;
35                 a[i + j + half] = u - v;
36                 w = w * step;
37             }
38         }
39     }
40     if (invert)
41     {
42         Z inverse = Z(n).inv();
43         for (auto &x : a)
44             x = x * inverse;
45     }
46 }
47
48 static Poly multiply(Poly a, Poly b)
49 {
50     if (a.empty() || b.empty())
51         return {};
52     assert(a.size() + b.size() - 1 <= max_size);
53     int size = a.size() + b.size() - 1, n = 1;
54     while (n < size)
55         n *= 2;
56     a.resize(n);
57     b.resize(n);
58     ntt(a);
59     ntt(b);
60     for (int i = 0; i < n; i++)
61         a[i] = a[i] * b[i];
62     ntt(a, true);
63     a.resize(size);
64     return a;
65 }
66 };

```

第五十六章 多项式平移

56.1 多项式系数平移

`polynomial_shift<mod=998244353,primitive=3>(f,c)` 返回 $f(x+c)$ 的升幂系数，结果长度始终与 f 相同，保留末尾零；空输入返回空。参数 c 使用 `ModInt<mod>`。依赖参数化 NTT， mod 为素数且 primitive 为原根；非空长度 n 要求 $2n-1 \leq \text{NTT::max_size}$ ，因此阶乘均可逆。逆序的 $f[i]i!$ 与 $c^i/i!$ 卷积后提取对应项， $O(n \log n)$ 时间、 $O(n)$ 空间。 $c=0$ 合法。传统函数 `Polynomial_Shift`， c 类型为 `Mod_Int<mod>`。

```
1 template <int mod = 998244353, int primitive = 3>
2 vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
3 {
4     using N = NttConvolution<mod, primitive>;
5     using Z = typename N::Z;
6     using Poly = typename N::Poly;
7     if (f.empty())
8         return {};
9     assert(f.size() <= (N::max_size + 1ULL) / 2);
10    int n = f.size();
11    Poly fact(n), inv(n), a(n), b(n);
12    fact[0] = 1;
13    for (int i = 1; i < n; i++)
14        fact[i] = fact[i - 1] * i;
15    inv[n - 1] = fact[n - 1].inv();
16    for (int i = n - 1; i > 0; i--)
17        inv[i - 1] = inv[i] * i;
18    Z power = 1;
19    for (int i = 0; i < n; i++)
20    {
21        a[n - 1 - i] = f[i] * fact[i];
22        b[i] = power * inv[i];
23        power = power * c;
24    }
25    auto product = N::multiply(a, b);
26    for (int i = 0; i < n; i++)
27        a[i] = product[n - 1 - i] * inv[i];
28    return a;
29 }
```

第五十七章 等比点求值

57.1 Chirp Z: 等比点求值

`chirp_z(f,a,r,m)` 返回 $f(a \cdot r^i)$, $i=0..m-1$; f 为升幂系数数组, a, r 为 `ModInt<mod>`。 $m \geq 0$, $f.size() \leq INT_MAX$; 空 f 表示零多项式, $m=0$ 返回空。约定 $r^0=1$, 因此 $r=0$ 时首项为 $f(a)$ 、其余为 $f(0)$; $a=0$ 或 $r=1$ 也单独处理。普通分支依赖 NTT 素数及原根, 非空长度 n 要求 $n+m-1 \leq NTT::max_size$ 。利用三角数指数与循环卷积, 避免平方指数的整数溢出; $O((n+m) \log(n+m))$ 时间、 $O(n+m)$ 空间, 特殊分支 $O(n+m)$ 。默认模板参数 `mod=998244353`, `primitive=3`。输出顺序允许重复点。传统函数 `Chirp_Z`, 参数类型 `Mod_Int<mod>`。

```
1 template <int mod = 998244353, int primitive = 3>
2 vector<ModInt<mod>>
3 chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
4 {
5     using N = NttConvolution<mod, primitive>;
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     assert(m >= 0 && f.size() <= INT_MAX);
9     if (m == 0 || f.empty())
10         return Poly(m);
11     if (a.v == 0 || r.v == 0 || r.v == 1)
12     {
13         Poly answer(m, f[0]);
14         Z value = 0;
15         for (int i = int(f.size()) - 1; i >= 0; i--)
16             value = value * a + f[i];
17         answer[0] = value;
18         if (r.v == 1)
19             fill(answer.begin(), answer.end(), value);
20         return answer;
21     }
22     assert(f.size() + m - 1 <= N::max_size);
23     int n = f.size(), size = 1;
24     while (size < n + m - 1)
25         size *= 2;
26     Poly x(size), y(size), inverse(max(n, m));
27     Z power = 1, triangle = 1, ri = r.inv();
28     for (auto &v : inverse)
29     {
30         v = triangle;
31         triangle = triangle * power;
32         power = power * ri;
33     }
```


构造循环卷积并取点值（接上页同一函数）：

```
34     power = 1;
35     for (int j = 0; j < n; j++)
36     {
37         x[n - 1 - j] = f[j] * power * inverse[j];
38         power = power * a;
39     }
40     power = 1;
41     triangle = 1;
42     for (int j = 0; j < n + m - 1; j++)
43     {
44         y[j] = triangle;
45         triangle = triangle * power;
46         power = power * r;
47     }
48     N::ntt(x);
49     N::ntt(y);
50     for (int j = 0; j < size; j++)
51         x[j] = x[j] * y[j];
52     N::ntt(x, true);
53     Poly answer(m);
54     for (int i = 0; i < m; i++)
55         answer[i] = x[n - 1 + i] * inverse[i];
56     return answer;
57 }
```

第五十八章 斯特林数整行计算

58.1 第二类斯特林数：同一行前缀

`stirling_second_row<mod,primitive>(n,m)` 返回 $S(n,0..m)$ ，默认 `mod=998244353`、`primitive=3`。模数必须是素数，`primitive` 是原根。`n` 支持完整 `unsigned long long`；`2m+1` 不能超过 NTT 的 `max_size`。 $S(n,k)$ 为无标号非空集合划分数；`n=0` 时仅第零项为 1，`k>n` 为零。一次卷积配合逆阶乘， $O(m \log m + m \log \text{mod})$ 时间、 $O(m)$ 空间。依赖 `NttConvolution`；模 167772161 可直接使用模板参数 `<167772161>`。传统接口 `Stirling_Second_Row`。

```
1 template <int mod = 998244353, int primitive = 3>
2 inline vector<ModInt<mod>> stirling_second_row(unsigned long long n, int m)
3 {
4     using N = NttConvolution<mod, primitive>;
5     assert(m >= 0 && m <= (N::max_size - 1) / 2);
6     using Z = typename N::Z;
7     typename N::Poly a(m + 1), b(m + 1);
8     b[0] = 1;
9     for (int i = 1; i <= m; i++)
10         b[i] = b[i - 1] * i;
11     b[m] = b[m].inv();
12     for (int i = m; i > 0; i--)
13         b[i - 1] = b[i] * i;
14     for (int i = 0; i <= m; i++)
15     {
16         Z value = i ? Z(i).pow(n % (mod - 1)) : Z(n == 0);
17         a[i] = value * b[i];
18         if (i % 2)
19             b[i] = Z(0) - b[i];
20     }
21     auto answer = N::multiply(a, b);
22     answer.resize(m + 1);
23     return answer;
24 }
```

58.2 无符号第一类斯特林数：同一行

`stirling_first_row<mod,primitive>(n)` 返回无符号 $c(n,0..n)$ ，默认模 998244353、原根 3。 $c(n,k)$ 为 n 元排列恰有 k 个循环的个数。 $n \geq 0$ ， $n+1 \leq \text{NTT 的 max_size}$ ； $n=0$ 返回 $\{1\}$ 。倍增配合多项式平移计算 $x(x+1)\cdots(x+n-1)$ ， $O(n \log n)$ 时间、 $O(n)$ 空间。带符号版本再乘 $(-1)^{(n-k)}$ 。依赖 `NttConvolution`；模 167772161 可用 `<167772161>`。传统接口 `Stirling_First_Row`。

```

1 template <int mod = 998244353, int primitive = 3>
2 inline vector<ModInt<mod>> stirling_first_row(int n)
3 {
4     using N = NttConvolution<mod, primitive>;
5     assert(n >= 0 && n < N::max_size);
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     Poly fact(n + 1), inv(n + 1);
9     fact[0] = 1;
10    for (int i = 1; i <= n; i++)
11        fact[i] = fact[i - 1] * i;
12    inv[n] = fact[n].inv();
13    for (int i = n; i > 0; i--)
14        inv[i - 1] = inv[i] * i;
15    function<Poly(int)> solve = [&](int len) -> Poly
16    {
17        if (len == 0)
18            return {1};
19        int m = len / 2;
20        Poly f = solve(m);
21        Poly a(m + 1), b(m + 1);
22        Z power = 1;
23        for (int i = 0; i <= m; i++)
24        {
25            a[m - i] = f[i] * fact[i];
26            b[i] = power * inv[i];
27            power = power * m;
28        }
29        auto c = N::multiply(a, b);
30        for (int i = 0; i <= m; i++)
31            a[i] = c[m - i] * inv[i];
32        f = N::multiply(f, a);
33        if (len % 2)
34        {
35            f.push_back(0);
36            for (int i = len; i > 0; i--)
37                f[i] = f[i - 1] + f[i] * (len - 1);
38            f[0] = f[0] * (len - 1);
39        }
40        return f;
41    };
42    return solve(n);
43 }
```

第五十九章 集合变换与卷积

59.1 按位 OR、AND、XOR 卷积

SetConvolution<mod=998244353>; Z 为 ModInt<mod>, Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且 $\leq \text{INT_MAX}$, 表示固定二进制位数的全集, 长度 1 合法。transform(a,op,invert=false) 原地变换, op 只能为 |、&、^; | 为子集和, & 为超集和, ^ 为 Walsh 变换。multiply(a,b,op) 返回 $\sum a[i]b[j]$ (满足 $i \text{ op } j = s$), a、b 长度必须相同。mod 可为 ≥ 2 的 int 范围合数; XOR 要求 mod 为奇数, 逆 2 用 $(\text{mod}+1)/2$, 不使用费马逆元。 $O(N \log N)$ 时间, 原地变换 $O(1)$ 额外空间, 卷积 $O(N)$ 空间。传统接口 Set_Convolution::Transform / Multiply。

```
1 template <int mod = 998244353> struct SetConvolution
2 {
3     static_assert(mod >= 2);
4     using Z = ModInt<mod>;
5     using Poly = vector<Z>;
6
7     static void transform(Poly &a, char op, bool invert = false)
8     {
9         assert(!a.empty() && a.size() <= INT_MAX);
10        int n = a.size();
11        assert((n & (n - 1)) == 0);
12        assert(op == '|' || op == '&' || op == '^');
13        assert(op != '^' || mod % 2 == 1);
14        Z half = (mod + 1LL) / 2;
15        for (int len = 1; len < n; len *= 2)
16            for (int i = 0; i < n; i += len * 2)
17                for (int j = 0; j < len; j++)
18                {
19                    Z &u = a[i + j], &v = a[i + j + len];
20                    if (op == '|')
21                        v = invert ? v - u : v + u;
22                    else if (op == '&')
23                        u = invert ? u - v : u + v;
24                    else
25                    {
26                        Z x = u, y = v;
27                        u = x + y;
28                        v = x - y;
29                        if (invert)
30                        {
31                            u = u * half;
32                            v = v * half;
33                        }
34                    }
35                }
36    }
```

点乘并逆变换（接上页同一结构体）：

```
38     static Poly multiply(Poly a, Poly b, char op)
39     {
40         assert(a.size() == b.size());
41         transform(a, op);
42         transform(b, op);
43         for (int i = 0; i < int(a.size()); i++)
44             a[i] = a[i] * b[i];
45         transform(a, op, true);
46         return a;
47     }
48 };
```

第六十章 不相交子集卷积

60.1 不相交子集卷积

`subset_convolution<mod=998244353>(a,b)` 返回 $c[s] = \sum a[t]b[s \text{ xor } t]$, 求和枚举 $t \subseteq s$, 等价于 $i|j=s$ 且 $i \& j=0$ 。依赖 `SetConvolution`, 只使用 OR 变换; `mod` 为 ≥ 2 的 `int` 范围整数, 允许偶数与合数。两个数组长度相同、非空, $N=2^m \leq \text{INT_MAX}$; 长度 1 的结果为 $a[0]b[0]$ 。按 `popcount` 分层, 层间普通卷积再逆 OR 变换, 保留总层数等于目标集合大小的项。 $O(m^2 2^m)$ 时间、 $O(m 2^m)$ 空间; 两组分层数组约占 $8(m+1)2^m$ 字节, 另有线性辅助数组。 $m=20$ 时仅分层数组约 168 MiB, 使用前核对内存。传统函数 `Subset_Convolution`。

```
1 // c[s] = sum_{t subset s} a[t] * b[s xor t].
2 template <int mod = 998244353>
3 vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
4                                         const vector<ModInt<mod>> &b)
5 {
6     using F = SetConvolution<mod>;
7     using Poly = typename F::Poly;
8     assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
9     int n = a.size(), m = 0;
10    assert((n & (n - 1)) == 0);
11    while ((1 << m) < n)
12        m++;
13    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
14    vector<int> rank(n);
15    for (int s = 0; s < n; s++)
16    {
17        rank[s] = __builtin_popcount(unsigned(s));
18        f[rank[s]][s] = a[s];
19        g[rank[s]][s] = b[s];
20    }
21    for (int k = 0; k <= m; k++)
22    {
23        F::transform(f[k], '|');
24        F::transform(g[k], '|');
25    }
26    Poly answer(n), h(n);
27    for (int k = 0; k <= m; k++)
28    {
29        fill(h.begin(), h.end(), 0);
30        for (int i = 0; i <= k; i++)
31            for (int s = 0; s < n; s++)
32                h[s] = h[s] + f[i][s] * g[k - i][s];
33        F::transform(h, '|', true);
34        for (int s = 0; s < n; s++)
35            if (rank[s] == k)
36                answer[s] = h[s];
37    }
38    return answer;
39 }
```

第六十一章 多项式

61.1 NTT、形式幂级数、FWT 与递推

依赖 number_theory.hpp。模数固定 998244353，乘法结果长度 $\leq 2^{23}$ ；inverse 要求 $a[0] \neq 0$ ，log 要求 $a[0]=1$ ，exp 要求 $a[0]=0$ ，截断长度 $n>0$ 。朴素逐层 NTT 实现的 FPS 为 $O(n \log n)$ 。FWT 的 op 为 $\wedge$ 、 $|$ 、 $\&$ ；独立且支持其他模数的接口见 SetConvolution。BM 返回 $s[n]=\sum c[i-1]s[n-i]$ 的系数；recurrence 用 $O(k^2 \log n)$ 求第 n 项。BM 对有限样本给出的递推不自动保证未知后续项成立。

```
1 struct Polynomial
2 {
3     static constexpr int mod = 998244353;
4     using Z = ModInt<mod>;
5     using Poly = vector<Z>;
6
7     static void ntt(Poly &a, bool invert = false)
8     {
9         int n = (int)a.size();
10        assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
11        for (int i = 1, j = 0; i < n; i++)
12        {
13            int bit = n >> 1;
14            for (; j & bit; bit >>= 1)
15                j ^= bit;
16            j ^= bit;
17            if (i < j)
18                swap(a[i], a[j]);
19        }
20        for (int len = 2; len <= n; len *= 2)
21        {
22            Z step = Z(3).pow((mod - 1) / len);
23            if (invert)
24                step = step.inv();
25            for (int i = 0; i < n; i += len)
26            {
27                Z w = 1;
28                for (int j = 0; j < len / 2; j++, w = w * step)
29                {
30                    Z u = a[i + j], v = a[i + j + len / 2] * w;
31                    a[i + j] = u + v;
32                    a[i + j + len / 2] = u - v;
33                }
34            }
35        }
36        if (invert)
37        {
38            Z inv = Z(n).inv();
39            for (auto &x : a)
40                x = x * inv;
41        }
42    }
43 }
```

```

44 static Poly multiply(Poly a, Poly b)
45 {
46     if (a.empty() || b.empty())
47         return {};
48     int size = (int)a.size() + (int)b.size() - 1, n = 1;
49     assert(size <= (1 << 23));
50     while (n < size)
51         n *= 2;
52     a.resize(n);
53     b.resize(n);
54     ntt(a);
55     ntt(b);
56     for (int i = 0; i < n; i++)
57         a[i] = a[i] * b[i];
58     ntt(a, true);
59     a.resize(size);
60     return a;
61 }
62
63 static Poly inverse(const Poly &a, int n)
64 {
65     assert(n > 0 && !a.empty() && a[0].v);
66     Poly b{a[0].inv()};
67     while ((int)b.size() < n)
68     {
69         int m = min(n, 2 * (int)b.size());
70         Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
71         auto t = multiply(f, b);
72         t.resize(m);
73         for (auto &x : t)
74             x = Z(0) - x;
75         t[0] = t[0] + 2;
76         b = multiply(b, t);
77         b.resize(m);
78     }
79     return b;
80 }
81
82 static Poly derivative(const Poly &a)
83 {
84     Poly b;
85     for (int i = 1; i < (int)a.size(); i++)
86         b.push_back(a[i] * i);
87     return b;
88 }
89
90 static Poly integral(const Poly &a)
91 {
92     Poly b(a.size() + 1);
93     vector<Z> inv(a.size() + 1);
94     if (!a.empty())
95         inv[1] = 1;
96     for (int i = 2; i <= (int)a.size(); i++)
97         inv[i] = Z(mod - mod / i) * inv[mod % i];
98     for (int i = 0; i < (int)a.size(); i++)
99         b[i + 1] = a[i] * inv[i + 1];

```



```

100     return b;
101 }
102
103 static Poly log(const Poly &a, int n)
104 {
105     assert(n > 0 && !a.empty() && a[0].v == 1);
106     auto b = multiply(derivative(a), inverse(a, n));
107     b.resize(n - 1);
108     return integral(b);
109 }
110
111 static Poly exp(const Poly &a, int n)
112 {
113     assert(n > 0 && (a.empty() || a[0].v == 0));
114     Poly b{1};
115     while ((int)b.size() < n)
116     {
117         int m = min(n, 2 * (int)b.size());
118         auto t = log(b, m);
119         for (int i = 0; i < m; i++)
120             t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
121         t[0] = t[0] + 1;
122         b = multiply(b, t);
123         b.resize(m);
124     }
125     return b;
126 }
127
128 static void fwt(Poly &a, char op, bool invert = false)
129 {
130     int n = (int)a.size();
131     assert(n > 0 && (n & (n - 1)) == 0);
132     const Z inv2 = (mod + 1) / 2;
133     for (int len = 1; len < n; len *= 2)
134         for (int i = 0; i < n; i += len * 2)
135             for (int j = 0; j < len; j++)
136             {
137                 Z &u = a[i + j], &v = a[i + j + len], x = u, y = v;
138                 if (op == '^')
139                 {
140                     u = x + y;
141                     v = x - y;
142                     if (invert)
143                     {
144                         u = u * inv2;
145                         v = v * inv2;
146                     }
147                 }
148                 else if (op == '|')
149                     v = invert ? y - x : y + x;
150                 else
151                 {
152                     assert(op == '&');
153                     u = invert ? x - y : x + y;
154                 }
155             }

```

```

156     }
157
158     // c: s[n]=sum c[i-1]*s[n-i], i=1..L.
159     static Poly berlekamp_massey(const Poly &s)
160     {
161         Poly c{1}, b{1};
162         int l = 0, m = 1;
163         Z last = 1;
164         for (int n = 0; n < (int)s.size(); n++)
165         {
166             Z d = s[n];
167             for (int i = 1; i <= l; i++)
168                 d = d + c[i] * s[n - i];
169             if (!d.v)
170             {
171                 ++m;
172                 continue;
173             }
174             auto old = c;
175             Z coef = d / last;
176             if (c.size() < b.size() + m)
177                 c.resize(b.size() + m);
178             for (int j = 0; j < (int)b.size(); j++)
179                 c[j + m] = c[j + m] - coef * b[j];
180             if (2 * l <= n)
181             {
182                 l = n + 1 - l;
183                 b = old;
184                 last = d;
185                 m = 1;
186             }
187             else
188                 ++m;
189         }
190         c.erase(c.begin());
191         for (auto &x : c)
192             x = Z(0) - x;
193         return c;
194     }
195
196     static Z recurrence(Poly init, const Poly &c, unsigned long long n)
197     {
198         int k = (int)c.size();
199         if (n < init.size())
200             return init[n];
201         if (!k)
202             return 0;
203         assert((int)init.size() >= k);
204         auto combine = [&](const Poly &a, const Poly &b)
205         {
206             Poly t(2 * k - 1);
207             for (int i = 0; i < k; i++)
208                 for (int j = 0; j < k; j++)
209                     t[i + j] = t[i + j] + a[i] * b[j];
210             for (int i = 2 * k - 2; i >= k; i--)
211                 for (int j = 1; j <= k; j++)

```

```
212         t[i - j] = t[i - j] + t[i] * c[j - 1];
213     t.resize(k);
214     return t;
215 };
216 Poly a(k), r(k);
217 r[0] = 1;
218 if (k == 1)
219     a[0] = c[0];
220 else
221     a[1] = 1;
222 for (; n; n >= 1, a = combine(a, a))
223     if (n & 1)
224         r = combine(r, a);
225 Z ans = 0;
226 for (int i = 0; i < k; i++)
227     ans = ans + r[i] * init[i];
228 return ans;
229 }
230 };
```

第六十二章 代数与数论进阶

62.1 模高斯消元、行列式、矩阵树定理

素数模数。solve(a,n) 输入增广矩阵，返回 consistent、rank、特解 particular 和零空间基 kernel。determinant(empty)=1。spanning_trees(n,edges) 使用 0-based 无向图，允许重边，忽略自环；返回生成树数模 p。高斯与行列式 $O(n^3)$ ，矩阵幂 $O(n^3 \log e)$ 。

```
1 template <int mod> struct LinearAlgebra
2 {
3     using Z = ModInt<mod>;
4     using Matrix = vector<vector<Z>>>;
5
6     struct Solution
7     {
8         bool consistent;
9         int rank;
10        vector<Z> particular;
11        Matrix kernel;
12    };
13
14    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
15    static Solution solve(Matrix a, int n)
16    {
17        int m = (int)a.size(), row = 0;
18        vector<int> where(n, -1);
19        for (const auto &v : a)
20            assert((int)v.size() == n + 1);
21        for (int col = 0; col < n && row < m; col++)
22        {
23            int p = row;
24            while (p < m && !a[p][col].v)
25                ++p;
26            if (p == m)
27                continue;
28            swap(a[p], a[row]);
29            Z inv = a[row][col].inv();
30            for (int j = col; j <= n; j++)
31                a[row][j] = a[row][j] * inv;
32            for (int i = 0; i < m; i++)
33                if (i != row && a[i][col].v)
34                {
35                    Z f = a[i][col];
36                    for (int j = col; j <= n; j++)
37                        a[i][j] = a[i][j] - f * a[row][j];
38                }
39            where[col] = row++;
40        }
41        for (int i = row; i < m; i++)
42            if (a[i][n].v)
43                return {false, row, {}, {}};
44        Solution ans{true, row, vector<Z>(n), {}};
```

```

45     for (int j = 0; j < n; j++)
46         if (where[j] != -1)
47             ans.particular[j] = a[where[j]][n];
48     for (int j = 0; j < n; j++)
49         if (where[j] == -1)
50         {
51             vector<Z> v(n);
52             v[j] = 1;
53             for (int k = 0; k < n; k++)
54                 if (where[k] != -1)
55                     v[k] = Z(0) - a[where[k]][j];
56             ans.kernel.push_back(v);
57         }
58     return ans;
59 }
60
61 static Z determinant(Matrix a)
62 {
63     int n = (int)a.size();
64     Z ans = 1;
65     for (int i = 0; i < n; i++)
66     {
67         assert((int)a[i].size() == n);
68         int p = i;
69         while (p < n && !a[p][i].v)
70             ++p;
71         if (p == n)
72             return 0;
73         if (p != i)
74         {
75             swap(a[p], a[i]);
76             ans = Z(0) - ans;
77         }
78         ans = ans * a[i][i];
79         Z inv = a[i][i].inv();
80         for (int j = i + 1; j < n; j++)
81         {
82             Z f = a[j][i] * inv;
83             for (int k = i; k < n; k++)
84                 a[j][k] = a[j][k] - f * a[i][k];
85         }
86     }
87     return ans;
88 }
89
90 static Matrix multiply(const Matrix &a, const Matrix &b)
91 {
92     assert(!a.empty() && !b.empty());
93     int n = a.size(), m = b[0].size(), k = b.size();
94     assert((int)a[0].size() == k);
95     Matrix c(n, vector<Z>(m));
96     for (int i = 0; i < n; i++)
97         for (int t = 0; t < k; t++)
98             for (int j = 0; j < m; j++)
99                 c[i][j] = c[i][j] + a[i][t] * b[t][j];
100     return c;

```

```

101     }
102
103     static Matrix power(Matrix a, unsigned long long e)
104     {
105         int n = a.size();
106         assert(n > 0 && (int)a[0].size() == n);
107         Matrix r(n, vector<Z>(n));
108         for (int i = 0; i < n; i++)
109             r[i][i] = 1;
110         for (; e >>= 1, a = multiply(a, a))
111             if (e & 1)
112                 r = multiply(r, a);
113         return r;
114     }
115
116     // Undirected multigraph, vertices 0..n-1; loops ignored.
117     static Z spanning_trees(int n, const vector<pair<int, int>> &edges)
118     {
119         assert(n > 0);
120         Matrix lap(n, vector<Z>(n));
121         for (auto [u, v] : edges)
122             if (u != v)
123             {
124                 lap[u][u] = lap[u][u] + 1;
125                 lap[v][v] = lap[v][v] + 1;
126                 lap[u][v] = lap[u][v] - 1;
127                 lap[v][u] = lap[v][u] - 1;
128             }
129         lap.pop_back();
130         for (auto &row : lap)
131             row.pop_back();
132         return determinant(lap);
133     }
134 };

```

62.2 杜教筛

构造预筛上限 $1 \leq B < \text{INT_MAX}$ ，内存须可用； $\text{mertens}(n)$ 返回 μ 的前缀和， $\text{totient_sum}(n)$ 返回 φ 的前缀和 (int128)。约定 $0 \leq n \leq 10^{11}$ ， $n=0$ 返回 0。按整除分块递归并写入 mmu/mphi 缓存；重复查询复用结果，清空缓存可重算，不可修改 $\text{pmu}/\text{pphi}/\text{limit}$ 。传统接口 Mertens、Totient_Sum。单次最大参数 N 的常用均摊分析为 $O(B+N/\sqrt{B})$ (哈希按期望常数计算)， B 约 $N^{2/3}$ 时为 $O(N^{2/3})$ ；小 B 不构成大输入性能保证。多次不同 N 可能新增缓存，空间随实际缓存增长；递归深度 $O(\log N)$ 。

```

1 struct DuJiao
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int limit;
6     vector<ll> pmu, pphi;
7     unordered_map<ll, ll> mmu;
8     unordered_map<ll, I> mphi;
9
10    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
11    {
12        assert(limit >= 1);
13        LinearSieve s(limit);
14        for (int i = 1; i <= limit; i++)
15        {
16            pmu[i] = pmu[i - 1] + s.mu[i];
17            pphi[i] = pphi[i - 1] + s.phi[i];
18        }
19    }
20
21    ll mertens(ll n)
22    {
23        if (n <= limit)
24            return pmu[n];
25        if (mmu.count(n))
26            return mmu[n];
27        ll ans = 1;
28        for (ll l = 2, r; l <= n; l = r + 1)
29        {
30            r = n / (n / l);
31            ans -= (r - l + 1) * mertens(n / l);
32        }
33        return mmu[n] = ans;
34    }
35
36    I totient_sum(ll n)
37    {
38        if (n <= limit)
39            return pphi[n];
40        if (mphi.count(n))
41            return mphi[n];

```

```
42     I ans = I(n) * (n + 1) / 2;  
43     for (ll l = 2, r; l <= n; l = r + 1)  
44     {  
45         r = n / (n / l);  
46         ans -= I(r - l + 1) * totient_sum(n / l);  
47     }  
48     return mphi[n] = ans;  
49 }  
50 };
```


62.3 扩展 BSGS

依赖 `number_theory.hpp` 的求逆与快速幂。`solve(a,b,m)` 返回最小 $x \geq 0$ 使 $a^x \equiv b \pmod{m}$ ，无解返回 -1； $1 \leq m \leq 10^{12}$ ， a 、 b 可为完整 `long long` 范围，入口归一化，约定 $0^0=1$ 。先剥离 `gcd` 并逐次检查较小指数，再在互质剩余问题上 BSGS。`baby` 表对同余幂保留最小指数 j ，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间 $O(\sqrt{m} + \log m)$ ，空间 $O(\sqrt{m})$ ，哈希表先 `reserve`；基于 `unordered_map` 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```

1 struct DiscreteLog
2 {
3     using ll = long long;
4
5     // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
6     static ll solve(ll a, ll b, ll m)
7     {
8         assert(m >= 1 && m <= 1000000000000LL);
9         a = (a % m + m) % m;
10        b = (b % m + m) % m;
11        if (m == 1 || b == 1)
12            return 0;
13        if (a == 1)
14            return -1;
15        ll offset = 0, k = 1;
16        for (ll g; (g = gcd(a, m)) > 1;)
17        {
18            if (b == k)
19                return offset;
20            if (b % g)
21                return -1;
22            b /= g;
23            m /= g;
24            k = (__int128)k * (a / g) % m;
25            ++offset;
26        }
27        ll target = (__int128)b * NumberTheory::inverse(k, m) % m;
28        ll step = sqrtl(m) + 1;
29        unordered_map<ll, ll> baby;
30        baby.reserve(step);
31        ll cur = 1 % m;
32        for (ll j = 0; j < step; j++)
33        {
34            if (!baby.count(cur))
35                baby[cur] = j;
36            cur = (__int128)cur * a % m;
37        }
38        ll inv = NumberTheory::inverse(NumberTheory::power(a, step, m), m);
39        cur = target;

```

```
40     for (ll i = 0; i <= step; i++)
41     {
42         auto it = baby.find(cur);
43         if (it != baby.end())
44             return offset + i * step + it->second;
45         cur = (__int128)cur * inv % m;
46     }
47     return -1;
48 }
49 };
```

第六十三章 任意模数行列式

63.1 任意正模数行列式：欧几里得消元

`determinant_mod(a,mod)` 返回 $\det(a)$ 的最小非负余数。 a 为 `long long` 方阵，元素支持完整 signed 64 位； $1 \leq \text{mod} \leq \text{LLONG_MAX}$ ，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 $1 \% \text{mod}$ ， $\text{mod}=1$ 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 `int128`。 $O(n^3 \log \text{mod})$ 时间、 $O(n^2)$ 空间；素数模通常优先用已有高斯行列式。传统接口 `Determinant_Mod`。

```
1 inline long long determinant_mod(vector<vector<long long>> a, long long mod)
2 {
3     assert(mod >= 1);
4     int n = a.size();
5     for (auto &row : a)
6     {
7         assert((int)row.size() == n);
8         for (auto &x : row)
9         {
10             x %= mod;
11             if (x < 0)
12                 x += mod;
13         }
14     }
15     long long answer = 1 % mod;
16     for (int i = 0; i < n; i++)
17     {
18         for (int j = i + 1; j < n; j++)
19             while (a[j][i])
20             {
21                 long long quotient = a[i][i] / a[j][i];
22                 for (int k = i; k < n; k++)
23                 {
24                     __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
25                     value %= mod;
26                     if (value < 0)
27                         value += mod;
28                     a[i][k] = (long long)value;
29                 }
30                 swap(a[i], a[j]);
31                 answer = answer ? mod - answer : 0;
32             }
33         if (a[i][i] == 0)
34             return 0;
35         answer = (__int128)answer * a[i][i] % mod;
36     }
37     return answer;
38 }
```

第六十四章 带权生成树计数

64.1 带权矩阵树：无向树与内外向树形图

声明 `MatrixTree<mod>`，调用 `count(n, edges, root, kind)` 返回所有合法树的边权乘积之和，`mod` 必须是素数。`Edge=(u,v,Z)`，点号 `0..n-1`， $n \geq 1$ ；`Kind::undirected` 为无向生成树，`toward_root` 为每点沿边能到根，`away_from_root` 为根沿边能到每点。后两种使用输入边方向，不能混淆。重边按不同边计数，自环忽略；单点答案 1。支持模意义下的零权和负权，结果 0 不一定表示不存在树。无向图删除哪个根行列均可；有向图 `root` 决定答案。依赖 `LinearAlgebra` 的素数模行列式， $O(n^3+m)$ 时间、 $O(n^2)$ 额外空间。传统接口 `Matrix_Tree<mod>::Count`，`Kind` 成员名不变。

```
1 template <int mod> struct MatrixTree
2 {
3     using A = LinearAlgebra<mod>;
4     using Z = typename A::Z;
5     using Edge = tuple<int, int, Z>;
6     enum class Kind
7     {
8         undirected,
9         toward_root,
10        away_from_root
11    };
12
13    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
14    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
15    {
16        assert(n >= 1 && root >= 0 && root < n);
17        typename A::Matrix lap(n - 1, vector<Z>(n - 1));
18        auto add = [&](int u, int v, Z w)
19        {
20            if (u == root)
21                return;
22            int i = u - (u > root), j = v - (v > root);
23            lap[i][i] = lap[i][i] + w;
24            if (v != root)
25                lap[i][j] = lap[i][j] - w;
26        };
27        for (auto [u, v, w] : edges)
28        {
29            assert(u >= 0 && u < n && v >= 0 && v < n);
30            if (u == v)
31                continue;
32            if (kind == Kind::away_from_root)
33                swap(u, v);
34            add(u, v, w);
35            if (kind == Kind::undirected)
36                add(v, u, w);
37        }
38        return A::determinant(lap);
39    }
40};
```

第六十五章 合数模生成树计数

65.1 任意正模数带权矩阵树

`count(n, edges, root, kind, mod)` 返回树的边权乘积之和对 `mod` 的最小非负余数。`mod` 为任意正 signed 64 位数，边权支持完整 signed 64 位。`Kind` 的 `undirected`、`toward_root`、`away_from_root` 分别表示无向、内向和外向树；点号 `0..n-1`。自环忽略，重边按不同边计数，单点返回 `1%mod`，`mod=1` 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 `determinant_mod`； $O(n^3 \log \text{mod} + m)$ 时间、 $O(n^2)$ 额外空间。不使用模逆元。已知模数是素数时一般优先使用 `MatrixTree<mod>`。传统接口 `Matrix_Tree_Mod::Count`，`Kind` 名称不变。

```
1 struct MatrixTreeMod
2 {
3     using ll = long long;
4     using Edge = tuple<int, int, ll>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
13    {
14        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
15        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
16        auto add = [&](int u, int v, ll w)
17        {
18            if (u == root)
19                return;
20            int i = u - (u > root), j = v - (v > root);
21            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
22            if (v != root)
23                lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
24        };
25        for (auto [u, v, w] : edges)
26        {
27            assert(u >= 0 && u < n && v >= 0 && v < n);
28            if (u == v)
29                continue;
30            w %= mod;
31            if (w < 0)
32                w += mod;
33            if (kind == Kind::away_from_root)
34                swap(u, v);
35            add(u, v, w);
36            if (kind == Kind::undirected)
37                add(v, u, w);
38        }
39        return determinant_mod(move(lap), mod);
40    }
41};
```

第六十六章 计算几何

66.1 整数精确二维几何

坐标绝对值 $\leq 10^{12}$ ，点数 $\leq 10^6$ 。叉积、点积、线段相交、凸包、点在多边形、点在凸包、旋转卡壳直径。凸包逆时针，不保留共线内点，不重复首点。contains 返回 0 外部、1 边界、2 内部。convex_contains/diameter2 输入必须为本模板 strict hull。PolarLess 要求排除零向量。

```
1 struct IntegerGeometry
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
7     struct Point
8     {
9         ll x = 0, y = 0;
10
11         bool operator<(Point b) const
12         {
13             return tie(x, y) < tie(b.x, b.y);
14         }
15
16         bool operator==(Point b) const
17         {
18             return x == b.x && y == b.y;
19         }
20     };
21
22     static I cross(Point a, Point b, Point c)
23     {
24         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
25     }
26
27     static I dot(Point a, Point b, Point c)
28     {
29         return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
30     }
31
32     static I dist2(Point a, Point b)
33     {
34         return dot(a, b, b);
35     }
36
37     static int sign(I x)
38     {
39         return (x > 0) - (x < 0);
40     }
41
42     static bool on_segment(Point p, Point a, Point b)
43     {
44         return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
```

```

45     }
46
47     static bool intersect(Point a, Point b, Point c, Point d)
48     {
49         int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
50         int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
51         if (!x && on_segment(c, a, b))
52             return true;
53         if (!y && on_segment(d, a, b))
54             return true;
55         if (!z && on_segment(a, c, d))
56             return true;
57         if (!w && on_segment(b, c, d))
58             return true;
59         return x * y < 0 && z * w < 0;
60     }
61
62     static vector<Point> hull(vector<Point> p)
63     {
64         sort(p.begin(), p.end());
65         p.erase(unique(p.begin(), p.end(), p.end()));
66         int n = (int)p.size();
67         if (n < 3)
68             return p;
69         vector<Point> h;
70         for (Point v : p)
71         {
72             while (h.size() > 1 && cross(h[h.size() - 2], h.back(), v) <= 0)
73                 h.pop_back();
74             h.push_back(v);
75         }
76         int low = (int)h.size();
77         for (int i = n - 2; i >= 0; i--)
78         {
79             while ((int)h.size() > low && cross(h[h.size() - 2], h.back(), p[i]) <= 0)
80                 h.pop_back();
81             h.push_back(p[i]);
82         }
83         h.pop_back();
84         return h; // CCW, no repeated endpoint, no collinear interior points.
85     }
86
87     static I area2(const vector<Point> &p)
88     {
89         I ans = 0;
90         int n = (int)p.size();
91         for (int i = 0; i < n; i++)
92             ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
93         return ans;
94     }
95
96     // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
97     static int contains(const vector<Point> &p, Point q)
98     {
99         int winding = 0, n = (int)p.size();
100        for (int i = 0; i < n; i++)

```

```

101     {
102         Point a = p[i], b = p[(i + 1) % n];
103         if (on_segment(q, a, b))
104             return 1;
105         I c = cross(a, b, q);
106         if (a.y <= q.y && b.y > q.y && c > 0)
107             ++winding;
108         if (a.y > q.y && b.y <= q.y && c < 0)
109             --winding;
110     }
111     return winding ? 2 : 0;
112 }
113
114 // Input must be the strict CCW hull above. O(log n).
115 static int convex_contains(const vector<Point> &p, Point q)
116 {
117     int n = (int)p.size();
118     if (n < 3)
119         return contains(p, q);
120     I a = cross(p[0], p[1], q), b = cross(p[0], p[n - 1], q);
121     if (a < 0 || b > 0)
122         return 0;
123     if (!a)
124         return on_segment(q, p[0], p[1]) ? 1 : 0;
125     if (!b)
126         return on_segment(q, p[0], p[n - 1]) ? 1 : 0;
127     int l = 1, r = n - 1;
128     while (l + 1 < r)
129     {
130         int m = (l + r) / 2;
131         if (cross(p[0], p[m], q) >= 0)
132             l = m;
133         else
134             r = m;
135     }
136     I c = cross(p[l], p[r], q);
137     return c < 0 ? 0 : c == 0 ? 1 : 2;
138 }
139
140 static I diameter2(const vector<Point> &p)
141 {
142     int n = (int)p.size();
143     if (n < 2)
144         return 0;
145     if (n == 2)
146         return dist2(p[0], p[1]);
147     I ans = 0;
148     int j = 1;
149     for (int i = 0; i < n; i++)
150     {
151         int k = (i + 1) % n;
152         while (cross(p[i], p[k], p[(j + 1) % n]) > cross(p[i], p[k], p[j]))
153             j = (j + 1) % n;
154         ans = max({ans, dist2(p[i], p[j]), dist2(p[k], p[j])});
155     }
156     return ans;

```



```

157     }
158
159     struct PolarLess
160     {
161         static int half(Point p)
162         {
163             return p.y < 0 || (p.y == 0 && p.x < 0);
164         }
165
166         bool operator()(Point a, Point b) const
167         {
168             if (half(a) != half(b))
169                 return half(a) < half(b);
170             I c = cross({0, 0}, a, b);
171             if (c)
172                 return c > 0;
173             return dist2({0, 0}, a) < dist2({0, 0}, b);
174         }
175     };
176 };

```

66.2 浮点点线圆构造

long double, 显式 eps 默认 10^{-12} 。要求有限输入及适当尺度；接近相切时按容差分类，不能提供任意实数输入的精确拓扑保证。Result 区分 none/one/two/infinite/degenerate。圆圆交点仅在圆心和正半径完全相同时返回 infinite；零半径圆按点处理，不把近同心圆当作重合。圆相关交点容差按局部距离和半径缩放，不使用固定的单位长度下限。零长度线返回 degenerate；投影与点到线段距离将零线段视为点。overlap 返回交叠面积，小圆弓用级数避免三角函数相减消失。坐标缩放和误差预算仍须按题面复核。

```

1 struct RealGeometry
2 {
3     using R = long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6
7     RealGeometry(R eps = 1e-12L) : eps(eps)
8     {
9         assert(eps > 0);
10    }
11
12    struct Point
13    {
14        R x = 0, y = 0;
15
16        Point operator+(Point b) const
17        {
18            return {x + b.x, y + b.y};
19        }
20
21        Point operator-(Point b) const
22        {
23            return {x - b.x, y - b.y};
24        }
25

```

```

26     Point operator*(R k) const
27     {
28         return {x * k, y * k};
29     }
30
31     Point operator/(R k) const
32     {
33         return {x / k, y / k};
34     }
35 };
36
37 struct Circle
38 {
39     Point o;
40     R r;
41 };
42 enum class Kind
43 {
44     none,
45     one,
46     two,
47     infinite,
48     degenerate
49 };
50
51 struct Result
52 {
53     Kind kind;
54     vector<Point> p;
55 };
56
57 static R dot(Point a, Point b)
58 {
59     return a.x * b.x + a.y * b.y;
60 }
61
62 static R cross(Point a, Point b)
63 {
64     return a.x * b.y - a.y * b.x;
65 }
66
67 static R norm(Point a)
68 {
69     return hypotl(a.x, a.y);
70 }
71
72 static Point perp(Point a)
73 {
74     return {-a.y, a.x};
75 }
76
77 Point projection(Point p, Point a, Point b) const
78 {
79     Point v = b - a;
80     R d = dot(v, v);
81     if (d == 0)

```

```

82         return a;
83     return a + v * (dot(p - a, v) / d);
84 }
85
86 R segment_distance(Point p, Point a, Point b) const
87 {
88     Point v = b - a;
89     R d = dot(v, v);
90     if (d == 0)
91         return norm(p - a);
92     R t = clamp(dot(p - a, v) / d, R(0), R(1));
93     return norm(p - (a + v * t));
94 }
95
96 Result line_intersection(Point a, Point b, Point c, Point d) const
97 {
98     Point u = b - a, v = d - c;
99     R nu = norm(u), nv = norm(v);
100    if (nu == 0 || nv == 0)
101        return {Kind::degenerate, {}};
102    R det = cross(u, v);
103    if (fabsl(det) <= eps * nu * nv)
104        return {fabsl(cross(c - a, u)) <= eps * nu * max(R(1), norm(c - a))
105                ? Kind::infinite
106                : Kind::none,
107                {}};
108    return {Kind::one, {a + u * (cross(c - a, v) / det)}};
109 }
110
111 Result line_circle(Point a, Point b, Circle c) const
112 {
113     assert(c.r >= 0);
114     Point v = b - a;
115     R len = norm(v);
116     if (len == 0)
117         return {Kind::degenerate, {}};
118     Point h = projection(c.o, a, b);
119     R d = norm(h - c.o), tol = eps * max(d, c.r);
120     if (d > c.r + tol)
121         return {Kind::none, {}};
122     if (fabsl(d - c.r) <= tol)
123         return {Kind::one, {h}};
124     R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
125     Point w = v * (t / len);
126     return {Kind::two, {h - w, h + w}};
127 }
128
129 Result circles(Circle a, Circle b) const
130 {
131     assert(a.r >= 0 && b.r >= 0);
132     Point v = b.o - a.o;
133     R d = norm(v);
134     R tol = eps * max({d, a.r, b.r});
135     // Infinite intersections require identical positive-radius circles.
136     if (d == 0)
137         {

```

```

138     if (a.r != b.r)
139         return {Kind::none, {}};
140     if (a.r == 0)
141         return {Kind::one, {a.o}};
142     return {Kind::infinite, {}};
143 }
144 if (a.r == 0 && b.r == 0)
145     return {Kind::none, {}};
146 if (a.r == 0 || b.r == 0)
147 {
148     R radius = max(a.r, b.r);
149     if (fabsl(d - radius) > tol)
150         return {Kind::none, {}};
151     return {Kind::one, {a.r == 0 ? a.o : b.o}};
152 }
153 if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol)
154     return {Kind::none, {}};
155 R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
156 Point h = a.o + v * (x / d);
157 // Near-concentric circles must not be mistaken for internal tangency.
158 if (fabsl(x) > a.r + tol)
159     return {Kind::none, {}};
160 if (fabsl(fabsl(x) - a.r) <= tol)
161     return {Kind::one, {h}};
162 R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
163 Point w = perp(v) * (y / d);
164 return {Kind::two, {h - w, h + w}};
165 }
166
167 R overlap(Circle a, Circle b) const
168 {
169     assert(a.r >= 0 && b.r >= 0);
170     R d = norm(a.o - b.o), pi = acosl(-1.L);
171     if (d >= a.r + b.r)
172         return 0;
173     if (d <= fabsl(a.r - b.r))
174         return pi * min(a.r, b.r) * min(a.r, b.r);
175     R difference = (a.r - b.r) * (a.r + b.r);
176     R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
177     R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
178     auto segment = [](R angle)
179     {
180         if (angle < 1e-3L)
181         {
182             R z = angle * angle;
183             return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
184         }
185         return angle - sinl(2 * angle) / 2;
186     };
187     return a.r * a.r * segment(x) + b.r * b.r * segment(y);
188 }
189 };

```

第六十七章 优化与可持久化

67.1 Max-plus 矩阵幂

$a[i][j]$ 为转移收益，不存在的边为 neg ； $power(k)$ 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^{119} 。 $O(n^3 \log k)$ ；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```
1 struct MaxPlusMatrix
2 {
3     using I = __int128_t;
4     static constexpr I neg = -(I(1) << 120);
5     int n;
6     vector<vector<I>> a;
7
8     MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
9     {
10         if (identity)
11             for (int i = 0; i < n; i++)
12                 a[i][i] = 0;
13     }
14
15     MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
16     {
17         assert(n == b.n);
18         MaxPlusMatrix c(n);
19         for (int i = 0; i < n; i++)
20             for (int k = 0; k < n; k++)
21                 if (a[i][k] != neg)
22                     for (int j = 0; j < n; j++)
23                         if (b.a[k][j] != neg)
24                             c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
25         return c;
26     }
27
28     MaxPlusMatrix power(unsigned long long e) const
29     {
30         MaxPlusMatrix b = *this, r(n, true);
31         for (; e >>= 1, b = b * b)
32             if (e & 1)
33                 r = r * b;
34         return r;
35     }
36 };
```

67.2 离散李超树与最优直线编号

构造传入所有可能查询的横坐标， $add(\{k,b,id\})$ 插入直线， $query(x)$ 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 $int128$ ， k 、 b 、 x 为有符号 64 位且中间值可表示。单次 $O(\log n)$ 。

```
1 struct LiChao
2 {
```

```

3  using ll = long long;
4  using I = __int128_t;
5
6  struct Line
7  {
8      ll k = 0, b = 0;
9      int id = -1;
10
11      I value(ll x) const
12      {
13          return I(k) * x + b;
14      }
15  };
16
17  struct Node
18  {
19      Line line;
20      int left = -1, right = -1;
21  };
22
23  vector<ll> xs;
24  vector<Node> a{Node{}};
25
26  LiChao(vector<ll> x) : xs(x)
27  {
28      sort(xs.begin(), xs.end());
29      xs.erase(unique(xs.begin(), xs.end()), xs.end());
30      assert(!xs.empty());
31  }
32
33  bool better(Line u, Line v, ll x) const
34  {
35      return v.id == -1 ||
36             (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
37  }
38
39  void add(Line line)
40  {
41      assert(line.id >= 0);
42      add(0, 0, (int)xs.size() - 1, line);
43  }
44
45  void add(int p, int l, int r, Line line)
46  {
47      int m = (l + r) / 2;
48      if (better(line, a[p].line, xs[m]))
49          swap(line, a[p].line);
50      if (l == r || line.id == -1)
51          return;
52      bool left = better(line, a[p].line, xs[l]);
53      bool right = better(line, a[p].line, xs[r]);
54      if (!left && !right)
55          return;
56      int next = left ? a[p].left : a[p].right;
57      if (next == -1)
58          {

```

```

59     next = (int)a.size();
60     a.push_back(Node{});
61     if (left)
62         a[p].left = next;
63     else
64         a[p].right = next;
65 }
66 if (left)
67     add(next, l, m, line);
68 else
69     add(next, m + 1, r, line);
70 }
71
72 pair<I, int> query(ll x) const
73 {
74     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
75     assert(idx < (int)xs.size() && xs[idx] == x);
76     int p = 0, l = 0, r = (int)xs.size() - 1;
77     Line best;
78     while (p != -1)
79     {
80         if (better(a[p].line, best, x))
81             best = a[p].line;
82         if (l == r)
83             break;
84         int m = (l + r) / 2;
85         if (idx <= m)
86         {
87             p = a[p].left;
88             r = m;
89         }
90         else
91         {
92             p = a[p].right;
93             l = m + 1;
94         }
95     }
96     assert(best.id != -1);
97     return {best.value(x), best.id};
98 }
99 };

```

67.3 主席树：静态区间第 k 小

构造传入 0-based 数组， $\text{kth}(l, r, k)$ 的区间使用 1-based 闭区间；要求 $1 \leq k \leq r - l + 1$ 。离散化后每个前缀一个版本，单次 $O(\log n)$ ，总空间 $O(n \log n)$ 。

```

1 struct PersistentKth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     vector<Node> t{Node{}};

```

```

9     vector<int> root{0};
10    vector<long long> vals;
11
12    PersistentKth(const vector<long long> &a) : vals(a)
13    {
14        sort(vals.begin(), vals.end());
15        vals.erase(unique(vals.begin(), vals.end()), vals.end());
16        for (long long x : a)
17        {
18            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
19            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
20        }
21    }
22
23    int add(int old, int l, int r, int x)
24    {
25        int p = t.size();
26        Node copy = t[old];
27        t.push_back(copy);
28        ++t[p].sum;
29        if (l < r)
30        {
31            int m = (l + r) / 2;
32            if (x <= m)
33            {
34                int v = add(t[old].l, l, m, x);
35                t[p].l = v;
36            }
37            else
38            {
39                int v = add(t[old].r, m + 1, r, x);
40                t[p].r = v;
41            }
42        }
43        return p;
44    }
45
46    long long kth(int l, int r, int k) const
47    {
48        assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49        int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50        while (L < R)
51        {
52            int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53            if (k <= c)
54            {
55                u = t[u].l;
56                v = t[v].l;
57                R = m;
58            }
59            else

```



```
60         {
61             k -= c;
62             u = t[u].r;
63             v = t[v].r;
64             L = m + 1;
65         }
66     }
67     return vals[L];
68 }
69 };
```

第六十八章 可持久化数组

68.1 可持久化数组：历史版本分叉与单点修改

PersistentArray(a,changes=0), a 是非空 0-based 初值数组; 第 0 版为初始状态。set(version,pos,value) 基于任意已有版本修改 1-based 位置, 创建并返回新版本编号; copy(version) 以 $O(1)$ 时间复制版本; query(version,pos) 只读且不自动创建版本。values 支持 long long 全范围。每次修改复制根到叶的节点, 旧版本不变; build 为内部函数, 不可覆盖历史节点。建树 $O(n)$, 修改及查询 $O(\log n)$, 初始 $2n-1$ 个节点, 每次修改至多 $\text{ceil}(\log_2(n))+1$ 个; 总空间 $O(n + \text{修改数} * \log n + \text{版本数})$ 。changes 仅用于预留节点池, 低估后可扩容, 所有外部引用应使用下标, 不保存 vector 元素引用。节点下标用 int, 节点总量须可表示。

```
1 struct PersistentArray
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0;
8         ll value = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root;
14
15     PersistentArray(const vector<ll> &a, int changes = 0) : n(a.size())
16     {
17         assert(n > 0 && changes >= 0);
18         int levels = 1;
19         for (int x = n - 1; x; x >>= 1)
20             levels++;
21         t.reserve(2ULL * n + 1ULL * changes * levels + 1);
22         root.push_back(build(1, n, a));
23     }
24
25     int build(int l, int r, const vector<ll> &a)
26     {
27         int p = t.size();
28         t.push_back(Node{});
29         if (l == r)
30             t[p].value = a[l - 1];
31         else
32         {
33             int m = (l + r) / 2;
34             int left = build(l, m, a);
35             int right = build(m + 1, r, a);
36             t[p].l = left;
37             t[p].r = right;
38         }
39         return p;
40     }
41 }
```

```

42 int modify(int old, int l, int r, int pos, ll value)
43 {
44     int p = t.size();
45     Node copy = t[old];
46     t.push_back(copy);
47     if (l == r)
48         t[p].value = value;
49     else
50     {
51         int m = (l + r) / 2;
52         if (pos <= m)
53         {
54             int child = modify(t[old].l, l, m, pos, value);
55             t[p].l = child;
56         }
57         else
58         {
59             int child = modify(t[old].r, m + 1, r, pos, value);
60             t[p].r = child;
61         }
62     }
63     return p;
64 }
65
66 // Create a new version from any existing version. Positions are 1-based.
67 int set(int version, int pos, ll value)
68 {
69     assert(0 <= version && version < (int)root.size());
70     assert(1 <= pos && pos <= n);
71     int p = modify(root[version], 1, n, pos, value);
72     root.push_back(p);
73     return (int)root.size() - 1;
74 }
75
76 int copy(int version)
77 {
78     assert(0 <= version && version < (int)root.size());
79     root.push_back(root[version]);
80     return (int)root.size() - 1;
81 }
82
83 ll query(int version, int pos) const
84 {
85     assert(0 <= version && version < (int)root.size());
86     assert(1 <= pos && pos <= n);
87     int p = root[version], l = 1, r = n;
88     while (l < r)
89     {
90         int m = (l + r) / 2;
91         if (pos <= m)
92         {
93             p = t[p].l;
94             r = m;

```

```
95         }
96         else
97         {
98             p = t[p].r;
99             l = m + 1;
100        }
101    }
102    return t[p].value;
103 }
104 };
```

第六十九章 可持久化区间操作

69.1 可持久化区间加与版本拼接

PersistentRange(n) 建立长度 n 的全零版本 0, $n \geq 1$; 使用隐式节点, 不预建整棵树。add(v,l,r,x) 在版本 v 的 [l,r) 加 x; splice(dst,src,l,r) 将 dst 的 [l,r) 替换为 src 同一位置的值, 其他位置保留 dst; 二者均返回新版本编号。query(v,l,r) 返回区间和, const 且不分配节点。下标 0-based、区间半开, 允许空区间; 空修改及零增量仍创建版本但复用根, 整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先, 用 offset 修正跨版本拼接; 每次修改/拼接/查询 $O(\log n)$, 版本别名 $O(1)$, 总节点 $O(q \log n)$ 。sum、lazy、delta、carry、offset 及其中间和差和长度乘积均须在 long long 内; 节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根, 不要外部改写。传统接口 Persistent_Range::Add / Splice / Query。

```
1 struct PersistentRange
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int left = 0, right = 0;
8         ll sum = 0, lazy = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root{0};
14
15     explicit PersistentRange(int size) : n(size)
16     {
17         assert(n > 0);
18     }
19
20     int clone(int p)
21     {
22         Node node = t[p];
23         t.push_back(node);
24         return int(t.size()) - 1;
25     }
26
27     void pull(int p, int len)
28     {
29         t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
30     }
```

区间加与版本拼接（接上页同一结构体）：

```

32  int change(int p, int l, int r, int ql, int qr, ll value)
33  {
34      if (qr <= l || r <= ql)
35          return p;
36      int u = clone(p);
37      if (ql <= l && r <= qr)
38      {
39          t[u].lazy += value;
40          t[u].sum += value * (r - l);
41          return u;
42      }
43      int m = l + (r - l) / 2;
44      int a = change(t[p].left, l, m, ql, qr, value);
45      int b = change(t[p].right, m, r, ql, qr, value);
46      t[u].left = a;
47      t[u].right = b;
48      pull(u, r - l);
49      return u;
50  }
51
52  int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
53  {
54      if (qr <= l || r <= ql)
55          return p;
56      if (ql <= l && r <= qr)
57      {
58          if (offset == 0)
59              return q;
60          int u = clone(q);
61          t[u].lazy += offset;
62          t[u].sum += offset * (r - l);
63          return u;
64      }
65      int u = clone(p), m = l + (r - l) / 2;
66      offset += t[q].lazy - t[p].lazy;
67      int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
68      int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
69      t[u].left = a;
70      t[u].right = b;
71      pull(u, r - l);
72      return u;
73  }

```

只读查询与对外接口（接上页同一结构体）：

```

75     ll get(int p, int l, int r, int ql, int qr, ll carry) const
76     {
77         if (qr <= l || r <= ql)
78             return 0;
79         if (ql <= l && r <= qr)
80             return t[p].sum + carry * (r - l);
81         carry += t[p].lazy;
82         int m = l + (r - l) / 2;
83         return get(t[p].left, l, m, ql, qr, carry) +
84                get(t[p].right, m, r, ql, qr, carry);
85     }
86
87     void check(int version, int l, int r) const
88     {
89         assert(0 <= version && version < int(root.size()));
90         assert(0 <= l && l <= r && r <= n);
91     }
92
93     int add(int version, int l, int r, ll value)
94     {
95         check(version, l, r);
96         int p = l == r || value == 0 ? root[version]
97                : change(root[version], 0, n, l, r, value);
98         root.push_back(p);
99         return int(root.size()) - 1;
100    }
101
102    int splice(int destination, int source, int l, int r)
103    {
104        check(destination, l, r);
105        check(source, l, r);
106        int p = l == r ? root[destination]
107               : replace(root[destination], root[source], 0, n, l, r, 0);
108        root.push_back(p);
109        return int(root.size()) - 1;
110    }
111
112    ll query(int version, int l, int r) const
113    {
114        check(version, l, r);
115        return l == r ? 0 : get(root[version], 0, n, l, r, 0);
116    }
117 };

```

第七十章 树上路径第 k 小

70.1 树上路径点权第 k 小

TreePathKth(a) 传入非空 0-based 权值数组，节点编号 1..n；add(u,v) 加无向边，要求恰好构成一棵树。build(root=1) 递归 DFS 建立根到各节点的频数版本与倍增表，可换根重建，旧版本自动清空。kth(u,v,k) 返回简单路径上第 k 小的点权，包含两个端点，每个节点计一次； $1 \leq k \leq$ 路径点数。lca(u,v) 返回当前预处理根下的最近公共祖先。使用 $root[u] + root[v] - root[lca] - root[parent(lca)]$ ，编号 0 为全零版本，避免少算或重复算 LCA。支持重复值与完整 long long 权值，内部只统计频数；权值和树结构在查询阶段保持不变。预处理 $O(n \log n)$ ，每次查询 $O(\log n)$ ，空间 $O(n \log n)$ 。递归深度最坏 $O(n)$ ，保持递归写法。

```
1 struct TreePathKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, lg;
11    vector<vector<int>> g, up;
12    vector<int> root, dep;
13    vector<ll> value, vals;
14    vector<Node> t;
15
16    TreePathKth(const vector<ll> &a)
17        : n(a.size()), g(n + 1), root(n + 1), dep(n + 1), value(a)
18    {
19        assert(n > 0);
20        lg = 1;
21        while ((1LL << lg) <= n)
22            lg++;
23        up.assign(lg, vector<int>(n + 1));
24    }
25
26    void add(int u, int v)
27    {
28        g[u].push_back(v);
29        g[v].push_back(u);
30    }
31
32    int insert(int old, int l, int r, int x)
33    {
34        int p = t.size();
35        Node copy = t[old];
36        t.push_back(copy);
37        t[p].sum++;
38        if (l == r)
39            return p;
40        int m = (l + r) / 2;
41        if (x <= m)
```



```

42     {
43         int child = insert(t[old].l, l, m, x);
44         t[p].l = child;
45     }
46     else
47     {
48         int child = insert(t[old].r, m + 1, r, x);
49         t[p].r = child;
50     }
51     return p;
52 }
53
54 void dfs(int u, int p)
55 {
56     up[0][u] = p;
57     for (int j = 1; j < lg; j++)
58         up[j][u] = up[j - 1][up[j - 1][u]];
59     int x = lower_bound(vals.begin(), vals.end(), value[u - 1]) - vals.begin();
60     root[u] = insert(root[p], 0, (int)vals.size() - 1, x);
61     for (int v : g[u])
62     {
63         if (v == p)
64             continue;
65         dep[v] = dep[u] + 1;
66         dfs(v, u);
67     }
68 }
69
70 void build(int r = 1)
71 {
72     assert(1 <= r && r <= n);
73     vals = value;
74     sort(vals.begin(), vals.end());
75     vals.erase(unique(vals.begin(), vals.end()), vals.end());
76     t.assign(1, Node{});
77     dep[r] = 0;
78     dfs(r, 0);
79 }
80
81 int lca(int u, int v) const
82 {
83     if (dep[u] < dep[v])
84         swap(u, v);
85     int diff = dep[u] - dep[v];
86     for (int j = 0; j < lg; j++)
87         if (diff >> j & 1)
88             u = up[j][u];
89     if (u == v)
90         return u;
91     for (int j = lg - 1; j >= 0; j--)
92         if (up[j][u] != up[j][v])
93         {
94             u = up[j][u];
95             v = up[j][v];
96         }
97     return up[0][u];

```

```

98     }
99
100     ll kth(int u, int v, int k) const
101     {
102         int z = lca(u, v);
103         assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
104         int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];
105         int l = 0, r = (int)vals.size() - 1;
106         while (l < r)
107         {
108             int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
109             int m = (l + r) / 2;
110             if (k <= count)
111             {
112                 a = t[a].l;
113                 b = t[b].l;
114                 c = t[c].l;
115                 d = t[d].l;
116                 r = m;
117             }
118             else
119             {
120                 k -= count;
121                 a = t[a].r;
122                 b = t[b].r;
123                 c = t[c].r;
124                 d = t[d].r;
125                 l = m + 1;
126             }
127         }
128         return vals[l];
129     }
130 };

```

第七十一章 动态区间顺序统计

71.1 动态区间第 k 小 (树状数组套权值线段树)

DynamicKth(a,future) 传入非空 0-based 初值, future 包含所有将来可能赋入的新值; 内部自动合并初值并离散化。set(pos,value) 单点赋值, kth(l,r,k) 查询闭区间第 k 小, 下标均为 1-based; $1 \leq k \leq r-l+1$ 。允许重复值与完整 long long 权值; 新值必须已在离散化集合内, 操作可按原顺序执行, 不保留历史版本。每个树状数组节点对应一棵可变权值线段树; 删除旧值后插入新值。空节点回收到 free_head 链, 重复修改不会按操作次数累计泄漏节点。设 D 为不同值数, $S=n+\text{future.size()}$, 预处理 $O(S \log S+n \log n \log(D+1))$, 单次修改及查询 $O(\log n \log(D+1))$ 。节点数上界 $B=(\text{ceil}(\log_2(D))+1) \cdot \sum \text{lowbit}(i)$, 求和 $i=1..n$; 空间 $O(S+n+B)$ 。update、modify、id 和节点池均为内部接口; 不能手动减去不存在的值。

```
1 struct DynamicKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, free_head = 0;
11    vector<int> bit, rank;
12    vector<ll> vals;
13    vector<Node> t{Node{}};
14
15    DynamicKth(const vector<ll> &a, vector<ll> future = {})
16        : n(a.size()), bit(n + 1), rank(n + 1)
17    {
18        assert(n > 0);
19        vals = future;
20        vals.insert(vals.end(), a.begin(), a.end());
21        sort(vals.begin(), vals.end());
22        vals.erase(unique(vals.begin(), vals.end()), vals.end());
23        for (int i = 1; i <= n; i++)
24        {
25            rank[i] = id(a[i - 1]);
26            modify(i, rank[i], 1);
27        }
28    }
29
30    int id(ll value) const
31    {
32        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
33        assert(p < (int)vals.size() && vals[p] == value);
34        return p;
35    }
36
37    int new_node()
38    {
39        if (!free_head)
```

```

40     {
41         t.push_back(Node{});
42         return (int)t.size() - 1;
43     }
44     int p = free_head;
45     free_head = t[p].l;
46     t[p] = Node{};
47     return p;
48 }
49
50 int update(int p, int l, int r, int x, int delta)
51 {
52     if (!p)
53         p = new_node();
54     t[p].sum += delta;
55     if (l < r)
56     {
57         int m = (l + r) / 2;
58         if (x <= m)
59         {
60             int child = update(t[p].l, l, m, x, delta);
61             t[p].l = child;
62         }
63         else
64         {
65             int child = update(t[p].r, m + 1, r, x, delta);
66             t[p].r = child;
67         }
68     }
69     if (!t[p].sum)
70     {
71         t[p].l = free_head;
72         free_head = p;
73         return 0;
74     }
75     return p;
76 }
77
78 void modify(int pos, int x, int delta)
79 {
80     for (int i = pos; i <= n; i += i & -i)
81         bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
82 }

```

```

84 void set(int pos, ll value)
85 {
86     assert(1 <= pos && pos <= n);
87     int x = id(value);
88     if (rank[pos] == x)
89         return;
90     modify(pos, rank[pos], -1);
91     modify(pos, x, 1);
92     rank[pos] = x;
93 }
94
95 ll kth(int l, int r, int k) const
96 {
97     assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
98     vector<int> a, b;
99     for (int i = r; i; i -= i & -i)
100         a.push_back(bit[i]);
101     for (int i = l - 1; i; i -= i & -i)
102         b.push_back(bit[i]);
103     int L = 0, R = (int)vals.size() - 1;
104     while (L < R)
105     {
106         int count = 0;
107         for (int p : a)
108             count += t[t[p].l].sum;
109         for (int p : b)
110             count -= t[t[p].l].sum;
111         bool left = k <= count;
112         if (!left)
113             k -= count;
114         for (int &p : a)
115             p = left ? t[p].l : t[p].r;
116         for (int &p : b)
117             p = left ? t[p].l : t[p].r;
118         int m = (L + R) / 2;
119         if (left)
120             R = m;
121         else
122             L = m + 1;
123     }
124     return vals[L];
125 }
126 };

```

第七十二章 区间不同数统计

72.1 静态区间不同数计数（前驱位置主席树）

PersistentDistinct(a) 传入非空 0-based 数组，query(l,r) 返回闭区间内不同值的个数， $1 \leq l \leq r \leq n$ 。支持重复值与完整 long long 值域；建成后不支持改值，查询顺序任意。令 prev[i] 为同值上次出现的位置，未出现为 0；root[i] 在值域 0..n-1 中统计前 i 个 prev。区间内每种值恰有一次 prev[i]<l，因此答案为 root[r]-root[l-1] 中位置 0..l-1 的频数和。每个元素只复制一条节点路径，预处理 $O(n \log n)$ ，查询 $O(\log n)$ ，空间 $O(n \log n)$ 。动态版预留 $n \cdot (\text{ceil}(\log_2(n))+1)+1$ 个节点，避免扩容峰值；root 和节点池为内部数据。

```
1 struct PersistentDistinct
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     int n;
9     vector<Node> t{Node{}};
10    vector<int> root;
11
12    PersistentDistinct(const vector<long long> &a) : n(a.size()), root(n + 1)
13    {
14        assert(n > 0);
15        int height = 1;
16        while ((1LL << (height - 1)) < n)
17            height++;
18        t.reserve(size_t(n) * height + 1);
19        vector<long long> vals = a;
20        sort(vals.begin(), vals.end());
21        vals.erase(unique(vals.begin(), vals.end()), vals.end());
22        vector<int> last(vals.size());
23        for (int i = 1; i <= n; i++)
24        {
25            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
26            root[i] = insert(root[i - 1], 0, n - 1, last[x]);
27            last[x] = i;
28        }
29    }
30
31    int insert(int old, int l, int r, int x)
32    {
33        int p = t.size();
34        Node copy = t[old];
35        t.push_back(copy);
36        t[p].sum++;
37        if (l < r)
38        {
39            int m = (l + r) / 2;
40            if (x <= m)
41            {
42                int child = insert(t[old].l, l, m, x);
```

```

43         t[p].l = child;
44     }
45     else
46     {
47         int child = insert(t[old].r, m + 1, r, x);
48         t[p].r = child;
49     }
50 }
51 return p;
52 }
53
54 int query(int l, int r) const
55 {
56     assert(1 <= l && l <= r && r <= n);
57     int u = root[l - 1], v = root[r];
58     int L = 0, R = n - 1, answer = 0;
59     while (L < R)
60     {
61         int m = (L + R) / 2;
62         if (l - 1 <= m)
63         {
64             u = t[u].l;
65             v = t[v].l;
66             R = m;
67         }
68         else
69         {
70             answer += t[t[v].l].sum - t[t[u].l].sum;
71             u = t[u].r;
72             v = t[v].r;
73             L = m + 1;
74         }
75     }
76     return answer + t[v].sum - t[u].sum;
77 }
78 };

```

第七十三章 图论进阶

73.1 矩形二分图最小权匹配

`solve(cost)` 输入 $n \times m$ 完整权值矩阵, $0 \leq n \leq m$; 每行匹配不同列, 返回最小费用及列号 (0-based)。允许负费用, 使用 int128 势能; $O(n^2m)$ 。禁止边不能用未经证明的有限大常数随意替代, 当前接口仅表示完整二分图。

```
1 struct Hungarian
2 {
3     using I = __int128_t;
4
5     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
6     static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
7     {
8         int n = cost.size();
9         if (!n)
10             return {0, {}};
11         int m = cost[0].size();
12         assert(n <= m);
13         vector<I> u(n + 1), v(m + 1);
14         vector<int> p(m + 1), way(m + 1);
15         const I inf = I(1) << 120;
16         for (int i = 1; i <= n; i++)
17         {
18             assert((int)cost[i - 1].size() == m);
19             p[0] = i;
20             int j0 = 0;
21             vector<I> dis(m + 1, inf);
22             vector<bool> used(m + 1);
23             do
24             {
25                 used[j0] = true;
26                 int i0 = p[j0], j1 = 0;
27                 I delta = inf;
28                 for (int j = 1; j <= m; j++)
29                     if (!used[j])
30                     {
31                         I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
32                         if (cur < dis[j])
33                         {
34                             dis[j] = cur;
35                             way[j] = j0;
36                         }
37                         if (dis[j] < delta)
38                         {
39                             delta = dis[j];
40                             j1 = j;
41                         }
42                     }
43                 for (int j = 0; j <= m; j++)
44                     if (used[j])
45                     {
```



```
46         u[p[j]] += delta;
47         v[j] -= delta;
48     }
49     else
50         dis[j] -= delta;
51     j0 = j1;
52 } while (p[j0]);
53 do
54 {
55     int j1 = way[j0];
56     p[j0] = p[j1];
57     j0 = j1;
58 } while (j0);
59 }
60 vector<int> ans(n);
61 for (int j = 1; j <= m; j++)
62     if (p[j])
63         ans[p[j] - 1] = j - 1;
64 return {-v[0], ans};
65 }
66 };
```

73.2 有向最小树形图

`solve(n, root, edges)`, 点号 $0..n-1$, $n \geq 1$, $0 \leq \text{root} < n$; 返回以 `root` 为根向外可达的最小树形图费用, 无解 `nullopt`。允许负权、重边和自环 (自环不选), $n=1$ 的费用为 0; 费用 `-1` 与无解可区分。输入、缩环后边权及累计费用绝对值须小于 2^{120} , 类型为 `int128`。每个非根点选最小入边; 若成环则缩点, 跨环入边减去原终点已选入边费用, 以计算替代代价。 $O(VE)$, `edges` 按值传入, 原图保持不变。仅返回费用, 不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验, 暂无专门 OJ AC。

```

1 struct Arborescence
2 {
3     using I = __int128_t;
4
5     struct Edge
6     {
7         int u, v;
8         I w;
9     };
10
11     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
12     static optional<I> solve(int n, int root, vector<Edge> e)
13     {
14         const I inf = I(1) << 120;
15         I ans = 0;
16         while (true)
17         {
18             vector<I> in(n, inf);
19             vector<int> pre(n, -1), id(n, -1), vis(n, -1);
20             for (auto a : e)
21                 if (a.u != a.v && a.w < in[a.v])
22                 {
23                     in[a.v] = a.w;
24                     pre[a.v] = a.u;
25                 }
26             in[root] = 0;
27             for (I x : in)
28             {
29                 if (x == inf)
30                     return nullopt;
31                 ans += x;
32             }

```

判环与缩点（接上页同一函数）：

```
33     int cnt = 0;
34     for (int i = 0; i < n; i++)
35     {
36         int u = i;
37         while (vis[u] != i && id[u] == -1 && u != root)
38         {
39             vis[u] = i;
40             u = pre[u];
41         }
42         if (u != root && id[u] == -1)
43         {
44             id[u] = cnt;
45             for (int v = pre[u]; v != u; v = pre[v])
46                 id[v] = cnt;
47             ++cnt;
48         }
49     }
50     if (!cnt)
51         return ans;
52     for (int i = 0; i < n; i++)
53         if (id[i] == -1)
54             id[i] = cnt++;
55     for (auto &a : e)
56     {
57         int v = a.v;
58         a.u = id[a.u];
59         a.v = id[a.v];
60         if (a.u != a.v)
61             a.w -= in[v];
62     }
63     root = id[root];
64     n = cnt;
65 }
66 }
67 };
```

73.3 无向全局最小割

`solve(w)`, $n \geq 2$; w 为 $n \times n$ 对称非负权矩阵, 重边先累加, 主对角置零。返回 {割权, 一侧顶点列表}, 编号 $0..n-1$; 两侧非空, 不要求原图连通, 不连通时割权为 0。并列最优割返回其中一种, 不承诺点数或字典序。矩阵按值传入, 原图不变。 $O(n^3)$ 时间、 $O(n^2)$ 空间; 无向边总权 (每条只计一次) 严格小于 2^{120} , 内部 `int128`。负权不适用。已逐条核验返回点集在原图上的跨割费用, 暂无专门 OJ AC。

```

1 struct StoerWagner
2 {
3     using I = __int128_t;
4
5     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
6     static pair<I, vector<int>> solve(vector<vector<I>> w)
7     {
8         int n = w.size();
9         assert(n >= 2);
10        vector<int> active(n);
11        iota(active.begin(), active.end(), 0);
12        vector<vector<int>> group(n);
13        for (int i = 0; i < n; i++)
14            group[i] = {i};
15        I best = I(1) << 120;
16        vector<int> side;
17        while (active.size() > 1)
18        {
19            vector<I> dis(n);
20            vector<bool> used(n);
21            int prev = -1;
22            for (int step = 0; step < (int)active.size(); step++)
23            {
24                int sel = -1;
25                for (int v : active)
26                    if (!used[v] && (sel == -1 || dis[v] > dis[sel]))
27                        sel = v;

```

本轮割、合并与继续扩张（接上页同一函数）：

```
28         if (step + 1 == (int)active.size())
29         {
30             if (dis[sel] < best)
31             {
32                 best = dis[sel];
33                 side = group[sel];
34             }
35             for (int v : active)
36                 if (v != prev && v != sel)
37                     w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
38             group[prev].insert(
39                 group[prev].end(), group[sel].begin(), group[sel].end());
40             active.erase(find(active.begin(), active.end(), sel));
41             break;
42         }
43         used[sel] = true;
44         prev = sel;
45         for (int v : active)
46             if (!used[v])
47                 dis[v] += w[sel][v];
48     }
49 }
50 return {best, side};
51 }
52 };
```

第七十四章 最小割树

74.1 Gomory–Hu 最小割树

gomory_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边; $n \geq 1$, 原图点号 0..n-1, 输入为无向边 tuple 列表, 重边保留、自环忽略、容量非负。非自环边总容量须严格小于 LLONG_MAX。两点最小割是树路径最小边权; 删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边, $n=1$ 返回空树。运行 n-1 次 Dinic, 一般上界 $O(n^3m+n^2)$, 额外空间 $O(n+m)$, DFS 增广保持递归。传统接口 Gomory_Hu(n,edges,flow), 传入 Network_Flow<N,M> 工作区, $N \geq n$, $M \geq 2m$, 建议工作区全局或 static; 每轮 Init。P4897 输入 n 表示 n+1 个点, 驱动传 n+1; 传统工作区取 N=501、M=3000。双版本 P4897 已 AC, 最长单点分别 252、185 ms。

```
1 inline vector<tuple<int, int, long long>>
2 gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
3 {
4     assert(n >= 1);
5     vector<int> parent(n);
6     vector<long long> value(n);
7     for (int s = 1; s < n; s++)
8     {
9         int t = parent[s];
10        Dinic flow(n);
11        for (auto [u, v, w] : edges)
12            if (u != v)
13            {
14                flow.add(u + 1, v + 1, w);
15                flow.add(v + 1, u + 1, w);
16            }
17        long long cut = flow.flow(s + 1, t + 1);
18        vector<bool> side(n);
19        for (int u : flow.cut(s + 1))
20            side[u - 1] = true;
21        for (int v = 1; v < n; v++)
22            if (v != s && parent[v] == t && side[v])
23                parent[v] = s;
24        if (side[parent[t]])
25        {
26            parent[s] = parent[t];
27            parent[t] = s;
28            value[s] = value[t];
29            value[t] = cut;
30        }
31        else
32            value[s] = cut;
33    }
34    vector<tuple<int, int, long long>> tree;
35    for (int v = 1; v < n; v++)
36        tree.push_back({v, parent[v], value[v]});
37    return tree;
38 }
```

第七十五章 最小割树查询

75.1 最小割树全点对查询

`cut_tree_values(n, tree)` 对合法的连通无环加权树计算全点对路径最小边权；点号 $0..n-1$ 、 $n \geq 1$ 、恰有 $n-1$ 条边，边权为非负 `long long`。输入可直接使用 `gomory_hu` 的返回值。结果 `ans[u][v]` 在 $u \neq v$ 时为两点最小割；对角线为 `LLONG_MAX` 哨兵，不能解释成同点最小割。 $n=1$ 返回单个哨兵。 $O(n^2)$ 时间与空间，递归 DFS 深度 $O(n)$ ，查询 $O(1)$ 。传统接口 `Cut_Tree_Values`。只需要标准库，不依赖流模板；不是任意图的查询函数。

```
1 inline vector<vector<long long>>
2 cut_tree_values(int n, const vector<tuple<int, int, long long>> &tree)
3 {
4     assert(n >= 1 && tree.size() == (size_t)n - 1);
5     vector<vector<pair<int, long long>>> g(n);
6     for (auto [u, v, w] : tree)
7     {
8         assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
9         g[u].push_back({v, w});
10        g[v].push_back({u, w});
11    }
12    vector<vector<long long>> answer(n, vector<long long>(n, LLONG_MAX));
13    for (int s = 0; s < n; s++)
14    {
15        auto dfs = [&](auto &&self, int u, int parent, long long value) -> void
16        {
17            answer[s][u] = value;
18            for (auto [v, w] : g[u])
19                if (v != parent)
20                    self(self, v, u, min(value, w));
21        };
22        dfs(dfs, s, -1, LLONG_MAX);
23    }
24    return answer;
25 }
```

第七十六章 带权二分图匹配

76.1 二分图最大权匹配（缺边与可选匹配）

WeightedMatching(n,m), 两侧编号 1..n、1..m。add(x,y,w) 加边，重边保留最大权；边权支持完整 long long 范围，内部势能与答案为 int128。solve() 要求全部左点匹配不同右点，无解返回 nullopt；n=m 时即最大权完美匹配。solve(true) 允许点不匹配，只最大化总权，不优先最大化边数。两种模式均从空匹配重算；l[x]、r[y] 为方案，0 表示未匹配，无解时全部清零。缺边用独立于合法边权的 -2^{120} 哨兵，仅用 add 修改；不把缺边当零权边。允许空侧。全匹配 $O(n^2m)$ ，可选模式增加 n 个虚拟零权右点，为 $O(n^2(m+n))$ ；空间 $O(nm+n+m)$ 。按最短松弛量扩展交错树：每次至少访问一个新右点，最多 n 次后增广；无有限松弛量即无解。

```
1 struct WeightedMatching
2 {
3     using I = __int128_t;
4     static constexpr I inf = I(1) << 120;
5     int n, m;
6     vector<vector<I>> w;
7     vector<int> l, r;
8
9     WeightedMatching(int n, int m)
10         : n(n), m(m), w(n + 1, vector<I>(m + 1, -inf)), l(n + 1), r(m + 1)
11     {
12     }
13
14     void add(int x, int y, long long weight)
15     {
16         assert(1 <= x && x <= n && 1 <= y && y <= m);
17         w[x][y] = max(w[x][y], I(weight));
18     }
19
20     optional<I> solve(bool allow_unmatched = false)
21     {
22         fill(l.begin(), l.end(), 0);
23         fill(r.begin(), r.end(), 0);
24         if (!allow_unmatched && n > m)
25             return nullopt;
26         int cols = m + (allow_unmatched ? n : 0);
27         vector<I> u(n + 1), v(cols + 1);
28         vector<int> p(cols + 1), way(cols + 1);
29         for (int i = 1; i <= n; i++)
30         {
31             p[0] = i;
32             int j0 = 0;
33             vector<I> dis(cols + 1, inf);
34             vector<bool> used(cols + 1);
35             do
36             {
37                 used[j0] = true;
38                 int x = p[j0], j1 = 0;
39                 I delta = inf;
40                 for (int j = 1; j <= cols; j++)
41                 {
```



```

42         if (used[j])
43             continue;
44         I weight = j <= m ? w[x][j] : 0;
45         if (weight != -inf)
46         {
47             I cur = -weight - u[x] - v[j];
48             if (cur < dis[j])
49             {
50                 dis[j] = cur;
51                 way[j] = j0;
52             }
53         }
54         if (dis[j] < delta)
55         {
56             delta = dis[j];
57             j1 = j;
58         }
59     }
60     if (delta == inf)
61         return nullopt;
62     for (int j = 0; j <= cols; j++)
63     {
64         if (used[j])
65         {
66             u[p[j]] += delta;
67             v[j] -= delta;
68         }
69         else if (dis[j] != inf)
70             dis[j] -= delta;
71     }
72     j0 = j1;
73 } while (p[j0]);
74 do
75 {
76     int j1 = way[j0];
77     p[j0] = p[j1];
78     j0 = j1;
79 } while (j0);
80 }
81 I answer = 0;
82 for (int j = 1; j <= m; j++)
83 {
84     if (!p[j])
85         continue;
86     l[p[j]] = j;
87     r[j] = p[j];
88     answer += w[p[j]][j];
89 }
90 return answer;
91 }
92 };

```

第七十七章 精确有界半平面交

77.1 整数半平面的有界多边形交

`bounded_polygon(lines)` 求 $a*x+b*y \leq c$ 的交集闭包顶点, $\text{Line}=\{a,b,c\}$, 三个整数系数绝对值均 $\leq 10^9$ 。要求交集有界, 空集也允许; 只返回正面积多边形, 点或线段交集返回空向量。无界输入不在接口范围内。 $\text{Point}=\{x,y,d\}$ 表示 $(x/d,y/d)$, $d>0$, 分数不必约分; 按逆时针返回, 不重复首点, 相邻重复点已去除。排序与所有构造、判定使用整数和 `int128`, 不使用 `epsilon`。 $O(n \log n)$ 时间、 $O(n)$ 空间。传统接口 `Integer_Halfplanes::Bounded_Polygon`。浮点转换与面积计算需单独处理。

```
1 struct IntegerHalfplanes
2 {
3     using I = __int128_t;
4     using ll = long long;
5
6     struct Line
7     {
8         ll a, b, c;
9     };
10
11     struct Point
12     {
13         ll x, y, d;
14     };
15
16     static I cross(Line u, Line v)
17     {
18         return I(u.a) * v.b - I(u.b) * v.a;
19     }
20
21     static int half(Line u)
22     {
23         return u.b < 0 || (u.b == 0 && u.a < 0);
24     }
25
26     static Point meet(Line u, Line v)
27     {
28         I d = cross(u, v);
29         assert(d != 0);
30         I x = I(u.c) * v.b - I(u.b) * v.c;
31         I y = I(u.a) * v.c - I(u.c) * v.a;
32         if (d < 0)
33         {
34             d = -d;
35             x = -x;
36             y = -y;
37         }
38         return {(ll)x, (ll)y, (ll)d};
39     }
```

接上页同一结构体与函数：

```

41     static I value(Line u, Point p)
42     {
43         return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
44     }
45
46     static bool same(Point p, Point q)
47     {
48         return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
49     }
50
51     static vector<Point> bounded_polygon(vector<Line> lines)
52     {
53         vector<Line> h;
54         for (auto u : lines)
55         {
56             assert(-10000000000 <= u.a && u.a <= 10000000000);
57             assert(-10000000000 <= u.b && u.b <= 10000000000);
58             assert(-10000000000 <= u.c && u.c <= 10000000000);
59             if (!u.a && !u.b)
60             {
61                 if (u.c < 0)
62                     return {};
63             }
64             else
65                 h.push_back(u);
66         }
67         sort(h.begin(),
68             h.end(),
69             [](Line u, Line v)
70             {
71                 if (half(u) != half(v))
72                     return half(u) < half(v);
73                 I d = cross(u, v);
74                 if (d)
75                     return d > 0;
76                 ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
77                 ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
78                 return I(u.c) * y < I(v.c) * x;
79             });

```

接上页同一结构体与函数：

```

80     deque<Line> q;
81     for (int i = 0; i < (int)h.size(); i++)
82     {
83         Line u = h[i];
84         if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0)
85             continue;
86         while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
87             q.pop_back();
88         while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0)
89             q.pop_front();
90         if (!q.empty() && cross(q.back(), u) == 0)
91             return {};
92         q.push_back(u);
93     }
94     while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
95         q.pop_back();
96     while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0)
97         q.pop_front();
98     if (q.size() < 3 || cross(q.front(), q.back()) == 0)
99         return {};
100     vector<Point> answer;
101     for (int i = 0; i < (int)q.size(); i++)
102     {
103         Point p = meet(q[i], q[(i + 1) % q.size()]);
104         if (answer.empty() || !same(answer.back(), p))
105             answer.push_back(p);
106     }
107     if (answer.size() > 1 && same(answer.front(), answer.back()))
108         answer.pop_back();
109     if (answer.size() < 3)
110         return {};
111     return answer;
112 }
113 };

```

第七十八章 圆与多边形面积

78.1 圆与简单多边形相交面积

`area(points,circle)` 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。`edge(a,b,r)` 返回圆心在 origin 时有向边 `a` 到 `b` 的面积贡献，`r` 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和， $O(n)$ 时间、 $O(1)$ 额外空间。依赖浮点几何的 `Point/Circle` 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。传统接口 `Circle_Polygon::Area`、`Edge`。

```
1 struct CirclePolygon
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     // Signed contribution of edge a -> b; circle centered at the origin.
9     static R edge(Point a, Point b, R r)
10    {
11        if (r == 0)
12            return 0;
13        Point v = b - a;
14        R length = G::norm(v);
15        if (length == 0)
16            return 0;
17        Point u = v / length;
18        R h = G::cross(u, a);
19        Point cuts[4] = {a};
20        int count = 1;
21        if (fabs(h) <= r)
22        {
23            R delta = sqrtl(max(R(0), (r - fabs(h)) * (r + fabs(h))));
24            R t = -G::dot(a, u);
25            Point foot = G::perp(u) * h;
26            for (R s : {-delta, delta})
27                if (t + s > 0 && t + s < length)
28                    cuts[count++] = foot + u * s;
29        }
30        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```

31     R answer = 0;
32     for (int i = 1; i < count; i++)
33     {
34         Point p = cuts[i - 1], q = cuts[i];
35         R cross = G::cross(p, q);
36         if (G::norm((p + q) / 2) <= r)
37             answer += cross / 2;
38         else
39             answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
40     }
41     return answer;
42 }
43
44 // Simple polygon, either orientation; duplicate consecutive vertices allowed.
45 static R area(const vector<Point> &p, Circle c)
46 {
47     assert(c.r >= 0);
48     if (p.size() < 3 || c.r == 0)
49         return 0;
50     R answer = 0, error = 0;
51     for (int i = 0; i < (int)p.size(); i++)
52     {
53         R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
54         R y = value - error;
55         R sum = answer + y;
56         error = (sum - answer) - y;
57         answer = sum;
58     }
59     return fabs1(answer);
60 }
61 };

```

第七十九章 最小覆盖圆

79.1 随机增量最小覆盖圆

`solve(points,rng,eps)` 返回 `optional<Circle>`, `Circle={o,r}`; 空集返回 `nullopt`, 单点或重复同一点返回半径零。`rng` 是调用者提供的 `mt19937_64` 随机数引擎。期望 $O(n)$ 时间, 最坏 $O(n^3)$, $O(n)$ 空间; 随机化不可省略。`eps` 默认 32 倍 long double 机器精度, 用于相对半径的包含判断。`diameter(a,b)` 求直径圆; `through(a,b,c)` 求外接圆, 共线时返回最远点对的直径圆。依赖浮点几何的 `Point/Circle` 与向量运算。输入及中间运算须有限且尺度适当; 接近共线时圆心可能不稳定, 无法恢复读入前的精度。传统接口 `Enclosing_Circle::Solve`、`Diameter`、`Through`。

```
1 struct EnclosingCircle
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     static Circle diameter(Point a, Point b)
9     {
10         Point o = a + (b - a) / 2;
11         return {o, max(G::norm(o - a), G::norm(o - b))};
12     }
13
14     static Circle through(Point a, Point b, Point c)
15     {
16         Point u = b - a, v = c - a;
17         R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
18         if (scale == 0)
19             return {a, 0};
20         u = u / scale;
21         v = v / scale;
22         R det = 2 * G::cross(u, v);
23         if (det == 0)
24         {
25             Circle best = diameter(a, b);
26             for (Circle q : {diameter(a, c), diameter(b, c)})
27                 if (q.r > best.r)
28                     best = q;
29             return best;
30         }
31         R x = G::dot(u, u), y = G::dot(v, v);
32         Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
33         Point o = a + offset * scale;
34         return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
35     }
36 }
```

随机增量主过程（接上页同一结构体）：

```

37     static optional<Circle>
38     solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
39     {
40         assert(eps >= 0 && eps < 1);
41         if (p.empty())
42             return nullopt;
43         shuffle(p.begin(), p.end(), rng);
44         Circle c{p[0], 0};
45         auto outside = [&](Point q)
46         {
47             return G::norm(q - c.o) > c.r * (1 + eps);
48         };
49         for (int i = 1; i < (int)p.size(); i++)
50             if (outside(p[i]))
51             {
52                 c = {p[i], 0};
53                 for (int j = 0; j < i; j++)
54                     if (outside(p[j]))
55                     {
56                         c = diameter(p[i], p[j]);
57                         for (int k = 0; k < j; k++)
58                             if (outside(p[k]))
59                                 c = through(p[i], p[j], p[k]);
60                     }
61             }
62         return c;
63     }
64 };

```


第八十章 圆的切线

80.1 两圆公切线与过点切线

`solve(a,b,eps)` 返回 `Result`, 包含 `infinite` 和 `lines`; `infinite` 为真表示无穷多条, 否则 `lines` 给出全部切线。每条 `Tangent` 保存切点 `a`、`b`, 单位法向量 `normal`, 以及 `side`。`a`、`b` 对应输入两圆, 直线方程 $\text{dot}(\text{normal}, x-a)=0$; 切点相同时仍有效。`side=1` 外公切线、`-1` 内公切线、`0` 两个不同点。半径零表示点, `from_point(p,c,eps)` 等价于第一圆取点, 圆上切点读 `b`。输入有限, 坐标及半径绝对值 $\leq 1e100$, 半径非负。`eps` 默认 `1e-12`, 用于相对圆心距的近相切判断; 近相切可能合并为一条, 浮点坐标仍有舍入误差。严格同心等半径时 `infinite=true`, 包括同一点; 同心不等半径返回空。`O(1)`; 依赖浮点几何的 `Point/Circle`、点运算及 `norm/perp`。传统接口 `Circle_Tangents::Solve`、`From_Point`。

```
1 struct CircleTangents
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     struct Tangent
9     {
10         Point a, b, normal;
11         int side; // 1 outer, -1 inner, 0 two distinct point circles.
12     };
13
14     struct Result
15     {
16         bool infinite = false;
17         vector<Tangent> lines;
18     };
19
20     // Line: dot(normal, x - a) = 0. Radius zero represents a point.
21     static Result solve(Circle a, Circle b, R eps = 1e-12L)
22     {
23         assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
24         Point v = b.o - a.o;
25         R d = G::norm(v);
26         if (d == 0)
27             return {a.r == b.r, {}};
28         Point u = v / d;
29         if (a.r == 0 && b.r == 0)
30             return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```

31     Result answer;
32     for (int side : {1, -1})
33     {
34         if (side == -1 && (a.r == 0 || b.r == 0))
35             continue;
36         R gap = a.r - side * b.r;
37         if (fabsl(gap) > d * (1 + eps))
38             continue;
39         bool touch = fabsl(gap) >= d * (1 - eps);
40         R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
41         R s = sqrtl(max(R(0), (1 - c) * (1 + c)));
42         for (int sign : {-1, 1})
43         {
44             Point normal = u * c + G::perp(u) * (sign * s);
45             Point p = a.o + normal * a.r;
46             Point q = b.o + normal * (side * b.r);
47             answer.lines.push_back({p, q, normal, side});
48             if (touch)
49                 break;
50         }
51     }
52     return answer;
53 }
54
55 static Result from_point(Point p, Circle c, R eps = 1e-12L)
56 {
57     return solve({p, 0}, c, eps);
58 }
59 };

```

第八十一章 浮点最近点对

81.1 浮点平面最近点对

`solve(points)` 返回 long double 最短距离 (不是平方), 少于两个点返回 `nullopt`, 重复点返回零。Point={x,y}; 输入须有限, 坐标绝对值 $\leq 1e100$ 。递归分治, 按纵坐标归并, $O(n \log n)$ 时间、 $O(n)$ 额外空间。用 `hypotl` 求距离, 不先平方; 排序不使用 `epsilon`。浮点运算仍有舍入误差, 输入前已丢失的精度无法恢复。传统接口 `Closest_Pair::Solve`。

```
1 struct ClosestPair
2 {
3     using R = long double;
4
5     struct Point
6     {
7         R x, y;
8     };
9
10    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
11    static optional<R> solve(vector<Point> p)
12    {
13        int n = p.size();
14        if (n < 2)
15            return nullopt;
16        sort(p.begin(),
17             p.end(),
18             [](Point a, Point b)
19             {
20                 return tie(a.x, a.y) < tie(b.x, b.y);
21             });
22        for (int i = 1; i < n; i++)
23            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y)
24                return R(0);
25        auto by_y = [](Point a, Point b)
26        {
27            return tie(a.y, a.x) < tie(b.y, b.x);
28        };
29        auto distance = [](Point a, Point b)
30        {
31            return hypotl(a.x - b.x, a.y - b.y);
32        };
33        vector<Point> tmp(n);
34        function<R(int, int)> dfs = [&](int l, int r) -> R
35        {
36            if (r - l <= 3)
37            {
38                R best = numeric_limits<R>::infinity();
39                for (int i = l; i < r; i++)
40                    for (int j = l; j < i; j++)
41                        best = min(best, distance(p[i], p[j]));
42                sort(p.begin() + l, p.begin() + r, by_y);
43                return best;
44            }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
45     int m = (l + r) / 2;
46     R x = p[m].x;
47     R best = min(dfs(l, m), dfs(m, r));
48     merge(p.begin() + l,
49           p.begin() + m,
50           p.begin() + m,
51           p.begin() + r,
52           tmp.begin() + l,
53           by_y);
54     copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
55     vector<Point> strip;
56     for (int i = l; i < r; i++)
57         if (fabs1(p[i].x - x) <= best)
58         {
59             for (int j = (int)strip.size() - 1; j >= 0; j--)
60             {
61                 if (p[i].y - strip[j].y > best)
62                     break;
63                 best = min(best, distance(p[i], strip[j]));
64             }
65             strip.push_back(p[i]);
66         }
67     return best;
68 };
69 return dfs(0, n);
70 }
71 };
```

第八十二章 精确几何进阶

82.1 整数最近点对与 Minkowski 和

`closest_pair(points)` 返回最短距离平方，少于两个点为 `nullopt`，重复点返回 0， $O(n \log n)$ ，全程精确整数。
`minkowski` 输入 strict CCW 凸包，返回不重复首点的凸包；非退化 $O(n+m)$ ，退化时 $O((n+m)\log(n+m))$ 。坐标及坐标和须满足整数几何范围。

```
1 struct GeometryExtra
2 {
3     using G = IntegerGeometry;
4     using P = G::Point;
5     using I = G::I;
6
7     // No pair exists for n<2. Exact squared distance,  $O(n \log n)$ .
8     static optional<I> closest_pair(vector<P> p)
9     {
10         int n = p.size();
11         if (n < 2)
12             return nullopt;
13         sort(p.begin(), p.end());
14         for (int i = 1; i < n; i++)
15             if (p[i] == p[i - 1])
16                 return I(0);
17         vector<P> tmp(n);
18         auto by_y = [](P a, P b)
19         {
20             return tie(a.y, a.x) < tie(b.y, b.x);
21         };
22         function<I(int, int)> solve = [&](int l, int r) -> I
23         {
24             if (r - l <= 3)
25             {
26                 I best = I(1) << 120;
27                 for (int i = l; i < r; i++)
28                     for (int j = i + 1; j < r; j++)
29                         best = min(best, G::dist2(p[i], p[j]));
30                 sort(p.begin() + l, p.begin() + r, by_y);
31                 return best;
32             }
33             int m = (l + r) / 2;
34             long long x = p[m].x;
35             I best = min(solve(l, m), solve(m, r));
36             merge(p.begin() + l,
37                 p.begin() + m,
38                 p.begin() + m,
39                 p.begin() + r,
40                 tmp.begin() + l,
41                 by_y);
42             copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
43             vector<P> strip;
44             for (int i = l; i < r; i++)
```

```

45         if ((I(p[i].x) - x) * (I(p[i].x) - x) < best)
46         {
47             for (int j = (int)strip.size() - 1; j >= 0; j--)
48             {
49                 I dy = I(p[i].y) - strip[j].y;
50                 if (dy * dy >= best)
51                     break;
52                 best = min(best, G::dist2(p[i], strip[j]));
53             }
54             strip.push_back(p[i]);
55         }
56         return best;
57     };
58     return solve(0, n);
59 }
60
61 // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
62 static vector<P> minkowski(vector<P> a, vector<P> b)
63 {
64     if (a.empty() || b.empty())
65         return {};
66     auto plus = [](P u, P v)
67     {
68         return P{u.x + v.x, u.y + v.y};
69     };
70     if (a.size() < 3 || b.size() < 3)
71     {
72         vector<P> p;
73         for (P u : a)
74             for (P v : b)
75                 p.push_back(plus(u, v));
76         return G::hull(p);
77     }
78     auto start = [](vector<P> &p)
79     {
80         auto it = min_element(p.begin(),
81                               p.end(),
82                               [](P u, P v)
83                               {
84                                   return tie(u.y, u.x) < tie(v.y, v.x);
85                               });
86         rotate(p.begin(), it, p.end());
87     };
88     start(a);
89     start(b);
90     int n = a.size(), m = b.size(), i = 0, j = 0;
91     vector<P> result;
92     while (i < n || j < m)
93     {
94         result.push_back(plus(a[i % n], b[j % m]));
95         if (i == n)
96         {
97             ++j;
98             continue;
99         }
100         if (j == m)

```

```
101         {
102             ++i;
103             continue;
104         }
105         P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
106         P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
107         I c = G::cross({0, 0}, u, v);
108         if (c >= 0)
109             ++i;
110         if (c <= 0)
111             ++j;
112     }
113     return result;
114 }
115 };
```

82.2 整数三维共线、共面与方向

坐标绝对值 $\leq 10^9$ ；orient(a,b,c,d) 为有向六倍四面体体积，0 表示共面；collinear 判三点共线，on_segment 支持退化线段。int128 精确判定，不包含三维凸包构造。

```

1 struct IntegerGeometry3D
2 {
3     using I = __int128_t;
4
5     // |coordinates|<=1e9: triple products fit signed int128.
6     struct Point
7     {
8         long long x, y, z;
9     };
10
11     struct Vector
12     {
13         I x, y, z;
14     };
15
16     static Vector diff(Point a, Point b)
17     {
18         return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
19     }
20
21     static Vector cross(Vector a, Vector b)
22     {
23         return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
24     }
25
26     static I dot(Vector a, Vector b)
27     {
28         return a.x * b.x + a.y * b.y + a.z * b.z;
29     }
30
31     static I orient(Point a, Point b, Point c, Point d)
32     {
33         return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
34     }
35
36     static bool collinear(Point a, Point b, Point c)
37     {
38         auto v = cross(diff(b, a), diff(c, a));
39         return !v.x && !v.y && !v.z;
40     }
41
42     static bool on_segment(Point p, Point a, Point b)
43     {
44         return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
45     }
46 };

```


第八十三章 凸包支撑点查询

83.1 支撑点查询恢复下凸包

静态 `build(left,right,oracle)` 返回从左到右的严格下凸包顶点, `Point` 为 $\{x,y\}$ 。有限非空整数点集无需显式给出; 回调 `oracle(a,b)` 必须精确返回使 $a*x+b*y$ 最大的可行点, 允许任意最优解。 `left/right` 必须分别为最小/最大横坐标中纵坐标最低的点; 全体同横坐标时二者必须相同。所有坐标绝对值不超过 10^{18} , 方向差使用 `long long`, 叉积先提升至 `int128`; 回调内部也须保证数值精确且无溢出。对弦 $a \rightarrow b$ 查询方向 $(b.y-a.y, a.x-b.x)$, 严格位于弦下方则递归分治, 否则确认边; 输出时删除共线中间点。若分治找到 K 个严格改善支撑点, 查询次数为 $2K+1$ (同横坐标时为 0), 额外时间、空间和最坏递归深度为 $O(K+1)$ 。 K 包括可能返回的共线非顶点, 不将其等同于最终顶点数。不能直接传浮点近似优化器。二分图闭包组合: 点为 $(|S|, |N(S)|)$, 左点赋权 a 、右点赋权 b , 左 $\rightarrow$ 邻点依赖; 下凸包方向满足 $a \geq 0, b < 0$, 最优闭包的右点恰为邻集。端点为 $(0,0)$ 和 $(\text{左点数}, \text{全部左点的邻点数})$, 有孤立右点时后者不等于全部右点数。该组合已做子集枚举对拍, 尚不代表关联赛题整题通过。

```
1 struct SupportHull
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Point
7     {
8         ll x, y;
9     };
10
11     static I cross(Point a, Point b, Point c)
12     {
13         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
14     }
```

```

16 // oracle(a,b) maximizes a*x+b*y exactly; |coordinates| <= 1e18.
17 // left/right: lowest points at the minimum/maximum x.
18 template <class Oracle>
19 static vector<Point> build(Point left, Point right, Oracle &&oracle)
20 {
21     assert(left.x <= right.x);
22     vector<Point> hull{left};
23     auto dfs = [&](auto &&self, Point a, Point b) -> void
24     {
25         Point c = oracle(b.y - a.y, a.x - b.x);
26         I turn = cross(a, b, c);
27         assert(turn <= 0);
28         if (turn < 0)
29         {
30             assert(a.x < c.x && c.x < b.x);
31             self(self, a, c);
32             self(self, c, b);
33         }
34         else
35         {
36             while (hull.size() >= 2 &&
37                 cross(hull[hull.size() - 2], hull.back(), b) == 0)
38                 hull.pop_back();
39             hull.push_back(b);
40         }
41     };
42     if (left.x < right.x)
43         dfs(dfs, left, right);
44     else
45         assert(left.y == right.y);
46     return hull;
47 }
48 };

```

第八十四章 动态树

84.1 Link-cut tree: 动态森林与路径异或

构造 LinkCutTree(n), 点号 1..n, 初始权值为零。set(x,value) 修改点权; link(x,y) 仅在不同连通块之间连边并返回是否成功; cut(x,y) 只删除真实存在的边, 否则返回 false。connected 查询连通性; query(x,y) 返回路径点权异或, 要求两点连通, 支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 $O(\log n)$, $O(n)$ 空间, splay 下传使用显式栈, 支持长链。权值为 unsigned long long。仅维护路径异或, 不包含虚子树聚合或动态直径。

```
1 struct LinkCutTree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    vector<Node> a;
13    vector<int> st;
14
15    LinkCutTree(int n) : a(n + 1), st(n + 1)
16    {
17    }
18
19    bool is_root(int x) const
20    {
21        int f = a[x].fa;
22        return a[f].ch[0] != x && a[f].ch[1] != x;
23    }
24
25    void pull(int x)
26    {
27        a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum;
28    }
29
30    void reverse(int x)
31    {
32        if (!x)
33            return;
34        swap(a[x].ch[0], a[x].ch[1]);
35        a[x].rev = !a[x].rev;
36    }
37
38    void push(int x)
39    {
40        if (!a[x].rev)
41            return;
42        reverse(a[x].ch[0]);
```

```

43     reverse(a[x].ch[1]);
44     a[x].rev = false;
45 }
46
47 void rotate(int x)
48 {
49     int y = a[x].fa, z = a[y].fa;
50     int k = a[y].ch[1] == x;
51     int w = a[x].ch[k ^ 1];
52     if (!is_root(y))
53         a[z].ch[a[z].ch[1] == y] = x;
54     a[x].fa = z;
55     a[y].ch[k] = w;
56     if (w)
57         a[w].fa = y;
58     a[x].ch[k ^ 1] = y;
59     a[y].fa = x;
60     pull(y);
61     pull(x);
62 }
63
64 void splay(int x)
65 {
66     int top = 0, y = x;
67     st[top++] = y;
68     while (!is_root(y))
69     {
70         y = a[y].fa;
71         st[top++] = y;
72     }
73     while (top)
74         push(st[--top]);
75     while (!is_root(x))
76     {
77         int y = a[x].fa, z = a[y].fa;
78         if (!is_root(y))
79         {
80             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
81             rotate(same ? y : x);
82         }
83         rotate(x);
84     }
85 }
86
87 void access(int x)
88 {
89     int last = 0;
90     for (int y = x; y; y = a[y].fa)
91     {
92         splay(y);
93         // The old right child's fa remains y: it becomes a path-parent.
94         a[y].ch[1] = last;
95         if (last)
96             a[last].fa = y;
97         pull(y);
98         last = y;

```

```

99     }
100     splay(x);
101 }
102
103 void make_root(int x)
104 {
105     access(x);
106     reverse(x);
107 }
108
109 int find_root(int x)
110 {
111     access(x);
112     push(x);
113     while (a[x].ch[0])
114     {
115         x = a[x].ch[0];
116         push(x);
117     }
118     splay(x);
119     return x;
120 }
121
122 bool connected(int x, int y)
123 {
124     return x == y || find_root(x) == find_root(y);
125 }
126
127 bool link(int x, int y)
128 {
129     make_root(x);
130     if (find_root(y) == x)
131         return false;
132     a[x].fa = y;
133     return true;
134 }
135
136 bool cut(int x, int y)
137 {
138     make_root(x);
139     access(y);
140     if (a[y].ch[0] != x || a[x].ch[1])
141         return false;
142     a[y].ch[0] = 0;
143     a[x].fa = 0;
144     pull(y);
145     return true;
146 }
147
148 void set(int x, U value)
149 {
150     access(x);
151     a[x].val = value;
152     pull(x);

```

```
153     }
154
155     // x and y must be connected. This is a path aggregate, not a subtree sum.
156     U query(int x, int y)
157     {
158         make_root(x);
159         access(y);
160         return a[y].sum;
161     }
162 };
```

第八十五章 随机平衡树

85.1 FHQ Treap: 可重集合与顺序统计

`insert(x)` 插入一个副本, `erase(x)` 删除一个副本并返回是否存在; `rank(x)` 为严格小于 `x` 的元素数加一, `x` 不必存在; `kth(k)` 要求 $1 \leq k \leq \text{size}()$ 。 `prev/next` 为严格前驱/后继, 不存在时返回 `nullopt`。支持重复值、`LLONG_MIN` 和 `LLONG_MAX`, 分裂不使用 `x-1`。随机优先级下单次期望 $O(\log n)$, 并非确定性最坏界。节点池不回收已删除节点, 空间按累计插入次数计算; 可在构造时指定随机种子。

```
1 struct OrderedTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    mt19937_64 rng;
14    int root = 0;
15
16    OrderedTreap(unsigned long long seed = 712367821) : rng(seed)
17    {
18    }
19
20    void pull(int p)
21    {
22        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
23    }
24
25    int node(ll x)
26    {
27        int p = a.size();
28        a.push_back({0, 0, 1, x, rng()});
29        return p;
30    }
31
32    // left: val < x, or val <= x when equal is true. No x-1 overflow.
33    void split(int p, ll x, int &l, int &r, bool equal = false)
34    {
35        if (!p)
36        {
37            l = 0;
38            r = 0;
39            return;
40        }
41        if (a[p].val < x || (equal && a[p].val == x))
42        {
43            l = p;
```

```

44         split(a[p].r, x, a[p].r, r, equal);
45     }
46     else
47     {
48         r = p;
49         split(a[p].l, x, l, a[p].l, equal);
50     }
51     pull(p);
52 }
53
54 int merge(int l, int r)
55 {
56     if (!l || !r)
57         return l ? l : r;
58     if (a[l].pri > a[r].pri)
59     {
60         a[l].r = merge(a[l].r, r);
61         pull(l);
62         return l;
63     }
64     a[r].l = merge(l, a[r].l);
65     pull(r);
66     return r;
67 }
68
69 void insert(ll x)
70 {
71     int p = node(x), l, r;
72     split(root, x, l, r);
73     root = merge(merge(l, p), r);
74 }
75
76 bool erase(ll x)
77 {
78     int l, mid, r;
79     split(root, x, l, mid);
80     split(mid, x, mid, r, true);
81     bool found = mid != 0;
82     if (mid)
83         mid = merge(a[mid].l, a[mid].r);
84     root = merge(l, merge(mid, r));
85     return found;
86 }
87
88 int size() const
89 {
90     return a[root].siz;
91 }
92
93 int less(ll x, bool equal = false) const
94 {
95     int p = root, ans = 0;
96     while (p)
97     {
98         if (a[p].val < x || (equal && a[p].val == x))
99             {

```



```

100         ans += a[a[p].l].siz + 1;
101         p = a[p].r;
102     }
103     else
104         p = a[p].l;
105     }
106     return ans;
107 }
108
109 int rank(ll x) const
110 {
111     return less(x) + 1;
112 }
113
114 ll kth(int k) const
115 {
116     assert(1 <= k && k <= size());
117     int p = root;
118     while (true)
119     {
120         int left = a[a[p].l].siz;
121         if (k == left + 1)
122             return a[p].val;
123         if (k <= left)
124             p = a[p].l;
125         else
126         {
127             k -= left + 1;
128             p = a[p].r;
129         }
130     }
131 }
132
133 optional<ll> prev(ll x) const
134 {
135     int k = less(x);
136     if (!k)
137         return nullopt;
138     return kth(k);
139 }
140
141 optional<ll> next(ll x) const
142 {
143     int k = less(x, true);
144     if (k == size())
145         return nullopt;
146     return kth(k + 1);
147 }
148 };

```

85.2 隐式 Treap：序列翻转、加值与区间和

`insert(k,value)` 在前 k 个元素后插入, $0 \leq k \leq \text{size}()$; `erase(l,r)`、`reverse(l,r)`、`add(l,r,value)`、`query(l,r)` 均使用 1-based 非空闭区间。`values()` 返回当前完整序列, $O(n)$, 使用递归中序遍历。其余操作在随机优先级下期望 $O(\log$

n); 翻转与加法懒标记同时维护。所有值、和、标记及 $\text{value} \times \text{长度}$ 中间结果必须落在 `long long` 范围内。删除不回收节点，内存按累计插入数计算。

```

1 struct SequenceTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0, sum = 0, tag = 0;
9         unsigned long long pri = 0;
10        bool rev = false;
11    };
12
13    vector<Node> a{Node{}};
14    mt19937_64 rng;
15    int root = 0;
16
17    SequenceTreap(unsigned long long seed = 712367821) : rng(seed)
18    {
19    }
20
21    int size() const
22    {
23        return a[root].siz;
24    }
25
26    void pull(int p)
27    {
28        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
29        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
30    }
31
32    void apply_add(int p, ll x)
33    {
34        if (!p)
35            return;
36        a[p].val += x;
37        a[p].sum += x * a[p].siz;
38        a[p].tag += x;
39    }
40
41    void apply_reverse(int p)
42    {
43        if (!p)
44            return;
45        swap(a[p].l, a[p].r);
46        a[p].rev = !a[p].rev;
47    }
48
49    void push(int p)
50    {
51        if (a[p].rev)
52        {
53            apply_reverse(a[p].l);

```

```

54         apply_reverse(a[p].r);
55         a[p].rev = false;
56     }
57     if (a[p].tag)
58     {
59         apply_add(a[p].l, a[p].tag);
60         apply_add(a[p].r, a[p].tag);
61         a[p].tag = 0;
62     }
63 }
64
65 // Split into the first k elements and the rest.
66 void split(int p, int k, int &l, int &r)
67 {
68     if (!p)
69     {
70         l = 0;
71         r = 0;
72         return;
73     }
74     push(p);
75     int left = a[a[p].l].siz;
76     if (k > left)
77     {
78         l = p;
79         split(a[p].r, k - left - 1, a[p].r, r);
80     }
81     else
82     {
83         r = p;
84         split(a[p].l, k, l, a[p].l);
85     }
86     pull(p);
87 }
88
89 int merge(int l, int r)
90 {
91     if (!l || !r)
92         return l ? l : r;
93     if (a[l].pri > a[r].pri)
94     {
95         push(l);
96         a[l].r = merge(a[l].r, r);
97         pull(l);
98         return l;
99     }
100    push(r);
101    a[r].l = merge(l, a[r].l);
102    pull(r);
103    return r;
104 }
105
106 // Insert after the first k elements, 0 <= k <= size().
107 void insert(int k, ll value)
108 {
109     assert(0 <= k && k <= size());

```

```

110     int p = a.size(), l, r;
111     a.push_back({0, 0, 1, value, value, 0, rng(), false});
112     split(root, k, l, r);
113     root = merge(merge(l, p), r);
114 }
115
116 // All range operations use 1-based inclusive intervals.
117 void erase(int l, int r)
118 {
119     assert(1 <= l && l <= r && r <= size());
120     int x, y, z;
121     split(root, r, y, z);
122     split(y, l - 1, x, y);
123     root = merge(x, z);
124 }
125
126 void reverse(int l, int r)
127 {
128     assert(1 <= l && l <= r && r <= size());
129     int x, y, z;
130     split(root, r, y, z);
131     split(y, l - 1, x, y);
132     apply_reverse(y);
133     root = merge(merge(x, y), z);
134 }
135
136 void add(int l, int r, ll value)
137 {
138     assert(1 <= l && l <= r && r <= size());
139     int x, y, z;
140     split(root, r, y, z);
141     split(y, l - 1, x, y);
142     apply_add(y, value);
143     root = merge(merge(x, y), z);
144 }
145
146 ll query(int l, int r)
147 {
148     assert(1 <= l && l <= r && r <= size());
149     int x, y, z;
150     split(root, r, y, z);
151     split(y, l - 1, x, y);
152     ll ans = a[y].sum;
153     root = merge(merge(x, y), z);
154     return ans;
155 }
156
157 void collect(int p, vector<ll> &out)
158 {
159     if (!p)
160         return;
161     push(p);
162     collect(a[p].l, out);
163     out.push_back(a[p].val);

```

```
164         collect(a[p].r, out);
165     }
166
167     vector<ll> values()
168     {
169         vector<ll> answer;
170         collect(root, answer);
171         return answer;
172     }
173 };
```

第八十六章 伸展树

86.1 Splay: 可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义, 严格前驱后继不存在返回 nullopt。重复键计数存储; 失败查询也伸展最后访问节点, 单次均摊 $O(\log n)$ 。不回收节点, 空间按累计新建节点数计算。支持 long long 全范围; 不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
1 struct OrderedSplay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2]{}, fa = 0, cnt = 0, siz = 0;
8         ll val = 0;
9     };
10
11     vector<Node> a{Node{}};
12     int root = 0;
13
14     int size() const
15     {
16         return a[root].siz;
17     }
18
19     void pull(int x)
20     {
21         a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz;
22     }
23
24     int node(ll value, int parent)
25     {
26         int x = a.size();
27         a.push_back({{0, 0}, parent, 1, 1, value});
28         return x;
29     }
30
31     void rotate(int x)
32     {
33         int y = a[x].fa, z = a[y].fa;
34         int k = a[y].ch[1] == x;
35         int w = a[x].ch[k ^ 1];
36         if (z)
37             a[z].ch[a[z].ch[1] == y] = x;
38         else
39             root = x;
40         a[x].fa = z;
41         a[y].ch[k] = w;
42         if (w)
43             a[w].fa = y;
44         a[x].ch[k ^ 1] = y;
```

```

45     a[y].fa = x;
46     pull(y);
47     pull(x);
48 }
49
50 void splay(int x)
51 {
52     while (a[x].fa)
53     {
54         int y = a[x].fa, z = a[y].fa;
55         if (z)
56         {
57             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
58             rotate(same ? y : x);
59         }
60         rotate(x);
61     }
62 }
63
64 // Splay the last visited node even when the value does not exist.
65 int find(ll value)
66 {
67     int x = root, last = 0;
68     while (x && a[x].val != value)
69     {
70         last = x;
71         x = a[x].ch[value > a[x].val];
72     }
73     if (x)
74         splay(x);
75     else if (last)
76         splay(last);
77     return x;
78 }
79
80 void insert(ll value)
81 {
82     int x = root, parent = 0;
83     while (x && a[x].val != value)
84     {
85         parent = x;
86         x = a[x].ch[value > a[x].val];
87     }
88     if (x)
89     {
90         ++a[x].cnt;
91         pull(x);
92     }
93     else
94     {
95         x = node(value, parent);
96         if (parent)
97             a[parent].ch[value > a[parent].val] = x;
98         else
99             root = x;
100     }

```

```

101     splay(x);
102 }
103
104 bool erase(ll value)
105 {
106     int x = find(value);
107     if (!x)
108         return false;
109     if (a[x].cnt > 1)
110     {
111         --a[x].cnt;
112         pull(x);
113         return true;
114     }
115     int l = a[x].ch[0], r = a[x].ch[1];
116     if (l)
117         a[l].fa = 0;
118     if (r)
119         a[r].fa = 0;
120     if (!l)
121     {
122         root = r;
123         return true;
124     }
125     root = l;
126     x = l;
127     while (a[x].ch[1])
128         x = a[x].ch[1];
129     splay(x);
130     a[x].ch[1] = r;
131     if (r)
132         a[r].fa = x;
133     pull(x);
134     return true;
135 }
136
137 int rank(ll value)
138 {
139     int x = root, last = 0, ans = 1;
140     while (x)
141     {
142         last = x;
143         if (value <= a[x].val)
144             x = a[x].ch[0];
145         else
146         {
147             ans += a[a[x].ch[0]].siz + a[x].cnt;
148             x = a[x].ch[1];
149         }
150     }
151     if (last)
152         splay(last);
153     return ans;
154 }
155
156 ll kth(int k)

```



```

157 {
158     assert(1 <= k && k <= size());
159     int x = root;
160     while (true)
161     {
162         int left = a[a[x].ch[0]].siz;
163         if (k <= left)
164             x = a[x].ch[0];
165         else if (k <= left + a[x].cnt)
166         {
167             splay(x);
168             return a[x].val;
169         }
170         else
171         {
172             k -= left + a[x].cnt;
173             x = a[x].ch[1];
174         }
175     }
176 }
177
178 optional<ll> prev(ll value)
179 {
180     int x = root, last = 0, answer = 0;
181     while (x)
182     {
183         last = x;
184         if (a[x].val < value)
185         {
186             answer = x;
187             x = a[x].ch[1];
188         }
189         else
190             x = a[x].ch[0];
191     }
192     if (last)
193         splay(last);
194     if (!answer)
195         return nullopt;
196     splay(answer);
197     return a[answer].val;
198 }
199
200 optional<ll> next(ll value)
201 {
202     int x = root, last = 0, answer = 0;
203     while (x)
204     {
205         last = x;
206         if (a[x].val > value)
207         {
208             answer = x;
209             x = a[x].ch[0];
210         }
211         else

```

```
212         x = a[x].ch[1];
213     }
214     if (last)
215         splay(last);
216     if (!answer)
217         return nullopt;
218     splay(answer);
219     return a[answer].val;
220 }
221 };
```

第八十七章 状态序列维护

87.1 状态序列 Treap：分组区间 GCD

build(v) 用数值与状态组成的序列替换当前内容。数值为无符号 64 位，状态为 0 或 1。insert(k,value,tag) 在前 k 项后插入。erase(l,r)、query(l,r,tag) 使用 1-based 非空闭区间；set(k,value) 修改值，toggle(k) 翻转状态。query 返回选中状态的 GCD，无对应元素为 nullopt，实际全零则返回 0。回收删除节点，空间按峰值长度计算。建树 $O(n)$ 次 GCD，随机优先级下单次期望 $O(\log n)$ 次 GCD；删除另需 $O(\text{删除长度})$ 回收。一次 GCD 为 $O(\log U)$ 。覆盖 kuangbin 3.7 的序列操作，尚无该原题 OJ 记录。建树后的聚合与删除节点回收均使用递归遍历；笛卡尔树的构造单调栈保留。

```
1 struct GcdSequenceTreap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    vector<int> recycled;
14    mt19937_64 rng;
15    int root = 0;
16
17    GcdSequenceTreap(U seed = 712367821) : rng(seed)
18    {
19    }
20
21    int size() const
22    {
23        return a[root].siz;
24    }
25
26    int node(U value, int tag)
27    {
28        assert(tag == 0 || tag == 1);
29        int p;
30        if (recycled.empty())
31        {
32            p = a.size();
33            a.push_back(Node{});
34        }
35        else
36        {
37            p = recycled.back();
38            recycled.pop_back();
39            a[p] = Node{};
40        }
41        a[p].siz = 1;
```

```

42     a[p].val = value;
43     a[p].tag = tag;
44     a[p].cnt[tag] = 1;
45     a[p].g[tag] = value;
46     a[p].pri = rng();
47     return p;
48 }
49
50 void pull(int p)
51 {
52     int l = a[p].l, r = a[p].r;
53     a[p].siz = a[l].siz + a[r].siz + 1;
54     for (int t = 0; t < 2; t++)
55     {
56         a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
57         a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
58         if (a[p].tag == t)
59             a[p].g[t] = gcd(a[p].g[t], a[p].val);
60     }
61 }
62
63 void split(int p, int k, int &l, int &r)
64 {
65     if (!p)
66     {
67         l = 0;
68         r = 0;
69         return;
70     }
71     int left = a[a[p].l].siz;
72     if (k > left)
73     {
74         l = p;
75         split(a[p].r, k - left - 1, a[p].r, r);
76     }
77     else
78     {
79         r = p;
80         split(a[p].l, k, l, a[p].l);
81     }
82     pull(p);
83 }
84
85 int merge(int l, int r)
86 {
87     if (!l || !r)
88         return l ? l : r;
89     if (a[l].pri > a[r].pri)
90     {
91         a[l].r = merge(a[l].r, r);
92         pull(l);
93         return l;
94     }
95     a[r].l = merge(l, a[r].l);
96     pull(r);
97     return r;

```

```

98     }
99
100    void rebuild(int p)
101    {
102        if (!p)
103            return;
104        rebuild(a[p].l);
105        rebuild(a[p].r);
106        pull(p);
107    }
108
109    void release(int p)
110    {
111        if (!p)
112            return;
113        release(a[p].l);
114        release(a[p].r);
115        recycled.push_back(p);
116    }
117
118    // O(n) GCD calls; replaces the current sequence.
119    void build(const vector<pair<U, int>> &v)
120    {
121        a.assign(1, Node{});
122        recycled.clear();
123        root = 0;
124        vector<int> st;
125        for (auto [value, tag] : v)
126        {
127            int p = node(value, tag), last = 0;
128            while (!st.empty() && a[st.back()].pri < a[p].pri)
129            {
130                last = st.back();
131                st.pop_back();
132            }
133            a[p].l = last;
134            if (!st.empty())
135                a[st.back()].r = p;
136            st.push_back(p);
137        }
138        if (st.empty())
139            return;
140        root = st.front();
141        rebuild(root);
142    }
143
144    // Insert after the first k entries. Range operations below are 1-based.
145    void insert(int k, U value, int tag)
146    {
147        assert(0 <= k && k <= size());
148        int p = node(value, tag), l, r;
149        split(root, k, l, r);
150        root = merge(merge(l, p), r);
151    }
152
153    void erase(int l, int r)

```

```

154     {
155         assert(1 <= l && l <= r && r <= size());
156         int x, y, z;
157         split(root, r, y, z);
158         split(y, l - 1, x, y);
159         root = merge(x, z);
160         release(y);
161     }
162
163     void set(int k, U value)
164     {
165         assert(1 <= k && k <= size());
166         int x, y, z;
167         split(root, k, y, z);
168         split(y, k - 1, x, y);
169         a[y].val = value;
170         pull(y);
171         root = merge(merge(x, y), z);
172     }
173
174     void toggle(int k)
175     {
176         assert(1 <= k && k <= size());
177         int x, y, z;
178         split(root, k, y, z);
179         split(y, k - 1, x, y);
180         a[y].tag ^= 1;
181         pull(y);
182         root = merge(merge(x, y), z);
183     }
184
185     optional<U> query(int l, int r, int tag)
186     {
187         assert(1 <= l && l <= r && r <= size());
188         assert(tag == 0 || tag == 1);
189         int x, y, z;
190         split(root, r, y, z);
191         split(y, l - 1, x, y);
192         optional<U> answer;
193         if (a[y].cnt[tag])
194             answer = a[y].g[tag];
195         root = merge(merge(x, y), z);
196         return answer;
197     }
198 };

```

第八十八章 一般图匹配

88.1 一般图最大基数匹配：带花树

Blossom(n), 点号 0..n-1; add(u,v) 加无向边, 忽略自环, 允许重边。solve() 从空匹配重新计算, 返回匹配边数; mate[u] 为配对点, 未匹配为 -1。结果支持直接输出方案。缩花维护交错树, 单次增广 $O(n^2+m)$, 总计 $O(n^3+nm)$, 简单图为 $O(n^3)$; 空间 $O(n+m)$ 。重复调用 solve 或加边后重算均可。不处理边权, 不能代替最大权匹配。

```
1 struct Blossom
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> mate, pre, base, q;
6     vector<bool> outer, flower, seen;
7
8     Blossom(int n)
9         : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
10    {
11    }
12
13    void add(int u, int v)
14    {
15        assert(0 <= u && u < n && 0 <= v && v < n);
16        if (u == v)
17            return;
18        g[u].push_back(v);
19        g[v].push_back(u);
20    }
21
22    int lca(int u, int v)
23    {
24        fill(seen.begin(), seen.end(), false);
25        while (true)
26        {
27            u = base[u];
28            seen[u] = true;
29            if (mate[u] == -1)
30                break;
31            u = pre[mate[u]];
32        }
33        while (!seen[base[v]])
34            v = pre[mate[base[v]]];
35        return base[v];
36    }
37
38    void mark(int u, int b, int v)
39    {
40        while (base[u] != b)
41        {
42            flower[base[u]] = true;
43            flower[base[mate[u]]] = true;
44            pre[u] = v;
```

```

45         v = mate[u];
46         u = pre[v];
47     }
48 }
49
50 bool augment(int root)
51 {
52     fill(outer.begin(), outer.end(), false);
53     fill(pre.begin(), pre.end(), -1);
54     iota(base.begin(), base.end(), 0);
55     q.assign(1, root);
56     outer[root] = true;
57     for (int head = 0; head < (int)q.size(); head++)
58     {
59         int u = q[head];
60         for (int v : g[u])
61         {
62             if (base[u] == base[v] || mate[u] == v)
63                 continue;
64             if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
65             {
66                 int b = lca(u, v);
67                 fill(flower.begin(), flower.end(), false);
68                 mark(u, b, v);
69                 mark(v, b, u);
70                 for (int x = 0; x < n; x++)
71                 {
72                     if (!flower[base[x]])
73                         continue;
74                     base[x] = b;
75                     if (!outer[x])
76                     {
77                         outer[x] = true;
78                         q.push_back(x);
79                     }
80                 }
81             }
82             else if (pre[v] == -1)
83             {
84                 pre[v] = u;
85                 if (mate[v] == -1)
86                 {
87                     while (v != -1)
88                     {
89                         int p = pre[v];
90                         int next = mate[p];
91                         mate[v] = p;
92                         mate[p] = v;
93                         v = next;
94                     }
95                     return true;
96                 }
97                 int x = mate[v];
98                 outer[x] = true;
99                 q.push_back(x);
100             }

```



```
101         }
102     }
103     return false;
104 }
105
106 // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
107 int solve()
108 {
109     fill(mate.begin(), mate.end(), -1);
110     int answer = 0;
111     for (int u = 0; u < n; u++)
112     {
113         if (mate[u] == -1 && augment(u))
114             answer++;
115     }
116     return answer;
117 }
118 };
```

第八十九章 线性递推

89.1 Berlekamp–Massey: 最短线性递推

`berlekamp_massey(s,p)` 要求 p 为素数, $2 \leq p \leq \text{INT_MAX}$, 输入均在 $[0,p)$; 返回 c , 使 $s[i] = \sum_{j=1..k} c[j-1]s[i-j]$ ($j=1..k, i \geq k$)。空序列或全零序列返回空系数。有限前缀可能对应多种未来, 不能无条件外推; 若已知原序列满足阶数至多 d 的齐次线性递推, 提供至少 $2d$ 项才能恢复可外推的递推。模运算次数 $O(Nk+N \log p)$, 后项来自逐次求逆; 空间 $O(N)$ 。不适用于合数模数或浮点拟合。

```
1 // Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
2 vector<int> berlekamp_massey(const vector<int> &s, int p = 998244353)
3 {
4     assert(p >= 2);
5     vector<int> c{1}, b{1};
6     int len = 0, shift = 1, last = 1;
7     for (int i = 0; i < (int)s.size(); i++)
8     {
9         assert(0 <= s[i] && s[i] < p);
10        long long d = s[i];
11        for (int j = 1; j <= len; j++)
12            d = (d + 1LL * c[j] * s[i - j]) % p;
13        if (d == 0)
14        {
15            shift++;
16            continue;
17        }
18        long long inv = 1, x = last;
19        for (int e = p - 2; e; e >>= 1)
20        {
21            if (e & 1)
22                inv = inv * x % p;
23            x = x * x % p;
24        }
25        long long factor = d * inv % p;
26        auto old = c;
27        if (c.size() < b.size() + shift)
28            c.resize(b.size() + shift);
29        for (int j = 0; j < (int)b.size(); j++)
30            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
31        if (2 * len <= i)
32        {
33            len = i + 1 - len;
34            b = old;
35            last = d;
36            shift = 1;
37        }
38    }
```

```
38         else
39             shift++;
40     }
41     c.resize(len + 1);
42     c.erase(c.begin());
43     for (int &x : c)
44         x = x == 0 ? 0 : p - x;
45     return c;
46 }
```

89.2 特征多项式求线性递推第 k 项

`recurrence_nth(init,c,n,p)` 使用 0-based 下标, n 为 unsigned long long。init 与 c 长度均为阶数 k ; $a[i] = \sum_{c[j-1]a[i-j]}$, $2 \leq p \leq \text{INT_MAX}$, 数值均在 $[0,p)$, 本函数允许合数模数。空系数约定全零序列。计算 x^n 对特征多项式取余再与初值点乘, $O(k^2 \log(n+1))$ 时间、 $O(k)$ 空间; 不依赖 BM 或其他模板。较大阶数可使用本库 `BostonMori::recurrence`, 须满足其 NTT 素数模数与容量条件; 本函数允许合数模数。

```

1 // init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
2 int recurrence_nth(const vector<int> &init,
3                   const vector<int> &c,
4                   unsigned long long n,
5                   int p = 998244353)
6 {
7     assert(p >= 2 && init.size() == c.size());
8     int k = c.size();
9     if (!k)
10         return 0;
11     for (int x : init)
12         assert(0 <= x && x < p);
13     for (int x : c)
14         assert(0 <= x && x < p);
15     if (n < (unsigned)k)
16         return init[n];
17     auto multiply = [&](const vector<int> &a, const vector<int> &b)
18     {
19         vector<int> t(2 * k - 1);
20         for (int i = 0; i < k; i++)
21         {
22             for (int j = 0; j < k; j++)
23                 t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
24         }
25         for (int i = 2 * k - 2; i >= k; i--)
26         {
27             for (int j = 1; j <= k; j++)
28                 t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
29         }
30         t.resize(k);
31         return t;
32     };
33     vector<int> answer(k), x(k);
34     answer[0] = 1;
35     if (k == 1)
36         x[0] = c[0];
37     else
38         x[1] = 1;
39     while (n)
40     {
41         if (n & 1)

```

```
42         answer = multiply(answer, x);
43         x = multiply(x, x);
44         n >>= 1;
45     }
46     long long value = 0;
47     for (int i = 0; i < k; i++)
48         value = (value + 1LL * answer[i] * init[i]) % p;
49     return value;
50 }
```

第九十章 快速线性递推

90.1 快速递推与有理生成函数单项

BostanMori<mod=998244353,primitive=3>, 依赖参数化 NTT; 素数模数与原根条件同 NttConvolution。Poly 为升幂系数 vector<Z>。coefficient(p,q,n) 返回 $[x^n]p(x)/q(x)$, q 非空且 $q[0] \neq 0$, p 可空或次数不小于 q; 令 $d=\max(p.size(),q.size())$, 要求 $2d-1 \leq \text{NTT::max_size}$ 。recurrence(init,c,n) 返回 $a[n]$, 从下标 k 起 $a[i]=c[0]a[i-1]+\dots+c[k-1]a[i-k]$, init、c 长度均为 k; $k=0$ 约定全零。非初值查询要求 $2k+1 \leq \text{NTT::max_size}$ 。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项, $O(d \log d \log(n+1))$ 时间、 $O(d)$ 空间; 递推接口 $d=k+1$ 。传统接口 Bostan_Mori::Coefficient / Recurrence。

```
1 template <int mod = 998244353, int primitive = 3> struct BostanMori
2 {
3     using N = NttConvolution<mod, primitive>;
4     using Z = typename N::Z;
5     using Poly = typename N::Poly;
6     using U = unsigned long long;
7
8     // Coefficient of  $x^n$  in  $p(x) / q(x)$ , with  $q[0] \neq 0$ .
9     static Z coefficient(Poly p, Poly q, U n)
10    {
11        assert(!q.empty() && q[0].v != 0);
12        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
13        while (n && !p.empty())
14        {
15            Poly r = q;
16            for (int i = 1; i < int(r.size()); i += 2)
17                r[i] = Z(0) - r[i];
18            Poly a = N::multiply(p, r);
19            Poly b = N::multiply(q, r);
20            p.clear();
21            q.clear();
22            for (int i = n & 1; i < int(a.size()); i += 2)
23                p.push_back(a[i]);
24            for (int i = 0; i < int(b.size()); i += 2)
25                q.push_back(b[i]);
26            n >>= 1;
27        }
28        return p.empty() ? Z(0) : p[0] / q[0];
29    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
31 // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
32 static Z recurrence(const Poly &init, const Poly &c, U n)
33 {
34     assert(init.size() == c.size());
35     int k = c.size();
36     if (k == 0)
37         return 0;
38     if (n < init.size())
39         return init[n];
40     assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
41     Poly q(k + 1);
42     q[0] = 1;
43     for (int i = 0; i < k; i++)
44         q[i + 1] = Z(0) - c[i];
45     Poly p = N::multiply(init, q);
46     p.resize(k);
47     return coefficient(p, q, n);
48 }
49 };
```

第二部分

传统接口风格

第九十一章 Tarjan 与缩点

91.1 Tarjan 强连通分量与缩点

算法约束见动态版本第 2 页。本节代码独立可抄；采用下列实际接口名。

声明 Tarjan_SCC<N>, 先 Init(n), Insert 加边, Run 求分量, Dag 输出缩点图。bel 与 cnt 可直接读取；大对象放在全局或 static。

```
1 template <int N> struct Tarjan_SCC
2 {
3     int n, timer = 0, cnt = 0;
4     vector<int> g[N + 1], st;
5     int dfn[N + 1], low[N + 1], bel[N + 1];
6
7     void Init(int vertices)
8     {
9         assert(0 <= vertices && vertices <= N);
10        n = vertices;
11        for ( int u = 1; u <= n; u++ )
12            g[u].clear();
13    }
14
15    void Insert(int u, int v)
16    {
17        assert(1 <= u && u <= n && 1 <= v && v <= n);
18        g[u].push_back(v);
19    }
20
21    void Dfs(int u)
22    {
23        dfn[u] = low[u] = ++timer;
24        st.push_back(u);
25        for ( int v : g[u] )
26        {
27            if ( !dfn[v] )
28            {
29                Dfs(v);
30                low[u] = min(low[u], low[v]);
31            }
32            else if ( !bel[v] )
33                low[u] = min(low[u], dfn[v]);
34        }
35        if ( low[u] == dfn[u] )
36        {
37            ++cnt;
38            while ( true )
39            {
40                int v = st.back();
41                st.pop_back();
42                bel[v] = cnt;
43                if ( v == u )
44                    break;
```

```

45         }
46     }
47 }
48
49 // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
50 void Run()
51 {
52     timer = 0;
53     cnt = 0;
54     fill(dfn, dfn + n + 1, 0);
55     fill(low, low + n + 1, 0);
56     fill-bel, bel + n + 1, 0);
57     st.clear();
58     for ( int u = 1; u <= n; u++ )
59     {
60         if ( !dfn[u] )
61             Dfs(u);
62     }
63 }
64
65 // Retains parallel edges. Use after Run().
66 vector<vector<int>> Dag() const
67 {
68     vector<vector<int>> result(cnt + 1);
69     for ( int u = 1; u <= n; u++ )
70     {
71         for ( int v : g[u] )
72         {
73             if ( bel[u] != bel[v] )
74                 result[bel[u]].push_back(bel[v]);
75         }
76     }
77     return result;
78 }
79 };

```

第九十二章 函数图

92.1 函数图分解、跳转与到达

算法约束见动态版本第 4 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Functional_Graph
2 {
3     using U = unsigned long long;
4     int n;
5     vector<int> component, position, entry, depth, order;
6     vector<vector<int>> cycles, up;
7
8     explicit Functional_Graph(vector<int> to = {})
9     {
10         Build(move(to));
11     }
12
13     void Build(vector<int> to)
14     {
15         assert(to.size() <= INT_MAX);
16         n = to.size();
17         vector<int> degree(n);
18         for ( int v : to )
19         {
20             assert(0 <= v && v < n);
21             degree[v]++;
22         }
23         order.clear();
24         for ( int u = 0; u < n; u++ )
25             if ( degree[u] == 0 )
26                 order.push_back(u);
27         for ( int i = 0; i < int(order.size()); i++ )
28         {
29             int v = to[order[i]];
30             if ( --degree[v] == 0 )
31                 order.push_back(v);
32         }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```

33     component.assign(n, -1);
34     position.assign(n, -1);
35     entry.resize(n);
36     depth.assign(n, 0);
37     cycles.clear();
38     for ( int u = 0; u < n; u++ )
39         if ( degree[u] && component[u] == -1 )
40         {
41             vector<int> cycle;
42             int v = u, id = cycles.size();
43             do
44             {
45                 component[v] = id;
46                 position[v] = cycle.size();
47                 entry[v] = v;
48                 cycle.push_back(v);
49                 v = to[v];
50             } while ( v != u );
51             cycles.push_back(move(cycle));
52         }
53     for ( int i = int(order.size()) - 1; i >= 0; i-- )
54     {
55         int u = order[i], v = to[u];
56         component[u] = component[v];
57         entry[u] = entry[v];
58         depth[u] = depth[v] + 1;
59     }
60     up.assign(1, to);
61     for ( int k = 1; (1ULL << k) <= U(n); k++ )
62     {
63         vector<int> row(n);
64         for ( int u = 0; u < n; u++ )
65             row[u] = up[k - 1][up[k - 1][u]];
66         up.push_back(move(row));
67     }
68 }
```

跳转与最少步数接口（接上页同一结构体）：

```

70     int Advance(int u, U k) const
71     {
72         assert(0 <= u && u < n);
73         if ( k >= U(depth[u]) )
74         {
75             k -= depth[u];
76             u = entry[u];
77             const auto &cycle = cycles[component[u]];
78             return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
79         }
80         for ( int bit = 0; k; bit++, k >>= 1 )
81             if ( k & 1 )
82                 u = up[bit][u];
83         return u;
84     }
85
86     int Steps(int u, int v) const
87     {
88         assert(0 <= u && u < n && 0 <= v && v < n);
89         if ( component[u] != component[v] || depth[u] < depth[v] )
90             return -1;
91         if ( depth[v] > 0 )
92         {
93             int d = depth[u] - depth[v];
94             return Advance(u, d) == v ? d : -1;
95         }
96         int offset = position[v] - position[entry[u]];
97         if ( offset < 0 )
98             offset += int(cycles[component[u]].size());
99         return depth[u] + offset;
100     }
101 };

```

第九十三章 双连通分量与圆方树

93.1 Tarjan 点双、边双与圆方树

算法约束见动态版本第 7 页。本节代码独立可抄；采用下列实际接口名。

声明 `Biconnected_Graph<N>`, `Init(n)` 初始化, `Insert` 加边, `Run` 分解, `Block_Forest` 和 `Bridge_Forest` 输出两种森林。 `blocks`、`cut`、`bridge`、`bel`、`cnt` 可直接读取。大对象放在全局或 `static`。

```
1 template <int N> struct Biconnected_Graph
2 {
3     int n, timer = 0, cnt = 0;
4     vector<pair<int, int>> g[N + 1], edges;
5     vector<vector<int>> blocks;
6     vector<int> bridge, st;
7     int dfn[N + 1], low[N + 1], cut[N + 1], bel[N + 1];
8
9     void Init(int vertices)
10    {
11        assert(0 <= vertices && vertices <= N);
12        n = vertices;
13        for ( int u = 1; u <= n; u++ )
14            g[u].clear();
15        edges.clear();
16    }
17
18    int Insert(int u, int v)
19    {
20        assert(1 <= u && u <= n && 1 <= v && v <= n);
21        int id = edges.size();
22        edges.push_back({u, v});
23        if ( u != v )
24        {
25            g[u].push_back({v, id});
26            g[v].push_back({u, id});
27        }
28        return id;
29    }
30
31    void Dfs(int u, int parent_edge)
32    {
33        dfn[u] = low[u] = ++timer;
34        st.push_back(u);
35        int children = 0;
36        for ( auto [v, id] : g[u] )
37        {
38            if ( id == parent_edge )
39                continue;
40            if ( !dfn[v] )
41            {
42                children++;
43                Dfs(v, id);
44                low[u] = min(low[u], low[v]);
```

```

45         if ( low[v] > dfn[u] )
46             bridge[id] = 1;
47         if ( low[v] >= dfn[u] )
48         {
49             if ( parent_edge != -1 || children > 1 )
50                 cut[u] = 1;
51             blocks.push_back({u});
52             while ( true )
53             {
54                 int x = st.back();
55                 st.pop_back();
56                 blocks.back().push_back(x);
57                 if ( x == v )
58                     break;
59             }
60         }
61     }
62     else
63         low[u] = min(low[u], dfn[v]);
64 }
65 if ( parent_edge == -1 && children == 0 )
66     blocks.push_back({u});
67 }
68
69 void Paint(int u)
70 {
71     bel[u] = cnt;
72     for ( auto [v, id] : g[u] )
73     {
74         if ( !bridge[id] && !bel[v] )
75             Paint(v);
76     }
77 }
78
79 void Run()
80 {
81     timer = 0;
82     cnt = 0;
83     fill(dfn, dfn + n + 1, 0);
84     fill(low, low + n + 1, 0);
85     fill(cut, cut + n + 1, 0);
86     fill(bel, bel + n + 1, 0);
87     bridge.assign(edges.size(), 0);
88     blocks.clear();
89     st.clear();
90     for ( int u = 1; u <= n; u++ )
91     {
92         if ( !dfn[u] )
93         {
94             Dfs(u, -1);
95             st.pop_back();
96         }
97     }
98     for ( int u = 1; u <= n; u++ )
99     {
100         if ( bel[u] )

```

```

101         continue;
102         ++cnt;
103         Paint(u);
104     }
105 }
106
107 // Original vertices 1..n; block i has node n+i+1. Includes isolated vertices.
108 vector<vector<int>> Block_Forest() const
109 {
110     vector<vector<int>> tree(n + blocks.size() + 1);
111     for ( int i = 0; i < (int)blocks.size(); i++ )
112     {
113         int b = n + i + 1;
114         for ( int u : blocks[i] )
115         {
116             tree[u].push_back(b);
117             tree[b].push_back(u);
118         }
119     }
120     return tree;
121 }
122
123 // Edge-connected component IDs 1..cnt; forest edges carry original edge IDs.
124 vector<vector<pair<int, int>>> Bridge_Forest() const
125 {
126     vector<vector<pair<int, int>>> tree(cnt + 1);
127     for ( int id = 0; id < (int)edges.size(); id++ )
128     {
129         if ( !bridge[id] )
130             continue;
131         auto [u, v] = edges[id];
132         u = bel[u];
133         v = bel[v];
134         tree[u].push_back({v, id});
135         tree[v].push_back({u, id});
136     }
137     return tree;
138 }
139 };

```


第九十四章 奇环顶点判定

94.1 属于简单奇环的全部顶点

算法约束见动态版本第 10 页。本节代码独立可抄；采用下列实际接口名。

```
1 // After Run(). Simple odd cycles have length >= 3; self-loops are ignored.
2 template <int N> vector<int> Odd_Cycle_Vertices(const Biconnected_Graph<N> &graph)
3 {
4     int n = graph.n;
5     auto forest = graph.Block_Forest();
6     vector<int> parent(forest.size()), depth(forest.size());
7     auto root_tree = [&](auto &&self, int u, int p) -> void
8     {
9         parent[u] = p;
10        for ( int v : forest[u] )
11            if ( v != p )
12            {
13                depth[v] = depth[u] + 1;
14                self(self, v, u);
15            }
16    };
17    for ( int u = 1; u <= n; u++ )
18        if ( !depth[u] )
19        {
20            depth[u] = 1;
21            root_tree(root_tree, u, 0);
22        }
```

```

23     vector<vector<pair<int, int>>> edges(graph.blocks.size());
24     for ( auto [u, v] : graph.edges )
25         if ( u != v )
26         {
27             int block = parent[depth[u] >= depth[v] ? u : v] - n - 1;
28             assert(0 <= block && block < (int)edges.size());
29             edges[block].push_back({u, v});
30         }
31     vector<vector<int>> g(n + 1);
32     vector<int> color(n + 1), answer(n + 1);
33     for ( int block = 0; block < (int)edges.size(); block++ )
34     {
35         for ( int u : graph.blocks[block] )
36         {
37             g[u].clear();
38             color[u] = 0;
39         }
40         for ( auto [u, v] : edges[block] )
41         {
42             g[u].push_back(v);
43             g[v].push_back(u);
44         }
45         bool bipartite = true;
46         auto paint = [&](auto &&self, int u) -> void
47         {
48             for ( int v : g[u] )
49                 if ( !color[v] )
50                 {
51                     color[v] = -color[u];
52                     self(self, v);
53                 }
54             else if ( color[v] == color[u] )
55                 bipartite = false;
56         };
57         int root = graph.blocks[block][0];
58         color[root] = 1;
59         paint(paint, root);
60         if ( !bipartite )
61             for ( int u : graph.blocks[block] )
62                 answer[u] = 1;
63     }
64     return answer;
65 }

```

第九十五章 删点连通性

95.1 由点双分解计算删点连通块数

算法约束见动态版本第 12 页。本节代码独立可抄；采用下列实际接口名。

```
1 // Call after Run(). Each vertex is deleted independently from the original graph.
2 template <int N>
3 inline pair<int, vector<int>> Removal_Components(const Biconnected_Graph<N> &graph)
4 {
5     int before = graph.cnt - count(graph.bridge.begin(), graph.bridge.end(), 1);
6     vector<int> after(graph.n + 1, before - 1);
7     after[0] = 0;
8     for ( const auto &block : graph.blocks )
9         if ( block.size() > 1 )
10             for ( int u : block )
11                 after[u]++;
12     return {before, after};
13 }
```

第九十六章 边双缩点与边定向

96.1 边双缩点的大小与内部边数

算法约束见动态版本第 13 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Edge_Compression
2 {
3     int n;
4     vector<int> size, inside;
5     vector<array<int, 3>> edges;
6
7     template <int N>
8     Edge_Compression(const Biconnected_Graph<N> &graph)
9         : n(graph.cnt), size(n + 1), inside(n + 1)
10    {
11        for ( int u = 1; u <= graph.n; u++ )
12            size[graph.bel[u]]++;
13        for ( int id = 0; id < (int)graph.edges.size(); id++ )
14        {
15            auto [u, v] = graph.edges[id];
16            u = graph.bel[u];
17            v = graph.bel[v];
18            if ( u == v )
19                inside[u]++;
20            else
21                edges.push_back({u, v, id});
22        }
23    }
24 };
```

96.2 无向边定向与强连通定向

算法约束见动态版本第 14 页。本节代码独立可抄；采用下列实际接口名。

```
1 // Tree edges go down; other edges go to ancestors. Handles each logical edge once.
2 inline vector<pair<int, int>> Orient_Edges(const Low_Link &graph)
3 {
4     vector<pair<int, int>> direction(graph.edges);
5     vector<int> seen(graph.n + 1);
6     auto dfs = [&](auto &&self, int u) -> void
7     {
8         seen[u] = 1;
9         for ( auto [v, id] : graph.g[u] )
10         {
11             if ( direction[id].first )
12                 continue;
13             direction[id] = {u, v};
14             if ( !seen[v] )
15                 self(self, v);
16         }
17     };
18     for ( int u = 1; u <= graph.n; u++ )
19         if ( !seen[u] )
20             dfs(dfs, u);
21     return direction;
22 }
```

第九十七章 桥树最少加边

97.1 连通无向图最少加边方案

算法约束见动态版本第 15 页。本节代码独立可抄；采用下列实际接口名。

```
1 // Connected, nonempty graph after Run(); new parallel edges are allowed.
2 template <int N>
3 inline vector<pair<int, int>> Bridge_Augmentation(const Biconnected_Graph<N> &graph)
4 {
5     assert(graph.n > 0 && graph.cnt > 0);
6     int bridges = count(graph.bridge.begin(), graph.bridge.end(), 1);
7     assert(graph.cnt - bridges == 1);
8     if ( graph.cnt == 1 )
9         return {};
10    auto tree = graph.Bridge_Forest();
11    vector<int> representative(graph.cnt + 1);
12    for ( int u = 1; u <= graph.n; u++ )
13        representative[graph.bel[u]] = u;
14    vector<int> leaves;
15    auto dfs = [&](auto &&self, int u, int parent) -> void
16    {
17        if ( tree[u].size() == 1 )
18            leaves.push_back(u);
19        for ( auto [v, id] : tree[u] )
20            if ( v != parent )
21                self(self, v, u);
22    };
23    dfs(dfs, 1, 0);
24    if ( leaves.size() % 2 )
25        leaves.push_back(leaves[0]);
26    int half = leaves.size() / 2;
27    vector<pair<int, int>> added;
28    for ( int i = 0; i < half; i++ )
29        added.push_back({representative[leaves[i]], representative[leaves[i + half]]});
30    return added;
31 }
```

第九十八章 数据结构

98.1 并查集

算法约束见动态版本第 16 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```
1 template <int N> struct Disjoint_Set
2 {
3     int n, fa[N + 1], siz[N + 1];
4
5     void Init(int size)
6     {
7         assert(size <= N);
8         n = size;
9         iota(fa, fa + n + 1, 0);
10        fill(siz, siz + n + 1, 1);
11    }
12
13    int Find(int x)
14    {
15        int root = x;
16        while ( root != fa[root] )
17            root = fa[root];
18        while ( x != root )
19        {
20            int next = fa[x];
21            fa[x] = root;
22            x = next;
23        }
24        return root;
25    }
26
27    bool Merge(int x, int y)
28    {
29        x = Find(x);
30        y = Find(y);
31        if ( x == y )
32            return false;
33        if ( siz[x] < siz[y] )
34            swap(x, y);
35        fa[y] = x;
36        siz[x] += siz[y];
37        return true;
38    }
39 };
```

98.2 可撤销并查集

算法约束见动态版本第 17 页。本节代码独立可抄；采用下列实际接口名。

```

1  template <int N> struct Rollback_DSU
2  {
3      int fa[N + 1], siz[N + 1];
4      vector<pair<int, int>> history;
5
6      void Init(int n)
7      {
8          assert(n <= N);
9          history.clear();
10         iota(fa, fa + n + 1, 0);
11         fill(siz, siz + n + 1, 1);
12     }
13
14     int Find(int x) const
15     {
16         while ( fa[x] != x )
17             x = fa[x];
18         return x;
19     }
20
21     int Snapshot() const
22     {
23         return (int)history.size();
24     }
25
26     bool Merge(int x, int y)
27     {
28         x = Find(x);
29         y = Find(y);
30         if ( x == y )
31             return false;
32         if ( siz[x] < siz[y] )
33             swap(x, y);
34         history.push_back({y, siz[x]});
35         fa[y] = x;
36         siz[x] += siz[y];
37         return true;
38     }
39
40     void Rollback(int time)
41     {
42         assert(0 <= time && time <= Snapshot());
43         while ( Snapshot() > time )
44         {
45             auto [y, size] = history.back();
46             history.pop_back();
47             siz[fa[y]] = size;
48             fa[y] = y;
49         }
50     }
51 };

```


98.3 树状数组与第 k 小

算法约束见动态版本第 18 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```

1 template <int N, class T = long long> struct Binary_Indexed_Tree
2 {
3     int n;
4     T tree[N + 1];
5
6     void Init(int size)
7     {
8         assert(size <= N);
9         n = size;
10        fill(tree, tree + n + 1, T(0));
11    }
12
13    void Insert(int x, T value)
14    {
15        assert(x >= 1);
16        for ( ; x <= n; x += x & -x )
17            tree[x] += value;
18    }
19
20    T Sum(int x) const
21    {
22        T answer = 0;
23        for ( ; x; x -= x & -x )
24            answer += tree[x];
25        return answer;
26    }
27
28    T Query(int L, int R) const
29    {
30        return Sum(R) - Sum(L - 1);
31    }
32
33    int Kth(T k) const
34    {
35        assert(k > 0);
36        int now = 0, step = 1;
37        while ( step <= n / 2 )
38            step <= 1;
39        for ( ; step; step >>= 1 )
40            if ( now + step <= n && tree[now + step] < k )
41            {
42                now += step;
43                k -= tree[now];
44            }
45        return now + 1;
46    }
47 };

```

98.4 区间加与区间和

算法约束见动态版本第 18 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```

1 template <int N> struct Segment_Tree
2 {
3     int n;
4     long long tree[4 * N + 5], tag[4 * N + 5];
5
6     void Init(int size)
7     {
8         assert(size > 0 && size <= N);
9         n = size;
10        fill(tree, tree + 4 * n + 5, 0);
11        fill(tag, tag + 4 * n + 5, 0);
12    }
13
14    void Update(int now, int L, int R, long long value)
15    {
16        tree[now] += (R - L + 1) * value;
17        tag[now] += value;
18    }
19
20    void Push_Up(int now)
21    {
22        tree[now] = tree[now << 1] + tree[now << 1 | 1];
23    }
24
25    void Push_Down(int now, int L, int R)
26    {
27        int mid = (L + R) >> 1;
28        Update(now << 1, L, mid, tag[now]);
29        Update(now << 1 | 1, mid + 1, R, tag[now]);
30        tag[now] = 0;
31    }
32
33    void Insert(int ql, int qr, long long value)
34    {
35        Insert(1, 1, n, ql, qr, value);
36    }
37
38    void Insert(int now, int L, int R, int ql, int qr, long long value)
39    {
40        if ( ql <= L && R <= qr )
41            return Update(now, L, R, value);
42        Push_Down(now, L, R);
43        int mid = (L + R) >> 1;
44        if ( ql <= mid )
45            Insert(now << 1, L, mid, ql, qr, value);
46        if ( qr > mid )
47            Insert(now << 1 | 1, mid + 1, R, ql, qr, value);
48        Push_Up(now);
49    }
50
51    long long Query(int ql, int qr)

```

```
52 {
53     return Query(1, 1, n, ql, qr);
54 }
55
56 long long Query(int now, int L, int R, int ql, int qr)
57 {
58     if ( ql <= L && R <= qr )
59         return tree[now];
60     Push_Down(now, L, R);
61     int mid = (L + R) >> 1;
62     long long answer = 0;
63     if ( ql <= mid )
64         answer += Query(now << 1, L, mid, ql, qr);
65     if ( qr > mid )
66         answer += Query(now << 1 | 1, mid + 1, R, ql, qr);
67     return answer;
68 }
69 };
```

98.5 64 位异或线性基

算法约束见动态版本第 21 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Xor_Basis
2 {
3     using U = unsigned long long;
4     using Wide = __uint128_t;
5     U base[64]{};
6     vector<U> p;
7     int rank = 0;
8     bool dependent = false, dirty = false;
9
10    void Init()
11    {
12        fill(base, base + 64, 0);
13        p.clear();
14        rank = 0;
15        dependent = false;
16        dirty = false;
17    }
18
19    bool Insert(U x)
20    {
21        for ( int i = 63; i >= 0; i-- )
22            if ( x >> i & 1 )
23            {
24                if ( !base[i] )
25                {
26                    base[i] = x;
27                    rank++;
28                    dirty = true;
29                    return true;
30                }
31                x ^= base[i];
32            }
33        dependent = true;
34        return false;
35    }
36
37    bool Contains(U x) const
38    {
39        for ( int i = 63; i >= 0; i-- )
40            if ( x >> i & 1 )
41                x ^= base[i];
42        return x == 0;
43    }
44
45    U Query(U x = 0) const
46    {
47        for ( int i = 63; i >= 0; i-- )
48            x = max(x, x ^ base[i]);
49        return x;
50    }

```

```

52 void Rebuild()
53 {
54     vector<U> b(base, base + 64);
55     for ( int i = 0; i < 64; i++ )
56         if ( b[i] )
57             for ( int j = i + 1; j < 64; j++ )
58                 if ( b[j] >> i & 1 )
59                     b[j] ^= b[i];
60     p.clear();
61     for ( U x : b )
62         if ( x )
63             p.push_back(x);
64     dirty = false;
65 }
66
67 // k is 1-based; nonempty excludes the empty input subset, not value zero.
68 optional<U> Kth(Wide k, bool nonempty = true)
69 {
70     if ( !k )
71         return nullopt;
72     Wide index = k;
73     if ( !nonempty || dependent )
74         index--;
75     if ( index >= (Wide(1) << rank) )
76         return nullopt;
77     if ( dirty )
78         Rebuild();
79     U answer = 0;
80     for ( int i = 0; i < rank; i++ )
81         if ( index >> i & 1 )
82             answer ^= p[i];
83     return answer;
84 }
85
86 optional<U> Minimum(bool nonempty = true)
87 {
88     return Kth(1, nonempty);
89 }
90
91 void Merge(const Xor_Basis &other)
92 {
93     dependent |= other.dependent;
94     for ( U x : other.base )
95         if ( x )
96             Insert(x);
97 }
98 };

```

第九十九章 带位置线性基

99.1 带位置限制的异或线性基

算法约束见动态版本第 24 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Position_Basis
2 {
3     using U = unsigned long long;
4     U base[64];
5     int pos[64];
6
7     Position_Basis()
8     {
9         Init();
10    }
11
12    void Init()
13    {
14        fill(base, base + 64, 0);
15        fill(pos, pos + 64, -1);
16    }
```

```

18 void Insert(U x, int at)
19 {
20     assert(at >= 0);
21     for ( int i = 63; i >= 0; i-- )
22         if ( x >> i & 1 )
23         {
24             if ( !base[i] )
25             {
26                 base[i] = x;
27                 pos[i] = at;
28                 return;
29             }
30             if ( at > pos[i] )
31             {
32                 swap(base[i], x);
33                 swap(pos[i], at);
34             }
35             x ^= base[i];
36         }
37 }
38
39 bool Contains(U x, long long lower = 0) const
40 {
41     assert(lower >= 0);
42     for ( int i = 63; i >= 0; i-- )
43         if ( (x >> i & 1) && pos[i] >= lower )
44             x ^= base[i];
45     return x == 0;
46 }
47
48 U Query(long long lower = 0, U x = 0) const
49 {
50     assert(lower >= 0);
51     for ( int i = 63; i >= 0; i-- )
52         if ( pos[i] >= lower )
53             x = max(x, x ^ base[i]);
54     return x;
55 }
56
57 int Rank(long long lower = 0) const
58 {
59     assert(lower >= 0);
60     int answer = 0;
61     for ( int i = 0; i < 64; i++ )
62         answer += pos[i] >= lower;
63     return answer;
64 }
65 };

```

第一百章 线性空间求交

100.1 异或线性空间求交

算法约束见动态版本第 26 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline Xor_Basis Basis_Intersection(const Xor_Basis &a, const Xor_Basis &b)
2 {
3     using U = unsigned long long;
4     U pivot[64], from_b[64]{};
5     copy(a.base, a.base + 64, pivot);
6     Xor_Basis result;
7     for ( U value : b.base )
8     {
9         U x = value, part = value;
10        for ( int i = 63; i >= 0 && x; i-- )
11            if ( x >> i & 1 )
12            {
13                if ( !pivot[i] )
14                {
15                    pivot[i] = x;
16                    from_b[i] = part;
17                    break;
18                }
19                x ^= pivot[i];
20                part ^= from_b[i];
21            }
22            if ( !x && part )
23                result.Insert(part);
24        }
25    return result;
26 }
```


100.2 Zassenhaus：同时求和空间与交空间

算法约束见动态版本第 27 页。本节代码独立可抄；采用下列实际接口名。

```

1 // Return independent generators of (sum space, intersection space).
2 inline pair<Xor_Basis, Xor_Basis> Basis_Sum_Intersection(const Xor_Basis &a,
3                                                         const Xor_Basis &b)
4 {
5     using W = __uint128_t;
6     W pivot[128]{};
7     auto insert = [&](W x)
8     {
9         for ( int i = 127; i >= 0 && x; i-- )
10             if ( x >> i & 1 )
11             {
12                 if ( !pivot[i] )
13                 {
14                     pivot[i] = x;
15                     return;
16                 }
17                 x ^= pivot[i];
18             }
19     };
20     for ( auto x : a.base )
21         insert((W(x) << 64) | x);
22     for ( auto x : b.base )
23         insert(W(x) << 64);
24     Xor_Basis sum, intersection;
25     for ( int i = 0; i < 128; i++ )
26         if ( pivot[i] )
27         {
28             if ( i < 64 )
29                 intersection.Insert((unsigned long long)pivot[i]);
30             else
31                 sum.Insert((unsigned long long)(pivot[i] >> 64));
32         }
33     return {sum, intersection};
34 }
```

第一百零一章 线段树常用修改

101.1 线段树：区间乘、加、赋值与模意义求和

算法约束见动态版本第 28 页。本节代码独立可抄；采用下列实际接口名。

声明 `Affine_Segment_Tree<N>` 后 `Init(a,mod)`, `N` 为最大数组长度。`Update(l,r,mul,add)` 修改, `Query(l,r)` 求和；支持重复初始化。大对象放在全局或 `static`。

```
1 template <int N> struct Affine_Segment_Tree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    Node t[4 * N + 4];
12
13    void Init(const vector<ll> &a, int modulus)
14    {
15        n = a.size();
16        mod = modulus;
17        assert(0 < n && n <= N && mod > 0);
18        Build(1, 1, n, a);
19    }
20
21    ll Norm(ll x) const
22    {
23        x %= mod;
24        if ( x < 0 )
25            x += mod;
26        return x;
27    }
28
29    void Push_Up(int p)
30    {
31        t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod;
32    }
33
34    void Build(int p, int l, int r, const vector<ll> &a)
35    {
36        t[p] = Node{};
37        if ( l == r )
38        {
39            t[p].sum = Norm(a[l - 1]);
40            return;
41        }
42        int m = (l + r) / 2;
43        Build(p * 2, l, m, a);
44        Build(p * 2 + 1, m + 1, r, a);
```

```

45     Push_Up(p);
46 }
47
48 // New transform follows the old pending transform: x -> x*a+b.
49 void Apply(int p, int len, ll a, ll b)
50 {
51     t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
52     t[p].mul = t[p].mul * a % mod;
53     t[p].add = (t[p].add * a % mod + b) % mod;
54 }
55
56 void Push_Down(int p, int l, int r)
57 {
58     if ( t[p].mul == 1 && t[p].add == 0 )
59         return;
60     int m = (l + r) / 2;
61     Apply(p * 2, m - l + 1, t[p].mul, t[p].add);
62     Apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
63     t[p].mul = 1;
64     t[p].add = 0;
65 }
66
67 void Update(int l, int r, ll a, ll b)
68 {
69     assert(1 <= l && l <= r && r <= n);
70     Update(1, 1, n, l, r, Norm(a), Norm(b));
71 }
72
73 void Update(int p, int l, int r, int ql, int qr, ll a, ll b)
74 {
75     if ( ql <= l && r <= qr )
76     {
77         Apply(p, r - l + 1, a, b);
78         return;
79     }
80     Push_Down(p, l, r);
81     int m = (l + r) / 2;
82     if ( ql <= m )
83         Update(p * 2, l, m, ql, qr, a, b);
84     if ( qr > m )
85         Update(p * 2 + 1, m + 1, r, ql, qr, a, b);
86     Push_Up(p);
87 }
88
89 ll Query(int l, int r)
90 {
91     assert(1 <= l && l <= r && r <= n);
92     return Query(1, 1, n, l, r);
93 }
94
95 ll Query(int p, int l, int r, int ql, int qr)
96 {
97     if ( ql <= l && r <= qr )
98         return t[p].sum;

```

```
99     Push_Down(p, l, r);
100     int m = (l + r) / 2;
101     ll answer = 0;
102     if ( ql <= m )
103         answer += Query(p * 2, l, m, ql, qr);
104     if ( qr > m )
105         answer += Query(p * 2 + 1, m + 1, r, ql, qr);
106     return answer % mod;
107 }
108 };
```

第一百零二章 网络流

102.1 最大流与方案

算法约束见动态版本第 31 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```
1 template <int N, int M> struct Network_Flow
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int to, next;
8         ll cap, initial;
9     } edge[2 * M];
10
11     int n, tot, head[N + 1], deep[N + 1], cur[N + 1], que[N + 1];
12
13     void Init(int size)
14     {
15         assert(size <= N);
16         n = size;
17         tot = 0;
18         fill(head, head + n + 1, -1);
19     }
20
21     int Add_Edge(int u, int v, ll cap)
22     {
23         assert(cap >= 0 && tot + 2 <= 2 * M);
24         int id = tot;
25         edge[tot] = {v, head[u], cap, cap};
26         head[u] = tot++;
27         edge[tot] = {u, head[v], 0, 0};
28         head[v] = tot++;
29         return id;
30     }
31
32     bool Bfs(int s, int t)
33     {
34         fill(deep, deep + n + 1, -1);
35         int L = 0, R = 0;
36         que[R++] = s;
37         deep[s] = 0;
38         while ( L < R )
39         {
40             int now = que[L++];
41             for ( int i = head[now]; i != -1; i = edge[i].next )
42             {
43                 int to = edge[i].to;
44                 if ( edge[i].cap && deep[to] == -1 )
45                 {
```

```

46         deep[to] = deep[now] + 1;
47         que[R++] = to;
48     }
49 }
50 }
51 return deep[t] != -1;
52 }
53
54 ll Dfs(int now, int t, ll flow)
55 {
56     if ( now == t )
57         return flow;
58     for ( int &i = cur[now]; i != -1; i = edge[i].next )
59     {
60         int to = edge[i].to;
61         if ( !edge[i].cap || deep[to] != deep[now] + 1 )
62             continue;
63         ll delta = Dfs(to, t, min(flow, edge[i].cap));
64         if ( delta )
65         {
66             edge[i].cap -= delta;
67             edge[i ^ 1].cap += delta;
68             return delta;
69         }
70     }
71     return 0;
72 }
73
74 ll Max_Flow(int s, int t, ll limit = LLONG_MAX)
75 {
76     assert(s != t && limit >= 0);
77     ll answer = 0;
78     while ( answer < limit && Bfs(s, t) )
79     {
80         copy(head, head + n + 1, cur);
81         while ( answer < limit )
82         {
83             ll delta = Dfs(s, t, limit - answer);
84             if ( !delta )
85                 break;
86             answer += delta;
87         }
88     }
89     return answer;
90 }
91
92 ll Used(int id) const
93 {
94     return edge[id].initial - edge[id].cap;
95 }
96
97 vector<int> Cut(int s)

```

```

98     {
99         Bfs(s, s);
100         vector<int> answer;
101         for ( int i = 1; i <= n; i++ )
102             if ( deep[i] != -1 )
103                 answer.push_back(i);
104         return answer;
105     }
106 };

```

102.2 势能最小费用最大流

算法约束见动态版本第 33 页。本节代码独立可抄；采用下列实际接口名。
使用 Insert 加边、Flow 求流、Used 查看方案。

```

1 struct Min_Cost_Flow
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5
6     struct Edge
7     {
8         int from, to;
9         ll cap, cost, initial;
10    };
11
12    int n;
13    vector<Edge> e;
14    vector<vector<int>>> g;
15
16    Min_Cost_Flow(int n) : n(n), g(n + 1)
17    {
18    }
19
20    int Insert(int u, int v, ll cap, ll cost)
21    {
22        assert(cap >= 0 && cost != LLONG_MIN);
23        int id = (int)e.size();
24        e.push_back({u, v, cap, cost, cap});
25        e.push_back({v, u, 0, -cost, 0});
26        g[u].push_back(id);
27        g[v].push_back(id + 1);
28        return id;
29    }
30
31    ll Used(int id) const
32    {
33        return e[id].initial - e[id].cap;
34    }
35
36    // Negative costs allowed; initial residual graph must have no negative cycle.
37    pair<ll, i128> Flow(int s, int t, ll limit = LLONG_MAX)
38    {
39        assert(s != t && limit >= 0);
40        const i128 inf = i128(1) << 120;

```

```

41     vector<i128> h(n + 1, 0), d(n + 1);
42     vector<int> pre(n + 1);
43     // Super-source Bellman-Ford: feasible potentials for every vertex.
44     for ( int it = 0; it < n; it++ )
45     {
46         bool changed = false;
47         for ( auto &a : e )
48             if ( a.cap && h[a.to] > h[a.from] + a.cost )
49             {
50                 h[a.to] = h[a.from] + a.cost;
51                 changed = true;
52             }
53         if ( !changed )
54             break;
55         if ( it == n - 1 )
56             throw invalid_argument("negative cycle");
57     }
58     ll f = 0;
59     i128 cost = 0;
60     while ( f < limit )
61     {
62         fill(d.begin(), d.end(), inf);
63         d[s] = 0;
64         priority_queue<pair<i128, int>,
65                       vector<pair<i128, int>>,
66                       greater<pair<i128, int>>>
67             q;
68         q.push({0, s});
69         while ( !q.empty() )
70         {
71             auto [du, u] = q.top();
72             q.pop();
73             if ( du != d[u] )
74                 continue;
75             for ( int id : g[u] )
76             {
77                 auto &a = e[id];
78                 i128 nd = du + a.cost + h[u] - h[a.to];
79                 if ( a.cap && nd < d[a.to] )
80                 {
81                     d[a.to] = nd;
82                     pre[a.to] = id;
83                     q.push({nd, a.to});
84                 }
85             }
86         }
87         if ( d[t] == inf )
88             break;
89         for ( int u = 1; u <= n; u++ )
90             if ( d[u] != inf )
91                 h[u] += d[u];
92         ll take = limit - f;
93         for ( int u = t; u != s; u = e[pre[u]].from )
94             take = min(take, e[pre[u]].cap);
95         for ( int u = t; u != s; u = e[pre[u]].from )
96         {

```



```
97         int id = pre[u];
98         e[id].cap -= take;
99         e[id ^ 1].cap += take;
100        cost += i128(take) * e[id].cost;
101    }
102    f += take;
103 }
104 return {f, cost};
105 }
106 };
```

102.3 上下界可行循环流

算法约束见动态版本第 35 页。本节代码独立可抄；采用下列实际接口名。

```

1 template <int N, int M> struct Bounded_Circulation
2 {
3     int n, tot;
4     Network_Flow<N + 2, M + N> graph;
5     long long balance[N + 1], lower[M];
6     int id[M];
7     bool solved, feasible;
8
9     void Init(int size)
10    {
11        assert(size <= N);
12        n = size;
13        tot = 0;
14        solved = feasible = false;
15        graph.Init(n + 2);
16        fill(balance, balance + n + 1, 0);
17    }
18
19    int Insert(int u, int v, long long L, long long R)
20    {
21        assert(!solved && 0 <= L && L <= R && tot < M);
22        id[tot] = graph.Add_Edge(u, v, R - L);
23        lower[tot] = L;
24        balance[u] -= L;
25        balance[v] += L;
26        return tot++;
27    }
28
29    bool Solve()
30    {
31        assert(!solved);
32        solved = true;
33        long long need = 0;
34        for (int u = 1; u <= n; u++)
35        {
36            if (balance[u] > 0)
37            {
38                graph.Add_Edge(n + 1, u, balance[u]);
39                need += balance[u];
40            }
41            else if (balance[u] < 0)
42                graph.Add_Edge(u, n + 2, -balance[u]);
43        }
44        return feasible = (graph.Max_Flow(n + 1, n + 2) == need);
45    }
46
47    long long Used(int index) const
48    {
49        assert(feasible);
50        return lower[index] + graph.Used(id[index]);
51    }
52 };

```

第一百零三章 最大权闭合子图

103.1 最大权闭合子图与选点方案

算法约束见动态版本第 37 页。本节代码独立可抄；采用下列实际接口名。

Maximum_Closure(weight,dependencies,graph) 额外接收 Network_Flow<N,M> 引用，并在每次调用中 Init 重建该网络。N 至少为原点数加 2，M 至少为依赖边数加非零权点数；这是原有向边容量，不是残量边总数。将网络对象放在全局或 static，模板参数可由对象自动推导。

```
1 template <int N, int M>
2 pair<long long, vector<int>> Maximum_Closure(const vector<long long> &weight,
3                                             const vector<pair<int, int>> &dependencies,
4                                             Network_Flow<N, M> &graph)
5 {
6     using I = __int128_t;
7     int n = weight.size(), s = n + 1, t = n + 2;
8     I positive = 0;
9     for ( long long w : weight )
10         if ( w > 0 )
11             positive += w;
12     assert(positive < LLONG_MAX);
13     long long inf = (long long)positive + 1;
14     graph.Init(n + 2);
15     for ( int u = 1; u <= n; u++ )
16     {
17         long long w = weight[u - 1];
18         if ( w > 0 )
19             graph.Add_Edge(s, u, w);
20         else if ( w < 0 )
21             graph.Add_Edge(u, t, (long long)min(-I(w), I(inf)));
22     }
23     for ( auto [u, v] : dependencies )
24     {
25         assert(1 <= u && u <= n && 1 <= v && v <= n);
26         graph.Add_Edge(u, v, inf);
27     }
28     long long answer = (long long)positive - graph.Max_Flow(s, t);
29     vector<int> selected;
30     for ( int u : graph.Cut(s) )
31         if ( u <= n )
32             selected.push_back(u);
33     return {answer, selected};
34 }
```

第一百零四章 图论

104.1 非负权最短路

算法约束见动态版本第 38 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Shortest_Path
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9
10    Shortest_Path(int n) : n(n), g(n + 1)
11    {
12    }
13
14    void Insert(int u, int v, ll w)
15    {
16        assert(w >= 0);
17        g[u].push_back({v, w});
18    }
19
20    void Run(int s)
21    {
22        dis.assign(n + 1, inf);
23        pre.assign(n + 1, -1);
24        dis[s] = 0;
25        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
26        q.push({0, s});
27        while ( !q.empty() )
28        {
29            auto [d, u] = q.top();
30            q.pop();
31            if ( d != dis[u] )
32                continue;
33            for ( auto [v, w] : g[u] )
34                if ( w < inf - d && d + w < dis[v] )
35                {
36                    dis[v] = d + w;
37                    pre[v] = u;
38                    q.push({dis[v], v});
39                }
40        }
41    }
42
43    vector<int> Path(int t) const
44    {
```

```

45     if ( dis[t] == inf )
46         return {};
47     vector<int> ans;
48     for ( ; t != -1; t = pre[t] )
49         ans.push_back(t);
50     reverse(ans.begin(), ans.end());
51     return ans;
52 }
53 };

```

104.2 强连通分量

算法约束见动态版本第 39 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Strong_Component
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
6
7     Strong_Component(int n) : n(n), g(n + 1), rg(n + 1)
8     {
9     }
10
11     void Insert(int u, int v)
12     {
13         g[u].push_back(v);
14         rg[v].push_back(u);
15     }
16
17     void Dfs1(int u, vector<int> &vis, vector<int> &order)
18     {
19         vis[u] = 1;
20         for ( int v : g[u] )
21         {
22             if ( !vis[v] )
23                 Dfs1(v, vis, order);
24         }
25         order.push_back(u);
26     }
27
28     void Dfs2(int u)
29     {
30         bel[u] = cnt;
31         for ( int v : rg[u] )
32         {
33             if ( !bel[v] )
34                 Dfs2(v);
35         }
36     }
37
38     // Recursive Kosaraju; component IDs are in topological order, 1..cnt.
39     void Run()
40     {
41         vector<int> vis(n + 1), order;

```

```

42     cnt = 0;
43     bel.assign(n + 1, 0);
44     for ( int s = 1; s <= n; s++ )
45     {
46         if ( !vis[s] )
47             Dfs1(s, vis, order);
48     }
49     reverse(order.begin(), order.end());
50     for ( int s : order )
51     {
52         if ( !bel[s] )
53         {
54             ++cnt;
55             Dfs2(s);
56         }
57     }
58 }
59 };

```

104.3 2-SAT 与任意可行方案

算法约束见动态版本第 40 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Two_SAT
2 {
3     int n;
4     Strong_Component g;
5     vector<int> ans;
6
7     Two_SAT(int n) : n(n), g(2 * n)
8     {
9     }
10
11     int id(int x, bool value) const
12     {
13         return 2 * x - 1 + value;
14     }
15
16     void Insert(int x, bool a, int y, bool b)
17     {
18         g.Insert(id(x, !a), id(y, b));
19         g.Insert(id(y, !b), id(x, a));
20     }
21
22     bool Solve()
23     {
24         g.Run();
25         ans.assign(n + 1, 0);

```

```

26     for ( int x = 1; x <= n; x++ )
27     {
28         if ( g.bel[id(x, 0)] == g.bel[id(x, 1)] )
29             return false;
30         ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
31     }
32     return true;
33 }
34 };

```

104.4 Hopcroft–Karp 最大匹配与最小点覆盖

算法约束见动态版本第 41 页。本节代码独立可抄；采用下列实际接口名。

声明 `Bipartite_Matching<N,M>` 后 `Init(n,m)`，两侧容量分别为 `N`、`M`。Insert 加边，Solve 求匹配，Cover 返回最小点覆盖；`l`、`r` 保存方案。大对象放在全局或 `static`。

```

1  template <int N, int M> struct Bipartite_Matching
2  {
3      int n, m;
4      vector<int> g[N + 1];
5      int l[N + 1], r[M + 1], dep[N + 1], cur[N + 1], que[N + 1];
6
7      void Init(int left_size, int right_size)
8      {
9          assert(0 <= left_size && left_size <= N && 0 <= right_size && right_size <= M);
10         n = left_size;
11         m = right_size;
12         fill(l, l + n + 1, 0);
13         fill(r, r + m + 1, 0);
14         for ( int u = 1; u <= n; u++ )
15             g[u].clear();
16     }
17
18     void Insert(int u, int v)
19     {
20         assert(1 <= u && u <= n && 1 <= v && v <= m);
21         g[u].push_back(v);
22     }
23
24     bool Bfs()
25     {
26         int head = 0, tail = 0;
27         fill(dep, dep + n + 1, n + 1);
28         fill(cur, cur + n + 1, 0);
29         for ( int u = 1; u <= n; u++ )
30         {
31             if ( !l[u] )
32             {
33                 dep[u] = 0;
34                 que[tail++] = u;
35             }
36         }
37         while ( head < tail )
38         {
39             int u = que[head++];

```

```

40         if ( dep[u] >= dep[0] )
41             continue;
42         for ( int v : g[u] )
43         {
44             int next = r[v];
45             if ( dep[next] != n + 1 )
46                 continue;
47             dep[next] = dep[u] + 1;
48             if ( next )
49                 que[tail++] = next;
50         }
51     }
52     return dep[0] != n + 1;
53 }
54
55 bool Dfs(int u)
56 {
57     if ( !u )
58         return true;
59     for ( int &i = cur[u]; i < (int)g[u].size(); i++ )
60     {
61         int v = g[u][i];
62         if ( dep[r[v]] == dep[u] + 1 && Dfs(r[v]) )
63         {
64             l[u] = v;
65             r[v] = u;
66             return true;
67         }
68     }
69     dep[u] = n + 1;
70     return false;
71 }
72
73 // Retains an existing matching; edges may be added between calls.
74 int Solve()
75 {
76     int answer = 0;
77     for ( int u = 1; u <= n; u++ )
78         answer += (l[u] != 0);
79     while ( Bfs() )
80     {
81         for ( int u = 1; u <= n; u++ )
82         {
83             if ( !l[u] && Dfs(u) )
84                 answer++;
85         }
86     }
87     return answer;
88 }
89
90 // Call after Solve(), before adding more edges. Returns left/right vertex IDs.
91 pair<vector<int>, vector<int>> Cover() const
92 {
93     vector<int> left, right;
94     for ( int u = 1; u <= n; u++ )
95     {

```



```

96         if ( dep[u] == n + 1 )
97             left.push_back(u);
98     }
99     for ( int v = 1; v <= m; v++ )
100     {
101         if ( r[v] && dep[r[v]] != n + 1 )
102             right.push_back(v);
103     }
104     return {left, right};
105 }
106 };

```

104.5 割点与桥

算法约束见动态版本第 [43](#) 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Low_Link
2 {
3     int n, timer = 0, components = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge, delta;
6     int edges = 0;
7
8     Low_Link(int n) : n(n), g(n + 1)
9     {
10    }
11
12    int Insert(int u, int v)
13    {
14        g[u].push_back({v, edges});
15        g[v].push_back({u, edges});
16        return edges++;
17    }

```

```

19 void Dfs(int u, int pe)
20 {
21     dfn[u] = low[u] = ++timer;
22     int children = 0;
23     for ( auto [v, id] : g[u] )
24     {
25         if ( id == pe )
26             continue;
27         if ( !dfn[v] )
28         {
29             ++children;
30             Dfs(v, id);
31             low[u] = min(low[u], low[v]);
32             if ( low[v] > dfn[u] )
33                 bridge[id] = 1;
34             if ( pe != -1 && low[v] >= dfn[u] )
35             {
36                 cut[u] = 1;
37                 delta[u]++;
38             }
39         }
40         else
41             low[u] = min(low[u], dfn[v]);
42     }
43     if ( pe == -1 && children > 1 )
44         cut[u] = 1;
45     if ( pe == -1 )
46         delta[u] = children - 1;
47 }
48
49 void Run()
50 {
51     timer = 0;
52     components = 0;
53     dfn.assign(n + 1, 0);
54     low = dfn;
55     cut = dfn;
56     delta = dfn;
57     bridge.assign(edges, 0);
58     for ( int u = 1; u <= n; u++ )
59         if ( !dfn[u] )
60         {
61             components++;
62             Dfs(u, -1);
63         }
64 }
65 };

```

第一百零五章 字典序最小 2-SAT

105.1 递归染色与回滚求最小赋值

算法约束见动态版本第 45 页。本节代码独立可抄；采用下列实际接口名。

声明 Lex_Two_SAT<N>, Init(n) 清空子句; Add、Force、Solve 与动态版对应, answer[1..n] 为答案。内部 Id 的字面量编号从 0 开始, 大对象放全局或 static。

```
1 template <int N> struct Lex_Two_SAT
2 {
3     int n;
4     vector<int> g[2 * N], history, answer;
5     int marked[2 * N];
6
7     void Init(int variables)
8     {
9         assert(0 <= variables && variables <= N);
10        n = variables;
11        for ( int u = 0; u < 2 * n; u++ )
12            g[u].clear();
13        history.clear();
14        answer.clear();
15    }
16
17    int Id(int x, bool value) const
18    {
19        assert(1 <= x && x <= n);
20        return 2 * (x - 1) + value;
21    }
22
23    // (x == a) OR (y == b). Internal literal IDs are 0-based.
24    void Add(int x, bool a, int y, bool b)
25    {
26        int u = Id(x, a), v = Id(y, b);
27        g[u ^ 1].push_back(v);
28        g[v ^ 1].push_back(u);
29        answer.clear();
30    }
31
32    void Force(int x, bool value)
33    {
34        Add(x, value, x, value);
35    }
```

```

37  bool Paint(int u)
38  {
39      if ( marked[u ^ 1] )
40          return false;
41      if ( marked[u] )
42          return true;
43      marked[u] = 1;
44      history.push_back(u);
45      for ( int v : g[u] )
46          if ( !Paint(v) )
47              return false;
48      return true;
49  }
50
51  bool Solve()
52  {
53      fill(marked, marked + 2 * n, 0);
54      history.clear();
55      answer.clear();
56      for ( int x = 1; x <= n; x++ )
57      {
58          int u = Id(x, false);
59          if ( marked[u] || marked[u ^ 1] )
60              continue;
61          int checkpoint = history.size();
62          if ( !Paint(u) )
63          {
64              while ( (int)history.size() > checkpoint )
65              {
66                  marked[history.back()] = 0;
67                  history.pop_back();
68              }
69              if ( !Paint(u ^ 1) )
70                  return false;
71          }
72      }
73      answer.assign(n + 1, 0);
74      for ( int x = 1; x <= n; x++ )
75          answer[x] = marked[Id(x, true)];
76      return true;
77  }
78  };

```

第一百零六章 有向欧拉路

106.1 有向图字典序最小欧拉路

算法约束见动态版本第 47 页。本节代码独立可抄；采用下列实际接口名。

声明 Directed_Euler<N>, Init(n) 清空图, Insert 加边并返回从 0 开始的边编号, Run 求解（参数默认值同动态版）。vertices、edge_ids、edges 可直接读取；N 为最大点数，大对象放全局或 static。

```
1 template <int N> struct Directed_Euler
2 {
3     int n;
4     vector<pair<int, int>> g[N + 1], edges;
5     vector<int> vertices, edge_ids;
6     int in[N + 1], out[N + 1], next[N + 1];
7
8     void Init(int count)
9     {
10         assert(0 < count && count <= N);
11         n = count;
12         for ( int u = 1; u <= n; u++ )
13         {
14             g[u].clear();
15             in[u] = 0;
16             out[u] = 0;
17         }
18         edges.clear();
19         vertices.clear();
20         edge_ids.clear();
21     }
```

```
23  int Insert(int u, int v)
24  {
25      assert(1 <= u && u <= n && 1 <= v && v <= n);
26      int id = edges.size();
27      edges.push_back({u, v});
28      g[u].push_back({v, id});
29      out[u]++;
30      in[v]++;
31      vertices.clear();
32      edge_ids.clear();
33      return id;
34  }
35
36  void Dfs(int u, int incoming)
37  {
38      while ( next[u] < (int)g[u].size() )
39      {
40          auto [v, id] = g[u][next[u]++];
41          Dfs(v, id);
42      }
43      vertices.push_back(u);
44      if ( incoming != -1 )
45          edge_ids.push_back(incoming);
46  }
```

```

48 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
49 bool Run(int start = 0, bool lexicographic = true)
50 {
51     assert(0 <= start && start <= n);
52     vertices.clear();
53     edge_ids.clear();
54     int source = 0, sink = 0;
55     for ( int u = 1; u <= n; u++ )
56     {
57         int delta = out[u] - in[u];
58         if ( delta == 1 && !source )
59             source = u;
60         else if ( delta == -1 && !sink )
61             sink = u;
62         else if ( delta != 0 )
63             return false;
64     }
65     if ( bool(source) != bool(sink) )
66         return false;
67     if ( source )
68     {
69         if ( start && start != source )
70             return false;
71         start = source;
72     }
73     if ( !start )
74     {
75         start = 1;
76         for ( int u = 1; u <= n; u++ )
77             if ( out[u] )
78             {
79                 start = u;
80                 break;
81             }
82     }
83     if ( lexicographic )
84         for ( int u = 1; u <= n; u++ )
85             sort(g[u].begin(), g[u].end());
86     fill(next, next + n + 1, 0);
87     Dfs(start, -1);
88     if ( edge_ids.size() != edges.size() )
89     {
90         vertices.clear();
91         edge_ids.clear();
92         return false;
93     }
94     reverse(vertices.begin(), vertices.end());
95     reverse(edge_ids.begin(), edge_ids.end());
96     return true;
97 }
98 };

```

第一百零七章 字典序单词链

107.1 字典序最小单词链

算法约束见动态版本第 49 页。本节代码独立可抄；采用下列实际接口名。

```
1 // Lowercase nonempty words. Returns original 0-based IDs; duplicates stay distinct.
2 inline optional<vector<int>> Word_Chain(const vector<string> &words)
3 {
4     int n = words.size();
5     vector<int> order(n);
6     iota(order.begin(), order.end(), 0);
7     for ( const auto &word : words )
8     {
9         assert(!word.empty());
10        for ( char c : word )
11            assert('a' <= c && c <= 'z');
12    }
13    sort(order.begin(),
14         order.end(),
15         [&](int a, int b)
16         {
17             if ( words[a] != words[b] )
18                 return words[a] < words[b];
19             return a < b;
20         });
21    Directed_Euler<26> graph;
22    graph.Init(26);
23    for ( int id : order )
24        graph.Insert(words[id].front() - 'a' + 1, words[id].back() - 'a' + 1);
25    // Preserve word order in each adjacency list; do not sort by destination.
26    if ( !graph.Run(0, false) )
27        return nullopt;
28    vector<int> result;
29    for ( int id : graph.edge_ids )
30        result.push_back(order[id]);
31    return result;
32 }
```


第一百零八章 无向欧拉路

108.1 无向图欧拉路与边方向

算法约束见动态版本第 50 页。本节代码独立可抄；采用下列实际接口名。

声明 `Undirected_Euler<N>`，`Init(n)` 清空图，`Insert` 返回从 0 开始的逻辑边编号，`Run` 参数与动态版一致。
`vertices`、`edge_ids` 和原边 `edges` 可直接读取；`N` 为最大点数，大对象放全局或 `static`。

```
1 template <int N> struct Undirected_Euler
2 {
3     int n;
4     vector<pair<int, int>> g[N + 1], edges;
5     vector<int> used, vertices, edge_ids;
6     int next[N + 1];
7
8     void Init(int count)
9     {
10         assert(0 < count && count <= N);
11         n = count;
12         for ( int u = 1; u <= n; u++ )
13         {
14             g[u].clear();
15         }
16         edges.clear();
17         vertices.clear();
18         edge_ids.clear();
19     }
```

```
21  int Insert(int u, int v)
22  {
23      assert(1 <= u && u <= n && 1 <= v && v <= n);
24      int id = edges.size();
25      edges.push_back({u, v});
26      g[u].push_back({v, id});
27      g[v].push_back({u, id});
28      vertices.clear();
29      edge_ids.clear();
30      return id;
31  }
32
33  void Dfs(int u, int incoming)
34  {
35      while ( next[u] < (int)g[u].size() )
36      {
37          auto [v, id] = g[u][next[u]++];
38          if ( used[id] )
39              continue;
40          used[id] = 1;
41          Dfs(v, id);
42      }
43      vertices.push_back(u);
44      if ( incoming != -1 )
45          edge_ids.push_back(incoming);
46  }
```

```

48 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
49 bool Run(int start = 0, bool lexicographic = true)
50 {
51     assert(0 <= start && start <= n);
52     vertices.clear();
53     edge_ids.clear();
54     vector<int> odd;
55     for ( int u = 1; u <= n; u++ )
56         if ( g[u].size() % 2 )
57             odd.push_back(u);
58     if ( !odd.empty() && odd.size() != 2 )
59         return false;
60     if ( !odd.empty() )
61     {
62         if ( start && start != odd[0] && start != odd[1] )
63             return false;
64         if ( !start )
65             start = odd[0];
66     }
67     if ( !start )
68     {
69         start = 1;
70         for ( int u = 1; u <= n; u++ )
71             if ( !g[u].empty() )
72             {
73                 start = u;
74                 break;
75             }
76     }
77     if ( lexicographic )
78         for ( int u = 1; u <= n; u++ )
79             sort(g[u].begin(), g[u].end());
80     fill(next, next + n + 1, 0);
81     used.assign(edges.size(), 0);
82     Dfs(start, -1);
83     if ( edge_ids.size() != edges.size() )
84     {
85         vertices.clear();
86         edge_ids.clear();
87         return false;
88     }
89     reverse(vertices.begin(), vertices.end());
90     reverse(edge_ids.begin(), edge_ids.end());
91     return true;
92 }
93 };

```

第一百零九章 混合图欧拉定向

109.1 欧拉回路与指定端点定向

算法约束见动态版本第 52 页。本节代码独立可抄；采用下列实际接口名。

调用时额外传入 `Network_Flow<N,M>` 工作区作为临时最大流图；`N` 至少为 `n+2`，`M` 至少为可反转边数 `+n`。
大流对象放全局或 `static`。

```
1 // Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
2 // start=finish=0 requests a circuit; positive endpoints request a specified trail.
3 template <int N, int M>
4 optional<vector<pair<int, int>>>
5 Mixed_Euler_Orientation(int n,
6     const vector<array<int, 3>> &edges,
7     Network_Flow<N, M> &flow,
8     int start = 0,
9     int finish = 0)
10 {
11     assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
12     assert(bool(start) == bool(finish));
13     vector<int> parent(n + 1), id;
14     vector<long long> degree(n + 1), balance(n + 1);
15     iota(parent.begin(), parent.end(), 0);
16     auto find = [&](auto &&self, int u) -> int
17     {
18         if ( parent[u] != u )
19             parent[u] = self(self, parent[u]);
20         return parent[u];
21     };
22     flow.Init(n + 2);
23     vector<pair<int, int>> direction;
24     for ( auto [u, v, type] : edges )
25     {
26         assert(1 <= u && u <= n && 1 <= v && v <= n);
27         assert(type == 0 || type == 1);
28         parent[find(find, u)] = find(find, v);
29         degree[u]++;
30         degree[v]++;
31         balance[u]++;
32         balance[v]--;
33         id.push_back(type == 0 ? flow.Add_Edge(u, v, 1) : -1);
34         direction.push_back({u, v});
35     }
```

```

36     if ( !edges.empty() )
37     {
38         int root = find(find, edges[0][0]);
39         for ( int u = 1; u <= n; u++ )
40             if ( degree[u] && find(find, u) != root )
41                 return nullopt;
42         if ( start && (!degree[start] || !degree[finish]) )
43             return nullopt;
44     }
45     if ( start )
46     {
47         balance[start]--;
48         balance[finish]++;
49     }
50     int source = n + 1, sink = n + 2;
51     long long need = 0;
52     for ( int u = 1; u <= n; u++ )
53     {
54         if ( balance[u] % 2 )
55             return nullopt;
56         if ( balance[u] > 0 )
57         {
58             flow.Add_Edge(source, u, balance[u] / 2);
59             need += balance[u] / 2;
60         }
61         else if ( balance[u] < 0 )
62             flow.Add_Edge(u, sink, -balance[u] / 2);
63     }
64     if ( flow.Max_Flow(source, sink) != need )
65         return nullopt;
66     for ( int i = 0; i < (int)edges.size(); i++ )
67         if ( id[i] != -1 && flow.Used(id[i]) )
68             swap(direction[i].first, direction[i].second);
69     return direction;
70 }

```

109.2 任意起终点的混合欧拉路径

算法约束见动态版本第 54 页。本节代码独立可抄；采用下列实际接口名。

调用时额外传入 `Network_Flow<N,M>` 工作区作为临时最大流图；`N` 至少为 $n+2$ ，`M` 至少为可反转边数 $+n$ 。
大流对象放全局或 `static`。

```

1 // Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
2 // orders.
3 template <int N, int M>
4 optional<vector<pair<int, int>>>
5 Mixed_Euler_Trail(int n, const vector<array<int, 3>> &edges, Network_Flow<N, M> &flow)
6 {
7     assert(n > 0);
8     vector<long long> degree(n + 1);
9     vector<int> odd;
10    for ( auto [u, v, type] : edges )
11    {
12        assert(1 <= u && u <= n && 1 <= v && v <= n);
13        degree[u]++;
14        degree[v]++;
15    }
16    for ( int u = 1; u <= n; u++ )
17        if ( degree[u] % 2 )
18            odd.push_back(u);
19    if ( odd.empty() )
20        return Mixed_Euler_Orientation(n, edges, flow);
21    if ( odd.size() != 2 )
22        return nullopt;
23    auto answer = Mixed_Euler_Orientation(n, edges, flow, odd[0], odd[1]);
24    if ( answer )
25        return answer;
26    return Mixed_Euler_Orientation(n, edges, flow, odd[1], odd[0]);
27 }

```

第一百一十章 图上异或行走

110.1 无向图最大异或行走

算法约束见动态版本第 55 页。本节代码独立可抄；采用下列实际接口名。

声明 `Xor_Walk<N>`，`Init(n)` 清空，`Insert` 加边，`Build(root)` 预处理指定连通块，`Query(u,v)` 返回可选最大值。N 为最大点号，大对象置于全局或 `static`；空间 $O(N+m+64)$ 。DFS 保持递归。

```
1 template <int N> struct Xor_Walk
2 {
3     using U = unsigned long long;
4     int n = 0;
5     bool built = false;
6     vector<pair<int, U>> g[N + 1];
7     int vis[N + 1];
8     U dist[N + 1];
9     Xor_Basis cycles;
10
11     void Init(int size)
12     {
13         assert(0 <= size && size <= N);
14         for ( int u = 1; u <= n; u++ )
15             g[u] = vector<pair<int, U>>();
16         n = size;
17         cycles.Init();
18         built = false;
19     }
```

```

21 void Insert(int u, int v, U w)
22 {
23     assert(1 <= u && u <= n && 1 <= v && v <= n);
24     g[u].push_back({v, w});
25     g[v].push_back({u, w});
26     built = false;
27 }
28
29 void Dfs(int u)
30 {
31     vis[u] = 1;
32     for ( auto [v, w] : g[u] )
33     {
34         if ( !vis[v] )
35         {
36             dist[v] = dist[u] ^ w;
37             Dfs(v);
38         }
39         else
40             cycles.Insert(dist[u] ^ dist[v] ^ w);
41     }
42 }
43
44 void Build(int root = 1)
45 {
46     assert(1 <= root && root <= n);
47     fill(vis, vis + n + 1, 0);
48     fill(dist, dist + n + 1, 0);
49     cycles.Init();
50     Dfs(root);
51     built = true;
52 }
53
54 optional<U> Query(int u, int v) const
55 {
56     assert(built && 1 <= u && u <= n && 1 <= v && v <= n);
57     if ( !vis[u] || !vis[v] )
58         return nullopt;
59     return cycles.Query(dist[u] ^ dist[v]);
60 }
61 };

```


第一百一十一章 树上算法

111.1 重链剖分与 LCA

算法约束见动态版本第 57 页。本节代码独立可抄；采用下列实际接口名。

声明 Heavy_Light-Decomposition<N> 后 Init(n), Insert 加边, Build(root) 剖分; Lca 与 Path 对应动态接口。大对象放在全局或 static。

```
1 template <int N> struct Heavy_Light-Decomposition
2 {
3     int n, timer = 0;
4     vector<int> g[N + 1];
5     int fa[N + 1], dep[N + 1], siz[N + 1], son[N + 1];
6     int top[N + 1], dfn[N + 1], rk[N + 1];
7
8     void Init(int vertices)
9     {
10         assert(0 < vertices && vertices <= N);
11         n = vertices;
12         for ( int u = 1; u <= n; u++ )
13             g[u].clear();
14     }
15
16     void Insert(int u, int v)
17     {
18         g[u].push_back(v);
19         g[v].push_back(u);
20     }
21
22     void Dfs1(int u, int p)
23     {
24         fa[u] = p;
25         siz[u] = 1;
26         son[u] = 0;
27         for ( int v : g[u] )
28         {
29             if ( v == p )
30                 continue;
31             dep[v] = dep[u] + 1;
32             Dfs1(v, u);
33             siz[u] += siz[v];
34             if ( !son[u] || siz[v] > siz[son[u]] )
35                 son[u] = v;
36         }
37     }
38
39     void Dfs2(int u, int t)
40     {
41         top[u] = t;
42         dfn[u] = ++timer;
43         rk[timer] = u;
44         if ( son[u] )
```

```

45     Dfs2(son[u], t);
46     for ( int v : g[u] )
47     {
48         if ( v != fa[u] && v != son[u] )
49             Dfs2(v, v);
50     }
51 }
52
53 // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
54 void Build(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     timer = 0;
58     dep[root] = 0;
59     Dfs1(root, root);
60     Dfs2(root, root);
61     assert(timer == n);
62 }
63
64 int Lca(int u, int v) const
65 {
66     while ( top[u] != top[v] )
67     {
68         if ( dep[top[u]] < dep[top[v]] )
69             swap(u, v);
70         u = fa[top[u]];
71     }
72     return dep[u] < dep[v] ? u : v;
73 }
74
75 // Commutative operations only; edge=true excludes the LCA's position.
76 template <class F> void Path(int u, int v, F work, bool edge = false) const
77 {
78     while ( top[u] != top[v] )
79     {
80         if ( dep[top[u]] < dep[top[v]] )
81             swap(u, v);
82         work(dfn[top[u]], dfn[u]);
83         u = fa[top[u]];
84     }
85     if ( dep[u] > dep[v] )
86         swap(u, v);
87     if ( dfn[u] + edge <= dfn[v] )
88         work(dfn[u] + edge, dfn[v]);
89 }
90 };

```

第一百一十二章 离线最近公共祖先

112.1 离线 Tarjan LCA

算法约束见动态版本第 59 页。本节代码独立可抄；采用下列实际接口名。

声明 `Offline_LCA<N>`，`Init(n)` 清空树与查询；`Insert` 加无向树边，`Add_Query` 返回从 0 开始的查询编号，`Run(root)` 统一计算，`answer[id]` 为答案。大对象放全局或 `static`。

```
1 template <int N> struct Offline_LCA
2 {
3     int n;
4     vector<int> g[N + 1], answer;
5     vector<pair<int, int>> queries[N + 1];
6     int parent[N + 1], size[N + 1], ancestor[N + 1], done[N + 1];
7
8     void Init(int vertices)
9     {
10         assert(0 <= vertices && vertices <= N);
11         n = vertices;
12         for ( int u = 1; u <= n; u++ )
13         {
14             g[u].clear();
15             queries[u].clear();
16         }
17         answer.clear();
18     }
19
20     void Insert(int u, int v)
21     {
22         assert(1 <= u && u <= n && 1 <= v && v <= n);
23         g[u].push_back(v);
24         g[v].push_back(u);
25     }
26
27     int Add_Query(int u, int v)
28     {
29         assert(1 <= u && u <= n && 1 <= v && v <= n);
30         int id = answer.size();
31         answer.push_back(0);
32         queries[u].push_back({v, id});
33         queries[v].push_back({u, id});
34         return id;
35     }
```

```

37     int Find(int u)
38     {
39         if ( parent[u] != u )
40             parent[u] = Find(parent[u]);
41         return parent[u];
42     }
43
44     void Merge(int u, int v)
45     {
46         u = Find(u);
47         v = Find(v);
48         if ( size[u] < size[v] )
49             swap(u, v);
50         parent[v] = u;
51         size[u] += size[v];
52     }
53
54     void Dfs(int u, int p)
55     {
56         for ( int v : g[u] )
57             if ( v != p )
58             {
59                 Dfs(v, u);
60                 Merge(u, v);
61                 ancestor[Find(u)] = u;
62             }
63         done[u] = 1;
64         for ( auto [v, id] : queries[u] )
65             if ( done[v] )
66                 answer[id] = ancestor[Find(v)];
67     }
68
69     // Input must be a nonempty tree. Query IDs are 0-based.
70     void Run(int root = 1)
71     {
72         assert(1 <= root && root <= n);
73         iota(parent, parent + n + 1, 0);
74         iota(ancestor, ancestor + n + 1, 0);
75         fill(size, size + n + 1, 1);
76         fill(done, done + n + 1, 0);
77         fill(answer.begin(), answer.end(), 0);
78         Dfs(root, 0);
79     }
80 };

```

第一百一十三章 在线最近公共祖先

113.1 欧拉序与 ST 表 LCA

算法约束见动态版本第 61 页。本节代码独立可抄；采用下列实际接口名。

声明 Euler_LCA<N,LOG>, Init(n) 清空树；Insert 加边，Build(root) 预处理，Lca 与 Distance 在线查询，Weighted_Distance 返回带权距离。LOG 层必须满足 2 的 LOG 次方大于 $2n-1$ ，默认 20 支持 50 万点；st 固定容量为 LOG 乘以 $2N$ 。大对象放全局或 static。

```
1 template <int N, int LOG = 20> struct Euler_LCA
2 {
3     static_assert(1 <= LOG && LOG <= 31);
4     using ll = long long;
5     int n;
6     bool ready = false;
7     vector<int> g[N + 1], euler;
8     vector<ll> cost[N + 1];
9     ll root_dist[N + 1];
10    int first[N + 1], depth[N + 1], lg[2 * N + 1];
11    int st[LOG][2 * N];
12
13    void Init(int vertices)
14    {
15        assert(0 < vertices && vertices <= N);
16        assert((1LL << LOG) > 2LL * vertices - 1);
17        n = vertices;
18        for (int u = 1; u <= n; u++)
19        {
20            g[u].clear();
21            cost[u].clear();
22        }
23        euler.clear();
24        ready = false;
25    }
26
27    void Insert(int u, int v, ll w = 1)
28    {
29        assert(1 <= u && u <= n && 1 <= v && v <= n);
30        assert(w >= 0);
31        g[u].push_back(v);
32        g[v].push_back(u);
33        cost[u].push_back(w);
34        cost[v].push_back(w);
35        ready = false;
36    }
37
38    int Shallower(int u, int v) const
39    {
40        return depth[u] <= depth[v] ? u : v;
41    }
```

递归欧拉序与 RMQ (接上页同一结构体):

```

43 void Dfs(int u, int parent)
44 {
45     first[u] = euler.size();
46     euler.push_back(u);
47     for ( int i = 0; i < int(g[u].size()); i++ )
48     {
49         int v = g[u][i];
50         if ( v != parent )
51         {
52             depth[v] = depth[u] + 1;
53             root_dist[v] = root_dist[u] + cost[u][i];
54             Dfs(v, u);
55             euler.push_back(u);
56         }
57     }
58 }
59
60 // Input must be a nonempty tree. Rebuilding may change the root.
61 void Build(int root = 1)
62 {
63     assert(1 <= root && root <= n);
64     euler.clear();
65     depth[root] = 0;
66     root_dist[root] = 0;
67     Dfs(root, 0);
68     int m = euler.size();
69     lg[0] = 0;
70     lg[1] = 0;
71     for ( int i = 2; i <= m; i++ )
72         lg[i] = lg[i / 2] + 1;
73     copy(euler.begin(), euler.end(), st[0]);
74     for ( int k = 1; k <= lg[m]; k++ )
75     {
76         for ( int i = 0; i + (1 << k) <= m; i++ )
77             st[k][i] = Shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
78     }
79     ready = true;
80 }

```

LCA、边数距离与带权距离（接上页同一结构体）：

```
82     int Lca(int u, int v) const
83     {
84         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
85         int l = first[u], r = first[v];
86         if ( l > r )
87             swap(l, r);
88         int k = lg[r - l + 1];
89         return Shallower(st[k][l], st[k][r - (1 << k) + 1]);
90     }
91
92     int Distance(int u, int v) const
93     {
94         int z = Lca(u, v);
95         return depth[u] + depth[v] - 2 * depth[z];
96     }
97
98     ll Weighted_Distance(int u, int v) const
99     {
100         int z = Lca(u, v);
101         return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
102     }
103 };
```

第一百一十四章 树上点集摘要

114.1 树上点集直径摘要

算法约束见动态版本第 64 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Tree_Diameter
2 {
3     using ll = long long;
4     int a = -1, b = -1;
5     ll length = 0;
6
7     // distance is the metric of one fixed tree with nonnegative edge weights.
8     template <class Distance> void Insert(int v, const Distance &distance)
9     {
10         assert(v >= 0);
11         if ( a == -1 )
12         {
13             a = v;
14             b = v;
15             return;
16         }
17         ll x = distance(a, v), y = distance(b, v);
18         if ( x >= y && x > length )
19         {
20             b = v;
21             length = x;
22         }
23         else if ( y > length )
24         {
25             a = v;
26             length = y;
27         }
28     }
29
30     template <class Distance> void Merge(Tree_Diameter other, const Distance &distance)
31     {
32         if ( other.a == -1 )
33             return;
34         Insert(other.a, distance);
35         Insert(other.b, distance);
36     }
37
38     template <class Distance>
39     optional<pair<int, ll>> Farthest(int v, const Distance &distance) const
40     {
41         if ( a == -1 )
42             return nullopt;
43         ll x = distance(a, v), y = distance(b, v);
44         return x >= y ? make_pair(a, x) : make_pair(b, y);
45     }
46 };
```


第一百一十五章 倍增祖先与路径最值

115.1 倍增 LCA、祖先与最大边权

算法约束见动态版本第 66 页。本节代码独立可抄；采用下列实际接口名。

声明 `Lifting_LCA<N,LOG>`, `Init(n)` 清空树；`Insert`、`Build`、`Jump`、`Lca`、`Distance`、`Max_Edge` 对应动态接口。LOG 层要求 2 的 LOG 次方大于 n，默认 20；大对象放全局或 static。

```
1 template <int N, int LOG = 20> struct Lifting_LCA
2 {
3     using ll = long long;
4     static constexpr ll LOW = numeric_limits<ll>::lowest();
5     static_assert(1 <= LOG && LOG <= 31);
6     int n;
7     bool ready = false;
8     vector<pair<int, ll>> g[N + 1];
9     int up[LOG][N + 1], depth[N + 1];
10    ll mx[LOG][N + 1];
11
12    void Init(int vertices)
13    {
14        assert(0 < vertices && vertices <= N && (1LL << LOG) > vertices);
15        n = vertices;
16        for (int u = 1; u <= n; u++)
17            g[u].clear();
18        for (int k = 0; k < LOG; k++)
19        {
20            up[k][0] = 0;
21            mx[k][0] = LOW;
22        }
23        ready = false;
24    }
25
26    void Insert(int u, int v, ll w = 1)
27    {
28        assert(1 <= u && u <= n && 1 <= v && v <= n);
29        g[u].push_back({v, w});
30        g[v].push_back({u, w});
31        ready = false;
32    }
```

```

34 void Dfs(int u, int parent, ll weight)
35 {
36     up[0][u] = parent;
37     mx[0][u] = weight;
38     for ( int k = 1; k < LOG; k++ )
39     {
40         int p = up[k - 1][u];
41         up[k][u] = up[k - 1][p];
42         mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
43     }
44     for ( auto [v, w] : g[u] )
45         if ( v != parent )
46         {
47             depth[v] = depth[u] + 1;
48             Dfs(v, u, w);
49         }
50 }
51
52 // Nonempty tree only. Node 0 means no ancestor.
53 void Build(int root = 1)
54 {
55     assert(1 <= root && root <= n);
56     depth[root] = 0;
57     Dfs(root, 0, LOW);
58     ready = true;
59 }
60
61 int Jump(int u, long long steps) const
62 {
63     assert(ready && 1 <= u && u <= n && steps >= 0);
64     if ( steps > depth[u] )
65         return 0;
66     for ( int k = 0; k < LOG; k++ )
67         if ( steps >> k & 1 )
68             u = up[k][u];
69     return u;
70 }

```

```

72  int Lca(int u, int v) const
73  {
74      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
75      if ( depth[u] < depth[v] )
76          swap(u, v);
77      u = Jump(u, depth[u] - depth[v]);
78      if ( u == v )
79          return u;
80      for ( int k = LOG - 1; k >= 0; k-- )
81          if ( up[k][u] != up[k][v] )
82              {
83                  u = up[k][u];
84                  v = up[k][v];
85              }
86      return up[0][u];
87  }
88
89  int Distance(int u, int v) const
90  {
91      return depth[u] + depth[v] - 2 * depth[Lca(u, v)];
92  }
93
94  optional<ll> Max_Edge(int u, int v) const
95  {
96      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
97      if ( u == v )
98          return nullopt;
99      if ( depth[u] < depth[v] )
100          swap(u, v);
101      ll answer = LOW;
102      int delta = depth[u] - depth[v];
103      for ( int k = 0; k < LOG; k++ )
104          if ( delta >> k & 1 )
105              {
106                  answer = max(answer, mx[k][u]);
107                  u = up[k][u];
108              }
109      if ( u == v )
110          return answer;
111      for ( int k = LOG - 1; k >= 0; k-- )
112          if ( up[k][u] != up[k][v] )
113              {
114                  answer = max(answer, max(mx[k][u], mx[k][v]));
115                  u = up[k][u];
116                  v = up[k][v];
117              }
118      return max(answer, max(mx[0][u], mx[0][v]));
119  }
120 };

```

第一百一十六章 树上路径交

116.1 两条树路径的交集

算法约束见动态版本第 69 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Path_Intersection_Result
2 {
3     int u, v, vertices;
4 };
5
6 // Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
7 template <class Depth, class Lca>
8 Path_Intersection_Result
9 Path_Intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
10 {
11     int z = lca(a, b);
12     auto project = [&](int x)
13     {
14         int p = z;
15         for ( int v : {lca(a, x), lca(b, x)} )
16             if ( depth[v] > depth[p] )
17                 p = v;
18         return p;
19     };
20     auto distance = [&](int u, int v)
21     {
22         return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
23     };
24     int p = project(c), q = project(d);
25     if ( p == q && distance(c, p) + distance(p, d) != distance(c, d) )
26         return {0, 0, 0};
27     return {p, q, int(distance(p, q) + 1)};
28 }
```

第一百一十七章 点分治

117.1 点分治：树上距离点对计数

算法约束见动态版本第 70 页。本节代码独立可抄；采用下列实际接口名。

声明 Centroid_Pairs<N>, Init(n) 初始化, Insert 加边, Build 预处理; Count_Leq 和 Count_Exact 返回计数, parent 保存点分树父节点。大对象放在全局或 static。

```
1 template <int N> struct Centroid_Pairs
2 {
3     using ll = long long;
4     int n;
5     vector<pair<int, ll>> g[N + 1];
6     int siz[N + 1], removed[N + 1], parent[N + 1];
7     vector<ll> all[N + 1];
8     vector<vector<ll>> part[N + 1];
9
10    void Init(int vertices)
11    {
12        assert(0 <= vertices && vertices <= N);
13        n = vertices;
14        for ( int u = 1; u <= n; u++ )
15            g[u].clear();
16    }
17
18    void Insert(int u, int v, ll w)
19    {
20        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
21        g[u].push_back({v, w});
22        g[v].push_back({u, w});
23    }
24
25    void Size_Dfs(int u, int p)
26    {
27        siz[u] = 1;
28        for ( auto [v, w] : g[u] )
29        {
30            if ( v == p || removed[v] )
31                continue;
32            Size_Dfs(v, u);
33            siz[u] += siz[v];
34        }
35    }
36
37    int Centroid(int u, int p, int total)
38    {
39        for ( auto [v, w] : g[u] )
40        {
41            if ( v != p && !removed[v] && siz[v] > total / 2 )
42                return Centroid(v, u, total);
43        }
44        return u;
```

```

45     }
46
47     void Collect(int u, int p, ll d, vector<ll> &out)
48     {
49         out.push_back(d);
50         for ( auto [v, w] : g[u] )
51         {
52             if ( v == p || removed[v] )
53                 continue;
54             assert(d <= numeric_limits<ll>::max() - w);
55             Collect(v, u, d + w, out);
56         }
57     }
58
59     void Decompose(int entry, int p)
60     {
61         Size_Dfs(entry, 0);
62         int c = Centroid(entry, 0, siz[entry]);
63         removed[c] = 1;
64         parent[c] = p;
65         all[c] = {0};
66         for ( auto [v, w] : g[c] )
67         {
68             if ( removed[v] )
69                 continue;
70             part[c].push_back({});
71             auto &d = part[c].back();
72             Collect(v, c, w, d);
73             sort(d.begin(), d.end());
74             all[c].insert(all[c].end(), d.begin(), d.end());
75         }
76         sort(all[c].begin(), all[c].end());
77         for ( auto [v, w] : g[c] )
78         {
79             if ( !removed[v] )
80                 Decompose(v, c);
81         }
82     }
83
84     // Input must be a forest. Rebuild after changing edges.
85     void Build()
86     {
87         fill(removed, removed + n + 1, 0);
88         for ( int u = 1; u <= n; u++ )
89         {
90             all[u].clear();
91             part[u].clear();
92         }
93         for ( int u = 1; u <= n; u++ )
94         {
95             if ( !removed[u] )
96                 Decompose(u, 0);
97         }
98     }
99
100     ll Pairs(const vector<ll> &d, ll k) const

```

```

101 {
102     int l = 0, r = (int)d.size() - 1;
103     ll answer = 0;
104     while ( l < r )
105     {
106         if ( d[l] <= k - d[r] )
107         {
108             answer += r - l;
109             l++;
110         }
111         else
112             r--;
113     }
114     return answer;
115 }
116
117 // Unordered Pairs of distinct connected vertices. Nonnegative edge weights.
118 ll Count_Leq(ll k) const
119 {
120     if ( k < 0 )
121         return 0;
122     ll answer = 0;
123     for ( int c = 1; c <= n; c++ )
124     {
125         answer += Pairs(all[c], k);
126         for ( const auto &d : part[c] )
127             answer -= Pairs(d, k);
128     }
129     return answer;
130 }
131
132 ll Count_Exact(ll k) const
133 {
134     if ( k < 0 )
135         return 0;
136     return Count_Leq(k) - (k ? Count_Leq(k - 1) : 0);
137 }
138 };

```

第一百一十八章 树上启发式合并

118.1 树上启发式合并：子树众数颜色和

算法约束见动态版本第 73 页。本节代码独立可抄；采用下列实际接口名。

声明 Subtree_Colors<N>, Init(n) 初始化, Insert 加边, Run(colors,root) 计算; answer 与 max_freq 为最终结果。Add_Color 为内部统计函数。大对象放在全局或 static。

```
1 template <int N> struct Subtree_Colors
2 {
3     int n, best = 0;
4     long long sum = 0;
5     vector<int> g[N + 1], order, value, freq;
6     int siz[N + 1], heavy[N + 1], tin[N + 1], tout[N + 1];
7     int color[N + 1], max_freq[N + 1];
8     long long answer[N + 1];
9
10    void Init(int vertices)
11    {
12        assert(0 <= vertices && vertices <= N);
13        n = vertices;
14        for ( int u = 1; u <= n; u++ )
15            g[u].clear();
16    }
17
18    void Insert(int u, int v)
19    {
20        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
21        g[u].push_back(v);
22        g[v].push_back(u);
23    }
24
25    void Prepare(int u, int p)
26    {
27        siz[u] = 1;
28        tin[u] = order.size();
29        order.push_back(u);
30        for ( int v : g[u] )
31        {
32            if ( v == p )
33                continue;
34            Prepare(v, u);
35            siz[u] += siz[v];
36            if ( siz[v] > siz[heavy[u]] )
37                heavy[u] = v;
38        }
39        tout[u] = (int)order.size() - 1;
40    }
41
42    void Add_Color(int u)
43    {
44        int c = color[u];
```



```

45     freq[c]++;
46     if ( freq[c] > best )
47     {
48         best = freq[c];
49         sum = value[c];
50     }
51     else if ( freq[c] == best )
52         sum += value[c];
53 }
54
55 void Dfs(int u, int p, bool keep)
56 {
57     for ( int v : g[u] )
58     {
59         if ( v != p && v != heavy[u] )
60             Dfs(v, u, false);
61     }
62     if ( heavy[u] )
63         Dfs(heavy[u], u, true);
64     for ( int v : g[u] )
65     {
66         if ( v == p || v == heavy[u] )
67             continue;
68         for ( int i = tin[v]; i <= tout[v]; i++ )
69             Add_Color(order[i]);
70     }
71     Add_Color(u);
72     max_freq[u] = best;
73     answer[u] = sum;
74     if ( !keep )
75     {
76         for ( int i = tin[u]; i <= tout[u]; i++ )
77             freq[color[order[i]]]--;
78         best = 0;
79         sum = 0;
80     }
81 }
82
83 // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
84 void Run(const vector<int> &colors, int root = 1)
85 {
86     assert((int)colors.size() == n + 1);
87     fill(siz, siz + n + 1, 0);
88     fill(heavy, heavy + n + 1, 0);
89     fill(tin, tin + n + 1, 0);
90     fill(tout, tout + n + 1, 0);
91     fill(color, color + n + 1, 0);
92     fill(max_freq, max_freq + n + 1, 0);
93     fill(answer, answer + n + 1, 0);
94     order.clear();
95     value.assign(colors.begin() + 1, colors.end());
96     sort(value.begin(), value.end());
97     value.erase(unique(value.begin(), value.end()), value.end());
98     freq.assign(value.size(), 0);
99     best = 0;
100    sum = 0;

```

```
101     if ( !n )
102         return;
103     assert(1 <= root && root <= n);
104     for ( int u = 1; u <= n; u++ )
105         color[u] =
106             lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
107     Prepare(root, 0);
108     Dfs(root, 0, false);
109 }
110 };
```

第一百一十九章 虚树

119.1 虚树：关键点压缩与路径信息

算法约束见动态版本第 76 页。本节代码独立可抄；采用下列实际接口名。

声明 `Virtual_Tree<N,LOG>`，要求 2 的 LOG 次方大于实际点数，默认 LOG=20。Init(n)、Insert、Prepare、Build 对应初始化、加边、预处理和查询；Lca、Ancestor、Path 为对应辅助接口。大对象放在全局或 static。

```
1 template <int N, int LOG = 20> struct Virtual_Tree
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll distance, minimum;
9     };
10
11     struct Result
12     {
13         vector<int> vertices;
14         vector<Edge> edges;
15     };
16
17     int n, log = 1, timer = 0;
18     vector<pair<int, ll>> g[N + 1];
19     int up[LOG][N + 1], depth[N + 1], tin[N + 1], tout[N + 1];
20     ll mn[LOG][N + 1], dist[N + 1];
21
22     void Init(int vertices)
23     {
24         assert(0 < vertices && vertices <= N);
25         n = vertices;
26         for (int u = 1; u <= n; u++)
27             g[u].clear();
28     }
29
30     void Insert(int u, int v, ll w = 1)
31     {
32         assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
33         g[u].push_back({v, w});
34         g[v].push_back({u, w});
35     }
36
37     void Dfs(int u, int p)
38     {
39         tin[u] = ++timer;
40         up[0][u] = p;
41         for (int j = 1; j < log; j++)
42         {
43             up[j][u] = up[j - 1][up[j - 1][u]];
44             mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
```

```

45     }
46     for ( auto [v, w] : g[u] )
47     {
48         if ( v == p )
49             continue;
50         assert(dist[u] <= numeric_limits<ll>::max() - w);
51         depth[v] = depth[u] + 1;
52         dist[v] = dist[u] + w;
53         mn[0][v] = w;
54         Dfs(v, u);
55     }
56     tout[u] = timer;
57 }
58
59 void Prepare(int root = 1)
60 {
61     assert(1 <= root && root <= n);
62     log = 1;
63     while ( (1LL << log) <= n )
64         log++;
65     assert(log <= LOG);
66     for ( int j = 0; j < log; j++ )
67     {
68         fill(up[j], up[j] + n + 1, 0);
69         fill(mn[j], mn[j] + n + 1, numeric_limits<ll>::max());
70     }
71     fill(depth, depth + n + 1, 0);
72     fill(tin, tin + n + 1, 0);
73     fill(tout, tout + n + 1, 0);
74     fill(dist, dist + n + 1, 0);
75     timer = 0;
76     Dfs(root, root);
77 }
78
79 bool Ancestor(int u, int v) const
80 {
81     return tin[u] <= tin[v] && tout[v] <= tout[u];
82 }
83
84 int Lca(int u, int v) const
85 {
86     if ( Ancestor(u, v) )
87         return u;
88     if ( Ancestor(v, u) )
89         return v;
90     for ( int j = log - 1; j >= 0; j-- )
91     {
92         if ( !Ancestor(up[j][u], v) )
93             u = up[j][u];
94     }
95     return up[0][u];
96 }
97
98 Edge Path(int u, int v) const
99 {
100     assert(u != v && Ancestor(u, v));

```

```

101     ll minimum = numeric_limits<ll>::max();
102     int x = v, steps = depth[v] - depth[u];
103     for ( int j = 0; j < log; j++ )
104     {
105         if ( steps >> j & 1 )
106         {
107             minimum = min(minimum, mn[j][x]);
108             x = up[j][x];
109         }
110     }
111     return {u, v, dist[v] - dist[u], minimum};
112 }
113
114 // Uses original vertex IDs. Empty input gives an empty result.
115 Result Build(vector<int> keys) const
116 {
117     if ( keys.empty() )
118         return {};
119     for ( int u : keys )
120         assert(1 <= u && u <= n);
121     auto cmp = [&](int u, int v)
122     {
123         return tin[u] < tin[v];
124     };
125     sort(keys.begin(), keys.end(), cmp);
126     keys.erase(unique(keys.begin(), keys.end()), keys.end());
127     int k = keys.size();
128     for ( int i = 1; i < k; i++ )
129         keys.push_back(Lca(keys[i - 1], keys[i]));
130     sort(keys.begin(), keys.end(), cmp);
131     keys.erase(unique(keys.begin(), keys.end()), keys.end());
132     Result result{keys, {}};
133     vector<int> st;
134     for ( int u : keys )
135     {
136         while ( !st.empty() && !Ancestor(st.back(), u) )
137             st.pop_back();
138         if ( !st.empty() )
139             result.edges.push_back(Path(st.back(), u));
140         st.push_back(u);
141     }
142     return result;
143 }
144 };

```

第一百二十章 字符串

120.1 前缀函数

算法约束见动态版本第 79 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<int> Prefix_Function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for ( int i = 1; i < n; i++ )
6     {
7         int j = p[i - 1];
8         while ( j && s[i] != s[j] )
9             j = p[j - 1];
10        p[i] = j + (s[i] == s[j]);
11    }
12    return p;
13 }
```

120.2 KMP 匹配

算法约束见动态版本第 79 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<int> Kmp_Match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = Prefix_Function(t);
5     vector<int> ans;
6     for ( int i = 0, j = 0; i < (int)s.size(); i++ )
7     {
8         while ( j && s[i] != t[j] )
9             j = p[j - 1];
10        j += (s[i] == t[j]);
11        if ( j == (int)t.size() )
12        {
13            ans.push_back(i - j + 1);
14            j = p[j - 1];
15        }
16    }
17    return ans;
18 }
```

120.3 Z 函数

算法约束见动态版本第 80 页。本节代码独立可抄；采用下列实际接口名。

```

1 inline vector<int> Z_Function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if ( n )
6         a[0] = n;
7     for ( int i = 1, l = 0, r = 0; i < n; i++ )
8     {
9         if ( i < r )
10            a[i] = min(r - i, a[i - l]);
11        while ( i + a[i] < n && s[a[i]] == s[i + a[i]] )
12            ++a[i];
13        if ( i + a[i] > r )
14        {
15            l = i;
16            r = i + a[i];
17        }
18    }
19    return a;
20 }
```

120.4 Manacher 回文半径

算法约束见动态版本第 80 页。本节代码独立可抄；采用下列实际接口名。

```

1 // odd[i] includes the center; even[i] is centered before i.
2 inline pair<vector<int>, vector<int>> Manacher(const string &s)
3 {
4     int n = (int)s.size();
5     vector<int> odd(n), even(n);
6     for ( int i = 0, l = 0, r = -1; i < n; i++ )
7     {
8         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
9         while ( i - k >= 0 && i + k < n && s[i - k] == s[i + k] )
10            ++k;
11        odd[i] = k--;
12        if ( i + k > r )
13        {
14            l = i - k;
15            r = i + k;
16        }
17    }
18    for ( int i = 0, l = 0, r = -1; i < n; i++ )
19    {
20        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
21        while ( i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k] )
22            ++k;
```

```

23     even[i] = k--;
24     if ( i + k > r )
25     {
26         l = i - k - 1;
27         r = i + k;
28     }
29 }
30 return {odd, even};
31 }

```

120.5 最小循环表示

算法约束见动态版本第 81 页。本节代码独立可抄；采用下列实际接口名。

```

1 inline int Minimum_Rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if ( !n )
5         return 0;
6     while ( i < n && j < n && k < n )
7     {
8         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
9         if ( a == b )
10            {
11                ++k;
12                continue;
13            }
14         if ( a > b )
15            {
16                i += k + 1;
17                if ( i == j )
18                    ++i;
19            }
20         else
21            {
22                j += k + 1;
23                if ( i == j )
24                    ++j;
25            }
26         k = 0;
27     }
28     return min(i, j);
29 }

```

120.6 AC 自动机出现次数

算法约束见动态版本第 81 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct AC_Automaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int fail = 0;

```



```

7     };
8
9     vector<Node> a{Node{}};
10    vector<int> order;
11    bool built = false;
12
13    int Insert(const string &s)
14    {
15        assert(!built && !s.empty());
16        int u = 0;
17        for ( char ch : s )
18        {
19            int c = ch - 'a';
20            assert(0 <= c && c < 26);
21            if ( !a[u].go[c] )
22            {
23                int v = (int)a.size();
24                a[u].go[c] = v;
25                a.push_back(Node{});
26            }
27            u = a[u].go[c];
28        }
29        return u;
30    }
31
32    void Build()
33    {
34        assert(!built);
35        built = true;
36        queue<int> q;
37        for ( int v : a[0].go )
38            if ( v )
39                q.push(v);
40        while ( !q.empty() )
41        {
42            int u = q.front();
43            q.pop();
44            order.push_back(u);
45            for ( int c = 0; c < 26; c++ )
46            {
47                int v = a[u].go[c];
48                if ( v )
49                {
50                    a[v].fail = a[a[u].fail].go[c];
51                    q.push(v);
52                }
53                else
54                    a[u].go[c] = a[a[u].fail].go[c];
55            }
56        }
57    }
58
59    vector<long long> count(const string &s) const
60    {
61        assert(built);
62        vector<long long> ans(a.size());

```

```
63     int u = 0;
64     for ( char ch : s )
65     {
66         u = a[u].go[ch - 'a'];
67         ++ans[u];
68     }
69     for ( int i = (int)order.size() - 1; i >= 0; i-- )
70         ans[a[order[i]].fail] += ans[order[i]];
71     return ans;
72 }
73 };
```

120.7 倍增后缀数组与 LCP

算法约束见动态版本第 84 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Suffix_Array
2 {
3     vector<int> sa, rk, lcp;
4
5     Suffix_Array(const string &s)
6     {
7         vector<int> a;
8         a.reserve(s.size());
9         for ( unsigned char c : s )
10             a.push_back(c);
11         Init(a, 256);
12     }
13
14     Suffix_Array(const vector<int> &s, int alphabet)
15     {
16         Init(s, alphabet);
17     }
18
19     void Init(const vector<int> &s, int alphabet)
20     {
21         assert(s.size() < INT_MAX);
22         assert(1 <= alphabet && alphabet < INT_MAX);
23         int n = (int)s.size();
24         sa.resize(n);
25         rk.resize(n);
26         lcp.assign(n, 0);
27         if ( !n )
28             return;
29         vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
30         int m = alphabet;
31         for ( int i = 0; i < n; i++ )
32         {
33             assert(0 <= s[i] && s[i] < alphabet);
34             rk[i] = s[i] + 1;
35             cnt[rk[i]]++;
36         }
37         for ( int i = 1; i <= m; i++ )
38             cnt[i] += cnt[i - 1];
39         for ( int i = n - 1; i >= 0; i-- )
40             sa[--cnt[rk[i]]] = i;

```

```

41     for ( int k = 1; k < n; k *= 2 )
42     {
43         int p = 0;
44         for ( int i = n - k; i < n; i++ )
45             order[p++] = i;
46         for ( int i : sa )
47             if ( i >= k )
48                 order[p++] = i - k;
49         fill(cnt.begin(), cnt.begin() + m + 1, 0);
50         for ( int x : rk )
51             cnt[x]++;
52         for ( int i = 1; i <= m; i++ )
53             cnt[i] += cnt[i - 1];
54         for ( int i = n - 1; i >= 0; i-- )
55             sa[--cnt[rk[order[i]]]] = order[i];
56         old.swap(rk);
57         auto second = [&](int i)
58         {
59             return i < n - k ? old[i + k] : 0;
60         };
61         m = 1;
62         rk[sa[0]] = m;
63         for ( int i = 1; i < n; i++ )
64         {
65             int a = sa[i - 1], b = sa[i];
66             if ( old[a] != old[b] || second(a) != second(b) )
67                 m++;
68             rk[b] = m;
69         }
70         if ( m == n )
71             break;
72     }
73     for ( int i = 0; i < n; i++ )
74         rk[sa[i]] = i;
75     for ( int i = 0, k = 0; i < n; i++ )
76     {
77         if ( !rk[i] )
78         {
79             k = 0;
80             continue;
81         }
82         int j = sa[rk[i] - 1];
83         while ( k < n - i && k < n - j && s[i + k] == s[j + k] )
84             ++k;
85         lcp[rk[i]] = k;
86         if ( k )
87             --k;
88     }
89 }
90 };

```

120.8 后缀自动机

算法约束见动态版本第 86 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Suffix_Automaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
12
13    void Extend(int c)
14    {
15        assert(0 <= c && c < 26);
16        int cur = (int)a.size();
17        a.push_back(Node{});
18        a[cur].len = a[last].len + 1;
19        a[cur].occ = 1;
20        int p = last;
21        while ( p != -1 && !a[p].go[c] )
22        {
23            a[p].go[c] = cur;
24            p = a[p].link;
25        }
26        if ( p == -1 )
27            a[cur].link = 0;
28        else
29        {
30            int q = a[p].go[c];
31            if ( a[p].len + 1 == a[q].len )
32                a[cur].link = q;
33            else
34            {
35                int clone = (int)a.size();
36                Node copy = a[q];
37                a.push_back(copy);
38                a[clone].len = a[p].len + 1;
39                a[clone].occ = 0;
40                while ( p != -1 && a[p].go[c] == q )
41                {
42                    a[p].go[c] = clone;
43                    p = a[p].link;
44                }
45                a[q].link = a[cur].link = clone;
46            }
47        }
48        last = cur;
49    }
50
51    vector<long long> Counts() const
52    {

```

```
53     vector<int> bucket(a[last].len + 1), order(a.size());
54     vector<long long> ans(a.size());
55     for ( auto &v : a )
56         ++bucket[v.len];
57     partial_sum(bucket.begin(), bucket.end(), bucket.begin());
58     for ( int i = 0; i < (int)a.size(); i++ )
59     {
60         order[--bucket[a[i].len]] = i;
61         ans[i] = a[i].occ;
62     }
63     for ( int i = (int)order.size() - 1; i > 0; i-- )
64         ans[a[order[i]].link] += ans[order[i]];
65     return ans;
66 }
67
68 long long Distinct() const
69 {
70     long long ans = 0;
71     for ( int i = 1; i < (int)a.size(); i++ )
72         ans += a[i].len - a[a[i].link].len;
73     return ans;
74 }
75 };
```

第一百二十一章 后缀查询与重复子串

121.1 后缀 LCP 与子串比较

算法约束见动态版本第 88 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Suffix_LCP
2 {
3     int n;
4     vector<int> rk, lg;
5     vector<vector<int>> st;
6
7     Suffix_LCP(const Suffix_Array &suffix)
8     {
9         Init(suffix);
10    }
11
12    void Init(const Suffix_Array &suffix)
13    {
14        n = suffix.sa.size();
15        rk = suffix.rk;
16        lg.assign(n + 1, 0);
17        for ( int i = 2; i <= n; i++ )
18            lg[i] = lg[i / 2] + 1;
19        st.assign(lg[n] + 1, {});
20        st[0] = suffix.lcp;
21        for ( int k = 1; k <= lg[n]; k++ )
22        {
23            int len = 1 << k;
24            st[k].resize(n - len + 1);
25            for ( int i = 0; i <= n - len; i++ )
26                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
27        }
28    }
```

```

30  int Query(int x, int y) const
31  {
32      assert(0 <= x && x <= n && 0 <= y && y <= n);
33      if ( x == y || x == n || y == n )
34          return n - max(x, y);
35      int a = rk[x], b = rk[y];
36      if ( a > b )
37          swap(a, b);
38      int k = lg[b - a];
39      return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
40  }
41
42  // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
43  int Compare(int l1, int r1, int l2, int r2) const
44  {
45      assert(0 <= l1 && l1 <= r1 && r1 <= n);
46      assert(0 <= l2 && l2 <= r2 && r2 <= n);
47      int a = r1 - l1, b = r2 - l2;
48      if ( Query(l1, l2) >= min(a, b) )
49          return (a > b) - (a < b);
50      return rk[l1] < rk[l2] ? -1 : 1;
51  }
52 };

```

121.2 两个前缀的最长公共后缀

算法约束见动态版本第 89 页。本节代码独立可抄；采用下列实际接口名。

```

1  // reversed must be built on the reversed original sequence.
2  inline int Prefix_Lcs(const Suffix_LCP &reversed, int x, int y)
3  {
4      assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
5      return reversed.Query(reversed.n - x, reversed.n - y);
6  }

```


121.3 重复子串 AA 的起止位置计数

算法约束见动态版本第 90 页。本节代码独立可抄；采用下列实际接口名。

```

1 // Count nonempty squares AA by their starting and ending positions.
2 inline pair<vector<long long>, vector<long long>>
3 Square_Counts(const Suffix_LCP &forward, const Suffix_LCP &reversed)
4 {
5     int n = forward.n;
6     assert(reversed.n == n);
7     vector<long long> start(n + 1), finish(n + 1);
8     for ( int len = 1; len <= n / 2; len++ )
9         for ( int i = 0; i < n - len; i += len )
10            {
11                int j = i + len;
12                int left = min(len - 1, Prefix_Lcs(reversed, i, j));
13                int right = min(len, forward.Query(i, j));
14                if ( left + right < len )
15                    continue;
16                int l = i - left, r = i + right - len;
17                start[l]++;
18                start[r + 1]--;
19                finish[l + 2 * len - 1]++;
20                finish[r + 2 * len]--;
21            }
22     for ( int i = 1; i < n; i++ )
23     {
24         start[i] += start[i - 1];
25         finish[i] += finish[i - 1];
26     }
27     start.resize(n);
28     finish.resize(n);
29     return {start, finish};
30 }
```

第一百二十二章 回文树

122.1 回文树 PAM: 回文后缀与出现次数

算法约束见动态版本第 91 页。本节代码独立可抄；采用下列实际接口名。

声明 `Palindromic_Tree<N>` 后调用 `Init()`；`N` 为最大追加字符数，内部另留两个根节点。`Insert`、`Distinct`、`Occurrences` 对应动态版接口；`t`、`last`、`total` 可直接读。大对象放在全局或 `static`。

```
1 template <int N> struct Palindromic_Tree
2 {
3     struct Node
4     {
5         int next[26]{};
6         int len = 0, fail = 0, suffixes = 0, end = -1;
7         long long hits = 0;
8     };
9
10    Node t[N + 2];
11    int tot = 1;
12    vector<int> s;
13    int last = 0;
14    long long total = 0;
15
16    void Init()
17    {
18        tot = 1;
19        t[0] = Node{};
20        t[1] = Node{};
21        t[0].fail = 1;
22        t[1].len = -1;
23        s.assign(1, -1);
24        last = 0;
25        total = 0;
26    }
27
28    int Get_Fail(int u) const
29    {
30        int pos = s.size() - 1;
31        while ( s[pos - t[u].len - 1] != s[pos] )
32            u = t[u].fail;
33        return u;
34    }
35
36    // Append a lowercase letter; return the longest palindromic suffix node.
37    int Insert(char ch)
38    {
39        int c = ch - 'a';
40        assert(0 <= c && c < 26);
41        assert(s.size() <= N);
42        s.push_back(c);
43        int u = Get_Fail(last);
44        if ( !t[u].next[c] )
```

```

45     {
46         int v = ++tot;
47         t[v] = Node{};
48         t[v].len = t[u].len + 2;
49         t[v].fail = t[Get_Fail(t[u].fail)].next[c];
50         t[v].suffixes = t[t[v].fail].suffixes + 1;
51         t[v].end = (int)s.size() - 2;
52         t[u].next[c] = v;
53     }
54     last = t[u].next[c];
55     t[last].hits++;
56     total += t[last].suffixes;
57     return last;
58 }
59
60 int Distinct() const
61 {
62     return tot - 1;
63 }
64
65 // Copy-and-propagate: repeated calls and later appends remain valid.
66 vector<long long> Occurrences() const
67 {
68     vector<long long> count(tot + 1);
69     for ( int u = 2; u <= tot; u++ )
70         count[u] = t[u].hits;
71     for ( int u = tot; u >= 2; u-- )
72         count[t[u].fail] += count[u];
73     count[0] = 0;
74     count[1] = 0;
75     return count;
76 }
77 };

```

第一百二十三章 数论

123.1 模运算、素性、CRT 与类欧几里德

算法约束见动态版本第 93 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Number_Theory
2 {
3     using ll = long long;
4     using ull = unsigned long long;
5     using i128 = __int128_t;
6     using u128 = __uint128_t;
7
8     static ull Mul(ull a, ull b, ull m)
9     {
10         return u128(a) * b % m;
11     }
12
13     static ull Power(ull a, ull b, ull m)
14     {
15         assert(m);
16         ull r = 1 % m;
17         a %= m;
18         for ( ; b; b >>= 1, a = Mul(a, a, m) )
19             if ( b & 1 )
20                 r = Mul(r, a, m);
21         return r;
22     }
23
24     static bool Prime(ull n)
25     {
26         if ( n < 2 )
27             return false;
28         for ( ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37} )
29             if ( n % p == 0 )
30                 return n == p;
31         ull d = n - 1;
32         int s = 0;
33         while ( !(d & 1) )
34         {
35             d >>= 1;
36             ++s;
37         }
38         for ( ull a :
39             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL} )
40         {
41             if ( a % n == 0 )
42                 continue;
43             ull x = Power(a, d, n);
44             if ( x == 1 || x == n - 1 )
45                 continue;
46             bool ok = false;
```

```

47     for ( int j = 1; j < s; j++ )
48     {
49         x = Mul(x, x, n);
50         if ( x == n - 1 )
51         {
52             ok = true;
53             break;
54         }
55     }
56     if ( !ok )
57         return false;
58 }
59 return true;
60 }
61
62 static i128 Exgcd(i128 a, i128 b, i128 &x, i128 &y)
63 {
64     if ( !b )
65     {
66         x = 1;
67         y = 0;
68         return a;
69     }
70     i128 d = Exgcd(b, a % b, y, x);
71     y -= a / b * x;
72     return d;
73 }
74
75 static ll Inverse(ll a, ll m)
76 {
77     assert(m > 0);
78     a %= m;
79     if ( a < 0 )
80         a += m;
81     i128 x, y;
82     if ( Exgcd(a, m, x, y) != 1 )
83         return -1;
84     return (x % m + m) % m;
85 }
86
87 // x = r (mod m); false means inconsistent. Throws if lcm exceeds int64.
88 static bool Crt(ll &r, ll &m, ll b, ll n)
89 {
90     assert(m > 0 && n > 0);
91     r %= m;
92     if ( r < 0 )
93         r += m;
94     b %= n;
95     if ( b < 0 )
96         b += n;
97     i128 x, y;
98     ll g = (ll)Exgcd(m, n, x, y);
99     i128 diff = i128(b) - r;
100     if ( diff % g )
101         return false;
102     i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;

```

```

103     if ( mod > LLONG_MAX )
104         throw overflow_error("CRT modulus");
105     r = (r + i128(m) * k) % mod;
106     m = (ll)mod;
107     return true;
108 }
109
110 // sum_{i=0}^{n-1} floor((a*i+b)/m), signed a,b; 0<=n<=1e9, 1<=m<=1e9.
111 static i128 Floor_Sum(ll n, ll m, ll a, ll b)
112 {
113     assert(n >= 0 && m > 0);
114     i128 ans = 0;
115     auto norm = [&](ll &v)
116     {
117         ll q = v / m;
118         if ( v % m < 0 )
119             --q;
120         v = (ll)(i128(v) - i128(q) * m);
121         return q;
122     };
123     ans += i128(norm(a)) * n * (n - 1) / 2;
124     ans += i128(norm(b)) * n;
125     while ( true )
126     {
127         ans += i128(n) * (n - 1) / 2 * (a / m);
128         a %= m;
129         ans += i128(n) * (b / m);
130         b %= m;
131         i128 y = i128(a) * n + b;
132         if ( y < m )
133             break;
134         n = (ll)(y / m);
135         b = (ll)(y % m);
136         swap(a, m);
137     }
138     return ans;
139 }
140 };

```

123.2 Pollard-Rho 因数分解

算法约束见动态版本第 95 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Pollard_Rho
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     mt19937_64 rng;
6
7     Pollard_Rho(ull seed = 712367821) : rng(seed)
8     {
9     }
10
11     ull Rho(ull n)
12     {

```

```

13     if ( n % 2 == 0 )
14         return 2;
15     for ( ;; )
16     {
17         ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
18         auto f = [&](ull v)
19         {
20             return (Number_Theory::Mul(v, v, n) + u128(c)) % n;
21         };
22         // Retry bounded attempts; Las Vegas: only exact divisors returned.
23         for ( int i = 0; i < 200000 && d == 1; i++ )
24         {
25             x = f(x);
26             y = f(f(y));
27             d = gcd(x > y ? x - y : y - x, n);
28         }
29         if ( 1 < d && d < n )
30             return d;
31     }
32 }

33
34 void Split(ull n, vector<ull> &a)
35 {
36     if ( n == 1 )
37         return;
38     if ( Number_Theory::Prime(n) )
39     {
40         a.push_back(n);
41         return;
42     }
43     ull d = Rho(n);
44     Split(d, a);
45     Split(n / d, a);
46 }

47
48 vector<ull> Factor(ull n)
49 {
50     assert(n >= 1);
51     vector<ull> a;
52     Split(n, a);
53     sort(a.begin(), a.end());
54     return a;
55 }
56 };

```

123.3 线性筛、欧拉函数与 Möbius 函数

算法约束见动态版本第 97 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Linear_Sieve
2 {
3     vector<int> Prime, lp, phi, mu;
4
5     Linear_Sieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
6     {
7         if ( n )
8             phi[1] = mu[1] = 1;
9         for ( int i = 2; i <= n; i++ )
10            {
11                if ( !lp[i] )
12                {
13                    lp[i] = i;
14                    Prime.push_back(i);
15                    phi[i] = i - 1;
16                    mu[i] = -1;
17                }
18                for ( int p : Prime )
19                {
20                    if ( p > n / i )
21                        break;
22                    int j = i * p;
23                    lp[j] = p;
24                    if ( i % p == 0 )
25                    {
26                        phi[j] = phi[i] * p;
27                        mu[j] = 0;
28                        break;
29                    }
30                    phi[j] = phi[i] * (p - 1);
31                    mu[j] = -mu[i];
32                }
33            }
34     }
35 };

```


123.4 静态模整数

算法约束见动态版本第 98 页。本节代码独立可抄；采用下列实际接口名。

```

1  template <int mod> struct Mod_Int
2  {
3      int v;
4
5      Mod_Int(long long x = 0) : v((x % mod + mod) % mod)
6      {
7      }
8
9      Mod_Int operator+(Mod_Int b) const
10     {
11         return Mod_Int((long long)v + b.v);
12     }
13
14     Mod_Int operator-(Mod_Int b) const
15     {
16         return Mod_Int((long long)v - b.v);
17     }
18
19     Mod_Int operator*(Mod_Int b) const
20     {
21         return Mod_Int(1LL * v * b.v);
22     }
23
24     Mod_Int Pow(long long e) const
25     {
26         assert(e >= 0);
27         Mod_Int a = *this, r = 1;
28         for ( ; e >= 1, a = a * a )
29             if ( e & 1 )
30                 r = r * a;
31         return r;
32     }
33
34     Mod_Int Inv() const
35     {
36         assert(v);
37         return Pow(mod - 2);
38     } // prime modulus
39
40     Mod_Int operator/(Mod_Int b) const
41     {
42         return *this * b.Inv();
43     }
44 };

```

123.5 阶乘组合数

算法约束见动态版本第 99 页。本节代码独立可抄；采用下列实际接口名。

Combination<prime>(n=0) 构造，Init(n) 扩大上限；Choose、Permute 对应组合数和排列数，fac、ifac 可只读访问。

```

1  template <int mod> struct Combination
2  {
3      using Z = Mod_Int<mod>;
4      vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6      Combination(int n = 0)
7      {
8          Init(n);
9      }
10
11     void Init(int n)
12     {
13         assert(0 <= n && n < mod);
14         int old = (int)fac.size() - 1;
15         if ( n <= old )
16             return;
17         fac.resize(n + 1);
18         ifac.resize(n + 1);
19         for ( int i = old + 1; i <= n; i++ )
20             fac[i] = fac[i - 1] * i;
21         ifac[n] = fac[n].Inv();
22         for ( int i = n; i > old; i-- )
23             ifac[i - 1] = ifac[i] * i;
24     }
25
26     Z Choose(int n, int k) const
27     {
28         if ( k < 0 || k > n )
29             return 0;
30         assert(n < (int)fac.size());
31         return fac[n] * ifac[k] * ifac[n - k];
32     }
33
34     Z Permute(int n, int k) const
35     {
36         if ( k < 0 || k > n )
37             return 0;
38         assert(n < (int)fac.size());
39         return fac[n] * ifac[n - k];
40     }
41 };

```

第一百二十四章 二元一次不定方程

124.1 二元一次不定方程

算法约束见动态版本第 100 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline bool
2 Linear_Equation(long long a, long long b, long long c, __int128_t &x, __int128_t &y)
3 {
4     using i128 = __int128_t;
5     x = 0;
6     y = 0;
7     if ( !a && !b )
8         return c == 0;
9     i128 aa = a < 0 ? -i128(a) : i128(a);
10    i128 bb = b < 0 ? -i128(b) : i128(b);
11    i128 u, v;
12    i128 g = Number_Theory::Exgcd(aa, bb, u, v);
13    if ( c % g )
14        return false;
15    x = u * (c / g);
16    y = v * (c / g);
17    if ( a < 0 )
18        x = -x;
19    if ( b < 0 )
20        y = -y;
21    return true;
22 }
```

第一百二十五章 线性同余方程

125.1 线性同余方程

算法约束见动态版本第 101 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline pair<long long, long long>
2 Linear_Congruence(long long a, long long b, long long m)
3 {
4     assert(m > 0);
5     using i128 = __int128_t;
6     a %= m;
7     if ( a < 0 )
8         a += m;
9     i128 x, y;
10    i128 g = Number_Theory::Exgcd(a, m, x, y);
11    if ( i128(b) % g )
12        return {-1, -1};
13    i128 period = m / g;
14    x = x * (-i128(b) / g) % period;
15    if ( x < 0 )
16        x += period;
17    return {(long long)x, (long long)period};
18 }
```

第一百二十六章 区间筛素数

126.1 区间筛素数

算法约束见动态版本第 102 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<int> Segmented_Primes(int l, int r)
2 {
3     assert(0 <= l && l <= r);
4     int limit = 0;
5     while ( 1LL * (limit + 1) * (limit + 1) <= r )
6         limit++;
7     vector<char> composite(limit + 1);
8     vector<char> removed(1LL * r - l + 1);
9     for ( int p = 2; p <= limit; p++ )
10         if ( !composite[p] )
11         {
12             for ( long long j = 1LL * p * p; j <= limit; j += p )
13                 composite[j] = true;
14             long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
15             for ( long long j = start; j <= r; j += p )
16                 removed[j - l] = true;
17         }
18     vector<int> primes;
19     for ( long long x = max(l, 2); x <= r; x++ )
20         if ( !removed[x - l] )
21             primes.push_back(x);
22     return primes;
23 }
```

第一百二十七章 批量模逆元

127.1 任意数组的批量逆元

算法约束见动态版本第 103 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline optional<vector<long long>> Batch_Inverse(vector<long long> a, long long mod)
2 {
3     assert(mod > 0);
4     int n = a.size();
5     vector<long long> prefix(n + 1, 1 % mod);
6     for ( int i = 0; i < n; i++ )
7     {
8         a[i] %= mod;
9         if ( a[i] < 0 )
10             a[i] += mod;
11         prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
12     }
13     long long inverse = Number_Theory::Inverse(prefix[n], mod);
14     if ( inverse == -1 )
15         return nullopt;
16     for ( int i = n - 1; i >= 0; i-- )
17     {
18         long long value = a[i];
19         a[i] = __int128_t(inverse) * prefix[i] % mod;
20         inverse = __int128_t(inverse) * value % mod;
21     }
22     return a;
23 }
```

第一百二十八章 连续整数逆元表

128.1 连续整数的线性逆元表

算法约束见动态版本第 104 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<int> Inverse_Table(int n, int p)
2 {
3     assert(p >= 2 && 0 <= n && n < p);
4     vector<int> inverse(n + 1);
5     if ( n )
6         inverse[1] = 1;
7     for ( int i = 2; i <= n; i++ )
8         inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
9     return inverse;
10 }
```

第一百二十九章 混合进制中国剩余定理

129.1 Garner: 大周期 CRT 与混合进制

算法约束见动态版本第 105 页。本节代码独立可抄；采用下列实际接口名。

```
1 // Pairwise coprime positive moduli. Digits use the input order.
2 inline vector<long long> Garner_Digits(const vector<long long> &b,
3                                       const vector<long long> &moduli)
4 {
5     using ll = long long;
6     using i128 = __int128_t;
7     int n = b.size();
8     assert(moduli.size() == b.size());
9     vector<ll> value(n), coefficient(n, 1), digit(n);
10    for ( ll m : moduli )
11        assert(m > 0);
12    for ( int i = 0; i < n; i++ )
13    {
14        ll m = moduli[i];
15        ll inverse = Number_Theory::Inverse(coefficient[i], m);
16        if ( inverse == -1 )
17            throw invalid_argument("Garner moduli are not coprime");
18        i128 delta = (i128(b[i]) - value[i]) % m;
19        if ( delta < 0 )
20            delta += m;
21        digit[i] = delta * inverse % m;
22        for ( int j = i + 1; j < n; j++ )
23        {
24            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
25            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
26        }
27    }
28    return digit;
29 }
30
31 // Return the least nonnegative CRT solution modulo target, not all solutions.
32 inline long long
33 Garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
34 {
35     assert(target > 0);
36     auto digit = Garner_Digits(b, moduli);
37     long long answer = 0;
38     for ( int i = (int)digit.size() - 1; i >= 0; i-- )
39         answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
40     return answer;
41 }
```


第一百三十章 乘法阶与原根

130.1 乘法阶与原根

算法约束见动态版本第 107 页。本节代码独立可抄；采用下列实际接口名。

构造 `Primitive_Root(n)`, `Init`、`Order`、`Is_Root`、`Minimum`、`All` 对应动态接口；`Factors` 为 ϕ 的不同素因子，依赖 `Number_Theory::Power`。

```
1 struct Primitive_Root
2 {
3     int mod, phi;
4     bool exists;
5     vector<int> Factors;
6
7     static vector<int> Factor(int n)
8     {
9         vector<int> result;
10        for ( int p = 2; p <= n / p; p++ )
11            if ( n % p == 0 )
12            {
13                result.push_back(p);
14                while ( n % p == 0 )
15                    n /= p;
16            }
17        if ( n > 1 )
18            result.push_back(n);
19        return result;
20    }
21
22    Primitive_Root(int n)
23    {
24        Init(n);
25    }
```

```

27 void Init(int n)
28 {
29     assert(n >= 2);
30     mod = n;
31     phi = n;
32     for ( int p : Factor(n) )
33         phi -= phi / p;
34     int odd = n;
35     if ( odd % 2 == 0 )
36         odd /= 2;
37     exists = n == 2 || n == 4 || (odd % 2 && Factor(odd).size() == 1);
38     Factors = Factor(phi);
39 }
40
41 int Order(long long a) const
42 {
43     a %= mod;
44     if ( a < 0 )
45         a += mod;
46     if ( gcd(a, (long long)mod) != 1 )
47         return 0;
48     int result = phi;
49     for ( int p : Factors )
50         while ( result % p == 0 && Number_Theory::Power(a, result / p, mod) == 1 )
51             result /= p;
52     return result;
53 }

```

```

55     bool Is_Root(long long a) const
56     {
57         if ( !exists )
58             return false;
59         a %= mod;
60         if ( a < 0 )
61             a += mod;
62         if ( gcd(a, (long long)mod) != 1 )
63             return false;
64         for ( int p : Factors )
65             if ( Number_Theory::Power(a, phi / p, mod) == 1 )
66                 return false;
67         return true;
68     }
69
70     int Minimum() const
71     {
72         if ( !exists )
73             return -1;
74         for ( int g = 1; g < mod; g++ )
75             if ( Is_Root(g) )
76                 return g;
77         return -1;
78     }
79
80     vector<int> All() const
81     {
82         int g = Minimum();
83         vector<int> roots;
84         if ( g == -1 )
85             return roots;
86         long long value = 1;
87         for ( int k = 1; k <= phi; k++ )
88         {
89             value = value * g % mod;
90             if ( gcd(k, phi) == 1 )
91                 roots.push_back(value);
92         }
93         sort(roots.begin(), roots.end());
94         return roots;
95     }
96 };

```

第一百三十一章 矩形 GCD 计数

131.1 矩形 GCD 计数

算法约束见动态版本第 110 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Coprime_Pairs
2 {
3     vector<int> sum;
4
5     Coprime_Pairs(int n) : sum(Linear_Sieve(n).mu)
6     {
7         for ( int i = 1; i <= n; i++ )
8             sum[i] += sum[i - 1];
9     }
10
11     long long Count(int a, int b) const
12     {
13         assert(a >= 0 && b >= 0 && min(a, b) < (int)sum.size());
14         __int128_t answer = 0;
15         for ( int l = 1, r; l <= min(a, b); l = r + 1 )
16         {
17             int x = a / l;
18             int y = b / l;
19             r = min(a / x, b / y);
20             answer += __int128_t(sum[r] - sum[l - 1]) * x * y;
21         }
22         return (long long)answer;
23     }
24
25     long long Rectangle(int a, int b, int c, int d, int k) const
26     {
27         assert(1 <= a && a <= b && 1 <= c && c <= d && k >= 1);
28         return Count(b / k, d / k) - Count((a - 1) / k, d / k) -
29             Count(b / k, (c - 1) / k) + Count((a - 1) / k, (c - 1) / k);
30     }
31 };
```

第一百三十二章 带权类欧几里德

132.1 整除和、带权和与平方和

算法约束见动态版本第 111 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline array<Mod_Int<998244353>, 3>
2 Floor_Moments(long long n, long long m, long long a, long long b)
3 {
4     using Z = Mod_Int<998244353>;
5     assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
6     if ( !n )
7         return {0, 0, 0};
8     auto normalize = [&](long long &v)
9     {
10         long long q = v / m;
11         if ( v % m < 0 )
12             q--;
13         v = (__int128_t(v) - __int128_t(q) * m);
14         return Z(q);
15     };
16     Z x = normalize(a);
17     Z y = normalize(b);
18     Z count = n;
19     Z s1 = count * Z(n - 1) * Z(499122177);
20     Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
21     Z f = 0, g = 0, h = 0;
22     if ( a )
23     {
24         long long height = (a * (n - 1) + b) / m;
25         auto [u, v, w] = Floor_Moments(height, a, m, m + a - b - 1);
26         Z t = height;
27         f = count * t - u;
28         g = t * s1 - (w - u) * Z(499122177);
29         h = count * t * t - Z(2) * v - u;
30     }
31     return {f + x * s1 + y * count,
32            g + x * s2 + y * s1,
33            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
34            Z(2) * y * f};
35 }
```

第一百三十三章 模几何级数与约数和

133.1 模幂与几何级数同时求值

算法约束见动态版本第 112 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline pair<unsigned long long, unsigned long long>
2 Power_Sum(unsigned long long a, __uint128_t n, unsigned long long mod)
3 {
4     assert(mod >= 1);
5     if ( !n )
6         return {1 % mod, 0};
7     auto [p, s] = Power_Sum(a, n / 2, mod);
8     unsigned long long square = __uint128_t(p) * p % mod;
9     unsigned long long sum = __uint128_t(s) * (p + 1) % mod;
10    if ( n % 2 )
11        return {__uint128_t(square) * a % mod, (__uint128_t(sum) + square) % mod};
12    return {square, sum};
13 }
```

133.2 幂的约数和

算法约束见动态版本第 113 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline unsigned long long Divisor_Sum_Power(unsigned long long a,
2                                             unsigned long long b,
3                                             unsigned long long mod,
4                                             Pollard_Rho &solver)
5 {
6     assert(a >= 1 && mod >= 1);
7     if ( !b )
8         return 1 % mod;
9     auto factors = solver.Factor(a);
10    unsigned long long answer = 1 % mod;
11    for ( int i = 0, j; i < (int)factors.size(); i = j )
12    {
13        j = i + 1;
14        while ( j < (int)factors.size() && factors[j] == factors[i] )
15            j++;
16        __uint128_t length = __uint128_t(j - i) * b + 1;
17        auto sum = Power_Sum(factors[i], length, mod).second;
18        answer = __uint128_t(answer) * sum % mod;
19    }
20    return answer;
21 }
```

第一百三十四章 单个数的欧拉函数

134.1 单个数的欧拉函数

算法约束见动态版本第 114 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline unsigned long long Euler_Phi(unsigned long long n)
2 {
3     assert(n >= 1);
4     unsigned long long answer = n;
5     for ( unsigned long long p = 2; p <= n / p; p++ )
6         if ( n % p == 0 )
7         {
8             answer = answer / p * (p - 1);
9             while ( n % p == 0 )
10                 n /= p;
11         }
12     if ( n > 1 )
13         answer = answer / n * (n - 1);
14     return answer;
15 }
16
17 inline unsigned long long Euler_Phi(unsigned long long n, Pollard_Rho &solver)
18 {
19     assert(n >= 1);
20     auto factors = solver.Factor(n);
21     unsigned long long answer = n;
22     unsigned long long previous = 0;
23     for ( auto p : factors )
24         if ( p != previous )
25         {
26             answer = answer / p * (p - 1);
27             previous = p;
28         }
29     return answer;
30 }
```

第一百三十五章 Carmichael 函数

135.1 Carmichael：模乘法群的公共指数

算法约束见动态版本第 115 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline int Carmichael(int n)
2 {
3     assert(n >= 1);
4     long long answer = 1;
5     for ( int p = 2; p <= n / p; p++ )
6         if ( n % p == 0 )
7         {
8             int power = 1;
9             while ( n % p == 0 )
10             {
11                 n /= p;
12                 power *= p;
13             }
14             int value = power / p * (p - 1);
15             if ( p == 2 && power >= 8 )
16                 value /= 2;
17             answer = answer / gcd(answer, (long long)value) * value;
18         }
19     if ( n > 1 )
20         answer = answer / gcd(answer, (long long)(n - 1)) * (n - 1);
21     return answer;
22 }
```


第一百三十六章 整数分拆

136.1 整数分拆与重数上限

算法约束见动态版本第 116 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Integer_Partitions
2 {
3     int mod;
4     vector<int> p;
5
6     Integer_Partitions(int n, int mod) : mod(mod), p(n + 1)
7     {
8         assert(n >= 0 && mod >= 1);
9         p[0] = 1 % mod;
10        for ( int i = 1; i <= n; i++ )
11        {
12            long long value = 0;
13            for ( long long j = 1; j * (3 * j - 1) / 2 <= i; j++ )
14            {
15                long long a = j * (3 * j - 1) / 2;
16                long long b = j * (3 * j + 1) / 2;
17                int sign = j % 2 ? 1 : -1;
18                value += 1LL * sign * p[i - a];
19                if ( b <= i )
20                    value += 1LL * sign * p[i - b];
21            }
22            p[i] = (value % mod + mod) % mod;
23        }
24    }
25
26    int Limited(int n, int k) const
27    {
28        assert(0 <= n && n < (int)p.size() && k >= 0);
29        long long step = 1LL * k;
30        long long value = p[n];
31        for ( long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++ )
32        {
33            long long a = step * (j * (3 * j - 1) / 2);
34            long long b = step * (j * (3 * j + 1) / 2);
35            int sign = j % 2 ? -1 : 1;
36            value += 1LL * sign * p[n - a];
37            if ( b <= n )
38                value += 1LL * sign * p[n - b];
39        }
40        return (value % mod + mod) % mod;
41    }
42 };
```

第一百三十七章 素数模数组合数

137.1 Lucas：小素数模的大下标组合数

算法约束见动态版本第 117 页。本节代码独立可抄；采用下列实际接口名。

声明 Lucas_Theorem<N>, Init(p) 重建素数模数下的表, Choose(n,k) 查询。要求 p 不超过 N；大对象放全局或 static。传统版空间 $O(N)$ 。

```
1 template <int N> struct Lucas_Theorem
2 {
3     int mod;
4     int fac[N + 1], ifac[N + 1];
5
6     void Init(int p)
7     {
8         assert(2 <= p && p <= N);
9         mod = p;
10        fac[0] = 1;
11        for ( int i = 1; i < p; i++ )
12            fac[i] = 1LL * fac[i - 1] * i % p;
13        long long a = fac[p - 1], r = 1;
14        for ( int e = p - 2; e; e >>= 1 )
15        {
16            if ( e & 1 )
17                r = r * a % p;
18            a = a * a % p;
19        }
20        ifac[p - 1] = r;
21        for ( int i = p - 1; i; i-- )
22            ifac[i - 1] = 1LL * ifac[i] * i % p;
23    }
24
25    int Choose(unsigned long long n, unsigned long long k) const
26    {
27        if ( k > n )
28            return 0;
29        long long answer = 1;
30        while ( n || k )
31        {
32            int a = n % mod, b = k % mod;
33            if ( b > a )
34                return 0;
35            answer = answer * fac[a] % mod;
36            answer = answer * ifac[b] % mod;
37            answer = answer * ifac[a - b] % mod;
38            n /= mod;
39            k /= mod;
40        }
41        return answer;
42    }
43 };
```

第一百三十八章 合数模数组合数

138.1 合数模数组合数（扩展 Lucas）

算法约束见动态版本第 119 页。本节代码独立可抄；采用下列实际接口名。

声明 `Ex_Lucas<N>` 后 `Init(mod)`，要求 `mod` 不超过 `N`；`Choose` 为查询接口。`fac` 是共用静态前缀积数组，各素数幂通过 `start` 分段；各段总长度不超过 `mod+1`，所以 `N` 限制模数即可。`Init` 可切换模数并清空旧预处理，大对象放在全局或 `static`。

```
1 template <int N> struct Ex_Lucas
2 {
3     using ull = unsigned long long;
4     using ll = long long;
5
6     struct Factor
7     {
8         int p, q, exponent, start, crt;
9     };
10
11     int mod;
12     int fac[N + 1], used;
13     vector<Factor> factors;
14
15     static ll Power(ll a, ull b, int m)
16     {
17         ll result = 1;
18         while ( b )
19         {
20             if ( b & 1 )
21                 result = result * a % m;
22             a = a * a % m;
23             b >>= 1;
24         }
25         return result;
26     }
27
28     void Init(int m)
29     {
30         assert(1 <= m && m <= N);
31         mod = m;
32         used = 0;
33         factors.clear();
34         int x = m;
35         for ( int p = 2; p <= x / p; p++ )
36         {
37             if ( x % p )
38                 continue;
39             int q = 1, e = 0;
40             do
41             {
42                 x /= p;
43                 q *= p;
```

```

44         e++;
45     } while ( x % p == 0 );
46     Add_Factor(p, q, e);
47 }
48 if ( x > 1 )
49     Add_Factor(x, x, 1);
50 }
51
52 void Add_Factor(int p, int q, int e)
53 {
54     int start = used;
55     fac[used++] = 1;
56     for ( int i = 1; i < q; i++ )
57     {
58         fac[used] = ll(fac[used - 1]) * (i % p ? i : 1) % q;
59         used++;
60     }
61     fac[used] = fac[used - 1];
62     used++;
63     int phi = q - q / p;
64     int crt = ll(mod / q) * Power(mod / q % q, phi - 1, q) % mod;
65     factors.push_back({p, q, e, start, crt});
66 }
67
68 static ull Valuation(ull n, int p)
69 {
70     ull result = 0;
71     while ( n )
72     {
73         n /= p;
74         result += n;
75     }
76     return result;
77 }
78
79 ll Unit(ull n, const Factor &f) const
80 {
81     ll result = 1;
82     while ( n )
83     {
84         result = result * Power(fac[f.start + f.q], n / f.q, f.q) % f.q;
85         result = result * fac[f.start + n % f.q] % f.q;
86         n /= f.p;
87     }
88     return result;
89 }

```

```
91     int Choose(ull n, ull k) const
92     {
93         if ( k > n || mod == 1 )
94             return 0;
95         ll answer = 0;
96         for ( const auto &f : factors )
97         {
98             ull e = Valuation(n, f.p) - Valuation(k, f.p) - Valuation(n - k, f.p);
99             if ( e >= ull(f.exponent) )
100                 continue;
101             ll denominator = Unit(k, f) * Unit(n - k, f) % f.q;
102             ll residue =
103                 Unit(n, f) * Power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
104             residue = residue * Power(f.p, e, f.q) % f.q;
105             answer = (answer + residue * f.crt) % mod;
106         }
107         return answer;
108     }
109 };
```

第一百三十九章 二次剩余

139.1 素数模平方根 (Cipolla)

算法约束见动态版本第 122 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<long long> Mod_Sqrt(long long n, int p)
2 {
3     using ll = long long;
4     assert(p >= 2);
5     n = (n % p + p) % p;
6     if ( n == 0 || p == 2 )
7         return {n};
8     auto Power = [p](ll a, ll b)
9     {
10         ll result = 1;
11         while ( b )
12         {
13             if ( b & 1 )
14                 result = result * a % p;
15             a = a * a % p;
16             b >>= 1;
17         }
18         return result;
19     };
20     if ( Power(n, (p - 1) / 2) != 1 )
21         return {};
22     ll answer;
23     if ( p % 4 == 3 )
24         answer = Power(n, (ll(p) + 1) / 4);
25     else
26     {
27         static mt19937_64 rng(712367);
28         ll a, w;
29         do
30         {
31             a = rng() % p;
32             w = (a * a % p - n + p) % p;
33         } while ( Power(w, (p - 1) / 2) != p - 1 );
34         using P = pair<ll, ll>;
35         auto Mul = [p, w](P x, P y)
36         {
37             ll real = (x.first * y.first + x.second * y.second % p * w) % p;
38             ll imag = (x.first * y.second + x.second * y.first) % p;
39             return P{real, imag};
40         };
41         P result{1, 0}, base{a, 1};
42         ll exponent = (ll(p) + 1) / 2;
43         while ( exponent )
44         {
45             if ( exponent & 1 )
46                 result = Mul(result, base);
```

```
47         base = Mul(base, base);
48         exponent >>= 1;
49     }
50     answer = result.first;
51 }
52 ll other = p - answer;
53 if ( answer > other )
54     swap(answer, other);
55 return {answer, other};
56 }
```

第一百四十章 素数模高次剩余

140.1 素数模高次剩余：全部根的紧凑表示

算法约束见动态版本第 124 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Kth_Residue
2 {
3     using ll = long long;
4     ll first, ratio, count;
5
6     // Prime p <= 1e12; g is a primitive root of p; k > 0.
7     // All roots: first * ratio^i mod p, 0 <= i < count.
8     static optional<Kth_Residue> Solve(ll a, unsigned long long k, ll p, ll g)
9     {
10         assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
11         a %= p;
12         if ( a < 0 )
13             a += p;
14         if ( a == 0 )
15             return Kth_Residue{0, 1, 1};
16         ll logarithm = Discrete_Log::Solve(g, a, p);
17         if ( logarithm < 0 )
18             return nullopt;
19         auto [exponent, period] = Linear_Congruence(k % (p - 1), -logarithm, p - 1);
20         if ( exponent < 0 )
21             return nullopt;
22         ll first = Number_Theory::Power(g, exponent, p);
23         ll ratio = Number_Theory::Power(g, period, p);
24         return Kth_Residue{first, ratio, (p - 1) / period};
25     }
26 };
```


第一百四十一章 素数幂模高次剩余

141.1 素数幂模高次剩余：含非互素被开方数

算法约束见动态版本第 125 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Prime_Power_Roots
2 {
3     using ll = long long;
4     ll first = 0, step = 1, lifts = 1;
5     array<ll, 2> ratio{1, 1}, count{1, 1};
6
7     ll Size() const
8     {
9         return count[0] * count[1] * lifts;
10    }
11
12    ll Get(ll i, ll j, ll lift) const
13    {
14        assert(i >= 0 && i < count[0] && j >= 0 && j < count[1]);
15        assert(lift >= 0 && lift < lifts);
16        ll x = (__int128)first * Number_Theory::Power(ratio[0], i, step) % step;
17        x = (__int128)x * Number_Theory::Power(ratio[1], j, step) % step;
18        return x + lift * step;
19    }
20
21    // p prime, 1 <= e, p^e <= 1e12, k > 0.
22    // For odd p, g must be a primitive root modulo p^e. For p=2, g is unused.
```

接上页同一结构体（第三页继续同一函数）:

```

23  static optional<Prime_Power_Roots>
24  Solve(ll a, unsigned long long k, ll p, int e, ll g = 0)
25  {
26      assert(p >= 2 && p <= 10000000000000LL && e >= 1 && e <= 39 && k > 0);
27      array<ll, 40> power{1};
28      for ( int i = 1; i <= e; i++ )
29      {
30          assert(power[i - 1] <= 10000000000000LL / p);
31          power[i] = power[i - 1] * p;
32      }
33      ll mod = power[e];
34      a %= mod;
35      if ( a < 0 )
36          a += mod;
37      Prime_Power_Roots answer;
38      if ( a == 0 )
39      {
40          answer.step = power[(e - 1) / k + 1];
41          answer.lifts = mod / answer.step;
42          return answer;
43      }
44      int t = 0;
45      while ( a % p == 0 )
46      {
47          a /= p;
48          ++t;
49      }
50      if ( t % k )
51          return nullopt;

```

接上页同一结构体（第三页继续同一函数）:

```

52     ll scale = power[t / k], q = power[e - t];
53     answer.first = 1;
54     answer.step = scale * q;
55     answer.lifts = mod / answer.step;
56     auto cyclic = [&](ll base, ll target, ll order, int slot)
57     {
58         ll logarithm = Discrete_Log::Solve(base, target, q);
59         if ( logarithm < 0 )
60             return false;
61         auto [x, period] = Linear_Congruence(k % order, -logarithm, order);
62         if ( x < 0 )
63             return false;
64         answer.first =
65             (__int128)answer.first * Number_Theory::Power(base, x, q) % q;
66         answer.ratio[slot] = Number_Theory::Power(base, period, q);
67         answer.count[slot] = order / period;
68         return true;
69     };
70     if ( p != 2 )
71     {
72         assert(g >= 1 && g < mod);
73         if ( !cyclic(g % q, a, q / p * (p - 1), 0) )
74             return nullopt;
75     }
76     else if ( q >= 4 )
77     {
78         int sign = a % 4 == 3;
79         auto [x, period] = Linear_Congruence(k % 2, -sign, 2);
80         if ( x < 0 )
81             return nullopt;
82         answer.first = x ? q - 1 : 1;
83         answer.ratio[0] = period == 1 ? q - 1 : 1;
84         answer.count[0] = 2 / period;
85         if ( q >= 8 && !cyclic(5, sign ? q - a : a, q / 4, 1) )
86             return nullopt;
87     }
88     answer.first *= scale;
89     return answer;
90 }
91 };

```

第一百四十二章 合数求根的准备

142.1 合数求根准备：分解模数与素数幂原根

算法约束见动态版本第 128 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<tuple<long long, int, long long>> Root_Factors(long long mod,
2                                     Pollard_Rho &rho)
3 {
4     assert(mod >= 1 && mod <= 10000000000000LL);
5     auto primes = rho.Factor(mod);
6     vector<tuple<long long, int, long long>> answer;
7     for ( int i = 0; i < (int)primes.size(); )
8     {
9         long long p = primes[i], power = 1;
10        int e = 0;
11        while ( i < (int)primes.size() && primes[i] == (unsigned long long)p )
12        {
13            power *= p;
14            ++e;
15            ++i;
16        }
17        long long g = 0;
18        if ( p != 2 )
19        {
20            long long phi = power / p * (p - 1);
21            auto factors = rho.Factor(phi);
22            factors.erase(unique(factors.begin(), factors.end()), factors.end());
23            auto primitive = [&](long long candidate)
24            {
25                if ( candidate % p == 0 )
26                    return false;
27                for ( auto q : factors )
28                    if ( Number_Theory::Power(candidate, phi / q, power) == 1 )
29                        return false;
30                return true;
31            };
32            g = 2;
33            while ( !primitive(g) )
34                ++g;
35        }
36        answer.push_back({p, e, g});
37    }
38    return answer;
39 }
```

第一百四十三章 合数模高次剩余

143.1 一般合数模高次剩余：CRT 组合与按索引取根

算法约束见动态版本第 129 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Composite_Roots
2 {
3     using ll = long long;
4     ll mod = 1, total = 1;
5     vector<Prime_Power_Roots> parts;
6     vector<ll> weight;
7
8     ll Get(ll index) const
9     {
10         assert(index >= 0 && index < total);
11         ll answer = 0;
12         for ( int t = 0; t < (int)parts.size(); t++ )
13         {
14             const auto &p = parts[t];
15             ll local = index % p.Size();
16             index /= p.Size();
17             ll lift = local % p.lifts;
18             local /= p.lifts;
19             ll j = local % p.count[1], i = local / p.count[1];
20             ll root = p.Get(i, j, lift);
21             answer = (answer + (__int128)root * weight[t]) % mod;
22         }
23         return answer;
24     }
25
26     // Factors are distinct primes with positive exponents: (p, e, g).
27     // Odd-prime g is a primitive root modulo p^e; for p=2 use g=0.
```

局部求根与 CRT 系数（接上页同一结构体）：

```

28     static optional<Composite_Roots>
29     Solve(ll a, unsigned long long k, const vector<tuple<ll, int, ll>> &factors)
30     {
31         assert(k > 0);
32         Composite_Roots answer;
33         for ( auto [p, e, g] : factors )
34         {
35             auto roots = Prime_Power_Roots::Solve(a, k, p, e, g);
36             if ( !roots )
37                 return nullopt;
38             ll m = roots->step * roots->lifts;
39             assert(gcd(answer.mod, m) == 1);
40             assert(answer.mod <= 1000000000000LL / m);
41             answer.mod *= m;
42             answer.total += roots->Size();
43             answer.parts.push_back(*roots);
44         }
45         for ( const auto &p : answer.parts )
46         {
47             ll m = p.step * p.lifts, rest = answer.mod / m;
48             ll inverse = Number_Theory::Inverse(rest % m, m);
49             answer.weight.push_back((__int128)rest * inverse % answer.mod);
50         }
51         return answer;
52     }
53
54     static optional<Composite_Roots>
55     Solve(ll a, unsigned long long k, ll mod, Pollard_Rho &rho)
56     {
57         return Solve(a, k, Root_Factors(mod, rho));
58     }
59 };

```

第一百四十四章 多项式插值

144.1 拉格朗日插值与连续点优化

算法约束见动态版本第 131 页。本节代码独立可抄；采用下列实际接口名。

声明 `Lagrange_Interpolation<N>`；`Init`、`Consecutive`、`Query` 对应动态接口，`N` 为最大样本数。坐标、权值与工作数组均为静态数组，大对象放在全局或 `static`；`Query` 会改写工作数组。

```
1 template <int N> struct Lagrange_Interpolation
2 {
3     using ll = long long;
4     int n, mod;
5     ll x[N], w[N], invfac[N], pre[N + 1];
6
7     ll Norm(ll a) const
8     {
9         a %= mod;
10        if ( a < 0 )
11            a += mod;
12        return a;
13    }
14
15    ll Power(ll a, int b) const
16    {
17        ll result = 1;
18        while ( b )
19        {
20            if ( b & 1 )
21                result = result * a % mod;
22            a = a * a % mod;
23            b >>= 1;
24        }
25        return result;
26    }
27
28    void Init(const vector<ll> &xs, const vector<ll> &ys, int p)
29    {
30        n = ys.size();
31        mod = p;
32        assert(p >= 2 && n <= p && xs.size() == ys.size());
33        assert(n <= N);
34        for ( int i = 0; i < n; i++ )
35            x[i] = Norm(xs[i]);
36        for ( int i = 0; i < n; i++ )
37        {
38            ll denominator = 1;
39            for ( int j = 0; j < n; j++ )
40                if ( i != j )
41                    denominator = denominator * Norm(x[i] - x[j]) % mod;
42            assert(denominator != 0);
43            w[i] = Norm(ys[i]) * Power(denominator, mod - 2) % mod;
44        }
```

```

45     }
46
47     void Consecutive(const vector<ll> &ys, int p)
48     {
49         n = ys.size();
50         mod = p;
51         assert(p >= 2 && n <= p);
52         assert(n <= N);
53         if ( !n )
54             return;
55
56         ll factorial = 1;
57         for ( int i = 1; i < n; i++ )
58             factorial = factorial * i % mod;
59         invfac[n - 1] = Power(factorial, mod - 2);
60         for ( int i = n - 1; i > 0; i-- )
61             invfac[i - 1] = invfac[i] * i % mod;
62         for ( int i = 0; i < n; i++ )
63         {
64             x[i] = i;
65             w[i] = Norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
66             if ( (n - 1 - i) & 1 )
67                 w[i] = Norm(-w[i]);
68         }
69     }
70
71     ll Query(ll k)
72     {
73         k = Norm(k);
74         pre[0] = 1;
75         for ( int i = 0; i < n; i++ )
76             pre[i + 1] = pre[i] * Norm(k - x[i]) % mod;
77         ll answer = 0, suffix = 1;
78         for ( int i = n - 1; i >= 0; i-- )
79         {
80             answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
81             suffix = suffix * Norm(k - x[i]) % mod;
82         }
83         return answer;
84     }
85 };

```


第一百四十五章 参数化 NTT 卷积

145.1 可指定素数与原根的 NTT 卷积

算法约束见动态版本第 133 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod, int primitive = 3> struct Ntt_Convolution
2 {
3     static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
4     static constexpr int max_size = (mod - 1) & -(mod - 1);
5     using Z = Mod_Int<mod>;
6     using Poly = vector<Z>;
7
8     // mod is prime, primitive is a primitive root modulo mod.
9     static void Ntt(Poly &a, bool invert = false)
10    {
11        int n = a.size();
12        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
13        for ( int i = 1, j = 0; i < n; i++ )
14        {
15            int bit = n >> 1;
16            for ( ; j & bit; bit >>= 1 )
17                j ^= bit;
18            j ^= bit;
19            if ( i < j )
20                swap(a[i], a[j]);
21        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```

22     for ( int half = 1; half < n; half *= 2 )
23     {
24         int len = half * 2;
25         Z step = Z(primitive).Pow((mod - 1) / len);
26         if ( invert )
27             step = step.Inv();
28         for ( int i = 0; i < n; i += len )
29         {
30             Z w = 1;
31             for ( int j = 0; j < half; j++ )
32             {
33                 Z u = a[i + j], v = a[i + j + half] * w;
34                 a[i + j] = u + v;
35                 a[i + j + half] = u - v;
36                 w = w * step;
37             }
38         }
39     }
40     if ( invert )
41     {
42         Z inverse = Z(n).Inv();
43         for ( auto &x : a )
44             x = x * inverse;
45     }
46 }
47
48 static Poly Multiply(Poly a, Poly b)
49 {
50     if ( a.empty() || b.empty() )
51         return {};
52     assert(a.size() + b.size() - 1 <= max_size);
53     int size = a.size() + b.size() - 1, n = 1;
54     while ( n < size )
55         n *= 2;
56     a.resize(n);
57     b.resize(n);
58     Ntt(a);
59     Ntt(b);
60     for ( int i = 0; i < n; i++ )
61         a[i] = a[i] * b[i];
62     Ntt(a, true);
63     a.resize(size);
64     return a;
65 }
66 };

```

第一百四十六章 多项式平移

146.1 多项式系数平移

算法约束见动态版本第 135 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod = 998244353, int primitive = 3>
2 vector<Mod_Int<mod>> Polynomial_Shift(const vector<Mod_Int<mod>> &f, Mod_Int<mod> c)
3 {
4     using N = Ntt_Convolution<mod, primitive>;
5     using Z = typename N::Z;
6     using Poly = typename N::Poly;
7     if ( f.empty() )
8         return {};
9     assert(f.size() <= (N::max_size + 1ULL) / 2);
10    int n = f.size();
11    Poly fact(n), inv(n), a(n), b(n);
12    fact[0] = 1;
13    for ( int i = 1; i < n; i++ )
14        fact[i] = fact[i - 1] * i;
15    inv[n - 1] = fact[n - 1].Inv();
16    for ( int i = n - 1; i > 0; i-- )
17        inv[i - 1] = inv[i] * i;
18    Z power = 1;
19    for ( int i = 0; i < n; i++ )
20    {
21        a[n - 1 - i] = f[i] * fact[i];
22        b[i] = power * inv[i];
23        power = power * c;
24    }
25    auto product = N::Multiply(a, b);
26    for ( int i = 0; i < n; i++ )
27        a[i] = product[n - 1 - i] * inv[i];
28    return a;
29 }
```

第一百四十七章 等比点求值

147.1 Chirp Z: 等比点求值

算法约束见动态版本第 136 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod = 998244353, int primitive = 3>
2 vector<Mod_Int<mod>>
3 Chirp_Z(const vector<Mod_Int<mod>> &f, Mod_Int<mod> a, Mod_Int<mod> r, int m)
4 {
5     using N = Ntt_Convolution<mod, primitive>;
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     assert(m >= 0 && f.size() <= INT_MAX);
9     if ( m == 0 || f.empty() )
10         return Poly(m);
11     if ( a.v == 0 || r.v == 0 || r.v == 1 )
12     {
13         Poly answer(m, f[0]);
14         Z value = 0;
15         for ( int i = int(f.size()) - 1; i >= 0; i-- )
16             value = value * a + f[i];
17         answer[0] = value;
18         if ( r.v == 1 )
19             fill(answer.begin(), answer.end(), value);
20         return answer;
21     }
22     assert(f.size() + m - 1 <= N::max_size);
23     int n = f.size(), size = 1;
24     while ( size < n + m - 1 )
25         size *= 2;
26     Poly x(size), y(size), inverse(max(n, m));
27     Z power = 1, triangle = 1, ri = r.Inv();
28     for ( auto &v : inverse )
29     {
30         v = triangle;
31         triangle = triangle * power;
32         power = power * ri;
33     }
```

构造循环卷积并取点值（接上页同一函数）：

```
34     power = 1;
35     for ( int j = 0; j < n; j++ )
36     {
37         x[n - 1 - j] = f[j] * power * inverse[j];
38         power = power * a;
39     }
40     power = 1;
41     triangle = 1;
42     for ( int j = 0; j < n + m - 1; j++ )
43     {
44         y[j] = triangle;
45         triangle = triangle * power;
46         power = power * r;
47     }
48     N::Ntt(x);
49     N::Ntt(y);
50     for ( int j = 0; j < size; j++ )
51         x[j] = x[j] * y[j];
52     N::Ntt(x, true);
53     Poly answer(m);
54     for ( int i = 0; i < m; i++ )
55         answer[i] = x[n - 1 + i] * inverse[i];
56     return answer;
57 }
```

第一百四十八章 斯特林数整行计算

148.1 第二类斯特林数：同一行前缀

算法约束见动态版本第 138 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod = 998244353, int primitive = 3>
2 inline vector<Mod_Int<mod>> Stirling_Second_Row(unsigned long long n, int m)
3 {
4     using N = Ntt_Convolution<mod, primitive>;
5     assert(m >= 0 && m <= (N::max_size - 1) / 2);
6     using Z = typename N::Z;
7     typename N::Poly a(m + 1), b(m + 1);
8     b[0] = 1;
9     for (int i = 1; i <= m; i++)
10         b[i] = b[i - 1] * i;
11     b[m] = b[m].Inv();
12     for (int i = m; i > 0; i--)
13         b[i - 1] = b[i] * i;
14     for (int i = 0; i <= m; i++)
15     {
16         Z value = i ? Z(i).Pow(n % (mod - 1)) : Z(n == 0);
17         a[i] = value * b[i];
18         if (i % 2)
19             b[i] = Z(0) - b[i];
20     }
21     auto answer = N::Multiply(a, b);
22     answer.resize(m + 1);
23     return answer;
24 }
```

148.2 无符号第一类斯特林数：同一行

算法约束见动态版本第 139 页。本节代码独立可抄；采用下列实际接口名。

```

1 template <int mod = 998244353, int primitive = 3>
2 inline vector<Mod_Int<mod>> Stirling_First_Row(int n)
3 {
4     using N = Ntt_Convolution<mod, primitive>;
5     assert(n >= 0 && n < N::max_size);
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     Poly fact(n + 1), inv(n + 1);
9     fact[0] = 1;
10    for ( int i = 1; i <= n; i++ )
11        fact[i] = fact[i - 1] * i;
12    inv[n] = fact[n].Inv();
13    for ( int i = n; i > 0; i-- )
14        inv[i - 1] = inv[i] * i;
15    function<Poly(int)> Solve = [&](int len) -> Poly
16    {
17        if ( len == 0 )
18            return {1};
19        int m = len / 2;
20        Poly f = Solve(m);
21        Poly a(m + 1), b(m + 1);
22        Z power = 1;
23        for ( int i = 0; i <= m; i++ )
24        {
25            a[m - i] = f[i] * fact[i];
26            b[i] = power * inv[i];
27            power = power * m;
28        }
29        auto c = N::Multiply(a, b);
30        for ( int i = 0; i <= m; i++ )
31            a[i] = c[m - i] * inv[i];
32        f = N::Multiply(f, a);
33        if ( len % 2 )
34        {
35            f.push_back(0);
36            for ( int i = len; i > 0; i-- )
37                f[i] = f[i - 1] + f[i] * (len - 1);
38            f[0] = f[0] * (len - 1);
39        }
40        return f;
41    };
42    return Solve(n);
43 }
```

第一百四十九章 集合变换与卷积

149.1 按位 OR、AND、XOR 卷积

算法约束见动态版本第 140 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod = 998244353> struct Set_Convolution
2 {
3     static_assert(mod >= 2);
4     using Z = Mod_Int<mod>;
5     using Poly = vector<Z>;
6
7     static void Transform(Poly &a, char op, bool invert = false)
8     {
9         assert(!a.empty() && a.size() <= INT_MAX);
10        int n = a.size();
11        assert((n & (n - 1)) == 0);
12        assert(op == '|' || op == '&' || op == '^');
13        assert(op != '^' || mod % 2 == 1);
14        Z half = (mod + 1LL) / 2;
15        for ( int len = 1; len < n; len *= 2 )
16            for ( int i = 0; i < n; i += len * 2 )
17                for ( int j = 0; j < len; j++ )
18                    {
19                        Z &u = a[i + j], &v = a[i + j + len];
20                        if ( op == '|' )
21                            v = invert ? v - u : v + u;
22                        else if ( op == '&' )
23                            u = invert ? u - v : u + v;
24                        else
25                            {
26                                Z x = u, y = v;
27                                u = x + y;
28                                v = x - y;
29                                if ( invert )
30                                    {
31                                        u = u * half;
32                                        v = v * half;
33                                    }
34                            }
35                    }
36    }
```


点乘并逆变换（接上页同一结构体）：

```
38 static Poly Multiply(Poly a, Poly b, char op)
39 {
40     assert(a.size() == b.size());
41     Transform(a, op);
42     Transform(b, op);
43     for ( int i = 0; i < int(a.size()); i++ )
44         a[i] = a[i] * b[i];
45     Transform(a, op, true);
46     return a;
47 }
48 };
```

第一百五十章 不相交子集卷积

150.1 不相交子集卷积

算法约束见动态版本第 142 页。本节代码独立可抄；采用下列实际接口名。

```
1 // c[s] = sum_{t subset s} a[t] * b[s xor t].
2 template <int mod = 998244353>
3 vector<Mod_Int<mod>> Subset_Convolution(const vector<Mod_Int<mod>> &a,
4                                         const vector<Mod_Int<mod>> &b)
5 {
6     using F = Set_Convolution<mod>;
7     using Poly = typename F::Poly;
8     assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
9     int n = a.size(), m = 0;
10    assert((n & (n - 1)) == 0);
11    while ( (1 << m) < n )
12        m++;
13    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
14    vector<int> rank(n);
15    for ( int s = 0; s < n; s++ )
16    {
17        rank[s] = __builtin_popcount(unsigned(s));
18        f[rank[s]][s] = a[s];
19        g[rank[s]][s] = b[s];
20    }
21    for ( int k = 0; k <= m; k++ )
22    {
23        F::Transform(f[k], '|');
24        F::Transform(g[k], '|');
25    }
26    Poly answer(n), h(n);
27    for ( int k = 0; k <= m; k++ )
28    {
29        fill(h.begin(), h.end(), 0);
30        for ( int i = 0; i <= k; i++ )
31            for ( int s = 0; s < n; s++ )
32                h[s] = h[s] + f[i][s] * g[k - i][s];
33        F::Transform(h, '|', true);
34        for ( int s = 0; s < n; s++ )
35            if ( rank[s] == k )
36                answer[s] = h[s];
37    }
38    return answer;
39 }
```

第一百五十一章 多项式

151.1 NTT、形式幂级数、FWT 与递推

算法约束见动态版本第 143 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Polynomial
2 {
3     static constexpr int mod = 998244353;
4     using Z = Mod_Int<mod>;
5     using Poly = vector<Z>;
6
7     static void Ntt(Poly &a, bool invert = false)
8     {
9         int n = (int)a.size();
10        assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
11        for ( int i = 1, j = 0; i < n; i++ )
12        {
13            int bit = n >> 1;
14            for ( ; j & bit; bit >>= 1 )
15                j ^= bit;
16            j ^= bit;
17            if ( i < j )
18                swap(a[i], a[j]);
19        }
20        for ( int len = 2; len <= n; len *= 2 )
21        {
22            Z step = Z(3).Pow((mod - 1) / len);
23            if ( invert )
24                step = step.Inv();
25            for ( int i = 0; i < n; i += len )
26            {
27                Z w = 1;
28                for ( int j = 0; j < len / 2; j++, w = w * step )
29                {
30                    Z u = a[i + j], v = a[i + j + len / 2] * w;
31                    a[i + j] = u + v;
32                    a[i + j + len / 2] = u - v;
33                }
34            }
35        }
36        if ( invert )
37        {
38            Z Inv = Z(n).Inv();
39            for ( auto &x : a )
40                x = x * Inv;
41        }
42    }
43
44    static Poly Multiply(Poly a, Poly b)
45    {
46        if ( a.empty() || b.empty() )
```

```

47     return {};
48     int size = (int)a.size() + (int)b.size() - 1, n = 1;
49     assert(size <= (1 << 23));
50     while ( n < size )
51         n *= 2;
52     a.resize(n);
53     b.resize(n);
54     Ntt(a);
55     Ntt(b);
56     for ( int i = 0; i < n; i++ )
57         a[i] = a[i] * b[i];
58     Ntt(a, true);
59     a.resize(size);
60     return a;
61 }
62
63 static Poly Inverse(const Poly &a, int n)
64 {
65     assert(n > 0 && !a.empty() && a[0].v);
66     Poly b{a[0].Inv()};
67     while ( (int)b.size() < n )
68     {
69         int m = min(n, 2 * (int)b.size());
70         Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
71         auto t = Multiply(f, b);
72         t.resize(m);
73         for ( auto &x : t )
74             x = Z(0) - x;
75         t[0] = t[0] + 2;
76         b = Multiply(b, t);
77         b.resize(m);
78     }
79     return b;
80 }
81
82 static Poly Derivative(const Poly &a)
83 {
84     Poly b;
85     for ( int i = 1; i < (int)a.size(); i++ )
86         b.push_back(a[i] * i);
87     return b;
88 }
89
90 static Poly Integral(const Poly &a)
91 {
92     Poly b(a.size() + 1);
93     vector<Z> Inv(a.size() + 1);
94     if ( !a.empty() )
95         Inv[1] = 1;
96     for ( int i = 2; i <= (int)a.size(); i++ )
97         Inv[i] = Z(mod - mod / i) * Inv[mod % i];
98     for ( int i = 0; i < (int)a.size(); i++ )
99         b[i + 1] = a[i] * Inv[i + 1];
100     return b;
101 }
102

```

```

103 static Poly Log(const Poly &a, int n)
104 {
105     assert(n > 0 && !a.empty() && a[0].v == 1);
106     auto b = Multiply(Derivative(a), Inverse(a, n));
107     b.resize(n - 1);
108     return Integral(b);
109 }
110
111 static Poly Exp(const Poly &a, int n)
112 {
113     assert(n > 0 && (a.empty() || a[0].v == 0));
114     Poly b{1};
115     while ( (int)b.size() < n )
116     {
117         int m = min(n, 2 * (int)b.size());
118         auto t = Log(b, m);
119         for ( int i = 0; i < m; i++ )
120             t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
121         t[0] = t[0] + 1;
122         b = Multiply(b, t);
123         b.resize(m);
124     }
125     return b;
126 }
127
128 static void Fwt(Poly &a, char op, bool invert = false)
129 {
130     int n = (int)a.size();
131     assert(n > 0 && (n & (n - 1)) == 0);
132     const Z inv2 = (mod + 1) / 2;
133     for ( int len = 1; len < n; len *= 2 )
134         for ( int i = 0; i < n; i += len * 2 )
135             for ( int j = 0; j < len; j++ )
136             {
137                 Z &u = a[i + j], &v = a[i + j + len], x = u, y = v;
138                 if ( op == '^' )
139                 {
140                     u = x + y;
141                     v = x - y;
142                     if ( invert )
143                     {
144                         u = u * inv2;
145                         v = v * inv2;
146                     }
147                 }
148                 else if ( op == '|' )
149                     v = invert ? y - x : y + x;
150                 else
151                 {
152                     assert(op == '&');
153                     u = invert ? x - y : x + y;
154                 }
155             }
156 }
157
158 // c: s[n]=sum c[i-1]*s[n-i], i=1..L.

```

```

159 static Poly Berlekamp_Massey(const Poly &s)
160 {
161     Poly c{1}, b{1};
162     int l = 0, m = 1;
163     Z last = 1;
164     for ( int n = 0; n < (int)s.size(); n++ )
165     {
166         Z d = s[n];
167         for ( int i = 1; i <= l; i++ )
168             d = d + c[i] * s[n - i];
169         if ( !d.v )
170         {
171             ++m;
172             continue;
173         }
174         auto old = c;
175         Z coef = d / last;
176         if ( c.size() < b.size() + m )
177             c.resize(b.size() + m);
178         for ( int j = 0; j < (int)b.size(); j++ )
179             c[j + m] = c[j + m] - coef * b[j];
180         if ( 2 * l <= n )
181         {
182             l = n + 1 - l;
183             b = old;
184             last = d;
185             m = 1;
186         }
187         else
188             ++m;
189     }
190     c.erase(c.begin());
191     for ( auto &x : c )
192         x = Z(0) - x;
193     return c;
194 }
195
196 static Z Recurrence(Poly init, const Poly &c, unsigned long long n)
197 {
198     int k = (int)c.size();
199     if ( n < init.size() )
200         return init[n];
201     if ( !k )
202         return 0;
203     assert((int)init.size() >= k);
204     auto combine = [&](const Poly &a, const Poly &b)
205     {
206         Poly t(2 * k - 1);
207         for ( int i = 0; i < k; i++ )
208             for ( int j = 0; j < k; j++ )
209                 t[i + j] = t[i + j] + a[i] * b[j];
210         for ( int i = 2 * k - 2; i >= k; i-- )
211             for ( int j = 1; j <= k; j++ )
212                 t[i - j] = t[i - j] + t[i] * c[j - 1];
213         t.resize(k);
214         return t;

```

```
215     };
216     Poly a(k), r(k);
217     r[0] = 1;
218     if ( k == 1 )
219         a[0] = c[0];
220     else
221         a[1] = 1;
222     for ( ; n; n >>= 1, a = combine(a, a) )
223         if ( n & 1 )
224             r = combine(r, a);
225     Z ans = 0;
226     for ( int i = 0; i < k; i++ )
227         ans = ans + r[i] * init[i];
228     return ans;
229 }
230 };
```

第一百五十二章 代数与数论进阶

152.1 模高斯消元、行列式、矩阵树定理

算法约束见动态版本第 148 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod> struct Linear_Algebra
2 {
3     using Z = Mod_Int<mod>;
4     using Matrix = vector<vector<Z>>>;
5
6     struct Solution
7     {
8         bool consistent;
9         int rank;
10        vector<Z> particular;
11        Matrix kernel;
12    };
13
14    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
15    static Solution Solve(Matrix a, int n)
16    {
17        int m = (int)a.size(), row = 0;
18        vector<int> where(n, -1);
19        for ( const auto &v : a )
20            assert((int)v.size() == n + 1);
21        for ( int col = 0; col < n && row < m; col++ )
22        {
23            int p = row;
24            while ( p < m && !a[p][col].v )
25                ++p;
26            if ( p == m )
27                continue;
28            swap(a[p], a[row]);
29            Z Inv = a[row][col].Inv();
30            for ( int j = col; j <= n; j++ )
31                a[row][j] = a[row][j] * Inv;
32            for ( int i = 0; i < m; i++ )
33                if ( i != row && a[i][col].v )
34                {
35                    Z f = a[i][col];
36                    for ( int j = col; j <= n; j++ )
37                        a[i][j] = a[i][j] - f * a[row][j];
38                }
39            where[col] = row++;
40        }
41        for ( int i = row; i < m; i++ )
42            if ( a[i][n].v )
43                return {false, row, {}, {}};
44        Solution ans{true, row, vector<Z>(n), {}};
45        for ( int j = 0; j < n; j++ )
46            if ( where[j] != -1 )
```



```

47         ans.particular[j] = a[where[j]][n];
48     for ( int j = 0; j < n; j++ )
49         if ( where[j] == -1 )
50         {
51             vector<Z> v(n);
52             v[j] = 1;
53             for ( int k = 0; k < n; k++ )
54                 if ( where[k] != -1 )
55                     v[k] = Z(0) - a[where[k]][j];
56             ans.kernel.push_back(v);
57         }
58     return ans;
59 }
60
61 static Z Determinant(Matrix a)
62 {
63     int n = (int)a.size();
64     Z ans = 1;
65     for ( int i = 0; i < n; i++ )
66     {
67         assert((int)a[i].size() == n);
68         int p = i;
69         while ( p < n && !a[p][i].v )
70             ++p;
71         if ( p == n )
72             return 0;
73         if ( p != i )
74         {
75             swap(a[p], a[i]);
76             ans = Z(0) - ans;
77         }
78         ans = ans * a[i][i];
79         Z Inv = a[i][i].Inv();
80         for ( int j = i + 1; j < n; j++ )
81         {
82             Z f = a[j][i] * Inv;
83             for ( int k = i; k < n; k++ )
84                 a[j][k] = a[j][k] - f * a[i][k];
85         }
86     }
87     return ans;
88 }
89
90 static Matrix Multiply(const Matrix &a, const Matrix &b)
91 {
92     assert(!a.empty() && !b.empty());
93     int n = a.size(), m = b[0].size(), k = b.size();
94     assert((int)a[0].size() == k);
95     Matrix c(n, vector<Z>(m));
96     for ( int i = 0; i < n; i++ )
97         for ( int t = 0; t < k; t++ )
98             for ( int j = 0; j < m; j++ )
99                 c[i][j] = c[i][j] + a[i][t] * b[t][j];
100     return c;
101 }
102

```

```

103 static Matrix Power(Matrix a, unsigned long long e)
104 {
105     int n = a.size();
106     assert(n > 0 && (int)a[0].size() == n);
107     Matrix r(n, vector<Z>(n));
108     for ( int i = 0; i < n; i++ )
109         r[i][i] = 1;
110     for ( ; e; e >>= 1, a = Multiply(a, a) )
111         if ( e & 1 )
112             r = Multiply(r, a);
113     return r;
114 }
115
116 // Undirected multigraph, vertices 0..n-1; loops ignored.
117 static Z Spanning_Trees(int n, const vector<pair<int, int>> &edges)
118 {
119     assert(n > 0);
120     Matrix lap(n, vector<Z>(n));
121     for ( auto [u, v] : edges )
122         if ( u != v )
123         {
124             lap[u][u] = lap[u][u] + 1;
125             lap[v][v] = lap[v][v] + 1;
126             lap[u][v] = lap[u][v] - 1;
127             lap[v][u] = lap[v][u] - 1;
128         }
129     lap.pop_back();
130     for ( auto &row : lap )
131         row.pop_back();
132     return Determinant(lap);
133 }
134 };

```

152.2 杜教筛

算法约束见动态版本第 151 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Du_Jiao
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int limit;
6     vector<ll> pmu, pphi;
7     unordered_map<ll, ll> mmu;
8     unordered_map<ll, I> mphi;
9
10    Du_Jiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
11    {
12        assert(limit >= 1);
13        Linear_Sieve s(limit);
14        for ( int i = 1; i <= limit; i++ )
15        {
16            pmu[i] = pmu[i - 1] + s.mu[i];
17            pphi[i] = pphi[i - 1] + s.phi[i];
18        }
19    }
20
21    ll Mertens(ll n)
22    {
23        if ( n <= limit )
24            return pmu[n];
25        if ( mmu.count(n) )
26            return mmu[n];
27        ll ans = 1;
28        for ( ll l = 2, r; l <= n; l = r + 1 )
29        {
30            r = n / (n / l);
31            ans -= (r - l + 1) * Mertens(n / l);
32        }
33        return mmu[n] = ans;
34    }
35
36    I Totient_Sum(ll n)
37    {
38        if ( n <= limit )
39            return pphi[n];
40        if ( mphi.count(n) )
41            return mphi[n];
42        I ans = I(n) * (n + 1) / 2;
43        for ( ll l = 2, r; l <= n; l = r + 1 )
44        {
45            r = n / (n / l);
46            ans -= I(r - l + 1) * Totient_Sum(n / l);
47        }
48        return mphi[n] = ans;
49    }
50 };

```

152.3 扩展 BSGS

算法约束见动态版本第 153 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Discrete_Log
2 {
3     using ll = long long;
4
5     // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
6     static ll Solve(ll a, ll b, ll m)
7     {
8         assert(m >= 1 && m <= 1000000000000LL);
9         a = (a % m + m) % m;
10        b = (b % m + m) % m;
11        if ( m == 1 || b == 1 )
12            return 0;
13        if ( a == 1 )
14            return -1;
15        ll offset = 0, k = 1;
16        for ( ll g; (g = gcd(a, m)) > 1; )
17        {
18            if ( b == k )
19                return offset;
20            if ( b % g )
21                return -1;
22            b /= g;
23            m /= g;
24            k = (__int128)k * (a / g) % m;
25            ++offset;
26        }
27        ll target = (__int128)b * Number_Theory::Inverse(k, m) % m;
28        ll step = sqrtl(m) + 1;
29        unordered_map<ll, ll> baby;
30        baby.reserve(step);
31        ll cur = 1 % m;
32        for ( ll j = 0; j < step; j++ )
33        {
34            if ( !baby.count(cur) )
35                baby[cur] = j;
36            cur = (__int128)cur * a % m;
37        }
38        ll Inv = Number_Theory::Inverse(Number_Theory::Power(a, step, m), m);
39        cur = target;
40        for ( ll i = 0; i <= step; i++ )
41        {
42            auto it = baby.find(cur);
43            if ( it != baby.end() )
44                return offset + i * step + it->second;
45            cur = (__int128)cur * Inv % m;
46        }
47        return -1;
48    }
49 };

```

第一百五十三章 任意模数行列式

153.1 任意正模数行列式：欧几里得消元

算法约束见动态版本第 155 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline long long Determinant_Mod(vector<vector<long long>> a, long long mod)
2 {
3     assert(mod >= 1);
4     int n = a.size();
5     for ( auto &row : a )
6     {
7         assert((int)row.size() == n);
8         for ( auto &x : row )
9         {
10             x %= mod;
11             if ( x < 0 )
12                 x += mod;
13         }
14     }
15     long long answer = 1 % mod;
16     for ( int i = 0; i < n; i++ )
17     {
18         for ( int j = i + 1; j < n; j++ )
19             while ( a[j][i] )
20             {
21                 long long quotient = a[i][i] / a[j][i];
22                 for ( int k = i; k < n; k++ )
23                 {
24                     __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
25                     value %= mod;
26                     if ( value < 0 )
27                         value += mod;
28                     a[i][k] = (long long)value;
29                 }
30                 swap(a[i], a[j]);
31                 answer = answer ? mod - answer : 0;
32             }
33         if ( a[i][i] == 0 )
34             return 0;
35         answer = (__int128)answer * a[i][i] % mod;
36     }
37     return answer;
38 }
```

第一百五十四章 带权生成树计数

154.1 带权矩阵树：无向树与内外向树形图

算法约束见动态版本第 156 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod> struct Matrix_Tree
2 {
3     using A = Linear_Algebra<mod>;
4     using Z = typename A::Z;
5     using Edge = tuple<int, int, Z>;
6     enum class Kind
7     {
8         undirected,
9         toward_root,
10        away_from_root
11    };
12
13    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
14    static Z Count(int n, const vector<Edge> &edges, int root, Kind kind)
15    {
16        assert(n >= 1 && root >= 0 && root < n);
17        typename A::Matrix lap(n - 1, vector<Z>(n - 1));
18        auto add = [&](int u, int v, Z w)
19        {
20            if ( u == root )
21                return;
22            int i = u - (u > root), j = v - (v > root);
23            lap[i][i] = lap[i][i] + w;
24            if ( v != root )
25                lap[i][j] = lap[i][j] - w;
26        };
27        for ( auto [u, v, w] : edges )
28        {
29            assert(u >= 0 && u < n && v >= 0 && v < n);
30            if ( u == v )
31                continue;
32            if ( kind == Kind::away_from_root )
33                swap(u, v);
34            add(u, v, w);
35            if ( kind == Kind::undirected )
36                add(v, u, w);
37        }
38        return A::Determinant(lap);
39    }
40};
```

第一百五十五章 合数模生成树计数

155.1 任意正模数带权矩阵树

算法约束见动态版本第 157 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Matrix_Tree_Mod
2 {
3     using ll = long long;
4     using Edge = tuple<int, int, ll>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    static ll Count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
13    {
14        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
15        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
16        auto add = [&](int u, int v, ll w)
17        {
18            if ( u == root )
19                return;
20            int i = u - (u > root), j = v - (v > root);
21            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
22            if ( v != root )
23                lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
24        };
25        for ( auto [u, v, w] : edges )
26        {
27            assert(u >= 0 && u < n && v >= 0 && v < n);
28            if ( u == v )
29                continue;
30            w %= mod;
31            if ( w < 0 )
32                w += mod;
33            if ( kind == Kind::away_from_root )
34                swap(u, v);
35            add(u, v, w);
36            if ( kind == Kind::undirected )
37                add(v, u, w);
38        }
39        return Determinant_Mod(move(lap), mod);
40    }
41 };
```

第一百五十六章 计算几何

156.1 整数精确二维几何

算法约束见动态版本第 158 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Integer_Geometry
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
7     struct Point
8     {
9         ll x = 0, y = 0;
10
11         bool operator<(Point b) const
12         {
13             return tie(x, y) < tie(b.x, b.y);
14         }
15
16         bool operator==(Point b) const
17         {
18             return x == b.x && y == b.y;
19         }
20     };
21
22     static I Cross(Point a, Point b, Point c)
23     {
24         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
25     }
26
27     static I Dot(Point a, Point b, Point c)
28     {
29         return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
30     }
31
32     static I Distance_Squared(Point a, Point b)
33     {
34         return Dot(a, b, b);
35     }
36
37     static int Sign(I x)
38     {
39         return (x > 0) - (x < 0);
40     }
41
42     static bool On_Segment(Point p, Point a, Point b)
43     {
44         return Cross(a, b, p) == 0 && Dot(p, a, b) <= 0;
45     }
46 }
```



```

47 static bool Intersect(Point a, Point b, Point c, Point d)
48 {
49     int x = Sign(Cross(a, b, c)), y = Sign(Cross(a, b, d));
50     int z = Sign(Cross(c, d, a)), w = Sign(Cross(c, d, b));
51     if ( !x && On_Segment(c, a, b) )
52         return true;
53     if ( !y && On_Segment(d, a, b) )
54         return true;
55     if ( !z && On_Segment(a, c, d) )
56         return true;
57     if ( !w && On_Segment(b, c, d) )
58         return true;
59     return x * y < 0 && z * w < 0;
60 }
61
62 static vector<Point> Convex_Hull(vector<Point> p)
63 {
64     sort(p.begin(), p.end());
65     p.erase(unique(p.begin(), p.end()), p.end());
66     int n = (int)p.size();
67     if ( n < 3 )
68         return p;
69     vector<Point> h;
70     for ( Point v : p )
71     {
72         while ( h.size() > 1 && Cross(h[h.size() - 2], h.back(), v) <= 0 )
73             h.pop_back();
74         h.push_back(v);
75     }
76     int low = (int)h.size();
77     for ( int i = n - 2; i >= 0; i-- )
78     {
79         while ( (int)h.size() > low && Cross(h[h.size() - 2], h.back(), p[i]) <= 0 )
80             h.pop_back();
81         h.push_back(p[i]);
82     }
83     h.pop_back();
84     return h; // CCW, no repeated endpoint, no collinear interior points.
85 }
86
87 static I Area_Twice(const vector<Point> &p)
88 {
89     I ans = 0;
90     int n = (int)p.size();
91     for ( int i = 0; i < n; i++ )
92         ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
93     return ans;
94 }
95
96 // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
97 static int Contains(const vector<Point> &p, Point q)
98 {
99     int winding = 0, n = (int)p.size();
100     for ( int i = 0; i < n; i++ )
101     {
102         Point a = p[i], b = p[(i + 1) % n];

```

```

103         if ( On_Segment(q, a, b) )
104             return 1;
105         I c = Cross(a, b, q);
106         if ( a.y <= q.y && b.y > q.y && c > 0 )
107             ++winding;
108         if ( a.y > q.y && b.y <= q.y && c < 0 )
109             --winding;
110     }
111     return winding ? 2 : 0;
112 }
113
114 // Input must be the strict CCW hull above. O(log n).
115 static int Convex_Contains(const vector<Point> &p, Point q)
116 {
117     int n = (int)p.size();
118     if ( n < 3 )
119         return Contains(p, q);
120     I a = Cross(p[0], p[1], q), b = Cross(p[0], p[n - 1], q);
121     if ( a < 0 || b > 0 )
122         return 0;
123     if ( !a )
124         return On_Segment(q, p[0], p[1]) ? 1 : 0;
125     if ( !b )
126         return On_Segment(q, p[0], p[n - 1]) ? 1 : 0;
127     int l = 1, r = n - 1;
128     while ( l + 1 < r )
129     {
130         int m = (l + r) / 2;
131         if ( Cross(p[0], p[m], q) >= 0 )
132             l = m;
133         else
134             r = m;
135     }
136     I c = Cross(p[l], p[r], q);
137     return c < 0 ? 0 : c == 0 ? 1 : 2;
138 }
139
140 static I Diameter_Squared(const vector<Point> &p)
141 {
142     int n = (int)p.size();
143     if ( n < 2 )
144         return 0;
145     if ( n == 2 )
146         return Distance_Squared(p[0], p[1]);
147     I ans = 0;
148     int j = 1;
149     for ( int i = 0; i < n; i++ )
150     {
151         int k = (i + 1) % n;
152         while ( Cross(p[i], p[k], p[(j + 1) % n]) > Cross(p[i], p[k], p[j]) )
153             j = (j + 1) % n;
154         ans =
155             max({ans, Distance_Squared(p[i], p[j]), Distance_Squared(p[k], p[j])});
156     }
157     return ans;
158 }

```

```

159
160 struct PolarLess
161 {
162     static int half(Point p)
163     {
164         return p.y < 0 || (p.y == 0 && p.x < 0);
165     }
166
167     bool operator()(Point a, Point b) const
168     {
169         if ( half(a) != half(b) )
170             return half(a) < half(b);
171         I c = Cross({0, 0}, a, b);
172         if ( c )
173             return c > 0;
174         return Distance_Squared({0, 0}, a) < Distance_Squared({0, 0}, b);
175     }
176 };
177 };

```

156.2 浮点点线圆构造

算法约束见动态版本第 161 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Real_Geometry
2 {
3     using R = long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6
7     Real_Geometry(R eps = 1e-12L) : eps(eps)
8     {
9         assert(eps > 0);
10    }
11
12    struct Point
13    {
14        R x = 0, y = 0;
15
16        Point operator+(Point b) const
17        {
18            return {x + b.x, y + b.y};
19        }
20
21        Point operator-(Point b) const
22        {
23            return {x - b.x, y - b.y};
24        }
25
26        Point operator*(R k) const
27        {
28            return {x * k, y * k};
29        }
30
31        Point operator/(R k) const

```

```

32     {
33         return {x / k, y / k};
34     }
35 };
36
37 struct Circle
38 {
39     Point o;
40     R r;
41 };
42 enum class Kind
43 {
44     none,
45     one,
46     two,
47     infinite,
48     degenerate
49 };
50
51 struct Result
52 {
53     Kind kind;
54     vector<Point> p;
55 };
56
57 static R Dot(Point a, Point b)
58 {
59     return a.x * b.x + a.y * b.y;
60 }
61
62 static R Cross(Point a, Point b)
63 {
64     return a.x * b.y - a.y * b.x;
65 }
66
67 static R norm(Point a)
68 {
69     return hypotl(a.x, a.y);
70 }
71
72 static Point perp(Point a)
73 {
74     return {-a.y, a.x};
75 }
76
77 Point Projection(Point p, Point a, Point b) const
78 {
79     Point v = b - a;
80     R d = Dot(v, v);
81     if ( d == 0 )
82         return a;
83     return a + v * (Dot(p - a, v) / d);
84 }
85
86 R Segment_Distance(Point p, Point a, Point b) const
87 {

```

```

88     Point v = b - a;
89     R d = Dot(v, v);
90     if ( d == 0 )
91         return norm(p - a);
92     R t = clamp(Dot(p - a, v) / d, R(0), R(1));
93     return norm(p - (a + v * t));
94 }
95
96 Result Line_Intersection(Point a, Point b, Point c, Point d) const
97 {
98     Point u = b - a, v = d - c;
99     R nu = norm(u), nv = norm(v);
100    if ( nu == 0 || nv == 0 )
101        return {Kind::degenerate, {}};
102    R det = Cross(u, v);
103    if ( fabsl(det) <= eps * nu * nv )
104        return {fabsl(Cross(c - a, u)) <= eps * nu * max(R(1), norm(c - a))
105                ? Kind::infinite
106                : Kind::none,
107                {}};
108    return {Kind::one, {a + u * (Cross(c - a, v) / det)}};
109 }
110
111 Result Line_Circle(Point a, Point b, Circle c) const
112 {
113     assert(c.r >= 0);
114     Point v = b - a;
115     R len = norm(v);
116     if ( len == 0 )
117         return {Kind::degenerate, {}};
118     Point h = Projection(c.o, a, b);
119     R d = norm(h - c.o), tol = eps * max(d, c.r);
120     if ( d > c.r + tol )
121         return {Kind::none, {}};
122     if ( fabsl(d - c.r) <= tol )
123         return {Kind::one, {h}};
124     R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
125     Point w = v * (t / len);
126     return {Kind::two, {h - w, h + w}};
127 }
128
129 Result Circles(Circle a, Circle b) const
130 {
131     assert(a.r >= 0 && b.r >= 0);
132     Point v = b.o - a.o;
133     R d = norm(v);
134     R tol = eps * max({d, a.r, b.r});
135     // Infinite intersections require identical positive-radius circles.
136     if ( d == 0 )
137     {
138         if ( a.r != b.r )
139             return {Kind::none, {}};
140         if ( a.r == 0 )
141             return {Kind::one, {a.o}};
142         return {Kind::infinite, {}};
143     }

```

```

144     if ( a.r == 0 && b.r == 0 )
145         return {Kind::none, {}};
146     if ( a.r == 0 || b.r == 0 )
147     {
148         R radius = max(a.r, b.r);
149         if ( fabsl(d - radius) > tol )
150             return {Kind::none, {}};
151         return {Kind::one, {a.r == 0 ? a.o : b.o}};
152     }
153     if ( d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol )
154         return {Kind::none, {}};
155     R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
156     Point h = a.o + v * (x / d);
157     // Near-concentric circles must not be mistaken for internal tangency.
158     if ( fabsl(x) > a.r + tol )
159         return {Kind::none, {}};
160     if ( fabsl(fabsl(x) - a.r) <= tol )
161         return {Kind::one, {h}};
162     R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
163     Point w = perp(v) * (y / d);
164     return {Kind::two, {h - w, h + w}};
165 }
166
167 R Overlap(Circle a, Circle b) const
168 {
169     assert(a.r >= 0 && b.r >= 0);
170     R d = norm(a.o - b.o), pi = acosl(-1.L);
171     if ( d >= a.r + b.r )
172         return 0;
173     if ( d <= fabsl(a.r - b.r) )
174         return pi * min(a.r, b.r) * min(a.r, b.r);
175     R difference = (a.r - b.r) * (a.r + b.r);
176     R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
177     R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
178     auto segment = [](R angle)
179     {
180         if ( angle < 1e-3L )
181         {
182             R z = angle * angle;
183             return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
184         }
185         return angle - sinl(2 * angle) / 2;
186     };
187     return a.r * a.r * segment(x) + b.r * b.r * segment(y);
188 }
189 };

```

第一百五十七章 优化与持久化

157.1 Max-plus 矩阵幂

算法约束见动态版本第 165 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Max_Plus_Matrix
2 {
3     using I = __int128_t;
4     static constexpr I neg = -(I(1) << 120);
5     int n;
6     vector<vector<I>> a;
7
8     Max_Plus_Matrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
9     {
10         if ( identity )
11             for ( int i = 0; i < n; i++ )
12                 a[i][i] = 0;
13     }
14
15     Max_Plus_Matrix operator*(const Max_Plus_Matrix &b) const
16     {
17         assert(n == b.n);
18         Max_Plus_Matrix c(n);
19         for ( int i = 0; i < n; i++ )
20             for ( int k = 0; k < n; k++ )
21                 if ( a[i][k] != neg )
22                     for ( int j = 0; j < n; j++ )
23                         if ( b.a[k][j] != neg )
24                             c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
25         return c;
26     }
27
28     Max_Plus_Matrix Power(unsigned long long e) const
29     {
30         Max_Plus_Matrix b = *this, r(n, true);
31         for ( ; e >>= 1, b = b * b )
32             if ( e & 1 )
33                 r = r * b;
34         return r;
35     }
36 };
```

157.2 离散李超树与最优直线编号

算法约束见动态版本第 165 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Li_Chao_Tree
2 {
3     using ll = long long;
4     using I = __int128_t;
5 }
```

```

6   struct Line
7   {
8       ll k = 0, b = 0;
9       int id = -1;
10
11       I value(ll x) const
12       {
13           return I(k) * x + b;
14       }
15   };
16
17   struct Node
18   {
19       Line line;
20       int left = -1, right = -1;
21   };
22
23   vector<ll> xs;
24   vector<Node> a{Node{}};
25
26   Li_Chao_Tree(vector<ll> x) : xs(x)
27   {
28       sort(xs.begin(), xs.end());
29       xs.erase(unique(xs.begin(), xs.end()), xs.end());
30       assert(!xs.empty());
31   }
32
33   bool better(Line u, Line v, ll x) const
34   {
35       return v.id == -1 ||
36              (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
37   }
38
39   void Insert(Line line)
40   {
41       assert(line.id >= 0);
42       Insert(0, 0, (int)xs.size() - 1, line);
43   }
44
45   void Insert(int p, int l, int r, Line line)
46   {
47       int m = (l + r) / 2;
48       if ( better(line, a[p].line, xs[m]) )
49           swap(line, a[p].line);
50       if ( l == r || line.id == -1 )
51           return;
52       bool left = better(line, a[p].line, xs[l]);
53       bool right = better(line, a[p].line, xs[r]);
54       if ( !left && !right )
55           return;
56       int next = left ? a[p].left : a[p].right;
57       if ( next == -1 )
58       {
59           next = (int)a.size();
60           a.push_back(Node{});
61           if ( left )

```



```

62         a[p].left = next;
63     else
64         a[p].right = next;
65 }
66 if ( left )
67     Insert(next, l, m, line);
68 else
69     Insert(next, m + 1, r, line);
70 }
71
72 pair<I, int> Query(ll x) const
73 {
74     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
75     assert(idx < (int)xs.size() && xs[idx] == x);
76     int p = 0, l = 0, r = (int)xs.size() - 1;
77     Line best;
78     while ( p != -1 )
79     {
80         if ( better(a[p].line, best, x) )
81             best = a[p].line;
82         if ( l == r )
83             break;
84         int m = (l + r) / 2;
85         if ( idx <= m )
86         {
87             p = a[p].left;
88             r = m;
89         }
90         else
91         {
92             p = a[p].right;
93             l = m + 1;
94         }
95     }
96     assert(best.id != -1);
97     return {best.value(x), best.id};
98 }
99 };

```

157.3 主席树：静态区间第 k 小

算法约束见动态版本第 167 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Persistent_Kth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     vector<Node> t{Node{}};
9     vector<int> root{0};
10    vector<long long> vals;
11
12    Persistent_Kth(const vector<long long> &a) : vals(a)

```

```

13 {
14     sort(vals.begin(), vals.end());
15     vals.erase(unique(vals.begin(), vals.end()), vals.end());
16     for ( long long x : a )
17     {
18         int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
19         root.push_back(Insert(root.back(), 0, (int)vals.size() - 1, p));
20     }
21 }
22
23 int Insert(int old, int l, int r, int x)
24 {
25     int p = t.size();
26     Node copy = t[old];
27     t.push_back(copy);
28     ++t[p].sum;
29     if ( l < r )
30     {
31         int m = (l + r) / 2;
32         if ( x <= m )
33         {
34             int v = Insert(t[old].l, l, m, x);
35             t[p].l = v;
36         }
37         else
38         {
39             int v = Insert(t[old].r, m + 1, r, x);
40             t[p].r = v;
41         }
42     }
43     return p;
44 }
45
46 long long Kth(int l, int r, int k) const
47 {
48     assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49     int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50     while ( L < R )
51     {
52         int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53         if ( k <= c )
54         {
55             u = t[u].l;
56             v = t[v].l;
57             R = m;
58         }
59         else

```

```
60         {
61             k -= c;
62             u = t[u].r;
63             v = t[v].r;
64             L = m + 1;
65         }
66     }
67     return vals[L];
68 }
69 };
```

第一百五十八章 可持久化数组

158.1 可持久化数组：历史版本分叉与单点修改

算法约束见动态版本第 170 页。本节代码独立可抄；采用下列实际接口名。

声明 Persistent_Array<NODES>, Init(a) 初始化；NODES 限制节点总数，不是数组长度。建议按 $2n-1$ 加上修改数乘以 $(\text{ceil}(\log_2(n))+1)$ 分配；Copy 不分配节点。Set、Copy、Query 对应动态接口。大对象放在全局或 static，Init 会丢弃全部旧版本。

```
1 template <int NODES> struct Persistent_Array
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0;
8         ll value = 0;
9     };
10
11     int n;
12     Node t[NODES + 1];
13     int tot = 0;
14     vector<int> root;
15
16     void Init(const vector<ll> &a)
17     {
18         n = a.size();
19         assert(n > 0);
20         tot = 0;
21         root.clear();
22         root.push_back(Build(1, n, a));
23     }
24
25     int Build(int l, int r, const vector<ll> &a)
26     {
27         assert(tot < NODES);
28         int p = ++tot;
29         t[p] = Node{};
30         if ( l == r )
31             t[p].value = a[l - 1];
32         else
33         {
34             int m = (l + r) / 2;
35             int left = Build(l, m, a);
36             int right = Build(m + 1, r, a);
37             t[p].l = left;
38             t[p].r = right;
39         }
40         return p;
41     }
42
43     int Modify(int old, int l, int r, int pos, ll value)
```

```

44     {
45         assert(tot < NODES);
46         int p = ++tot;
47         t[p] = t[old];
48         if ( l == r )
49             t[p].value = value;
50         else
51         {
52             int m = (l + r) / 2;
53             if ( pos <= m )
54             {
55                 int child = Modify(t[old].l, l, m, pos, value);
56                 t[p].l = child;
57             }
58             else
59             {
60                 int child = Modify(t[old].r, m + 1, r, pos, value);
61                 t[p].r = child;
62             }
63         }
64         return p;
65     }
66
67     // Create a new version from any existing version. Positions are 1-based.
68     int Set(int version, int pos, ll value)
69     {
70         assert(0 <= version && version < (int)root.size());
71         assert(1 <= pos && pos <= n);
72         int p = Modify(root[version], 1, n, pos, value);
73         root.push_back(p);
74         return (int)root.size() - 1;
75     }
76
77     int Copy(int version)
78     {
79         assert(0 <= version && version < (int)root.size());
80         root.push_back(root[version]);
81         return (int)root.size() - 1;
82     }
83
84     ll Query(int version, int pos) const
85     {
86         assert(0 <= version && version < (int)root.size());
87         assert(1 <= pos && pos <= n);
88         int p = root[version], l = 1, r = n;
89         while ( l < r )
90         {
91             int m = (l + r) / 2;
92             if ( pos <= m )
93             {
94                 p = t[p].l;
95                 r = m;

```

```
96         }
97         else
98         {
99             p = t[p].r;
100             l = m + 1;
101         }
102     }
103     return t[p].value;
104 }
105 };
```

第一百五十九章 可持久化区间操作

159.1 可持久化区间加与版本拼接

算法约束见动态版本第 173 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Persistent_Range
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int left = 0, right = 0;
8         ll sum = 0, lazy = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root{0};
14
15     explicit Persistent_Range(int size) : n(size)
16     {
17         assert(n > 0);
18     }
19
20     int Clone(int p)
21     {
22         Node node = t[p];
23         t.push_back(node);
24         return int(t.size()) - 1;
25     }
26
27     void Pushup(int p, int len)
28     {
29         t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
30     }
```

区间加与版本拼接（接上页同一结构体）：

```

32  int Change(int p, int l, int r, int ql, int qr, ll value)
33  {
34      if ( qr <= l || r <= ql )
35          return p;
36      int u = Clone(p);
37      if ( ql <= l && r <= qr )
38      {
39          t[u].lazy += value;
40          t[u].sum += value * (r - l);
41          return u;
42      }
43      int m = l + (r - l) / 2;
44      int a = Change(t[p].left, l, m, ql, qr, value);
45      int b = Change(t[p].right, m, r, ql, qr, value);
46      t[u].left = a;
47      t[u].right = b;
48      Pushup(u, r - l);
49      return u;
50  }
51
52  int Replace(int p, int q, int l, int r, int ql, int qr, ll offset)
53  {
54      if ( qr <= l || r <= ql )
55          return p;
56      if ( ql <= l && r <= qr )
57      {
58          if ( offset == 0 )
59              return q;
60          int u = Clone(q);
61          t[u].lazy += offset;
62          t[u].sum += offset * (r - l);
63          return u;
64      }
65      int u = Clone(p), m = l + (r - l) / 2;
66      offset += t[q].lazy - t[p].lazy;
67      int a = Replace(t[p].left, t[q].left, l, m, ql, qr, offset);
68      int b = Replace(t[p].right, t[q].right, m, r, ql, qr, offset);
69      t[u].left = a;
70      t[u].right = b;
71      Pushup(u, r - l);
72      return u;
73  }

```


只读查询与对外接口（接上页同一结构体）：

```

75     ll Get(int p, int l, int r, int ql, int qr, ll carry) const
76     {
77         if ( qr <= l || r <= ql )
78             return 0;
79         if ( ql <= l && r <= qr )
80             return t[p].sum + carry * (r - l);
81         carry += t[p].lazy;
82         int m = l + (r - l) / 2;
83         return Get(t[p].left, l, m, ql, qr, carry) +
84             Get(t[p].right, m, r, ql, qr, carry);
85     }
86
87     void Check(int version, int l, int r) const
88     {
89         assert(0 <= version && version < int(root.size()));
90         assert(0 <= l && l <= r && r <= n);
91     }
92
93     int Add(int version, int l, int r, ll value)
94     {
95         Check(version, l, r);
96         int p = l == r || value == 0 ? root[version]
97             : Change(root[version], 0, n, l, r, value);
98         root.push_back(p);
99         return int(root.size()) - 1;
100     }
101
102     int Splice(int destination, int source, int l, int r)
103     {
104         Check(destination, l, r);
105         Check(source, l, r);
106         int p = l == r ? root[destination]
107             : Replace(root[destination], root[source], 0, n, l, r, 0);
108         root.push_back(p);
109         return int(root.size()) - 1;
110     }
111
112     ll Query(int version, int l, int r) const
113     {
114         Check(version, l, r);
115         return l == r ? 0 : Get(root[version], 0, n, l, r, 0);
116     }
117 };

```

第一百六十章 树上路径第 k 小

160.1 树上路径点权第 k 小

算法约束见动态版本第 176 页。本节代码独立可抄；采用下列实际接口名。

声明 `Tree_Path_Kth<N,NODES,LOG>`, `Init(a)` 清空图并保存权值; `Insert`、`Build`、`Kth`、`Lca` 对应动态接口。

要求 2 的 LOG 次方大于实际点数，默认 LOG=20。若不同权值数为 D，NODES 至少取 n 乘以 $(\text{ceil}(\log_2(D))+1)$ ，另有编号 0 空节点。大对象放在全局或 static；Init 与 Build 分别清空图和重建版本。

```
1 template <int N, int NODES, int LOG = 20> struct Tree_Path_Kth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    static_assert(1 <= LOG && LOG <= 31);
11    int n, tot;
12    vector<int> g[N + 1];
13    int up[LOG][N + 1], root[N + 1], dep[N + 1];
14    ll value[N + 1];
15    vector<ll> vals;
16    Node t[NODES + 1];
17
18    void Init(const vector<ll> &a)
19    {
20        n = a.size();
21        assert(0 < n && n <= N && (1LL << LOG) > n);
22        for (int u = 1; u <= n; u++)
23        {
24            value[u] = a[u - 1];
25            g[u].clear();
26        }
27        root[0] = 0;
28        for (int j = 0; j < LOG; j++)
29            up[j][0] = 0;
30    }
31
32    void Insert(int u, int v)
33    {
34        g[u].push_back(v);
35        g[v].push_back(u);
36    }
37
38    int Update(int old, int l, int r, int x)
39    {
40        assert(tot < NODES);
41        int p = ++tot;
42        t[p] = t[old];
43        t[p].sum++;
```

```

44     if ( l == r )
45         return p;
46     int m = (l + r) / 2;
47     if ( x <= m )
48     {
49         int child = Update(t[old].l, l, m, x);
50         t[p].l = child;
51     }
52     else
53     {
54         int child = Update(t[old].r, m + 1, r, x);
55         t[p].r = child;
56     }
57     return p;
58 }
59
60 void Dfs(int u, int p)
61 {
62     up[0][u] = p;
63     for ( int j = 1; j < LOG; j++ )
64         up[j][u] = up[j - 1][up[j - 1][u]];
65     int x = lower_bound(vals.begin(), vals.end(), value[u]) - vals.begin();
66     root[u] = Update(root[p], 0, (int)vals.size() - 1, x);
67     for ( int v : g[u] )
68     {
69         if ( v == p )
70             continue;
71         dep[v] = dep[u] + 1;
72         Dfs(v, u);
73     }
74 }
75
76 void Build(int r = 1)
77 {
78     assert(1 <= r && r <= n);
79     vals.assign(value + 1, value + n + 1);
80     sort(vals.begin(), vals.end());
81     vals.erase(unique(vals.begin(), vals.end()), vals.end());
82     tot = 0;
83     t[0] = Node{};
84     dep[r] = 0;
85     Dfs(r, 0);
86 }
87
88 int Lca(int u, int v) const
89 {
90     if ( dep[u] < dep[v] )
91         swap(u, v);
92     int diff = dep[u] - dep[v];
93     for ( int j = 0; j < LOG; j++ )
94         if ( diff >> j & 1 )
95             u = up[j][u];
96     if ( u == v )
97         return u;
98     for ( int j = LOG - 1; j >= 0; j-- )
99         if ( up[j][u] != up[j][v] )

```

```

100         {
101             u = up[j][u];
102             v = up[j][v];
103         }
104         return up[0][u];
105     }
106
107 ll Kth(int u, int v, int k) const
108 {
109     int z = Lca(u, v);
110     assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
111     int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];
112     int l = 0, r = (int)vals.size() - 1;
113     while ( l < r )
114     {
115         int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
116         int m = (l + r) / 2;
117         if ( k <= count )
118         {
119             a = t[a].l;
120             b = t[b].l;
121             c = t[c].l;
122             d = t[d].l;
123             r = m;
124         }
125         else
126         {
127             k -= count;
128             a = t[a].r;
129             b = t[b].r;
130             c = t[c].r;
131             d = t[d].r;
132             l = m + 1;
133         }
134     }
135     return vals[l];
136 }
137 };

```

第一百六十一章 动态区间顺序统计

161.1 动态区间第 k 小 (树状数组套权值线段树)

算法约束见动态版本第 179 页。本节代码独立可抄；采用下列实际接口名。

声明 `Dynamic_Kth<N,NODES>` 后 `Init(a,future)`, `Set` 与 `Kth` 为公开操作。`N` 限制数组长度, `NODES` 限制分配节点的最大编号, 另有编号 0 空节点。按照上面的 `B` 分配一定足够; 例如 `n` 不超过 100000 且 `D` 不超过 200000 时, `B` 不超过 16682000。节点回收使此界与修改次数无关。大对象放在全局或 `static`, `Init` 清空旧状态。

```
1 template <int N, int NODES> struct Dynamic_Kth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, tot, free_head;
11    int bit[N + 1], rank[N + 1];
12    vector<ll> vals;
13    Node t[NODES + 1];
14
15    void Init(const vector<ll> &a, const vector<ll> &future = {})
16    {
17        n = a.size();
18        assert(0 < n && n <= N);
19        tot = 0;
20        free_head = 0;
21        t[0] = Node{};
22        fill(bit, bit + n + 1, 0);
23        vals = future;
24        vals.insert(vals.end(), a.begin(), a.end());
25        sort(vals.begin(), vals.end());
26        vals.erase(unique(vals.begin(), vals.end()), vals.end());
27        for (int i = 1; i <= n; i++)
28        {
29            rank[i] = Id(a[i - 1]);
30            Modify(i, rank[i], 1);
31        }
32    }
33
34    int Id(ll value) const
35    {
36        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
37        assert(p < (int)vals.size() && vals[p] == value);
38        return p;
39    }
40
41    int New_Node()
42    {
43        if (!free_head)
```

```

44     {
45         assert(tot < NODES);
46         int p = ++tot;
47         t[p] = Node{};
48         return p;
49     }
50     int p = free_head;
51     free_head = t[p].l;
52     t[p] = Node{};
53     return p;
54 }
55
56 int Update(int p, int l, int r, int x, int delta)
57 {
58     if ( !p )
59         p = New_Node();
60     t[p].sum += delta;
61     if ( l < r )
62     {
63         int m = (l + r) / 2;
64         if ( x <= m )
65         {
66             int child = Update(t[p].l, l, m, x, delta);
67             t[p].l = child;
68         }
69         else
70         {
71             int child = Update(t[p].r, m + 1, r, x, delta);
72             t[p].r = child;
73         }
74     }
75     if ( !t[p].sum )
76     {
77         t[p].l = free_head;
78         free_head = p;
79         return 0;
80     }
81     return p;
82 }
83
84 void Modify(int pos, int x, int delta)
85 {
86     for ( int i = pos; i <= n; i += i & -i )
87         bit[i] = Update(bit[i], 0, (int)vals.size() - 1, x, delta);
88 }

```

```

90 void Set(int pos, ll value)
91 {
92     assert(1 <= pos && pos <= n);
93     int x = Id(value);
94     if ( rank[pos] == x )
95         return;
96     Modify(pos, rank[pos], -1);
97     Modify(pos, x, 1);
98     rank[pos] = x;
99 }
100
101 ll Kth(int l, int r, int k) const
102 {
103     assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
104     vector<int> a, b;
105     for ( int i = r; i; i -= i & -i )
106         a.push_back(bit[i]);
107     for ( int i = l - 1; i; i -= i & -i )
108         b.push_back(bit[i]);
109     int L = 0, R = (int)vals.size() - 1;
110     while ( L < R )
111     {
112         int count = 0;
113         for ( int p : a )
114             count += t[t[p].l].sum;
115         for ( int p : b )
116             count -= t[t[p].l].sum;
117         bool left = k <= count;
118         if ( !left )
119             k -= count;
120         for ( int &p : a )
121             p = left ? t[p].l : t[p].r;
122         for ( int &p : b )
123             p = left ? t[p].l : t[p].r;
124         int m = (L + R) / 2;
125         if ( left )
126             R = m;
127         else
128             L = m + 1;
129     }
130     return vals[L];
131 }
132 };

```

第一百六十二章 区间不同数统计

162.1 静态区间不同数计数（前驱位置主席树）

算法约束见动态版本第 182 页。本节代码独立可抄；采用下列实际接口名。

声明 Persistent_Distinct<N,NODES> 后 Init(a), Query(l,r) 为查询。N 限制数组长度；NODES 至少取 $n \times \lceil \log_2(n) \rceil + 1$ ，另有编号 0 空节点。n 不超过 1000000 时可取 NODES=21000000。大对象放在全局或 static，Init 可重复调用并清空全部旧版本。

```
1 template <int N, int NODES> struct Persistent_Distinct
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     int n, tot;
9     Node t[NODES + 1];
10    int root[N + 1];
11
12    void Init(const vector<long long> &a)
13    {
14        n = a.size();
15        assert(0 < n && n <= N);
16        tot = 0;
17        t[0] = Node{};
18        root[0] = 0;
19        vector<long long> vals = a;
20        sort(vals.begin(), vals.end());
21        vals.erase(unique(vals.begin(), vals.end()), vals.end());
22        vector<int> last(vals.size());
23        for ( int i = 1; i <= n; i++ )
24        {
25            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
26            root[i] = Insert(root[i - 1], 0, n - 1, last[x]);
27            last[x] = i;
28        }
29    }
30
31    int Insert(int old, int l, int r, int x)
32    {
33        assert(tot < NODES);
34        int p = ++tot;
35        t[p] = t[old];
36        t[p].sum++;
37        if ( l < r )
38        {
39            int m = (l + r) / 2;
40            if ( x <= m )
41            {
42                int child = Insert(t[old].l, l, m, x);
43                t[p].l = child;
```



```

44         }
45     else
46     {
47         int child = Insert(t[old].r, m + 1, r, x);
48         t[p].r = child;
49     }
50 }
51 return p;
52 }
53
54 int Query(int l, int r) const
55 {
56     assert(1 <= l && l <= r && r <= n);
57     int u = root[l - 1], v = root[r];
58     int L = 0, R = n - 1, answer = 0;
59     while ( L < R )
60     {
61         int m = (L + R) / 2;
62         if ( l - 1 <= m )
63         {
64             u = t[u].l;
65             v = t[v].l;
66             R = m;
67         }
68         else
69         {
70             answer += t[t[v].l].sum - t[t[u].l].sum;
71             u = t[u].r;
72             v = t[v].r;
73             L = m + 1;
74         }
75     }
76     return answer + t[v].sum - t[u].sum;
77 }
78 };

```

第一百六十三章 图论进阶

163.1 矩形二分图最小权匹配

算法约束见动态版本第 184 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Kuhn_Munkres
2 {
3     using I = __int128_t;
4
5     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
6     static pair<I, vector<int>> Solve(const vector<vector<long long>> &cost)
7     {
8         int n = cost.size();
9         if ( !n )
10             return {0, {}};
11         int m = cost[0].size();
12         assert(n <= m);
13         vector<I> u(n + 1), v(m + 1);
14         vector<int> p(m + 1), way(m + 1);
15         const I inf = I(1) << 120;
16         for ( int i = 1; i <= n; i++ )
17         {
18             assert((int)cost[i - 1].size() == m);
19             p[0] = i;
20             int j0 = 0;
21             vector<I> dis(m + 1, inf);
22             vector<bool> Used(m + 1);
23             do
24             {
25                 Used[j0] = true;
26                 int i0 = p[j0], j1 = 0;
27                 I delta = inf;
28                 for ( int j = 1; j <= m; j++ )
29                     if ( !Used[j] )
30                     {
31                         I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
32                         if ( cur < dis[j] )
33                         {
34                             dis[j] = cur;
35                             way[j] = j0;
36                         }
37                         if ( dis[j] < delta )
38                         {
39                             delta = dis[j];
40                             j1 = j;
41                         }
42                     }
43                 for ( int j = 0; j <= m; j++ )
44                     if ( Used[j] )
45                     {
46                         u[p[j]] += delta;
```

```
47         v[j] -= delta;
48     }
49     else
50         dis[j] -= delta;
51     j0 = j1;
52 } while ( p[j0] );
53 do
54 {
55     int j1 = way[j0];
56     p[j0] = p[j1];
57     j0 = j1;
58 } while ( j0 );
59 }
60 vector<int> ans(n);
61 for ( int j = 1; j <= m; j++ )
62     if ( p[j] )
63         ans[p[j] - 1] = j - 1;
64 return {-v[0], ans};
65 }
66 };
```

163.2 有向最小树形图

算法约束见动态版本第 186 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Directed_MST
2 {
3     using I = __int128_t;
4
5     struct Edge
6     {
7         int u, v;
8         I w;
9     };
10
11     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
12     static optional<I> Solve(int n, int root, vector<Edge> e)
13     {
14         const I inf = I(1) << 120;
15         I ans = 0;
16         while ( true )
17         {
18             vector<I> in(n, inf);
19             vector<int> pre(n, -1), id(n, -1), vis(n, -1);
20             for ( auto a : e )
21                 if ( a.u != a.v && a.w < in[a.v] )
22                 {
23                     in[a.v] = a.w;
24                     pre[a.v] = a.u;
25                 }
26             in[root] = 0;
27             for ( I x : in )
28             {
29                 if ( x == inf )
30                     return nullopt;
31                 ans += x;
32             }

```

判环与缩点（接上页同一函数）：

```
33     int cnt = 0;
34     for ( int i = 0; i < n; i++ )
35     {
36         int u = i;
37         while ( vis[u] != i && id[u] == -1 && u != root )
38         {
39             vis[u] = i;
40             u = pre[u];
41         }
42         if ( u != root && id[u] == -1 )
43         {
44             id[u] = cnt;
45             for ( int v = pre[u]; v != u; v = pre[v] )
46                 id[v] = cnt;
47             ++cnt;
48         }
49     }
50     if ( !cnt )
51         return ans;
52     for ( int i = 0; i < n; i++ )
53         if ( id[i] == -1 )
54             id[i] = cnt++;
55     for ( auto &a : e )
56     {
57         int v = a.v;
58         a.u = id[a.u];
59         a.v = id[a.v];
60         if ( a.u != a.v )
61             a.w -= in[v];
62     }
63     root = id[root];
64     n = cnt;
65 }
66 }
67 };
```

163.3 无向全局最小割

算法约束见动态版本第 188 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Global_Min_Cut
2 {
3     using I = __int128_t;
4
5     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
6     static pair<I, vector<int>> Solve(vector<vector<I>> w)
7     {
8         int n = w.size();
9         assert(n >= 2);
10        vector<int> active(n);
11        iota(active.begin(), active.end(), 0);
12        vector<vector<int>> group(n);
13        for ( int i = 0; i < n; i++ )
14            group[i] = {i};
15        I best = I(1) << 120;
16        vector<int> side;
17        while ( active.size() > 1 )
18        {
19            vector<I> dis(n);
20            vector<bool> Used(n);
21            int prev = -1;
22            for ( int step = 0; step < (int)active.size(); step++ )
23            {
24                int sel = -1;
25                for ( int v : active )
26                    if ( !Used[v] && (sel == -1 || dis[v] > dis[sel]) )
27                        sel = v;
```

本轮割、合并与继续扩张（接上页同一函数）：

```
28         if ( step + 1 == (int)active.size() )
29         {
30             if ( dis[sel] < best )
31             {
32                 best = dis[sel];
33                 side = group[sel];
34             }
35             for ( int v : active )
36                 if ( v != prev && v != sel )
37                     w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
38             group[prev].insert(
39                 group[prev].end(), group[sel].begin(), group[sel].end());
40             active.erase(find(active.begin(), active.end(), sel));
41             break;
42         }
43         Used[sel] = true;
44         prev = sel;
45         for ( int v : active )
46             if ( !Used[v] )
47                 dis[v] += w[sel][v];
48     }
49 }
50 return {best, side};
51 }
52 };
```

第一百六十四章 最小割树

164.1 Gomory-Hu 最小割树

算法约束见动态版本第 190 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int N, int M>
2 vector<tuple<int, int, long long>> Gomory_Hu(
3     int n, const vector<tuple<int, int, long long>> &edges, Network_Flow<N, M> &flow)
4 {
5     assert(n >= 1);
6     vector<int> parent(n);
7     vector<long long> value(n);
8     for ( int s = 1; s < n; s++ )
9     {
10         int t = parent[s];
11         flow.Init(n);
12         for ( auto [u, v, w] : edges )
13             if ( u != v )
14             {
15                 flow.Add_Edge(u + 1, v + 1, w);
16                 flow.Add_Edge(v + 1, u + 1, w);
17             }
18         long long cut = flow.Max_Flow(s + 1, t + 1);
19         vector<bool> side(n);
20         for ( int u : flow.Cut(s + 1) )
21             side[u - 1] = true;
22         for ( int v = 1; v < n; v++ )
23             if ( v != s && parent[v] == t && side[v] )
24                 parent[v] = s;
25         if ( side[parent[t]] )
26         {
27             parent[s] = parent[t];
28             parent[t] = s;
29             value[s] = value[t];
30             value[t] = cut;
31         }
32         else
33             value[s] = cut;
34     }
35     vector<tuple<int, int, long long>> tree;
36     for ( int v = 1; v < n; v++ )
37         tree.push_back({v, parent[v], value[v]});
38     return tree;
39 }
```


第一百六十五章 最小割树查询

165.1 最小割树全点对查询

算法约束见动态版本第 191 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<vector<long long>>
2 Cut_Tree_Values(int n, const vector<tuple<int, int, long long>> &tree)
3 {
4     assert(n >= 1 && tree.size() == (size_t)n - 1);
5     vector<vector<pair<int, long long>>> g(n);
6     for ( auto [u, v, w] : tree )
7     {
8         assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
9         g[u].push_back({v, w});
10        g[v].push_back({u, w});
11    }
12    vector<vector<long long>> answer(n, vector<long long>(n, LLONG_MAX));
13    for ( int s = 0; s < n; s++ )
14    {
15        auto dfs = [&](auto &&self, int u, int parent, long long value) -> void
16        {
17            answer[s][u] = value;
18            for ( auto [v, w] : g[u] )
19                if ( v != parent )
20                    self(self, v, u, min(value, w));
21        };
22        dfs(dfs, s, -1, LLONG_MAX);
23    }
24    return answer;
25 }
```

第一百六十六章 带权二分图匹配

166.1 二分图最大权匹配（缺边与可选匹配）

算法约束见动态版本第 192 页。本节代码独立可抄；采用下列实际接口名。

声明 `Weighted_Matching<N,M>` 后 `Init(n,m)`, `N`、`M` 分别限制左右点数。`Insert` 加边, `Solve` 求解; `l`、`r` 保存方案, `Init` 清空原图。权值矩阵是静态数组, 内部增广辅助数组按实际规模分配, 大对象放在全局或 `static`。

```
1 template <int N, int M> struct Weighted_Matching
2 {
3     using I = __int128_t;
4     static constexpr I inf = I(1) << 120;
5     int n, m;
6     I w[N + 1][M + 1];
7     int l[N + 1], r[M + 1];
8
9     void Init(int left_size, int right_size)
10    {
11        assert(0 <= left_size && left_size <= N);
12        assert(0 <= right_size && right_size <= M);
13        n = left_size;
14        m = right_size;
15        for ( int i = 0; i <= n; i++ )
16            fill(w[i], w[i] + m + 1, -inf);
17        fill(l, l + n + 1, 0);
18        fill(r, r + m + 1, 0);
19    }
20
21    void Insert(int x, int y, long long weight)
22    {
23        assert(1 <= x && x <= n && 1 <= y && y <= m);
24        w[x][y] = max(w[x][y], I(weight));
25    }
26
27    optional<I> Solve(bool allow_unmatched = false)
28    {
29        fill(l, l + n + 1, 0);
30        fill(r, r + m + 1, 0);
31        if ( !allow_unmatched && n > m )
32            return nullopt;
33        int cols = m + (allow_unmatched ? n : 0);
34        vector<I> u(n + 1), v(cols + 1);
35        vector<int> p(cols + 1), way(cols + 1);
36        for ( int i = 1; i <= n; i++ )
37        {
38            p[0] = i;
39            int j0 = 0;
40            vector<I> dis(cols + 1, inf);
41            vector<bool> used(cols + 1);
42            do
43            {
44                used[j0] = true;
```

```

45     int x = p[j0], j1 = 0;
46     I delta = inf;
47     for ( int j = 1; j <= cols; j++ )
48     {
49         if ( used[j] )
50             continue;
51         I weight = j <= m ? w[x][j] : 0;
52         if ( weight != -inf )
53         {
54             I cur = -weight - u[x] - v[j];
55             if ( cur < dis[j] )
56             {
57                 dis[j] = cur;
58                 way[j] = j0;
59             }
60         }
61         if ( dis[j] < delta )
62         {
63             delta = dis[j];
64             j1 = j;
65         }
66     }
67     if ( delta == inf )
68         return nullopt;
69     for ( int j = 0; j <= cols; j++ )
70     {
71         if ( used[j] )
72         {
73             u[p[j]] += delta;
74             v[j] -= delta;
75         }
76         else if ( dis[j] != inf )
77             dis[j] -= delta;
78     }
79     j0 = j1;
80 } while ( p[j0] );
81 do
82 {
83     int j1 = way[j0];
84     p[j0] = p[j1];
85     j0 = j1;
86 } while ( j0 );
87 }
88 I answer = 0;
89 for ( int j = 1; j <= m; j++ )
90 {
91     if ( !p[j] )
92         continue;
93     l[p[j]] = j;
94     r[j] = p[j];
95     answer += w[p[j]][j];
96 }
97 return answer;
98 }
99 };

```

第一百六十七章 精确有界半平面交

167.1 整数半平面的有界多边形交

算法约束见动态版本第 194 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Integer_Halfplanes
2 {
3     using I = __int128_t;
4     using ll = long long;
5
6     struct Line
7     {
8         ll a, b, c;
9     };
10
11     struct Point
12     {
13         ll x, y, d;
14     };
15
16     static I Cross(Line u, Line v)
17     {
18         return I(u.a) * v.b - I(u.b) * v.a;
19     }
20
21     static int Half(Line u)
22     {
23         return u.b < 0 || (u.b == 0 && u.a < 0);
24     }
25
26     static Point Meet(Line u, Line v)
27     {
28         I d = Cross(u, v);
29         assert(d != 0);
30         I x = I(u.c) * v.b - I(u.b) * v.c;
31         I y = I(u.a) * v.c - I(u.c) * v.a;
32         if ( d < 0 )
33         {
34             d = -d;
35             x = -x;
36             y = -y;
37         }
38         return {(ll)x, (ll)y, (ll)d};
39     }
```

接上页同一结构体与函数：

```

41     static I Value(Line u, Point p)
42     {
43         return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
44     }
45
46     static bool Same(Point p, Point q)
47     {
48         return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
49     }
50
51     static vector<Point> Bounded_Polygon(vector<Line> lines)
52     {
53         vector<Line> h;
54         for ( auto u : lines )
55         {
56             assert(-10000000000 <= u.a && u.a <= 10000000000);
57             assert(-10000000000 <= u.b && u.b <= 10000000000);
58             assert(-10000000000 <= u.c && u.c <= 10000000000);
59             if ( !u.a && !u.b )
60             {
61                 if ( u.c < 0 )
62                     return {};
63             }
64             else
65                 h.push_back(u);
66         }
67         sort(h.begin(),
68             h.end(),
69             [](Line u, Line v)
70             {
71                 if ( Half(u) != Half(v) )
72                     return Half(u) < Half(v);
73                 I d = Cross(u, v);
74                 if ( d )
75                     return d > 0;
76                 ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
77                 ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
78                 return I(u.c) * y < I(v.c) * x;
79             });

```

接上页同一结构体与函数：

```

80     deque<Line> q;
81     for ( int i = 0; i < (int)h.size(); i++ )
82     {
83         Line u = h[i];
84         if ( i && Half(u) == Half(h[i - 1]) && Cross(u, h[i - 1]) == 0 )
85             continue;
86         while ( q.size() >= 2 && Value(u, Meet(q[q.size() - 2], q.back())) > 0 )
87             q.pop_back();
88         while ( q.size() >= 2 && Value(u, Meet(q[0], q[1])) > 0 )
89             q.pop_front();
90         if ( !q.empty() && Cross(q.back(), u) == 0 )
91             return {};
92         q.push_back(u);
93     }
94     while ( q.size() >= 3 && Value(q.front(), Meet(q[q.size() - 2], q.back())) > 0 )
95         q.pop_back();
96     while ( q.size() >= 3 && Value(q.back(), Meet(q[0], q[1])) > 0 )
97         q.pop_front();
98     if ( q.size() < 3 || Cross(q.front(), q.back()) == 0 )
99         return {};
100     vector<Point> answer;
101     for ( int i = 0; i < (int)q.size(); i++ )
102     {
103         Point p = Meet(q[i], q[(i + 1) % q.size()]);
104         if ( answer.empty() || !Same(answer.back(), p) )
105             answer.push_back(p);
106     }
107     if ( answer.size() > 1 && Same(answer.front(), answer.back()) )
108         answer.pop_back();
109     if ( answer.size() < 3 )
110         return {};
111     return answer;
112 }
113 };

```

第一百六十八章 圆与多边形面积

168.1 圆与简单多边形相交面积

算法约束见动态版本第 197 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Circle_Polygon
2 {
3     using G = Real_Geometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     // Signed contribution of edge a -> b; circle centered at the origin.
9     static R Edge(Point a, Point b, R r)
10    {
11        if ( r == 0 )
12            return 0;
13        Point v = b - a;
14        R length = G::norm(v);
15        if ( length == 0 )
16            return 0;
17        Point u = v / length;
18        R h = G::Cross(u, a);
19        Point cuts[4] = {a};
20        int count = 1;
21        if ( fabs1(h) <= r )
22        {
23            R delta = sqrt1(max(R(0), (r - fabs1(h)) * (r + fabs1(h))));
24            R t = -G::Dot(a, u);
25            Point foot = G::perp(u) * h;
26            for ( R s : {-delta, delta} )
27                if ( t + s > 0 && t + s < length )
28                    cuts[count++] = foot + u * s;
29        }
30        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```

31     R answer = 0;
32     for ( int i = 1; i < count; i++ )
33     {
34         Point p = cuts[i - 1], q = cuts[i];
35         R cross = G::Cross(p, q);
36         if ( G::norm((p + q) / 2) <= r )
37             answer += cross / 2;
38         else
39             answer += r * r * atan2l(cross, G::Dot(p, q)) / 2;
40     }
41     return answer;
42 }
43
44 // Simple polygon, either orientation; duplicate consecutive vertices allowed.
45 static R Area(const vector<Point> &p, Circle c)
46 {
47     assert(c.r >= 0);
48     if ( p.size() < 3 || c.r == 0 )
49         return 0;
50     R answer = 0, error = 0;
51     for ( int i = 0; i < (int)p.size(); i++ )
52     {
53         R value = Edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
54         R y = value - error;
55         R sum = answer + y;
56         error = (sum - answer) - y;
57         answer = sum;
58     }
59     return fabs1(answer);
60 }
61 };

```


第一百六十九章 最小覆盖圆

169.1 随机增量最小覆盖圆

算法约束见动态版本第 199 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Enclosing_Circle
2 {
3     using G = Real_Geometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     static Circle Diameter(Point a, Point b)
9     {
10         Point o = a + (b - a) / 2;
11         return {o, max(G::norm(o - a), G::norm(o - b))};
12     }
13
14     static Circle Through(Point a, Point b, Point c)
15     {
16         Point u = b - a, v = c - a;
17         R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
18         if ( scale == 0 )
19             return {a, 0};
20         u = u / scale;
21         v = v / scale;
22         R det = 2 * G::Cross(u, v);
23         if ( det == 0 )
24         {
25             Circle best = Diameter(a, b);
26             for ( Circle q : {Diameter(a, c), Diameter(b, c)} )
27                 if ( q.r > best.r )
28                     best = q;
29             return best;
30         }
31         R x = G::Dot(u, u), y = G::Dot(v, v);
32         Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
33         Point o = a + offset * scale;
34         return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
35     }
```

随机增量主过程（接上页同一结构体）：

```

37     static optional<Circle>
38     Solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
39     {
40         assert(eps >= 0 && eps < 1);
41         if ( p.empty() )
42             return nullopt;
43         shuffle(p.begin(), p.end(), rng);
44         Circle c{p[0], 0};
45         auto outside = [&](Point q)
46         {
47             return G::norm(q - c.o) > c.r * (1 + eps);
48         };
49         for ( int i = 1; i < (int)p.size(); i++ )
50             if ( outside(p[i]) )
51             {
52                 c = {p[i], 0};
53                 for ( int j = 0; j < i; j++ )
54                     if ( outside(p[j]) )
55                     {
56                         c = Diameter(p[i], p[j]);
57                         for ( int k = 0; k < j; k++ )
58                             if ( outside(p[k]) )
59                                 c = Through(p[i], p[j], p[k]);
60                     }
61             }
62         return c;
63     }
64 };

```

第一百七十章 圆的切线

170.1 两圆公切线与过点切线

算法约束见动态版本第 201 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Circle_Tangents
2 {
3     using G = Real_Geometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     struct Tangent
9     {
10         Point a, b, normal;
11         int side; // 1 outer, -1 inner, 0 two distinct point circles.
12     };
13
14     struct Result
15     {
16         bool infinite = false;
17         vector<Tangent> lines;
18     };
19
20     // Line: dot(normal, x - a) = 0. Radius zero represents a point.
21     static Result Solve(Circle a, Circle b, R eps = 1e-12L)
22     {
23         assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
24         Point v = b.o - a.o;
25         R d = G::norm(v);
26         if ( d == 0 )
27             return {a.r == b.r, {}};
28         Point u = v / d;
29         if ( a.r == 0 && b.r == 0 )
30             return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```

31     Result answer;
32     for ( int side : {1, -1} )
33     {
34         if ( side == -1 && (a.r == 0 || b.r == 0) )
35             continue;
36         R gap = a.r - side * b.r;
37         if ( fabsl(gap) > d * (1 + eps) )
38             continue;
39         bool touch = fabsl(gap) >= d * (1 - eps);
40         R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
41         R s = sqrtl(max(R(0), (1 - c) * (1 + c)));
42         for ( int sign : {-1, 1} )
43         {
44             Point normal = u * c + G::perp(u) * (sign * s);
45             Point p = a.o + normal * a.r;
46             Point q = b.o + normal * (side * b.r);
47             answer.lines.push_back({p, q, normal, side});
48             if ( touch )
49                 break;
50         }
51     }
52     return answer;
53 }
54
55 static Result From_Point(Point p, Circle c, R eps = 1e-12L)
56 {
57     return Solve({p, 0}, c, eps);
58 }
59 };

```

第一百七十一章 浮点最近点对

171.1 浮点平面最近点对

算法约束见动态版本第 203 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Closest_Pair
2 {
3     using R = long double;
4
5     struct Point
6     {
7         R x, y;
8     };
9
10    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
11    static optional<R> Solve(vector<Point> p)
12    {
13        int n = p.size();
14        if ( n < 2 )
15            return nullopt;
16        sort(p.begin(),
17             p.end(),
18             [](Point a, Point b)
19             {
20                 return tie(a.x, a.y) < tie(b.x, b.y);
21             });
22        for ( int i = 1; i < n; i++ )
23            if ( p[i].x == p[i - 1].x && p[i].y == p[i - 1].y )
24                return R(0);
25        auto by_y = [](Point a, Point b)
26        {
27            return tie(a.y, a.x) < tie(b.y, b.x);
28        };
29        auto distance = [](Point a, Point b)
30        {
31            return hypotl(a.x - b.x, a.y - b.y);
32        };
33        vector<Point> tmp(n);
34        function<R(int, int)> Dfs = [&](int l, int r) -> R
35        {
36            if ( r - l <= 3 )
37            {
38                R best = numeric_limits<R>::infinity();
39                for ( int i = l; i < r; i++ )
40                    for ( int j = l; j < i; j++ )
41                        best = min(best, distance(p[i], p[j]));
42                sort(p.begin() + l, p.begin() + r, by_y);
43                return best;
44            }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
45     int m = (l + r) / 2;
46     R x = p[m].x;
47     R best = min(Dfs(l, m), Dfs(m, r));
48     merge(p.begin() + l,
49           p.begin() + m,
50           p.begin() + m,
51           p.begin() + r,
52           tmp.begin() + l,
53           by_y);
54     copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
55     vector<Point> strip;
56     for ( int i = l; i < r; i++ )
57         if ( fabs1(p[i].x - x) <= best )
58         {
59             for ( int j = (int)strip.size() - 1; j >= 0; j-- )
60             {
61                 if ( p[i].y - strip[j].y > best )
62                     break;
63                 best = min(best, distance(p[i], strip[j]));
64             }
65             strip.push_back(p[i]);
66         }
67     return best;
68 };
69 return Dfs(0, n);
70 }
71 };
```

第一百七十二章 精确几何进阶

172.1 整数最近点对与 Minkowski 和

算法约束见动态版本第 205 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Geometry_Extra
2 {
3     using G = Integer_Geometry;
4     using P = G::Point;
5     using I = G::I;
6
7     // No pair exists for n<2. Exact squared distance, O(n log n).
8     static optional<I> Closest_Pair(vector<P> p)
9     {
10         int n = p.size();
11         if ( n < 2 )
12             return nullopt;
13         sort(p.begin(), p.end());
14         for ( int i = 1; i < n; i++ )
15             if ( p[i] == p[i - 1] )
16                 return I(0);
17         vector<P> tmp(n);
18         auto by_y = [](P a, P b)
19         {
20             return tie(a.y, a.x) < tie(b.y, b.x);
21         };
22         function<I(int, int)> Solve = [&](int l, int r) -> I
23         {
24             if ( r - l <= 3 )
25             {
26                 I best = I(1) << 120;
27                 for ( int i = l; i < r; i++ )
28                     for ( int j = i + 1; j < r; j++ )
29                         best = min(best, G::Distance_Squared(p[i], p[j]));
30                 sort(p.begin() + l, p.begin() + r, by_y);
31                 return best;
32             }
33             int m = (l + r) / 2;
34             long long x = p[m].x;
35             I best = min(Solve(l, m), Solve(m, r));
36             merge(p.begin() + l,
37                 p.begin() + m,
38                 p.begin() + m,
39                 p.begin() + r,
40                 tmp.begin() + l,
41                 by_y);
42             copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
43             vector<P> strip;
44             for ( int i = l; i < r; i++ )
45                 if ( (I(p[i].x) - x) * (I(p[i].x) - x) < best )
46                     {
```

```

47         for ( int j = (int)strip.size() - 1; j >= 0; j-- )
48         {
49             I dy = I(p[i].y) - strip[j].y;
50             if ( dy * dy >= best )
51                 break;
52             best = min(best, G::Distance_Squared(p[i], strip[j]));
53         }
54         strip.push_back(p[i]);
55     }
56     return best;
57 };
58 return Solve(0, n);
59 }
60
61 // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
62 static vector<P> Minkowski(vector<P> a, vector<P> b)
63 {
64     if ( a.empty() || b.empty() )
65         return {};
66     auto plus = [](P u, P v)
67     {
68         return P{u.x + v.x, u.y + v.y};
69     };
70     if ( a.size() < 3 || b.size() < 3 )
71     {
72         vector<P> p;
73         for ( P u : a )
74             for ( P v : b )
75                 p.push_back(plus(u, v));
76         return G::Convex_Hull(p);
77     }
78     auto start = [](vector<P> &p)
79     {
80         auto it = min_element(p.begin(),
81                               p.end(),
82                               [](P u, P v)
83                               {
84                                   return tie(u.y, u.x) < tie(v.y, v.x);
85                               });
86         rotate(p.begin(), it, p.end());
87     };
88     start(a);
89     start(b);
90     int n = a.size(), m = b.size(), i = 0, j = 0;
91     vector<P> result;
92     while ( i < n || j < m )
93     {
94         result.push_back(plus(a[i % n], b[j % m]));
95         if ( i == n )
96         {
97             ++j;
98             continue;
99         }
100         if ( j == m )
101         {
102             ++i;

```



```
103         continue;
104     }
105     P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
106     P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
107     I c = G::Cross({0, 0}, u, v);
108     if ( c >= 0 )
109         ++i;
110     if ( c <= 0 )
111         ++j;
112 }
113 return result;
114 }
115 };
```

172.2 整数三维共线、共面与方向

算法约束见动态版本第 208 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Integer_Geometry_3D
2 {
3     using I = __int128_t;
4
5     // |coordinates|<=1e9: triple products fit signed int128.
6     struct Point
7     {
8         long long x, y, z;
9     };
10
11     struct Vector
12     {
13         I x, y, z;
14     };
15
16     static Vector diff(Point a, Point b)
17     {
18         return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
19     }
20
21     static Vector Cross(Vector a, Vector b)
22     {
23         return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
24     }
25
26     static I Dot(Vector a, Vector b)
27     {
28         return a.x * b.x + a.y * b.y + a.z * b.z;
29     }
30
31     static I orient(Point a, Point b, Point c, Point d)
32     {
33         return Dot(Cross(diff(b, a), diff(c, a)), diff(d, a));
34     }
35
36     static bool collinear(Point a, Point b, Point c)
37     {
38         auto v = Cross(diff(b, a), diff(c, a));
39         return !v.x && !v.y && !v.z;
40     }
41
42     static bool On_Segment(Point p, Point a, Point b)
43     {
44         return collinear(p, a, b) && Dot(diff(a, p), diff(b, p)) <= 0;
45     }
46 };

```

第一百七十三章 凸包支撑点查询

173.1 支撑点查询恢复下凸包

算法约束见动态版本第 209 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Support_Hull
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Point
7     {
8         ll x, y;
9     };
10
11     static I Cross(Point a, Point b, Point c)
12     {
13         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
14     }
```

```

16 // oracle(a,b) maximizes a*x+b*y exactly; |coordinates| <= 1e18.
17 // left/right: lowest points at the minimum/maximum x.
18 template <class Oracle>
19 static vector<Point> Build(Point left, Point right, Oracle &&oracle)
20 {
21     assert(left.x <= right.x);
22     vector<Point> hull{left};
23     auto dfs = [&](auto &&self, Point a, Point b) -> void
24     {
25         Point c = oracle(b.y - a.y, a.x - b.x);
26         I turn = Cross(a, b, c);
27         assert(turn <= 0);
28         if ( turn < 0 )
29         {
30             assert(a.x < c.x && c.x < b.x);
31             self(self, a, c);
32             self(self, c, b);
33         }
34         else
35         {
36             while ( hull.size() >= 2 &&
37                     Cross(hull[hull.size() - 2], hull.back(), b) == 0 )
38                 hull.pop_back();
39             hull.push_back(b);
40         }
41     };
42     if ( left.x < right.x )
43         dfs(dfs, left, right);
44     else
45         assert(left.y == right.y);
46     return hull;
47 }
48 };

```

第一百七十四章 动态树

174.1 Link-cut tree: 动态森林与路径异或

算法约束见动态版本第 211 页。本节代码独立可抄；采用下列实际接口名。

声明 Link_Cut_Tree<N>，然后 Init(n) 初始化；N 为最大点号。Set、Link、Cut、Connected、Query 分别对应动态版的小写接口。大对象放在全局或 static。

```
1 template <int N> struct Link_Cut_Tree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2]{}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    Node tree[N + 1];
13    int st[N + 1];
14
15    void Init(int n)
16    {
17        assert(0 <= n && n <= N);
18        fill(tree, tree + n + 1, Node{});
19    }
20
21    bool Is_Root(int x) const
22    {
23        int f = tree[x].fa;
24        return tree[f].ch[0] != x && tree[f].ch[1] != x;
25    }
26
27    void Push_Up(int x)
28    {
29        tree[x].sum = tree[tree[x].ch[0]].sum ^ tree[x].val ^ tree[tree[x].ch[1]].sum;
30    }
31
32    void Reverse(int x)
33    {
34        if ( !x )
35            return;
36        swap(tree[x].ch[0], tree[x].ch[1]);
37        tree[x].rev = !tree[x].rev;
38    }
39
40    void Push_Down(int x)
41    {
42        if ( !tree[x].rev )
43            return;
44        Reverse(tree[x].ch[0]);
```

```

45     Reverse(tree[x].ch[1]);
46     tree[x].rev = false;
47 }
48
49 void Rotate(int x)
50 {
51     int y = tree[x].fa, z = tree[y].fa;
52     int k = tree[y].ch[1] == x;
53     int w = tree[x].ch[k ^ 1];
54     if ( !Is_Root(y) )
55         tree[z].ch[tree[z].ch[1] == y] = x;
56     tree[x].fa = z;
57     tree[y].ch[k] = w;
58     if ( w )
59         tree[w].fa = y;
60     tree[x].ch[k ^ 1] = y;
61     tree[y].fa = x;
62     Push_Up(y);
63     Push_Up(x);
64 }
65
66 void Splay(int x)
67 {
68     int top = 0, y = x;
69     st[top++] = y;
70     while ( !Is_Root(y) )
71     {
72         y = tree[y].fa;
73         st[top++] = y;
74     }
75     while ( top )
76         Push_Down(st[--top]);
77     while ( !Is_Root(x) )
78     {
79         int y = tree[x].fa, z = tree[y].fa;
80         if ( !Is_Root(y) )
81         {
82             bool same = (tree[y].ch[1] == x) == (tree[z].ch[1] == y);
83             Rotate(same ? y : x);
84         }
85         Rotate(x);
86     }
87 }
88
89 void Access(int x)
90 {
91     int last = 0;
92     for ( int y = x; y; y = tree[y].fa )
93     {
94         Splay(y);
95         // The old right child's fa remains y: it becomes a path-parent.
96         tree[y].ch[1] = last;
97         if ( last )
98             tree[last].fa = y;
99         Push_Up(y);
100        last = y;

```

```

101     }
102     Splay(x);
103 }
104
105 void Make_Root(int x)
106 {
107     Access(x);
108     Reverse(x);
109 }
110
111 int Find_Root(int x)
112 {
113     Access(x);
114     Push_Down(x);
115     while ( tree[x].ch[0] )
116     {
117         x = tree[x].ch[0];
118         Push_Down(x);
119     }
120     Splay(x);
121     return x;
122 }
123
124 bool Connected(int x, int y)
125 {
126     return x == y || Find_Root(x) == Find_Root(y);
127 }
128
129 bool Link(int x, int y)
130 {
131     Make_Root(x);
132     if ( Find_Root(y) == x )
133         return false;
134     tree[x].fa = y;
135     return true;
136 }
137
138 bool Cut(int x, int y)
139 {
140     Make_Root(x);
141     Access(y);
142     if ( tree[y].ch[0] != x || tree[x].ch[1] )
143         return false;
144     tree[y].ch[0] = 0;
145     tree[x].fa = 0;
146     Push_Up(y);
147     return true;
148 }
149
150 void Set(int x, U value)
151 {
152     Access(x);
153     tree[x].val = value;
154     Push_Up(x);

```

```
155     }
156
157     // x and y must be connected. This is a path aggregate, not a subtree sum.
158     U Query(int x, int y)
159     {
160         Make_Root(x);
161         Access(y);
162         return tree[y].sum;
163     }
164 };
```


第一百七十五章 随机平衡树

175.1 FHQ Treap: 可重集合与顺序统计

算法约束见动态版本第 215 页。本节代码独立可抄；采用下列实际接口名。

声明容量模板参数 N，然后 Init() 或 Init(seed)。N 限制累计插入次数，不是当前元素个数；大对象放在全局或 static。接口采用 Insert、Erase、Kth/Query 等命名。

```
1 template <int N> struct Ordered_Treap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    Node tree[N + 1];
13    int tot = 0;
14    mt19937_64 rng;
15    int root = 0;
16
17    void Init(unsigned long long seed = 712367821)
18    {
19        root = 0;
20        tot = 0;
21        tree[0] = Node{};
22        rng.seed(seed);
23    }
24
25    void Push_Up(int p)
26    {
27        tree[p].siz = tree[tree[p].l].siz + tree[tree[p].r].siz + 1;
28    }
29
30    int New_Node(ll x)
31    {
32        assert(tot < N);
33        int p = ++tot;
34        tree[p] = {0, 0, 1, x, rng()};
35        return p;
36    }
37
38    // left: val < x, or val <= x when equal is true. No x-1 overflow.
39    void Split(int p, ll x, int &l, int &r, bool equal = false)
40    {
41        if ( !p )
42        {
43            l = 0;
44            r = 0;
```

```

45         return;
46     }
47     if ( tree[p].val < x || (equal && tree[p].val == x) )
48     {
49         l = p;
50         Split(tree[p].r, x, tree[p].r, r, equal);
51     }
52     else
53     {
54         r = p;
55         Split(tree[p].l, x, l, tree[p].l, equal);
56     }
57     Push_Up(p);
58 }
59
60 int Merge(int l, int r)
61 {
62     if ( !l || !r )
63         return l ? l : r;
64     if ( tree[l].pri > tree[r].pri )
65     {
66         tree[l].r = Merge(tree[l].r, r);
67         Push_Up(l);
68         return l;
69     }
70     tree[r].l = Merge(l, tree[r].l);
71     Push_Up(r);
72     return r;
73 }
74
75 void Insert(ll x)
76 {
77     int p = New_Node(x), l, r;
78     Split(root, x, l, r);
79     root = Merge(Merge(l, p), r);
80 }
81
82 bool Erase(ll x)
83 {
84     int l, mid, r;
85     Split(root, x, l, mid);
86     Split(mid, x, mid, r, true);
87     bool found = mid != 0;
88     if ( mid )
89         mid = Merge(tree[mid].l, tree[mid].r);
90     root = Merge(l, Merge(mid, r));
91     return found;
92 }
93
94 int Size() const
95 {
96     return tree[root].siz;
97 }
98
99 int Count_Less(ll x, bool equal = false) const
100 {

```

```

101     int p = root, ans = 0;
102     while ( p )
103     {
104         if ( tree[p].val < x || (equal && tree[p].val == x) )
105         {
106             ans += tree[tree[p].l].siz + 1;
107             p = tree[p].r;
108         }
109         else
110             p = tree[p].l;
111     }
112     return ans;
113 }
114
115 int Rank(ll x) const
116 {
117     return Count_Less(x) + 1;
118 }
119
120 ll Kth(int k) const
121 {
122     assert(1 <= k && k <= Size());
123     int p = root;
124     while ( true )
125     {
126         int left = tree[tree[p].l].siz;
127         if ( k == left + 1 )
128             return tree[p].val;
129         if ( k <= left )
130             p = tree[p].l;
131         else
132         {
133             k -= left + 1;
134             p = tree[p].r;
135         }
136     }
137 }
138
139 optional<ll> Predecessor(ll x) const
140 {
141     int k = Count_Less(x);
142     if ( !k )
143         return nullopt;
144     return Kth(k);
145 }
146
147 optional<ll> Successor(ll x) const
148 {
149     int k = Count_Less(x, true);
150     if ( k == Size() )
151         return nullopt;
152     return Kth(k + 1);
153 }
154 };

```

175.2 隐式 Treap：序列翻转、加值与区间和

算法约束见动态版本第 217 页。本节代码独立可抄；采用下列实际接口名。

声明容量模板参数 N，然后 Init() 或 Init(seed)。N 限制累计插入次数，不是当前元素个数；大对象放在全局或 static。接口采用 Insert、Erase、Kth/Query 等命名。

```

1  template <int N> struct Sequence_Treap
2  {
3      using ll = long long;
4
5      struct Node
6      {
7          int l = 0, r = 0, siz = 0;
8          ll val = 0, sum = 0, tag = 0;
9          unsigned long long pri = 0;
10         bool rev = false;
11     };
12
13     Node tree[N + 1];
14     int tot = 0;
15     mt19937_64 rng;
16     int root = 0;
17
18     void Init(unsigned long long seed = 712367821)
19     {
20         root = 0;
21         tot = 0;
22         tree[0] = Node{};
23         rng.seed(seed);
24     }
25
26     int Size() const
27     {
28         return tree[root].siz;
29     }
30
31     void Push_Up(int p)
32     {
33         tree[p].siz = tree[tree[p].l].siz + tree[tree[p].r].siz + 1;
34         tree[p].sum = tree[tree[p].l].sum + tree[p].val + tree[tree[p].r].sum;
35     }
36
37     void Apply_Add(int p, ll x)
38     {
39         if ( !p )
40             return;
41         tree[p].val += x;
42         tree[p].sum += x * tree[p].siz;
43         tree[p].tag += x;
44     }
45
46     void Apply_Reverse(int p)
47     {
48         if ( !p )
49             return;
50         swap(tree[p].l, tree[p].r);

```

```

51     tree[p].rev = !tree[p].rev;
52 }
53
54 void Push_Down(int p)
55 {
56     if ( tree[p].rev )
57     {
58         Apply_Reverse(tree[p].l);
59         Apply_Reverse(tree[p].r);
60         tree[p].rev = false;
61     }
62     if ( tree[p].tag )
63     {
64         Apply_Add(tree[p].l, tree[p].tag);
65         Apply_Add(tree[p].r, tree[p].tag);
66         tree[p].tag = 0;
67     }
68 }
69
70 // Split into the first k elements and the rest.
71 void Split(int p, int k, int &l, int &r)
72 {
73     if ( !p )
74     {
75         l = 0;
76         r = 0;
77         return;
78     }
79     Push_Down(p);
80     int left = tree[tree[p].l].siz;
81     if ( k > left )
82     {
83         l = p;
84         Split(tree[p].r, k - left - 1, tree[p].r, r);
85     }
86     else
87     {
88         r = p;
89         Split(tree[p].l, k, l, tree[p].l);
90     }
91     Push_Up(p);
92 }
93
94 int Merge(int l, int r)
95 {
96     if ( !l || !r )
97         return l ? l : r;
98     if ( tree[l].pri > tree[r].pri )
99     {
100         Push_Down(l);
101         tree[l].r = Merge(tree[l].r, r);
102         Push_Up(l);
103         return l;
104     }
105     Push_Down(r);
106     tree[r].l = Merge(l, tree[r].l);

```

```

107     Push_Up(r);
108     return r;
109 }
110
111 // Insert after the first k elements, 0 <= k <= size().
112 void Insert(int k, ll value)
113 {
114     assert(0 <= k && k <= Size());
115     assert(tot < N);
116     int p = ++tot, l, r;
117     tree[p] = {0, 0, 1, value, value, 0, rng(), false};
118     Split(root, k, l, r);
119     root = Merge(Merge(l, p), r);
120 }
121
122 // All range operations use 1-based inclusive intervals.
123 void Erase(int l, int r)
124 {
125     assert(1 <= l && l <= r && r <= Size());
126     int x, y, z;
127     Split(root, r, y, z);
128     Split(y, l - 1, x, y);
129     root = Merge(x, z);
130 }
131
132 void Reverse(int l, int r)
133 {
134     assert(1 <= l && l <= r && r <= Size());
135     int x, y, z;
136     Split(root, r, y, z);
137     Split(y, l - 1, x, y);
138     Apply_Reverse(y);
139     root = Merge(Merge(x, y), z);
140 }
141
142 void Add(int l, int r, ll value)
143 {
144     assert(1 <= l && l <= r && r <= Size());
145     int x, y, z;
146     Split(root, r, y, z);
147     Split(y, l - 1, x, y);
148     Apply_Add(y, value);
149     root = Merge(Merge(x, y), z);
150 }
151
152 ll Query(int l, int r)
153 {
154     assert(1 <= l && l <= r && r <= Size());
155     int x, y, z;
156     Split(root, r, y, z);
157     Split(y, l - 1, x, y);
158     ll ans = tree[y].sum;
159     root = Merge(Merge(x, y), z);
160     return ans;
161 }
162

```

```
163 void Collect(int p, vector<ll> &out)
164 {
165     if ( !p )
166         return;
167     Push_Down(p);
168     Collect(tree[p].l, out);
169     out.push_back(tree[p].val);
170     Collect(tree[p].r, out);
171 }
172
173 vector<ll> Values()
174 {
175     vector<ll> answer;
176     Collect(root, answer);
177     return answer;
178 }
179 };
```

第一百七十六章 伸展树

176.1 Splay：可重集合与顺序统计

算法约束见动态版本第 222 页。本节代码独立可抄；采用下列实际接口名。

声明 `Ordered_Splay<N>` 后调用 `Init()`；`N` 为累计新建节点上限。接口采用 `Insert`、`Erase`、`Rank`、`Kth`、`Prev`、`Next`。大对象放在全局或 `static`。

```
1 template <int N> struct Ordered_Splay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0, cnt = 0, siz = 0;
8         ll val = 0;
9     };
10
11     Node tree[N + 1];
12     int tot = 0;
13     int root = 0;
14
15     void Init()
16     {
17         root = 0;
18         tot = 0;
19         tree[0] = Node{};
20     }
21
22     int Size() const
23     {
24         return tree[root].siz;
25     }
26
27     void Push_Up(int x)
28     {
29         tree[x].siz = tree[tree[x].ch[0]].siz + tree[x].cnt + tree[tree[x].ch[1]].siz;
30     }
31
32     int New_Node(ll value, int parent)
33     {
34         assert(tot < N);
35         int x = ++tot;
36         tree[x] = {0, 0, parent, 1, 1, value};
37         return x;
38     }
39
40     void Rotate(int x)
41     {
42         int y = tree[x].fa, z = tree[y].fa;
43         int k = tree[y].ch[1] == x;
44         int w = tree[x].ch[k ^ 1];
```



```

45     if ( z )
46         tree[z].ch[tree[z].ch[1] == y] = x;
47     else
48         root = x;
49     tree[x].fa = z;
50     tree[y].ch[k] = w;
51     if ( w )
52         tree[w].fa = y;
53     tree[x].ch[k ^ 1] = y;
54     tree[y].fa = x;
55     Push_Up(y);
56     Push_Up(x);
57 }
58
59 void Splay(int x)
60 {
61     while ( tree[x].fa )
62     {
63         int y = tree[x].fa, z = tree[y].fa;
64         if ( z )
65         {
66             bool same = (tree[y].ch[1] == x) == (tree[z].ch[1] == y);
67             Rotate(same ? y : x);
68         }
69         Rotate(x);
70     }
71 }
72
73 // Splay the last visited node even when the value does not exist.
74 int Find(ll value)
75 {
76     int x = root, last = 0;
77     while ( x && tree[x].val != value )
78     {
79         last = x;
80         x = tree[x].ch[value > tree[x].val];
81     }
82     if ( x )
83         Splay(x);
84     else if ( last )
85         Splay(last);
86     return x;
87 }
88
89 void Insert(ll value)
90 {
91     int x = root, parent = 0;
92     while ( x && tree[x].val != value )
93     {
94         parent = x;
95         x = tree[x].ch[value > tree[x].val];
96     }
97     if ( x )
98     {
99         ++tree[x].cnt;
100         Push_Up(x);

```

```

101     }
102     else
103     {
104         x = New_Node(value, parent);
105         if ( parent )
106             tree[parent].ch[value > tree[parent].val] = x;
107         else
108             root = x;
109     }
110     Splay(x);
111 }
112
113 bool Erase(ll value)
114 {
115     int x = Find(value);
116     if ( !x )
117         return false;
118     if ( tree[x].cnt > 1 )
119     {
120         --tree[x].cnt;
121         Push_Up(x);
122         return true;
123     }
124     int l = tree[x].ch[0], r = tree[x].ch[1];
125     if ( l )
126         tree[l].fa = 0;
127     if ( r )
128         tree[r].fa = 0;
129     if ( !l )
130     {
131         root = r;
132         return true;
133     }
134     root = l;
135     x = l;
136     while ( tree[x].ch[1] )
137         x = tree[x].ch[1];
138     Splay(x);
139     tree[x].ch[1] = r;
140     if ( r )
141         tree[r].fa = x;
142     Push_Up(x);
143     return true;
144 }
145
146 int Rank(ll value)
147 {
148     int x = root, last = 0, ans = 1;
149     while ( x )
150     {
151         last = x;
152         if ( value <= tree[x].val )
153             x = tree[x].ch[0];
154         else
155         {
156             ans += tree[tree[x].ch[0]].siz + tree[x].cnt;

```

```

157         x = tree[x].ch[1];
158     }
159 }
160 if ( last )
161     Splay(last);
162 return ans;
163 }
164
165 ll Kth(int k)
166 {
167     assert(1 <= k && k <= Size());
168     int x = root;
169     while ( true )
170     {
171         int left = tree[tree[x].ch[0]].siz;
172         if ( k <= left )
173             x = tree[x].ch[0];
174         else if ( k <= left + tree[x].cnt )
175         {
176             Splay(x);
177             return tree[x].val;
178         }
179         else
180         {
181             k -= left + tree[x].cnt;
182             x = tree[x].ch[1];
183         }
184     }
185 }
186
187 optional<ll> Predecessor(ll value)
188 {
189     int x = root, last = 0, answer = 0;
190     while ( x )
191     {
192         last = x;
193         if ( tree[x].val < value )
194         {
195             answer = x;
196             x = tree[x].ch[1];
197         }
198         else
199             x = tree[x].ch[0];
200     }
201     if ( last )
202         Splay(last);
203     if ( !answer )
204         return nullopt;
205     Splay(answer);
206     return tree[answer].val;
207 }
208
209 optional<ll> Successor(ll value)
210 {
211     int x = root, last = 0, answer = 0;
212     while ( x )

```

```
213     {
214         last = x;
215         if ( tree[x].val > value )
216         {
217             answer = x;
218             x = tree[x].ch[0];
219         }
220         else
221             x = tree[x].ch[1];
222     }
223     if ( last )
224         Splay(last);
225     if ( !answer )
226         return nullopt;
227     Splay(answer);
228     return tree[answer].val;
229 }
230 };
```

第一百七十七章 状态序列维护

177.1 状态序列 Treap：分组区间 GCD

算法约束见动态版本第 227 页。本节代码独立可抄；采用下列实际接口名。

声明 `Gcd_Sequence_Treap<N>` 后调用 `Init(seed)`；`N` 为峰值元素数。`Build` 重建序列，`Insert`、`Erase`、`Set`、`Toggle`、`Query` 对应动态版接口。大对象放在全局或 `static`。

```
1 template <int N> struct Gcd_Sequence_Treap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    Node tree[N + 1];
13    int tot = 0;
14    vector<int> recycled;
15    mt19937_64 rng;
16    int root = 0;
17
18    void Init(U seed = 712367821)
19    {
20        root = 0;
21        tot = 0;
22        tree[0] = Node{};
23        recycled.clear();
24        rng.seed(seed);
25    }
26
27    int Size() const
28    {
29        return tree[root].siz;
30    }
31
32    int New_Node(U value, int tag)
33    {
34        assert(tag == 0 || tag == 1);
35        int p;
36        if ( recycled.empty() )
37        {
38            assert(tot < N);
39            p = ++tot;
40            tree[p] = Node{};
41        }
42        else
43        {
44            p = recycled.back();
```

```

45         recycled.pop_back();
46         tree[p] = Node{};
47     }
48     tree[p].siz = 1;
49     tree[p].val = value;
50     tree[p].tag = tag;
51     tree[p].cnt[tag] = 1;
52     tree[p].g[tag] = value;
53     tree[p].pri = rng();
54     return p;
55 }
56
57 void Push_Up(int p)
58 {
59     int l = tree[p].l, r = tree[p].r;
60     tree[p].siz = tree[l].siz + tree[r].siz + 1;
61     for ( int t = 0; t < 2; t++ )
62     {
63         tree[p].cnt[t] = tree[l].cnt[t] + tree[r].cnt[t] + (tree[p].tag == t);
64         tree[p].g[t] = gcd(tree[l].g[t], tree[r].g[t]);
65         if ( tree[p].tag == t )
66             tree[p].g[t] = gcd(tree[p].g[t], tree[p].val);
67     }
68 }
69
70 void Split(int p, int k, int &l, int &r)
71 {
72     if ( !p )
73     {
74         l = 0;
75         r = 0;
76         return;
77     }
78     int left = tree[tree[p].l].siz;
79     if ( k > left )
80     {
81         l = p;
82         Split(tree[p].r, k - left - 1, tree[p].r, r);
83     }
84     else
85     {
86         r = p;
87         Split(tree[p].l, k, l, tree[p].l);
88     }
89     Push_Up(p);
90 }
91
92 int Merge(int l, int r)
93 {
94     if ( !l || !r )
95         return l ? l : r;
96     if ( tree[l].pri > tree[r].pri )
97     {
98         tree[l].r = Merge(tree[l].r, r);
99         Push_Up(l);
100        return l;

```

```

101     }
102     tree[r].l = Merge(l, tree[r].l);
103     Push_Up(r);
104     return r;
105 }
106
107 void Rebuild(int p)
108 {
109     if ( !p )
110         return;
111     Rebuild(tree[p].l);
112     Rebuild(tree[p].r);
113     Push_Up(p);
114 }
115
116 void Release(int p)
117 {
118     if ( !p )
119         return;
120     Release(tree[p].l);
121     Release(tree[p].r);
122     recycled.push_back(p);
123 }
124
125 // O(n) GCD calls; replaces the current sequence.
126 void Build(const vector<pair<U, int>> &v)
127 {
128     assert(v.size() <= N);
129     tot = 0;
130     tree[0] = Node{};
131     recycled.clear();
132     root = 0;
133     vector<int> st;
134     for ( auto [value, tag] : v )
135     {
136         int p = New_Node(value, tag), last = 0;
137         while ( !st.empty() && tree[st.back()].pri < tree[p].pri )
138         {
139             last = st.back();
140             st.pop_back();
141         }
142         tree[p].l = last;
143         if ( !st.empty() )
144             tree[st.back()].r = p;
145         st.push_back(p);
146     }
147     if ( st.empty() )
148         return;
149     root = st.front();
150     Rebuild(root);
151 }
152
153 // Insert after the first k entries. Range operations below are 1-based.
154 void Insert(int k, U value, int tag)
155 {
156     assert(0 <= k && k <= Size());

```

```

157     int p = New_Node(value, tag), l, r;
158     Split(root, k, l, r);
159     root = Merge(Merge(l, p), r);
160 }
161
162 void Erase(int l, int r)
163 {
164     assert(1 <= l && l <= r && r <= Size());
165     int x, y, z;
166     Split(root, r, y, z);
167     Split(y, l - 1, x, y);
168     root = Merge(x, z);
169     Release(y);
170 }
171
172 void Set(int k, U value)
173 {
174     assert(1 <= k && k <= Size());
175     int x, y, z;
176     Split(root, k, y, z);
177     Split(y, k - 1, x, y);
178     tree[y].val = value;
179     Push_Up(y);
180     root = Merge(Merge(x, y), z);
181 }
182
183 void Toggle(int k)
184 {
185     assert(1 <= k && k <= Size());
186     int x, y, z;
187     Split(root, k, y, z);
188     Split(y, k - 1, x, y);
189     tree[y].tag ^= 1;
190     Push_Up(y);
191     root = Merge(Merge(x, y), z);
192 }
193
194 optional<U> Query(int l, int r, int tag)
195 {
196     assert(1 <= l && l <= r && r <= Size());
197     assert(tag == 0 || tag == 1);
198     int x, y, z;
199     Split(root, r, y, z);
200     Split(y, l - 1, x, y);
201     optional<U> answer;
202     if ( tree[y].cnt[tag] )
203         answer = tree[y].g[tag];
204     root = Merge(Merge(x, y), z);
205     return answer;
206 }
207 };

```


第一百七十八章 一般图匹配

178.1 一般图最大基数匹配：带花树

算法约束见动态版本第 231 页。本节代码独立可抄；采用下列实际接口名。

声明 `General_Matching<N>` 后调用 `Init(n)`，`N` 为点数上限。`Insert` 加边，`Solve` 返回匹配边数，`mate` 保存配对点，点号仍为 0-based。大对象放在全局或 `static`。

```
1 template <int N> struct General_Matching
2 {
3     int n;
4     vector<int> g[N];
5     int mate[N], pre[N], base[N], q[N];
6     bool outer[N], flower[N], seen[N];
7
8     void Init(int vertices)
9     {
10         assert(0 <= vertices && vertices <= N);
11         n = vertices;
12         for ( int u = 0; u < n; u++ )
13             g[u].clear();
14         fill(mate, mate + n, -1);
15     }
16
17     void Insert(int u, int v)
18     {
19         assert(0 <= u && u < n && 0 <= v && v < n);
20         if ( u == v )
21             return;
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     int Lca(int u, int v)
27     {
28         fill(seen, seen + n, false);
29         while ( true )
30         {
31             u = base[u];
32             seen[u] = true;
33             if ( mate[u] == -1 )
34                 break;
35             u = pre[mate[u]];
36         }
37         while ( !seen[base[v]] )
38             v = pre[mate[base[v]]];
39         return base[v];
40     }
41
42     void Mark(int u, int b, int v)
43     {
44         while ( base[u] != b )
```

```

45     {
46         flower[base[u]] = true;
47         flower[base[mate[u]]] = true;
48         pre[u] = v;
49         v = mate[u];
50         u = pre[v];
51     }
52 }
53
54 bool Augment(int root)
55 {
56     fill(outer, outer + n, false);
57     fill(pre, pre + n, -1);
58     iota(base, base + n, 0);
59     int tail = 0;
60     q[tail++] = root;
61     outer[root] = true;
62     for ( int head = 0; head < tail; head++ )
63     {
64         int u = q[head];
65         for ( int v : g[u] )
66         {
67             if ( base[u] == base[v] || mate[u] == v )
68                 continue;
69             if ( v == root || (mate[v] != -1 && pre[mate[v]] != -1) )
70             {
71                 int b = Lca(u, v);
72                 fill(flower, flower + n, false);
73                 Mark(u, b, v);
74                 Mark(v, b, u);
75                 for ( int x = 0; x < n; x++ )
76                 {
77                     if ( !flower[base[x]] )
78                         continue;
79                     base[x] = b;
80                     if ( !outer[x] )
81                     {
82                         outer[x] = true;
83                         q[tail++] = x;
84                     }
85                 }
86             }
87             else if ( pre[v] == -1 )
88             {
89                 pre[v] = u;
90                 if ( mate[v] == -1 )
91                 {
92                     while ( v != -1 )
93                     {
94                         int p = pre[v];
95                         int next = mate[p];
96                         mate[v] = p;
97                         mate[p] = v;
98                         v = next;
99                     }
100                     return true;

```

```
101         }
102         int x = mate[v];
103         outer[x] = true;
104         q[tail++] = x;
105     }
106 }
107 }
108 return false;
109 }
110
111 // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
112 int Solve()
113 {
114     fill(mate, mate + n, -1);
115     int answer = 0;
116     for ( int u = 0; u < n; u++ )
117     {
118         if ( mate[u] == -1 && Augment(u) )
119             answer++;
120     }
121     return answer;
122 }
123 };
```

第一百七十九章 线性递推

179.1 Berlekamp–Massey：最短线性递推

算法约束见动态版本第 234 页。本节代码独立可抄；采用下列实际接口名。

```
1 // Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
2 vector<int> Berlekamp_Massey(const vector<int> &s, int p = 998244353)
3 {
4     assert(p >= 2);
5     vector<int> c{1}, b{1};
6     int len = 0, shift = 1, last = 1;
7     for ( int i = 0; i < (int)s.size(); i++ )
8     {
9         assert(0 <= s[i] && s[i] < p);
10        long long d = s[i];
11        for ( int j = 1; j <= len; j++ )
12            d = (d + 1LL * c[j] * s[i - j]) % p;
13        if ( d == 0 )
14        {
15            shift++;
16            continue;
17        }
18        long long inv = 1, x = last;
19        for ( int e = p - 2; e; e >>= 1 )
20        {
21            if ( e & 1 )
22                inv = inv * x % p;
23            x = x * x % p;
24        }
25        long long factor = d * inv % p;
26        auto old = c;
27        if ( c.size() < b.size() + shift )
28            c.resize(b.size() + shift);
29        for ( int j = 0; j < (int)b.size(); j++ )
30            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
31        if ( 2 * len <= i )
32        {
33            len = i + 1 - len;
34            b = old;
35            last = d;
36            shift = 1;
37        }
38        else
39            shift++;
40    }
41    c.resize(len + 1);
42    c.erase(c.begin());
43    for ( int &x : c )
44        x = x == 0 ? 0 : p - x;
45    return c;
46 }
```

179.2 特征多项式求线性递推第 k 项

算法约束见动态版本第 236 页。本节代码独立可抄；采用下列实际接口名。

```

1 // init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
2 int Recurrence_Nth(const vector<int> &init,
3                   const vector<int> &c,
4                   unsigned long long n,
5                   int p = 998244353)
6 {
7     assert(p >= 2 && init.size() == c.size());
8     int k = c.size();
9     if ( !k )
10         return 0;
11     for ( int x : init )
12         assert(0 <= x && x < p);
13     for ( int x : c )
14         assert(0 <= x && x < p);
15     if ( n < (unsigned)k )
16         return init[n];
17     auto multiply = [&](const vector<int> &a, const vector<int> &b)
18     {
19         vector<int> t(2 * k - 1);
20         for ( int i = 0; i < k; i++ )
21         {
22             for ( int j = 0; j < k; j++ )
23                 t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
24         }
25         for ( int i = 2 * k - 2; i >= k; i-- )
26         {
27             for ( int j = 1; j <= k; j++ )
28                 t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
29         }
30         t.resize(k);
31         return t;
32     };
33     vector<int> answer(k), x(k);
34     answer[0] = 1;
35     if ( k == 1 )
36         x[0] = c[0];
37     else
38         x[1] = 1;
39     while ( n )
40     {
41         if ( n & 1 )
42             answer = multiply(answer, x);
43         x = multiply(x, x);
44         n >>= 1;
45     }
46     long long value = 0;
47     for ( int i = 0; i < k; i++ )
48         value = (value + 1LL * answer[i] * init[i]) % p;
49     return value;
50 }

```

第一百八十章 快速线性递推

180.1 快速递推与有理生成函数单项

算法约束见动态版本第 238 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod = 998244353, int primitive = 3> struct Bostan_Mori
2 {
3     using N = Ntt_Convolution<mod, primitive>;
4     using Z = typename N::Z;
5     using Poly = typename N::Poly;
6     using U = unsigned long long;
7
8     // Coefficient of  $x^n$  in  $p(x) / q(x)$ , with  $q[0] \neq 0$ .
9     static Z Coefficient(Poly p, Poly q, U n)
10    {
11        assert(!q.empty() && q[0].v != 0);
12        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
13        while ( n && !p.empty() )
14        {
15            Poly r = q;
16            for ( int i = 1; i < int(r.size()); i += 2 )
17                r[i] = Z(0) - r[i];
18            Poly a = N::Multiply(p, r);
19            Poly b = N::Multiply(q, r);
20            p.clear();
21            q.clear();
22            for ( int i = n & 1; i < int(a.size()); i += 2 )
23                p.push_back(a[i]);
24            for ( int i = 0; i < int(b.size()); i += 2 )
25                q.push_back(b[i]);
26            n >>= 1;
27        }
28        return p.empty() ? Z(0) : p[0] / q[0];
29    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
31 // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
32 static Z Recurrence(const Poly &init, const Poly &c, U n)
33 {
34     assert(init.size() == c.size());
35     int k = c.size();
36     if ( k == 0 )
37         return 0;
38     if ( n < init.size() )
39         return init[n];
40     assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
41     Poly q(k + 1);
42     q[0] = 1;
43     for ( int i = 0; i < k; i++ )
44         q[i + 1] = Z(0) - c[i];
45     Poly p = N::Multiply(init, q);
46     p.resize(k);
47     return Coefficient(p, q, n);
48 }
49 };
```

第三部分

数学速查与维护

第一百八十一章 数学速查：条件先于公式

181.1 整除、卷积与筛法

对正整数定义狄利克雷卷积

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

单位元为 $\varepsilon(n) = [n = 1]$ ，常函数 $1(n) = 1$ ，恒等函数 $\text{id}(n) = n$ 。关键恒等式是 $\mu * 1 = \varepsilon$ 、 $\varphi * 1 = \text{id}$ 。若 $F(n) = \sum_{d|n} f(d)$ ，则 $f(n) = \sum_{d|n} \mu(d)F(n/d)$ 。

$$\sum_{i=1}^a \sum_{j=1}^b [\gcd(i, j) = 1] = \sum_{d=1}^{\min(a, b)} \mu(d) \lfloor a/d \rfloor \lfloor b/d \rfloor.$$

整除分块：当左端点为 l 、商为 $q = \lfloor n/l \rfloor$ ，右端点为 $r = \lfloor n/q \rfloor$ 。同时包含多个商时，取各自右端点的最小值。杜教筛由卷积恒等式获得

$$M(n) = 1 - \sum_{l=2}^n M(\lfloor n/l \rfloor), \quad \Phi(n) = \frac{n(n+1)}{2} - \sum_{l=2}^n \Phi(\lfloor n/l \rfloor),$$

求和按等商区间合并；第二式求和中没有额外乘以 l 。

181.2 杜教筛：预筛与缓存都不可省略

前节的两条卷积恒等式给出 DuJiao 的递推，见代码中的 `mertens` 与 `totient_sum`。把商相同的一段合并，系数为区间长度，递归参数严格变小；0 直接查表返回 0。超过预筛上限的结果在返回时写入缓存，后续相同参数直接复用。WIDA 收集的 `sumMu` 示例查询了 `fMu`，但未写入新结果；本库已有实现包含写回，不能照搬遗漏缓存的版本。

设最大查询为 N ，预筛上限为 B 。单次查询的常用上界为 $O(B + N/\sqrt{B})$ ，其中哈希访问按期望常数处理；取 B 约 $N^{2/3}$ 得到 $O(N^{2/3})$ 。多个不同查询可能继续增加缓存，不能把单次空间估计当成任意多询问的上界。保留缓存可复用，主动清空仅影响重算开销；预筛表与上限应视为只读。

接口范围为 $0 \leq n \leq 10^{11}$ ，返回精确整数；欧拉函数前缀和用 128 位。WIDA 的 Z 模数结果可在精确莫比乌斯前缀和返回后取模得到。负值先求余，仅在余数为负时加模数，避免无条件加模数的溢出风险。

两套实现以独立试除构建到 200000 的前缀参考，检查四种预筛上限、零输入、乱序与重复查询、缓存写回及清空重建。上限 10^{11} 使用 10^6 与 2×10^6 两种预筛规模核对双版本结果；这一项属于边界运行与一致性证据，不是独立答案对拍。普通与 ASan/UBSan 均通过，暂无该接口的在线 AC。

181.3 64 位判素与质因数分解

本库判素采用七个固定底数的 Miller-Rabin，覆盖无符号 64 位整数：

2, 325, 9375, 28178, 450775, 9780504, 1795265022.

将 $n-1 = d2^s$ 、 d 为奇数，检查 a^d 及后续平方链。小于 2 的输入不视为素数；底数模 n 为 0 时跳过该底数，不可误判为合数。模乘用无符号 128 位容纳完整的两个 64 位数之积。固定底数的适用范围依据公开 SPRP 记录，不外推到任意精度整数。

<https://miller-rabin.appspot.com/>

Pollard-Rho 用 $f(x) = x^2 + c \pmod n$ 迭代，并求 $\gcd(|x-y|, n)$ 。只有 $1 < d < n$ 的因子才返回；一次尝试超过轮数上限或得到 $d = n$ ，就换参数重试。其中模平方后的加法也在 128 位中执行，避免 n 接近 2^{64} 时加 c 溢

出。该过程保证返回真因子，但随机尝试次数没有确定的最坏上界。递归分解两侧，直到判为素数，最后排序并保留重复因子。

调用 `factor(1)` 得到空数组；对 $n \geq 2$ ，返回单元素说明 n 为素数。POJ 1811 所需的最小质因子可取排序结果首项；P4718 所需的最大质因子可取末项。不要对空数组取首尾，也不要绕过 `factor` 把素数传给内部 `rho`。

双版本对百万范围以独立埃氏筛核验判素，对 $1 \dots 10000$ 以试除核验完整因子列表。另验证三个随机种子下的大质数平方、重复因子、强伪素数和 $2^{64} - 1$ ；期望因子的素性均以试除独立验证。 $2^{61} - 1$ 使用完整分解 $n - 1$ 的 Lucas 素性证书检查；模乘与模幂另以任意精度运算对拍。完整 POJ 1811 与 P4718 驱动分别验证 350 组输入，检查素数标签与最小、最大质因子。P4718 压力批次含 300 个接近 10^{18} 的平衡半素数，本机普通编译运行约 1.99、1.83 秒；机器差异与随机路径都会影响耗时，本地运行时间不直接代表线上性能。两套代码随后通过 P4718 全部 20 个测试点，最长单点显示约 2.11、2.12 秒，低于当前页面的 4 秒限制。该 AC 检查素数标签与最大质因子，范围为 $2 \leq n \leq 10^{18}$ 。完整因子重数、最小质因子、 $n = 1$ 与更大的无符号 64 位输入仍依据本地验证；普通与 ASan/UBSan 均通过，不构成随机运行时间保证。

来源：kuangbin 2.6；固定底数记录见上方链接。

181.4 单个数的欧拉函数

实现见第 114 页。对 $n \geq 1$ ，由容斥得到

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right),$$

其中只枚举不同质因子，故质数幂 p^k 的贡献为 $p^{k-1}(p-1)$ 。每次更新按先除后乘的顺序： $\text{ans} \leftarrow (\text{ans}/p)(p-1)$ 。当前尚未处理的质因子仍整除答案，而且答案只会减小，乘法结果不会超过原始 n 。

无分解器参数的入口采用试除，循环上界用 $p \leq n/p$ ，剩下的 $n > 1$ 再处理一次。这能避免 p^2 溢出，但不消除 $O(\sqrt{n})$ 的耗时；不适合试除 64 位大素数。传入现有 PollardRho 对象时先分解，再在排序后的因子列表上去重更新；不重复构造分解器，后续调用沿用其随机状态。该入口没有确定的最坏运行时间。 $\varphi(1) = 1$ ，两个入口均接受完整的正无符号 64 位输入；0 不在定义域内。

本地双版本验证包含 $1 \dots 20000$ 的线性筛交叉检查、小范围直接计数互质整数，以及重复质因子、大质数平方、 $2^{64} - 1$ 和已独立认证的大素数。普通与 ASan/UBSan 测试均通过；P4718 的 AC 只验证底层分解用途，不是此欧拉函数的新 AC。来源：kuangbin 2.7.1、2.7.3；WIDA 的单个数欧拉函数。

181.5 线性筛：三张表的约定

LinearSieve 同时构造最小质因子、欧拉函数和莫比乌斯函数。内层遇到第一个整除 i 的素数后停止，因此每个合数 j 只由 $p = \text{lp}(j)$ 与 $i = j/p$ 生成一次，时间为 $O(n)$ 。若 $p \mid i$ ，则 $\varphi(ip) = p\varphi(i)$ 、 $\mu(ip) = 0$ ；否则二者分别为 $(p-1)\varphi(i)$ 和 $-\mu(i)$ 。

判素使用 $x \geq 2$ 且 $\text{lp}(x) = x$ ，不能漏掉前半条件。分解表内正整数时，反复取 $p = \text{lp}(x)$ 、令 $x \leftarrow x/p$ ，直到 $x = 1$ ；重复因子会重复输出。0 不可分解，1 的质因子列表为空。上限要求 $0 \leq n < \text{INT_MAX}$ ，避免构造时 $n+1$ 溢出，并保证内存可用。筛乘积前先检查 $p \leq n/i$ ，不直接计算可能溢出的 ip 。

双版本验证至 $n = 10^7$ ，包括小前缀、试除最小质因子、因子还原和逐项约数和证书：

$$\sum_{d|x} \varphi(d) = x, \quad \sum_{d|x} \mu(d) = [x = 1].$$

这些恒等式在 $d = x$ 项系数为 1，可按 x 递增唯一确定两张表。普通与 ASan/UBSan 均通过；不据此宣称莫比乌斯反演的所有应用已验证。

来源：kuangbin 2.1.1、2.1.2、2.7.2、2.7.4、2.13.1；WIDA 的全部欧拉函数表。

WIDA 的“使用莫比乌斯反演求解欧拉函数”先存储全部约数，再使用

$$\varphi(n) = n - \sum_{d|n, d < n} \varphi(d).$$

这与 $\varphi * 1 = \text{id}$ 等价；再卷积 μ 可得 $\varphi(n) = \sum_{d|n} \mu(d)(n/d)$ 。本库用线性筛提供同一张表，无需额外保存约数列表。百万范围另分别运行直接卷积和真约数扣减，逐项核对两套筛表；两种方法都不复用线性筛的递推分支。

181.6 区间筛：只保存目标区间

实现见第 102 页。设 $0 \leq L \leq R \leq \text{INT_MAX}$ ，先筛出不超过 $\lfloor \sqrt{R} \rfloor$ 的素数，再对每个素数 p 从

$$\max(p^2, \lceil L/p \rceil p)$$

开始标记区间内的倍数。任何合数都有不超过自身平方根的素因子，所以不会遗漏；从 p^2 开始也不会误删 p 自身。输出跳过 0 和 1，其余未标记数恰为素数。底筛与区间标记共用同一个外层素数循环，但使用两张独立的表。

令 $W = R - L + 1$ ，空间为 $O(\sqrt{R} + W)$ ，包含返回的素数数组；时间为 $O((\sqrt{R} + W) \log \log(R + 3))$ 。函数每次调用都重建底筛，适合题目中的少量区间查询。端点虽然是 `int`，向上取整的加法、 p^2 、标记变量和输出遍历变量均用 `long long`；否则 $R = \text{INT_MAX}$ 时最后一次递增也会溢出。区间长度仍受可用内存约束，不适合一次保存整个 32 位值域。

POJ 2689 与 UVa 10140 的驱动读到 EOF，扫描相邻素数差。最近、最远都只在严格更优时更新，距离并列保留最先出现的一对；少于两个素数时输出题目指定句子。两套实现以试除和 Miller–Rabin 判素进行独立对拍，并检查平方边界、单点区间及右端点为 `INT_MAX` 的百万长度区间；完整驱动验证 208 组输入，含官方样例。普通测试与 ASan/UBSan 均通过；尚无本库在线 AC。

来源：kuangbin 2.1.3；<https://onlinejudge.org/external/101/10140.pdf>。

181.7 矩形 GCD 计数：缩放后容斥

实现见第 110 页。由 $[g = 1] = \sum_{d|g} \mu(d)$ ，前缀矩形互质数对数为

$$F(A, B) = \sum_{i=1}^{\min(A, B)} \mu(i) \left\lfloor \frac{A}{i} \right\rfloor \left\lfloor \frac{B}{i} \right\rfloor.$$

两个商同时不变的右端点取 $r = \min(A/\lfloor A/l \rfloor, B/\lfloor B/l \rfloor)$ ，用莫比乌斯前缀和合并 $[l, r]$ 。不能只按一个商分块。只需筛到 $\min(A, B)$ ，不要求较长的一边也在预处理范围内。 $A = 0$ 或 $B = 0$ 时循环为空，返回 0；正整数条件不包含坐标轴。

矩形内 $\gcd(x, y) = k$ 等价于 x, y 均为 k 的倍数，且缩放后互质。因此答案为四个前缀矩形的容斥：

$$\begin{aligned} & F(\lfloor b/k \rfloor, \lfloor d/k \rfloor) - F(\lfloor (a-1)/k \rfloor, \lfloor d/k \rfloor) \\ & - F(\lfloor b/k \rfloor, \lfloor (c-1)/k \rfloor) + F(\lfloor (a-1)/k \rfloor, \lfloor (c-1)/k \rfloor). \end{aligned}$$

这里必须先减 1 再除以 k ；零商无需特殊补偿。输入端点为正 `int`，最终答案不超过矩形面积，能用有符号 64 位保存。分块累加使用 128 位，避免依赖正负中间项的额外界。

双版本以小矩形逐对求 $\gcd$ 为独立参考，另验证百万正方形的 $F(N, N) = 2 \sum_{i=1}^N \varphi(i) - 1$ ，以及按短边剩余类直接计数的 `INT_MAX` 长条。P2522 双版本完整程序验证五万次查询，包括缩放、窄边和无解；普通与 ASan/UBSan 均通过。尚无本库在线 AC。

来源：kuangbin 2.13.2 (BZOJ 2301)；<https://www.luogu.com.cn/problem/P2522>。

181.8 整数分拆：重数限制的差一问题

实现见第 116 页。分拆不区分加数顺序，部分均为正整数。无约束生成函数 $P(z) = \prod_{i \geq 1} (1 - z^i)^{-1}$ ，空分拆给出 $p(0) = 1$ 。五边形数定理为

$$\prod_{i \geq 1} (1 - z^i) = 1 + \sum_{j \geq 1} (-1)^j \left(z^{j(3j-1)/2} + z^{j(3j+1)/2} \right).$$

与 $P(z)$ 相乘后比较系数, 得到 $p(n)$ 的递推, 符号按 $+, +, -, -, +, +, \dots$ 成对出现。每项只依赖更小的下标, 每个 n 有 $O(\sqrt{n})$ 个非零偏移。实现先用 64 位累加, 再归一化到模数范围; 无需逆元, 合数模数与模数 1 均可使用。每个正整数最多出现 k 次的生成函数是

$$\prod_{i \geq 1} (1 + z^i + \dots + z^{ki}) = P(z) \prod_{i \geq 1} (1 - z^{(k+1)i}).$$

因此用已有 p 表与放大 $k+1$ 倍的五边形偏移相乘, 符号为 $-, -, +, +, \dots$ 。本库 `limited(n,k)` 的上限包含 k ; $k=0$ 时非零 n 无解, $k=1$ 为互异分拆。kuangbin 的注释写 “不超过 k 个”, 但其 `solve(n,k)` 使用放大 k 倍的公式, 实际表示每种数少于 k 次, 对应本库 `limited(n,k-1)` ($k \geq 1$)。HDU 原题页面本次无法读取, 不能由这条注释推断原题措辞。

两套实现以任意精度完全背包验证到 1000, 以逐种部分、逐个重数枚举验证有界情况到 100, 覆盖素数与合数模数、0/1 重数限制和大于总和的限制; 另检查 $N = 100000$ 的边界。普通与 ASan/UBSan 均通过, 暂无在线 AC。

来源: kuangbin 2.11; <https://oi-wiki.org/math/combinatorics/partition/>。

181.9 幂的约数和: 不除以 $p-1$

实现见第 113 页。若 $a = \prod p_i^{e_i}$, 则 a^b 的约数和是 $\prod_i (1 + p_i + \dots + p_i^{e_i b})$ 。几何级数的除法公式在模意义下要求分母可逆; 合数模数或 $p \equiv 1$ 时不能直接套用。

令 $G(a, n)$ 同时返回 $P = a^n$ 与 $S = \sum_{i=0}^{n-1} a^i$ 。先递归求一半项数 $h = \lfloor n/2 \rfloor$ 的 (P, S) ; 偶数项的结果为 $(P^2, S(1+P))$, 奇数项再补最后的 $a^{2h} = P^2$, 得到 $(P^2 a, S(1+P) + P^2)$, 全过程取模。 $n=0$ 的结果为 $(1, 0)$, 因此底数为 0 时, 非空级数的和仍为 1。本接口参数是项数, 不是最高指数; 约数和必须传 $e_i b + 1$ 。

把幂与和一起返回, 可使每层只做常数次运算, 总时间 $O(\log(n+1))$ 。长度允许无符号 128 位, 避免 $e_i b$ 在 64 位中溢出; 模数及底数允许完整无符号 64 位。所有模乘与必要的加法在 128 位中执行, 任意正模数均可用。底数 $a \geq 1$, $b=0$ 或 $a=1$ 都返回 $1 \bmod m$; 0 的约数和不在接口定义域内。

双版本以直接枚举约数、逐项几何求和和任意精度矩阵幂独立核验, 包括模数 1、合数模数、完整 128 位项数, 以及指数乘重数超过 64 位的案例。P1593 完整程序各验证 126 组输入, 包含 $p \equiv 0, 1 \pmod{9901}$ 和最大输入; 普通与 ASan/UBSan 均通过, 在线 AC 待补。

来源: kuangbin 2.12; <https://www.luogu.com.cn/problem/P1593>。

181.10 类欧几里德: 有符号整除求和

`NumberTheory::floor_sum(n,m,a,b)` 返回

$$F(n, m, a, b) = \sum_{i=0}^{n-1} \left\lfloor \frac{ai + b}{m} \right\rfloor.$$

传统接口为 `Number_Theory::Floor_Sum`。范围约定 $0 \leq n \leq 10^9$, $1 \leq m \leq 10^9$, a, b 支持完整有符号 64 位, 返回 128 位。 n 是项数; 若题目下标为 $0 \dots N$, 需传 $N+1$ 并检查仍在约定范围内。

先令 $a = q_a m + r_a$, $b = q_b m + r_b$, 其中余数属于 $[0, m)$, 答案先加上 $q_a n(n-1)/2 + q_b n$ 。C++ 的负整数除法向零截断, 因此余数为负时需要把商减 1, 再用 128 位乘法计算规范余数。这一步不能简单用两次取模替代后丢弃的贡献。

对规范化后的 $0 \leq a, b < m$, 设 $y = an + b$ 。若 $y < m$, 余下贡献为 0; 否则交换格点计数的求和方向可得

$$F(n, m, a, b) = F(\lfloor y/m \rfloor, a, m, y \bmod m).$$

下一轮先提取新的整商, 再继续交换。进入交换分支时 $a > 0$, 不会产生零分母。模数按欧几里得过程减小, 时间 $O(\log(m+1))$, 额外空间 $O(1)$ 。

双版本以任意精度向下取整逐项求和对拍, 覆盖负系数、零项数和整除边界; 十亿项的常数、连续整数、反向连续整数及整倍数斜率使用闭式公式独立核验。同时检查模数 1 下两个系数的 64 位极值。普通与 ASan/UBSan 均通过, 暂无在线 AC。本节只验证基本整除和; 下节单独给出带下标权与平方和的接口及证据。

181.11 类欧几里德的三个求和量

实现见第 111 页。令 $y_i = \lfloor (ai + b)/m \rfloor$ ，维护 $f = \sum y_i$ 、 $g = \sum iy_i$ 、 $h = \sum y_i^2$ ，下标均为 $0 \leq i < n$ 。结果固定模 998244353，不要更换模数却保留 2、6 的逆元常数。

先把 $a = xm + a'$ 、 $b = ym + b'$ 归一化；设规范化问题返回 (f_0, g_0, h_0) ，并记 $S_1 = \sum i$ 、 $S_2 = \sum i^2$ ，则

$$\begin{aligned} f &= f_0 + xS_1 + yn, \\ g &= g_0 + xS_2 + yS_1, \\ h &= h_0 + x^2S_2 + y^2n + 2xyS_1 + 2xg_0 + 2yf_0. \end{aligned}$$

负系数的商仍须向下取整。规范化后的 $a' = 0$ 时三个剩余量均为 0。

对 $0 < a' < m$ ，令 $t = \lfloor (a'(n-1) + b')/m \rfloor$ 。高度 $j = 1 \dots t$ 首次出现的下标为 $r_j = \lceil (mj - b')/a' \rceil$ 。以 $q = j-1$ 编号，递归参数为 $(t, a', m, m + a' - b' - 1)$ ；设返回 (u, v, w) ，即 $u = \sum r_j$ 、 $v = \sum (j-1)r_j$ 、 $w = \sum r_j^2$ ，则

$$f_0 = nt - u, \quad g_0 = tS_1 - (w - u)/2, \quad h_0 = nt^2 - 2v - u.$$

这来自按高度计算每一层的贡献，其中 $y_i^2 = \sum_{j=1}^{y_i} (2j-1)$ 。分母每次严格减小，所以只有 $O(\log(m+1))$ 层递归。

P5170 的上界包含 n ，驱动传入 $n+1$ ；题目输出顺序为 f, h, g ，与本接口不同。核心函数支持十亿加一项，并对完整有符号 64 位系数做任意精度逐项对拍；大项数用一次多项式的和与平方和闭式检查。两套完整程序各验证十万组输入，检查最大闭区间端点与全部三个输出。普通与 ASan/UBSan 均通过，在线 AC 待补；任意更高次矩尚不在这个接口内。

补充的大参数测试包含 3000 组随机参数、64 位系数端点和斐波那契型欧几里得链。反转 $i \mapsto n-1-i$ 应保持 f, h 不变，并使 g 变为 $(n-1)f - g$ 。将每项替换为 $-y_i - 1$ ，三个结果应变为 $-f - n$ 、 $-g - S_1$ 、 $h + 2f + n$ 。相邻前缀还应满足增量 (y_n, ny_n, y_n^2) ，其中新增项以任意精度计算。第一结果另与独立接口 `floor_sum` 交叉核对。这些恒等式测试是大参数一致性证据，与前述逐项独立答案对拍区分记录；普通与 ASan/UBSan 均通过。

来源与题目：<https://www.luogu.com.cn/problem/P5170>。

181.12 模运算的适用条件

模逆元存在当且仅当 $\gcd(a, m) = 1$ 。费马逆元 a^{p-2} 仅适用于素数模数及 $a \neq 0$ 。对正模数 m, n ，同余式 $x \equiv a \pmod{m}$ 、 $x \equiv b \pmod{n}$ 可合并当且仅当 $\gcd(m, n) \mid (b - a)$ ；新模数为 $\text{lcm}(m, n)$ 。

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p} \quad (p \text{ 为素数, } n_i, k_i \text{ 是 } p \text{ 进制位}).$$

Lucas 不能直接套在合数模数。阶乘与逆阶乘预处理要求上限小于模数；超过模数会出现阶乘为零。扩展欧拉降幂中，非互质情况不能直接将指数对 $\varphi(m)$ 取模；需要判断原指数是否达到足够阈值。大整数指数可以在读入时同时维护余数与是否超过阈值。

181.13 任意数组的批量逆元

设 $s_i = \prod_{j < i} a_j \pmod{m}$ ， $s_0 = 1$ 。仅对总乘积 s_n 求一次逆元，随后从右向左处理。若当前 $t = s_{i+1}^{-1}$ ，则

$$a_i^{-1} = ts_i \pmod{m}, \quad s_i^{-1} = ta_i \pmod{m}.$$

先保存原 a_i 再覆盖输出，避免误用已写入的逆元更新 t 。总乘积与 m 互素当且仅当每一项均与 m 互素，因此一次扩展欧几里得也能判断整个批次是否可行；模数无需为素数。实现见第 103 页。

接口返回可选向量：无解是空的可选值，空输入是有效的空向量，两者不同。模数为 1 时按接口约定返回全零。调用者可以移动不再使用的输入数组，避免额外复制。P5431 的驱动程序计算 $\sum_{i=1}^n k^i/a_i$ ，注意权重从 k^1 开始。五百万项规模采用缓冲的字符读取，并移动输入以控制峰值内存。双版本已做乘法证书和非互素拒绝的穷举检查，以及 64 位极值、空数组、模数 1 的检查；完整驱动另用逐项逆元和几何级数进行对拍。P5431 双版本均通过

全部 5 个测试点；动态版记录 297561197，传统版记录 297561804。最慢单点分别为 386、384 毫秒，峰值内存均为 76.80 MB。合数模数、不可逆输入和完整 64 位范围以上本地测试验证。

该方法可处理任意可逆数组；连续整数的专用递推见下一节。

来源：<https://oi-wiki.org/math/number-theory/inverse/>。

181.14 连续整数的线性逆元表

素数模数 p 下，对 $1 < i < p$ ，写 $p = qi + r$ ，其中 $q = \lfloor p/i \rfloor$ ， $r = p \bmod i$ 。 r 非零，且 $r < i$ 。将等式模 p 后乘以 $i^{-1}r^{-1}$ ，得到

$$i^{-1} \equiv -\lfloor p/i \rfloor (p \bmod i)^{-1} \pmod{p}.$$

从 $1^{-1} = 1$ 开始递推，右侧总是已算过的较小下标，故总时间为 $O(N)$ 。实现见第 104 页；第 0 项只作占位，允许 $N = 0$ 。 $N \geq p$ 时区间包含无逆元的 p ，不能套用该表。

递推还依赖 r 可逆，不能仅凭 i 与合数模数互素就照搬。例如 $i = 3, m = 8$ 时 $r = 2$ 不可逆。kuangbin 的“简洁写法”递归式也有这一限制，本书以一般扩展欧几里得接口代替其不成立的合数推广。若 $\gcd(a, m) = 1$ ，扩展欧几里得给出 $ax + my = 1$ ，归一化 x 即为逆元；若不互素，则整数方程无解。素数下也可用 a^{p-2} 求逆元；kuangbin 标为“利用欧拉函数”的代码实际使用这一费马形式。

P3811 双版本完整程序逐行输出逆元，已在本地对三百万行结果逐项检验 $ix_i \equiv 1$ 。P3811 双版本均通过全部 6 个测试点，单点最长分别为 221 ms 与 219 ms。单个逆元另用小模数穷举与费马乘法证书交叉验证；这部分不由 P3811 的 AC 覆盖。

来源：<https://oi-wiki.org/math/number-theory/inverse/>；WIDA 数论逆元节与 kuangbin 2.4。

181.15 一次不定方程：特解与正整数解范围

实现见第 100 页。对不全为零的 a, b ，令 $g = \gcd(|a|, |b|) > 0$ 。递归扩展欧几里得先求 $|a|u + |b|v = g$ ；当且仅当 $g \mid c$ 时有整数解。把系数乘以 c/g ，再恢复 a, b 的符号，就得到 $ax_0 + by_0 = c$ 。两个特解相减，再利用 $a/g, b/g$ 互素，可知所有解恰为

$$x = x_0 + \frac{b}{g}t, \quad y = y_0 - \frac{a}{g}t, \quad t \in \mathbb{Z}.$$

若 $a = b = 0$ ，则 $c = 0$ 时所有整数对都是解，否则无解；此时不能使用除以 g 的通解式。

底层 `exgcd` 的本库调用约定为非负系数，上限 2^{63} ；返回非负的 g ， $(0, 0)$ 约定返回 0。包装函数先转为 128 位再取相反数，因而可以处理 `LLONG_MIN`。缩放后的特解也用 128 位保存；不要提前转回 `long long`。即使特解可表示，直接验证 $ax + by$ 的两个乘积仍可能超过 128 位，本地验证使用任意精度整数。

对 P5656 的正系数 a, b, c ，设 $d_x = b/g, d_y = a/g$ ，先将 x_0 调整到 $x_{\min} \in [1, d_x]$ ，并计算与它配对的 $y_{\max} = (c - ax_{\min})/b$ 。若 $y_{\max} > 0$ ，正整数解个数为

$$N = \left\lfloor \frac{y_{\max} - 1}{d_y} \right\rfloor + 1, \quad x_{\max} = x_{\min} + (N - 1)d_x, \quad y_{\min} = y_{\max} - (N - 1)d_y.$$

否则不存在同时为正的解，但 x 和 y 仍各自有最小正值；分别按周期归一化即可，它们通常不是同一组解。正值归一化使用 $(v \% d + d - 1) \% d + 1$ ，不要把余数 0 当作正整数。驱动中 $a, b, c \leq 10^9$ ，归一化后的乘积不超过 10^{18} ，故该部分可用 64 位。

双版本以小范围有符号输入穷举、任意精度等式证书、64 位端点和斐波那契递归链验证。完整 P5656 驱动另验证 200000 组输入，包含正解数量与四个极值、无解和无正解分支；普通与 ASan/UBSan 测试均通过，在线 AC 待补。

来源：kuangbin 2.3；WIDA 数论打印稿的 `exgcd` 与不定方程例题。

<https://oi-wiki.org/math/number-theory/bezouts/>。

181.16 线性同余：最小解与最小周期

实现见第 101 页，接口约定为 $ax + b \equiv 0 \pmod{m}$ 。先把 a 归一化到 $[0, m)$ ，令 $g = \gcd(a, m)$ 。有解当且仅当 $g \mid b$ ；若 $au + mv = g$ ，则一个特解为 $u(-b/g)$ ，所有解为 $x = x_0 + (m/g)t$ 。归一化到 $[0, m/g)$ 后得到最小非负解。除以 g 后 a/g 与 m/g 互素，因而 m/g 是最小正周期。当 $a = 0$ 且 $m \mid b$ 时所有整数均为解，返回 $(0, 1)$ ；否则无解。模数 $m = 1$ 始终返回 $(0, 1)$ 。

输入常数项支持 `LLONG_MIN`，内部用 `-i128(b)`，不能先执行 64 位的 `-b` 再转换。若题目给出 $ax \equiv c \pmod{m}$ ，可先将 c 归一化到 $[0, m)$ 再取负传入，避免 $c = \text{LLONG_MIN}$ 时在调用点溢出。本接口依赖的 `exgcd` 与前节相同，不必重复抄写。

P1516 中相遇条件为 $(m - n)t + (x - y) \equiv 0 \pmod{L}$ 。传入速度差与起点差，第一返回值就是最早非负相遇时间；失败时输出 `Impossible`。不能擅自把负的速度差取绝对值，也不能把两者同余的初始位置强制判为需要跳跃。

小模数测试逐个枚举全部剩余类，检查解集、最小解和最小周期；大数用任意精度等式证书验证。完整驱动以直接模拟、交换青蛙和构造大周期最早相遇时间验证 242 组输入，包括等速和初始位置同余。两套代码均通过普通与 ASan/UBSan 测试，在线 AC 待补。

来源：WIDA 收集的 jiangly `exgcd` 文件内 `sol`；<https://www.luogu.com.cn/problem/P1516>。

181.17 扩展 CRT：逐式合并

kuangbin 2.5 的 `solve` 对应单次合并，MLES 对应下面的逐式循环。实现见第 93 页的 `crt`，传统版对应 `Crt`。初始取 $r = 0, m = 1$ ，依次合并 $x \equiv b \pmod{n}$ 。空方程组保持 $(0, 1)$ ，表示任意整数；一旦无解即停止，不能忽略失败继续合并。先把余数归一化。设 $g = \gcd(m, n)$ ，由扩展欧几里得得到 $mu + nv = g$ 。写 $x = r + mt$ ，原问题化为 $mt \equiv b - r \pmod{n}$ 。若 $g \nmid (b - r)$ 则无解，否则

$$q = n/g, \quad t \equiv \frac{b-r}{g}u \pmod{q}, \quad 0 \leq t < q.$$

新周期为 mq ，新余数为 $(r + mt) \bmod (mq)$ 。所有可行 t 相差 q ，故新余数加新周期的任意整数倍恰好是全部解；逐式合并时这一不变量保持成立。模数不必互素，模数 1 也可直接参与。

参考实现使用 `int` 保存合并状态，本库扩展为 64 位并显式报告周期越界；避免可能溢出的 `abs(a-a0)`。

代码要求正模数且最终周期不超过有符号 64 位上限；中间乘法与扩展欧几里得系数使用 128 位整数。无解返回假；有解而周期越界抛出异常。失败时旧周期不变，旧余数已归一化。

k 条方程的时间为 $O(k \log M)$ ，其中 M 是输入及中间模数的上界；递归欧几里得使用 $O(\log M)$ 栈空间。

P4777 的完整程序按“模数、余数”读入，题目保证有解且周期不超过 10^{18} 。本地验证包含双版本小模数穷举、任意精度整数对拍、有符号极值、无解与越界，以及十万条同余式的完整程序测试。另穷举至多三个小模数方程组成的所有有序系统，检查空系统与后续矛盾。双版本均已通过 P4777 全部 15 个测试点；动态版记录 297521102，传统版记录 297521173。

kuangbin 的“解同余方程组”是模 7 的线性方程组消元应用，见后面的工厂工期一节。来源：<https://oi-wiki.org/math/number-theory/crt/>；WIDA 数论模板的 `exCRT` 节。

181.18 Garner：总周期可以超过整数范围

设正模数 m_i 两两互素， $P = \prod_i m_i$ 。经典构造为 $x = \sum_i b_i P_i (P_i^{-1} \bmod m_i) \bmod P$ ，其中 $P_i = P/m_i$ 。模 m_i 时其他项都为零，本项为 b_i ；两解之差被所有 m_i 整除，因而被 P 整除。但直接写此式需要存储整个 P ，并防止乘法溢出。

Garner 改用混合进制

$$x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots, \quad 0 \leq d_i < m_i.$$

在确定第 i 位前，对每个后续模数维护已确定部分的余数 v_j ，以及当前位权 $c_j = \prod_{t < i} m_t \bmod m_j$ 。于是

$$d_i = (b_i - v_i) c_i^{-1} \bmod m_i.$$

把 $c_j d_i$ 加到 v_j , 再将 c_j 乘 m_i 即可进入下一位。互素性保证逆元存在; 模数 1 的位固定为 0, 可直接保留。位数范围保证重建的 x 恰在 $[0, P)$, 因此就是最小非负解。实现见第 105 页。

若只需 $x \bmod T$, 从高位向低位做 Horner 计算: 每步将当前值乘对应模数, 加上该位, 再对 T 取模。 T 无需与 m_i 互素, 因为这里不对 T 求逆。这返回的是规范解 x 的余数; 一般解 $x + kP$ 对 T 的余数可能不同。例如模 3 余 2、模 5 余 3, 规范解是 8; 对 7 取模为 1, 但另一个解 23 的余数是 2。若用多模 NTT 恢复整数系数, 仍须证明真实系数位于所选规范区间, 不能在乘积不够大时直接把 CRT 的代表元当成原整数。

双版本本地验证使用任意精度整数的经典 CRT 求和作为独立对拍, 检查混合进制的完整重建、负余数、有符号极值、非互素输入拒绝和 1000 个模数。P1495 完整程序按题目保证的 $P \leq 10^{18}$ 输出规范解, 双版本均通过全部 11 个测试点; 动态版记录 297521594, 传统版记录 297521658。超出该题范围的能力以上述本地测试验证。来源: <https://oi-wiki.org/math/number-theory/crt/>。

181.19 素数模平方根

奇素数 p 下, 对 $n \not\equiv 0 \pmod{p}$, Euler 判别式 $n^{(p-1)/2} \equiv 1 \pmod{p}$ 当且仅当存在平方根, 否则为 -1 。当 $p \equiv 3 \pmod{4}$ 且有解时, $r = n^{(p+1)/4}$ 为一根, 另一根是 $p - r$ 。

Cipolla 选择 a 使 $w = a^2 - n$ 是非平方, 令 $\omega^2 = w$ 。在扩域中取 $\alpha = a + \omega$, 有 $\alpha^p = a - \omega$, 故 $\alpha^{p+1} = n$ 。令 $\beta = \alpha^{(p+1)/2}$, 则 $\beta^2 = n$ 且 $\beta^{p-1} = n^{(p-1)/2} = 1$, 所以 β 落在原素数域中。实现见第 122 页; 零和模数 2 单独处理。模数为合数时不能照搬此判别和扩域构造。

181.20 乘法阶与原根

对 $\gcd(a, n) = 1$, 阶是最小的正整数 d 使 $a^d \equiv 1 \pmod{n}$, 且 $d \mid \varphi(n)$ 。从 $d = \varphi(n)$ 开始, 对每个素因子 q , 只要 $a^{d/q} \equiv 1$ 就继续除去 q , 所得即为实际阶。对于不可逆的 a , 接口返回 0, 表示此处没有定义乘法阶。

原根要求阶达到 $\varphi(n)$ 。只需对其每个不同素因子 q 检查 $a^{\varphi(n)/q} \not\equiv 1$: 若实际阶是真因子, 商中必含某个这样的 q 。必须先检查 $\gcd(a, n) = 1$; 例如模 9 的候选 3 虽通过上述非 1 判定, 却不是原根。 $n \geq 2$ 时, 原根恰存在于 $2, 4, p^k, 2p^k$, 其中 p 是奇素数。原根存在且取一个原根 g 后, g^t 的阶是 $\varphi(n)/\gcd(t, \varphi(n))$, 所以枚举与 $\varphi(n)$ 互素的指数即可得到全部原根, 共 $\varphi(\varphi(n))$ 个; 这些数按大小输出时还需排序。

实现见第 107 页。本地验证穷举小模数的幂循环, 与双版本的阶、最小值和完整集合逐项比较; 另验证大素数、奇素数幂、无原根合数和有符号输入边界。P6091 的完整程序输出排序后第 $d, 2d, \dots$ 个原根, 零个输出时仍保留空行。双版本均通过全部 11 个测试点; 动态版记录 297558740, 传统版记录 297559082。求阶、最小原根的最小性及更大输入范围仍以上述本地证据验证。若 $d = \text{ord}_n(a)$, 则 $\text{ord}_n(a^k) = d/\gcd(d, k)$ 。两个可逆元素乘积的阶整除各自阶的最小公倍数; 两阶互素时, 乘积的阶等于两阶之积。来源: <https://oi-wiki.org/math/number-theory/primitive-root/>; kuangbin 2.17。

181.21 Carmichael: 最小公共指数

$\lambda(n)$ 是模 n 的所有可逆元素之阶的最小公倍数, 约定 $\lambda(1) = 1$ 。它整除 $\varphi(n)$, 但可以严格更小, 例如 $\lambda(8) = 2$ 而 $\varphi(8) = 4$ 。按互素的素数幂分解 $n = \prod_i p_i^{e_i}$ 后, 由 CRT, 对每个分量同时成立的幂等式也对 n 成立, 因此

$$\lambda(n) = \text{lcm}_i \lambda(p_i^{e_i}), \quad \lambda(p^e) = \begin{cases} 2^{e-2}, & p = 2, e \geq 3, \\ p^{e-1}(p-1), & \text{其他情形} \end{cases}$$

奇素数幂的单位群有原根, 公共指数即其大小。模 2^e 的奇数可表示为 $\pm 5^k$, 当 $e \geq 3$ 时 5 的阶为 2^{e-2} , 这给出二幂情形的上界和达到上界的元素。实现见第 115 页, 试除版本适用于有符号 32 位正模数。

对可逆底数可把指数对 $\lambda(n)$ 取模; 非互素底数仍须另行处理。例如模 8 下 $2^2 = 4$, 但 $2^{2 \bmod \lambda(8)} = 1$, 直接降幂会错误。另有 $\lambda(\text{lcm}(a, b)) = \text{lcm}(\lambda(a), \lambda(b))$, 这也提供了独立的组合检查条件。

合数 n 是 Carmichael 数当且仅当 $\lambda(n) \mid (n-1)$ 。等价的 Korselt 判别是： n 无平方因子，且每个素因子 p 都满足 $(p-1) \mid (n-1)$ 。素数也满足前一整除式，判定时必须排除素数。这些合数对所有互素底数都通过 Fermat 判定，不能用单纯的 Fermat 检验代替素性判定。无平方因子再配合 CRT，也使所有整数 a 满足 $a^n \equiv a \pmod{n}$ 。

双版本以小模数的全部实际阶之最小公倍数对拍，另检验大二幂、奇素数幂、32 位边界及公共指数证书。

UVa10006 完整程序对全部合法输入进行本地 Korselt 对拍；在线 AC 待补。

来源：<https://oi-wiki.org/math/number-theory/primitive-root/>；

应用题：<https://onlinejudge.org/external/100/10006.pdf>。

181.22 Lucas：按素数进制逐位计算

在特征为素数 p 的域中， $(1+x)^p = 1+x^p$ 。若 $n = \sum_i n_i p^i$ ，则

$$(1+x)^n = \prod_i (1+x^{p^i})^{n_i}.$$

每个因子选择的次数在 $0, \dots, n_i$ 之间，严格小于 p ，因此位数表示唯一。比较 x^k 的系数即得 $\binom{n}{k} = \prod_i \binom{n_i}{k_i} \pmod{p}$ ；若某位 $k_i > n_i$ ，答案立即为零。仅预处理 $0!, \dots, (p-1)!$ 及其逆元，这些数均非零。 $p!$ 为零，不能把阶乘表延长后继续用除法公式。实现见第 117 页。

预处理后每位只做常数乘法，适合固定小素数模数下的多次查询。接口接收 n, k ，不接收“ n, m 表示选 n 个”的隐含约定；P3807 的驱动程序显式调用 $\text{choose}(n+m, n)$ 。题目范围内求和安全，完整 64 位输入自行求和时仍要检查溢出。本地验证包含 Pascal 三角、精确大整数、小模数重建和最大无符号 64 位下标；P3807 双版本均已通过全部 25 个测试点；动态版记录 297556209，传统版记录 297556596。完整 64 位参数及额外边界以上述本地对拍验证。来源：<https://oi-wiki.org/math/number-theory/lucas/>。

181.23 合数模数组组合数与插值

对素数幂 $q = p^e$ ，记 $v_p(n!) = \sum_{j \geq 1} \lfloor n/p^j \rfloor$ ， $U_p(n) = n!/p^{v_p(n!)}$ 。设 $t = v_p(n!) - v_p(k!) - v_p((n-k)!)$ 。当 $t \geq e$ 时组合数模 q 为零；否则

$$\binom{n}{k} \equiv U_p(n) U_p(k)^{-1} U_p(n-k)^{-1} p^t \pmod{q}.$$

这里仅对与 p 互素的单位部分求逆。各素数幂上的结果通过 CRT 合并，对应第 119 页的实现。

在素数域中， n 个模 p 两两不同的横坐标确定唯一的次数小于 n 的多项式。定义 $w_i = y_i / \prod_{j \neq i} (x_i - x_j)$ ，则

$$f(k) = \sum_{i=0}^{n-1} w_i \prod_{j < i} (k - x_j) \prod_{j > i} (k - x_j).$$

该式不除以 $k - x_i$ ，查询点命中样本点时仍然成立。当 $x_i = i$ 时，分母为 $i!(n-1-i)!(-1)^{n-1-i}$ ，要求 $n \leq p$ 。实现见第 131 页。对整数幂前缀和 $S_d(k) = \sum_{i=1}^k i^d$ ，当 $0 \leq d \leq p-2$ 时，它在模 p 意义下可用次数至多 $d+1$ 的多项式表示；取 $0, 1, \dots, d+1$ 的样本即可。不能只因某序列前几项能插值，就断言后续仍由该多项式给出。

181.24 阶乘表：组合数与排列数

当素数模数为 p 且 $0 \leq k \leq n < p$ 时，阶乘均非零，因此

$$C(n, k) = n! (k!)^{-1} ((n-k)!)^{-1}, \quad P(n, k) = n! ((n-k)!)^{-1} \pmod{p}.$$

将表从旧上限扩展到 N ，先正向补齐阶乘，只对 $N!$ 求一次逆元，再用 $(i-1)!^{-1} = i (i!)^{-1}$ 反向补齐新逆阶乘项即可。不需要每一项分别快速幂求逆。较小上限的初始化请求保留已有表。查询前显式扩表，之后查询为常数时间；内部两表重分配时旧元素引用不能继续使用。实现见第 99 页。

WIDA 的“组合数 (Lucas)”文件实际提供的是普通阶乘、逆阶乘、组合数和排列数，没有逐位 Lucas 递推。本书以此节覆盖该文件的数学功能，用显式扩表取代查询时自动扩表，适用范围仍须限制在模数以内。验证使用独立 Pascal 三角和下降阶乘，另检查百万项表及扩表前后的原查询结果。

181.25 组合计数

容斥: $|\bigcup A_i| = \sum_{\emptyset \neq S} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$ 。二项式反演: 若 $b_n = \sum_{k=0}^n \binom{n}{k} a_k$, 则 $a_n = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} b_k$ 。

$$\sum_k \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n}, \quad C_n = \binom{2n}{n} - \binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免直接除以 $n+1$, 在模数下尤其有用, 但组合数本身仍须正确求模。第二类 Stirling 数满足

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}, \quad S(0,0) = 1.$$

第一类无符号 Stirling 数把系数 k 换为 $n-1$ 。

球与盒	空盒允许	空盒禁止
不同球, 不同盒	k^n	$k!S(n,k)$
相同球, 不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球, 相同盒	$\sum_{j=0}^k S(n,j)$	$S(n,k)$

上表默认 $k \geq 1$; $n=0, k=0$ 等边界应按原问题约定单独处理。

181.26 群作用计数

有限群 G 作用在有限集合 X 上时, 轨道数为

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

对位置置换的 q 色染色, $|\text{Fix}(g)| = q^{c(g)}$, 其中 $c(g)$ 为循环数。环形项链只商去旋转; 手链还商去反射, 二者群不同, 不能共用分母。模数整除 $|G|$ 时不能使用模逆元, 须先在合适的整数模数下求和再做精确除法。

181.27 生成函数与递推

普通生成函数 $A(x) = \sum a_n x^n$ 的乘法对应普通卷积。指数生成函数 $A(x) = \sum a_n x^n / n!$ 的乘法对应 $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$ 。形式幂级数求逆要求常数项可逆; $\log A$ 取 $A(0) = 1$, $\exp A$ 取 $A(0) = 0$ 。有限域中积分涉及 $1/i$, 必须避免 i 被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2^m}}, \quad (\log A)' = A'/A.$$

BM 从样本得到最短线性递推。要对未来项使用, 必须另外证明序列确实具有不超过相应阶数的线性递推; 少量数值吻合不是证明。矩阵转移、有限状态计数常可提供阶数上界。

181.28 线性代数与生成树

域上的线性方程组一致时, 解集为一个特解加零空间, 零空间维数为未知数个数减秩。浮点消元中的秩依赖误差阈值; 模素数消元是精确域运算; 合数模数下不可随意除以非零元。无向图 Laplacian 的对角元为度数, 非对角元为负的边重数。删去同一行列后的行列式为生成树数。自环不参与生成树; 重边需要累加。只有一个顶点时空余子式行列式约定为 1。有向图需要重新明确入度/出度 Laplacian 与根向内/向外树的方向, 不能直接复用无向封装。

181.29 模线性方程组：工厂工期

POJ2947 (同题 UVa1564) 给出每位工人制造的产品类型和起止星期。设第 j 种产品需 x_j 天，第 i 条记录中出现 a_{ij} 次，以星期一为 0、星期日为 6，则

$$\sum_j a_{ij} x_j \equiv \text{finish}_i - \text{start}_i + 1 \pmod{7}.$$

起止两天均计入；同一种产品重复出现必须累加系数。跨星期无需特判，模数类型负责归一化负数。每个余数在 $[3, 9]$ 中恰有一个代表，因此模方程的解数就是原问题的解数。

直接使用第 148 页的模高斯消元，模数取 7。增广矩阵有 m 行、 $n+1$ 列，最后一列是右端项。先判断 **consistent**：假时输出无解；为真且秩小于 n 时输出多解；否则读取特解，余数小于 3 的分量加 7。不能先以秩不足判断多解，因为零系数行仍可能有非零右端项。本题所有非零系数均可逆；合数模数不具备这一性质，不能直接套用此消元。

完整双版本程序位于 `verify/poj/2947.*.cpp`。验证包含所有起止星期组合、产品重复 7 次导致系数归零、矛盾的零行、独立枚举所有 3~9 天赋值，以及用可逆行变换构造的 300 阶稠密系统。该应用验证覆盖分类及唯一解输出，不代替零空间基等其他接口的验证。来源：<https://onlinejudge.org/external/15/1564.pdf>；kuangbin 2.10.2。

181.30 概率、期望与博弈

期望线性性不要求独立： $\mathbb{E}[\sum X_i] = \sum \mathbb{E}[X_i]$ 。方差可加需要协方差为零；独立是充分条件。非负整数变量 $\mathbb{E}[X] = \sum_{k \geq 1} \Pr(X \geq k)$ 。吸收过程的期望方程 $E_i = 1 + \sum_j p_{ij} E_j$ 需要检查是否几乎必然吸收及期望是否有限；不能对闭合非吸收分量直接求唯一有限解。有限无环公平组合游戏在正常玩法下

$$\text{SG}(u) = \text{mex}\{\text{SG}(v) : u \rightarrow v\}.$$

独立子游戏的 SG 值异或为零当且仅当先手必败。反常玩法、含环游戏、双方可走步不同的游戏不能直接套这个结论。

181.31 数值算法与几何

排序必须使用严格弱序；不能以“差小于 eps 则相等”定义通用排序比较器。叉积与行列式容易发生抵消；整数题优先精确整数判定，浮点构造单独处理误差。对于简单格点多边形（无孔），Pick 定理 $A = I + B/2 - 1$ ，边界格点数 $B = \sum \gcd(|\Delta x|, |\Delta y|)$ ；不要重复计入边端点。自适应积分的局部误差估计需要函数足够规则；奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性，整数域最后枚举剩余区间；二分答案要求判定单调性。

来源索引 本章为重新组织的公式速查，扩展范围依据 <https://oi-wiki.org/math/>。小模数 exLucas 与有限域拉格朗日插值已有双风格实现和本地验证；素数模平方根已有 Cipolla 实现；仍需补齐二次剩余的其他方法与定理、Min25、Newton 插值、子集卷积、行列式进阶、数值积分等内容及验证。本章公式不能代替这些待完成代码。

181.32 线性递推：系数方向与外推条件

约定 $a_i = \sum_{j=1}^k c_{j-1} a_{i-j}$ ，初值是 $a_0, \dots, a_{k-1}$ 。对应特征多项式为

$$F(x) = x^k - c_0 x^{k-1} - c_1 x^{k-2} - \dots - c_{k-1}.$$

若 $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$ ，则 $a_n = \sum_{i=0}^{k-1} r_i a_i$ 。代码由高次向低次消去多项式项，第 j 个系数写入原次数减 j 的位置，避免倒置系数。例如 Fibonacci 初值为 $[0, 1]$ ，递推系数为 $[1, 1]$ 。

BM 在域上求有限前缀的最短递推。本书实现要求素数模数；一般的模合数环不能保证非零偏差存在逆元。仅凭若干样本无法证明真实序列以后一直遵循同一个规律。若事先知道序列满足阶数至多 d 的齐次线性递推，至少给出前 $2d$ 项，再用恢复出的系数求后续项。系数向量末尾的零不能擅自删除：它可能对应非平稳的初始段，改变阶数会改变递推开始适用的位置。

181.33 素数模高次剩余与单位根

实现见第 124 页；依赖本书离散对数、线性同余和快速幂。设 p 为素数， g 是原根， $k > 0$ 。 $a = 0$ 时根只有零；否则写 $a = g^b$ 、 $x = g^y$ ，方程转为

$$ky \equiv b \pmod{p-1}.$$

令 $d = \gcd(k, p-1)$ 。当 $d \nmid b$ 时无解；否则指数解为 $y = y_0 + i(p-1)/d$ ， $0 \leq i < d$ 。因此用三个数存全部根：

$$\text{first} = g^{y_0}, \quad \text{ratio} = g^{(p-1)/d}, \quad \text{count} = d.$$

依次乘 ratio 即可枚举；不保证数值递增，也不保证 first 是最小根。ratio 是阶为 d 的单位根，因而枚举恰有 d 个不同的根。模 $p = 2$ 同样有效；指数同余的模数是 1，唯一非零根为 1。

接口支持 $p \leq 10^{12}$ ，需要调用者提供已验证的素数与原根； k 支持完整 unsigned long long，先对 $p-1$ 取模后解同余。 a 支持完整 signed long long，先正规化。返回空 optional 表示无解，返回 $\{0, 1, 1\}$ 表示唯一零根。给定原根后的期望时间、空间为 $O(\sqrt{p})$ ；枚举全部根另外需要 $O(d)$ 时间。模乘须先扩展到 128 位，不可用 64 位乘积直接取模。

测试穷举小于 100 的每个素数、全部原根及小指数，用任意精度模幂逐个枚举候选根并比较整个集合；覆盖负数、64 位边界、模 2、无解与零根。另检查 998244352 个根的紧凑表示，以及素数 999999999989 上的植根测试；后一个素数与原根的条件由试除及乘法阶证书独立确认。普通与 ASan/UBSan 均通过，尚无专门在线 AC。

这是 [OI Wiki 高次剩余](#) 中离散对数转线性同余方法的素数模实现。素数幂及非互素被开方数见下一节；一般合数模数见后文；改良 Tonelli-Shanks 尚需另补，不能把本节状态解读为该页面全部覆盖。

181.34 素数幂模高次剩余与根的提升

实现见第 125 页。令 $M = p^e$ 。当 $a = 0$ 时，全部根是 $p^{\lceil e/k \rceil}$ 的倍数。否则令 $t = v_p(a) < e$ ，必须有 $k \mid t$ ，否则无解。取 $s = t/k$ ， $q = p^{e-t}$ ，写 $x = p^s y$ ，只需求单位方程

$$y^k \equiv a/p^t \pmod{q}.$$

每个单位根 y 对应原模数下的根

$$x = p^s y + \ell p^s q, \quad 0 \leq \ell < p^{t-s}.$$

这些提升保持互不相同，也解释了非互素被开方数可能产生大量根。

奇素数时，提供模 p^e 的原根，约化后仍是模 q 的原根，沿用离散对数转线性同余的方法。 $q = 2$ 时单位根只有 1； $q \geq 4$ 且为二的幂时，单位表示为 $(-1)^z 5^y$ 。符号指数模 2，5 的指数模 $q/4$ ； $q = 4$ 时只有符号分支。分别解两个指数同余即可，不可假设二的高次幂存在原根。

返回结构体保存 first、两个 ratio、两个 count，以及 step、lifts。全部根由下式给出：

$$(\text{first} \cdot \text{ratio}[0]^i \cdot \text{ratio}[1]^j \bmod \text{step}) + \ell \cdot \text{step}.$$

其中 $0 \leq i < \text{count}[0]$ 、 $0 \leq j < \text{count}[1]$ 、 $0 \leq \ell < \text{lifts}$ 。get 已处理模乘溢出，size 返回三者乘积；不保证根按数值排序。零根情形复用同一表示。

模数上限 10^{12} ， k 支持完整无符号 64 位。只检查范围，不检查 p 的素性和奇素数原根的阶，调用者须保证。默认参数 $g = 0$ 仅用于 $p = 2$ ；奇素数不能漏传原根。时间和空间由 BSGS 决定，期望 $O(\sqrt{q})$ ；单次按索引取根使用快速幂，需 $O(\log M)$ 。

已穷举多个素数的全部不超过 256 的素数幂、所有被开方数与小指数，用任意精度模幂直接枚举所有根，比较整个集合。另测有符号与无符号 64 位边界、 2^{39} 的非平凡离散对数、 3^{20} 的非互素方程，以及数万亿根的紧凑表示。普通与 ASan/UBSan 均通过；尚无专门在线 AC。一般合数的 CRT 组合见下一节；改良 Tonelli-Shanks 仍未完成，OI Wiki 页面继续标记部分覆盖。

181.35 一般合数模高次剩余：CRT 组合

实现见第 129 页。调用者提供完整分解 $M = \prod_{t=0}^{F-1} m_t$ ，其中 $m_t = p_t^{e_t}$ 两两互素，并提供每个奇素数幂的原根。先分别求局部根族；任一局部无解，整体即无解。若局部根数为 c_t ，则整体根数为 $\prod_t c_t$ 。只保存局部根族，不创建这个大小的数组。

对每个因子预处理

$$E_t = \frac{M}{m_t} \left(\left(\frac{M}{m_t} \right)^{-1} \bmod m_t \right) \bmod M.$$

对一组局部根 r_t ，唯一对应的标准代表是

$$x = \sum_t r_t E_t \bmod M.$$

逆元存在是因为因子两两互素。`get(index)` 将索引按各个 c_t 转成混合进制数字，再把局部索引拆成两个乘法根族索引和加法提升索引，最后用上述公式组合。CRT 双射保证不重复、不遗漏，输出顺序不保证递增。

$M \leq 10^{12}$ ， $k > 0$ ；空因子列表表示 $M = 1$ ，返回唯一代表 0。`mod` 保存模数，`total` 保存总解数，`parts` 和 `weight` 保存局部根族与 E_t 。模数乘积检查上界，CRT 的乘积先扩展至 128 位。单次按索引查询为 $O(F \log M)$ ；求解的主要时间和空间为局部 BSGS，因子列表接口不包含整数分解与原根搜索的成本；自动准备重载见下一节。

已对 $1 \leq M \leq 180$ 、所有被开方数与小指数直接枚举所有根，与两套代码的完整输出集合比较，并反转因子顺序重新检查。另覆盖有符号与无符号 64 位边界、混合素数幂上的非互素方程和大量零根，以及十一种不同素因子组成模数时的全部 1024 个平方根。普通与 ASan/UBSan 均通过；尚无专门在线 AC。至此已有已知分解及原根时的 BSGS 求根路线，更快的改良 Tonelli-Shanks 等内容仍未覆盖，OI Wiki 条目保留部分完成状态。

181.36 自动准备模数分解与原根

实现见第 128 页。`root_factors(M, rho)` 把 $M \leq 10^{12}$ 分解为互异素数的幂，返回合数求根需要的 (p, e, g) 列表；模数 1 返回空列表。二的幂不需要原根，固定填零。奇素数幂 $m = p^e$ 的乘法群是循环群，设 $\varphi = m/p \cdot (p-1)$ 。分解 φ 后，候选 g 为原根当且仅当

$$p \nmid g, \quad g^{\varphi/q} \not\equiv 1 \pmod{m} \quad \text{对每个不同素因子 } q \mid \varphi.$$

从 2 起枚举候选，返回最小原根；所有奇素数幂均有原根。这里检查的是模 p^e 的阶，不能仅取一个模 p 的原根而不验证提升。

重载 `CompositeRoots::solve(a, k, M, rho)` 自动调用此准备过程。传统版为 `Composite_Roots::Solve` 和 `Root_Factors`，随机分解工作区类型为 `Pollard_Rho`。如果需要在同一个模数下处理多个方程，先保存因子列表，调用原有列表接口，避免重复分解和原根搜索。准备成本包括 M 与各 φ 的因数分解，以及逐个候选的模幂检验；不能把这部分直接算作常数时间。

独立试除验证 $1 \dots 500$ 的全部模数与多组接近 10^{12} 的边界值，并用任意精度模幂验证返回原根的阶。小模数进一步检查更小候选不能生成全部单位。直接传模数的两套求根接口，在小模数上与全部候选根的枚举结果比较。普通与 `ASan/UBSan` 均通过；尚无自动准备入口的在线 AC。

181.37 带权矩阵树定理与方向约定

实现见第 156 页。返回的是每棵树的边权乘积，再对所有树求和，并对素数取模。无权计数只需把所有边权设为 1。重边是不同选择，自环不属于任何生成树；单点图的空树乘积为 1。零权和负权都按域上的运算处理，因此结果为零可能来自抵消，不能作为连通性判定。

无向边 uv 权值 w 对拉普拉斯矩阵贡献为 L_{uu}, L_{vv} 各加 w ， L_{uv}, L_{vu} 各减 w 。删除 root 行和列后求行列式，与 root 的选择无关。对于每个点都能沿有向边走到根的树形图，边 $u \rightarrow v$ 贡献 $L_{uu} += w$ 、 $L_{uv} -= w$ ；删除根行列后得到内向树形图的权值和。外向树形图先反转所有边，再使用同一构造。

例如只有一条边 $0 \rightarrow 1$ 时，内向根 1 的答案为该边权，内向根 0 为零；外向根 0 的答案为该边权。P6178 指定的是以 1 为根的外向树，驱动把点号减一，并选择 `away_from_root`。只直接构造 $(n-1) \times (n-1)$ 余子式，时间 $O(n^3 + m)$ 、额外空间 $O(n^2)$ 。行列式复用本书的素数模高斯消元，不支持将 mod 随意改成合数。

测试独立枚举 $n-1$ 条边的所有选择，检查连通性或根向约束，用任意精度整数计算边权乘积之和，再分别对多个素数取模。两套实现覆盖所有根、三种方向、自环重边、负权及 64 位权值边界，并通过 220 点完全图的加权 Cayley 公式检查。P6178 完整驱动另测 300 点、十万条边以及输入输出约定。普通与 ASan/UBSan 均通过。两套代码已在 P6178 通过全部 11 个测试点：[动态版记录 297619886](#)、[传统版记录 297620263](#)。单点最大用时分别为 64 ms、62 ms，内存峰值为 2.85 MB、2.40 MB。在线测试覆盖无向与根为 1 的外向树；其他根、内向树和负权仍使用本地证据。定理背景可参考 [OI Wiki 矩阵树定理](#)。

181.38 任意正模数行列式

实现见第 155 页。素数模高斯消元可以除以任意非零主元，合数模数没有这个性质。本节只使用倍加行和交换行，不求逆元。先把元素正规化到 $[0, m)$ ，对第 i 列的两个元素 a, b ，用整数商 $q = \lfloor a/b \rfloor$ 执行 $R_i \leftarrow R_i - qR_j$ ，再交换 R_i, R_j 。该列的新数对为 $(b, a \bmod b)$ ，所以循环按欧几里得算法终止。其余元素每步对 m 取模，只改变整数代表，不改变行列式模 m 的值。

倍加行不改变行列式；交换行改变符号。消去所有下三角元素后，乘上对角元素及累计符号即得结果。剩余部分的该列全零时直接返回零，不需要判断某个非零元素是否与模数互素。例如 $\text{diag}(2, 3)$ 在模 8 下的行列式为 6，不能因为第一个主元没有逆元就判定失败。

支持任意正 signed 64 位模数与完整 signed 64 位输入元素。正规化后单个乘积小于 2^{126} ，行更新差值和结果乘积都放入有符号 128 位。空矩阵按约定返回 $1 \bmod m$ ；模 1 的所有返回值为零。时间上界 $O(n^3 \log m)$ ，矩阵副本占 $O(n^2)$ 空间。本节可以用于合数模的拉普拉斯余子式，但之前的 `MatrixTree<mod>` 接口仍使用素数模高斯，不能仅把其模板参数改成合数；合数模使用下一节的独立接口。

测试用排列定义穷举所有元素属于 $\{-1, 0, 1\}$ 的三阶矩阵，并用任意精度 Bareiss 消元验证随机矩阵。覆盖不可逆主元、奇异矩阵、模 1、有符号 64 位边界、Fibonacci 长欧几里得过程，以及 300 阶对角矩阵和完全图余子式的 Cayley 公式。两套版本普通与 ASan/UBSan 均通过；尚无专门在线 AC。行变换依据可参考 [OI Wiki 行列式](#)。

181.39 任意正模数的矩阵树计数

实现见第 157 页。拉普拉斯余子式表示生成树权值和的恒等式本身不要求模数为素数；需要换掉的是依赖模逆元的行列式算法。本节沿用无向、内向与外向三种构造，但使用欧几里得行列式消元。输入边权和正模数均支持 signed 64 位，接口名为 `MatrixTreeMod::count`，传统版为 `Matrix_Tree_Mod::Count`。

先把边权正规化到 $[0, m)$ ，每次矩阵元素加减仍先扩展至 128 位，再取模存回 64 位。重边的权值和可能超过 64 位范围，不能先累加到普通整数中。只构造删除根后的余子式，并将其存储直接交给行列式函数。单点返回 $1 \bmod m$ ，模 1 返回零。负权与零权仍按代数权值处理，返回零不能直接解释为没有生成树。

独立参考程序逐个枚举边集，用任意精度整数计算权值和。已检查多个合数模数、所有根与三种方向、断图、重边自环及 64 位极值权重；另以大权值完全图的 Cayley 公式验证 160 点规模。普通与 ASan/UBSan 均通过，尚无该接口的在线 AC。此前 P6178 的 AC 属于素数模版本，不自动转移到本接口。

181.40 斯特林数的快速整行计算

第二类实现见第 138 页，第一类实现见第 139 页。本节默认模数 $p = 998244353$ ，允许用模板参数指定素数与原根，复用参数化 NTT。第二类 $S(n, k)$ 计数无标号非空集合划分；第一类 $c(n, k)$ 计数排列的循环数。两者不能互换，第一类本节返回无符号版本。

第二类的容斥公式为

$$S(n, k) = \sum_{i=0}^k \frac{i^n (-1)^{k-i}}{i! (k-i)!}.$$

分别构造 $a_i = i^n/i!$ 和 $b_i = (-1)^i/i!$ ，一次卷积取前 $m+1$ 项即可。 n 支持无符号 64 位； $i > 0$ 时指数对 $p-1$ 取模后快速幂， $i = 0$ 时单独区分 $n = 0$ 和 $n > 0$ ，不可直接套用费马降幂。 $S(0, 0) = 1$ ，正 n 的 $S(n, 0) = 0$ ， $k > n$ 时为零。令 L 为 NTT 长度上限，要求 $2m+1 \leq L$ ；默认模数下即 $m < 2^{22}$ 。

无符号第一类是升阶乘多项式的系数：

$$x(x+1)\cdots(x+n-1) = \sum_{k=0}^n c(n, k)x^k.$$

令 $F_m(x) = x(x+1)\cdots(x+m-1)$ ，倍增式为 $F_{2m}(x) = F_m(x)F_m(x+m)$ ；奇数长度再乘 $x+2m$ 。平移系数满足 $[x^i]F_m(x+m) = \sum_{j=i}^m f_j j! m^{j-i}/(i!(j-i)!)$ ，将 $f_j j!$ 倒序后与 $m^j/j!$ 卷积即可。每层只递归计算一次半长多项式，总时间 $O(n \log n)$ ，空间 $O(n)$ 。要求 $n+1 \leq L$ ，默认模数下即 $n < 2^{23}$ ； $n = 0$ 返回常数多项式 1。如果需要下降阶乘对应的带符号数，将第 k 项乘 $(-1)^{n-k}$ 。

验证使用小规模集合划分、排列循环的直接枚举，以及 300 行的递推表；大指数第二类另用独立转移矩阵快速幂验证。规模检查包含二十万项第二类行与两万项第一类行，用幂函数的下降阶乘展开和升阶乘求值恒等式检查。普通与 ASan/UBSan 均通过。P5395 和 P5408 使用模 167772161，现已支持指定模板参数 167772161。两题均获两套码风在线 AC、各 10 个测试点全过：第二类动态版与 传统版，第一类动态版与 传统版。OJ 覆盖题面模数与行长，其他模数和第二类完整无符号指数仍由本地证据支持。该模数已测试二十万项第二类行与 262143 阶第一类行，普通与消毒器均通过。按列计算及 OI Wiki 的其他斯特林数应用仍待补，[对应条目](#)保留部分覆盖状态。

181.41 参数化 NTT 与模数适配

实现见第 133 页。传入素数 p 和原根 g ；长度 n 必须为二次幂且整除 $p-1$ 。代码自动计算 $L = 2^{v_2(p-1)}$ 作为长度上限，不代表分配这个长度的数组没有成本。两个非空数组的普通卷积需要补齐到不小于长度和减一的二次幂。模 998244353 的 $L = 2^{23}$ ，模 167772161 的 $L = 2^{25}$ ，两者均可用原根 3。不能只改模数而忽略原根和长度条件；接口只检查基本范围，不检查素性或原根的阶。

正变换按 g 的相应单位根计算，逆变换使用逆单位根并乘 n^{-1} 。模数支持 `int` 范围内的素数，模整数的加减和乘积使用 64 位中间量。蝶形层以半长度递增，避免最后一层之后的有符号整数溢出。

独立参考用任意精度整数直接计算 DFT 和朴素卷积，同时验证正逆变换复原。已检查七组素数和原根，包括模 2、3、17、97，斯特林数的默认模数与洛谷题目模数，以及 2013265921、原根 31。这些测试中的素性及原根条件另用试除和乘法阶证书确认。大规模全一数组卷积逐项比较已知三角形系数，检查补齐长度的边界。两套实现普通与 ASan/UBSan 均通过。

181.42 Bostan–Mori 与快速递推单项

代码见第 238 页。对于 $a_i = \sum_{j=1}^k c_{j-1} a_{i-j}$ ，令 $Q(x) = 1 - \sum_{j=1}^k c_{j-1} x^j$ ， $P(x) = (Q(x) \sum_{i=0}^{k-1} a_i x^i) \bmod x^k$ ，则 $[x^n]P/Q = a_n$ 。注意系数按前一项、前两项的顺序给出，初值从 a_0 开始。

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)} = \frac{U_0(x^2) + xU_1(x^2)}{V(x^2)}.$$

分母的奇数次项抵消，因此只保留分子与 n 同奇偶的项，分母保留偶数项，再将 n 右移一位。最终 $n = 0$ 时答案是 $P(0)/Q(0)$ 。实现不需要分子次数小于分母次数，空分子直接返回零；分母常数项必须非零。每轮系数长度不增加，初始最大长度 d 必须满足 $2d - 1$ 不超过 NTT 容量。总时间 $O(d \log d \log(n + 1))$ ，空间 $O(d)$ ； $n = 0$ 直接取常数项。

独立验证包括按定义做形式幂级数除法、友矩阵快速幂，以及 $(1 - x)^{-32000}$ 的二项式系数闭式；覆盖完整无符号 64 位下标、常数分母、不等于 1 的非零常数项、空分子、不真分式、零递推系数和多种 NTT 素数。P4723 驱动读取有符号系数并归一化；两套版本均获在线 AC：动态版与 传统版 各 5 个测试点全部通过，最慢单点分别为 907 ms、905 ms。OJ 覆盖模 998244353、 $n \leq 10^9$ 、 $k \leq 32000$ ；其他模数、完整 64 位下标与通用有理函数接口的扩展范围由本地测试支持。OI Wiki 线性递推 中的连续区间取项与其他快速算法仍待补，不将单项算法视为整页覆盖。

181.43 集合变换与不相交子集卷积

代码见第 140 页与第 142 页。长度 $N = 2^m$ 确定全集的位数，数组第 s 项对应集合 s 。OR 变换计算 $\hat{a}_s = \sum_{t \subseteq s} a_t$ ，AND 变换计算所有超集的和；逆变换将逐位加法改成减法。XOR 变换的系数是 $(-1)^{\text{popcount}(s \& t)}$ ，蝶形将 (u, v) 变为 $(u + v, u - v)$ ，逆变换每层再除以 2。因此 OR、AND 可用于任意模数，XOR 需要 2 可逆；本实现限制为奇数模数，使用 $(\text{mod} + 1)/2$ ，不对合数模数使用费马小定理。三类卷积均是变换后逐项相乘，再逆变换。

$$c_s = \sum_{t \subseteq s} a_t b_{s \setminus t} \quad \text{要求两个集合不相交。}$$

普通 OR 卷积允许重叠，不能直接用于上式。将输入按集合大小分层， $A_{i,s} = [|s| = i]a_s$ ， B 同理。每层 OR 变换后，对层数做卷积： $H_{k,s} = \sum_{i=0}^k \hat{A}_{i,s} \hat{B}_{k-i,s}$ 。对每个 k 逆 OR 变换，得到并集恰为 s 、两集合大小之和为 k 的贡献。取 $k = |s|$ ，由 $|u| + |v| = |u \cup v| + |u \cap v|$ 得到交集为空。时间 $O(m^2 2^m)$ ，空间 $O(m 2^m)$ ；本实现逐层复用结果缓冲区，不用第三组二维数组。 $m = 20$ 时两组分层数组本身占 168 MiB，另计输入和线性缓冲。

验证独立枚举变换系数、所有下标对与子掩码，并检查复合模数与 `INT_MAX` 模数。大规模使用 $a_s = x^{|s|}$ 、 $b_s = y^{|s|}$ 的逐位乘积恒等式：OR 为 $(x + y + xy)^{|s|}$ ，AND 为 $(xy)^{|s|}(1 + x + y)^{m - |s|}$ ，XOR 为 $(x + y)^{|s|}(1 + xy)^{m - |s|}$ ，不相交子集卷积为 $(x + y)^{|s|}$ 。P4717 两套版本各 10 点在线 AC：动态版和 传统版，最慢单点 84 ms、81 ms。

P6097 两套版本各 5 点在线 AC：动态版和 传统版，最大测试点显示 2.70 s、2.69 s，内存 188.83 MB、189.00 MB。两题 OJ 仅验证各自题面模数和长度范围，其他模数及长度 1 仍由本地独立测试覆盖。OI Wiki FWT 的其他运算及更广泛应用仍待覆盖。

181.44 Taylor 平移与等比点求值

代码见第 135 页和第 136 页。多项式使用升幂系数，输入长度 n 表示次数小于 n ，允许末尾零。平移输出的第 i 项满足

$$[x^i]f(x+c) = \frac{1}{i!} \sum_{j=i}^{n-1} f_j j! \frac{c^{j-i}}{(j-i)!}.$$

倒序 $f_j j!$ ，与 $c^j/j!$ 卷积，再取第 $n-1-i$ 项并乘 $1/i!$ 。保留原数组长度；零多项式可以用空数组表示，返回空数组。使用 NTT 素数及原根，要求 $2n-1$ 不超过变换容量；此时所需阶乘非零。时间 $O(n \log n)$ 、空间 $O(n)$ 。这与给出 $f(0), f(1), \dots$ 后平移点值的任务不同，后者尚待补充。

Chirp Z 返回 $f(ar^i)$ ， $0 \leq i < m$ 。设 $T_j = j(j-1)/2$ ，由 $ij = T_{i+j} - T_i - T_j$ 得

$$f(ar^i) = r^{-T_i} \sum_{j=0}^{n-1} f_j a^j r^{-T_j} r^{T_{i+j}}.$$

将第一项数组倒序，与 r^{T_j} 卷积，抽取第 $n-1+i$ 项。三角数幂通过累乘生成，不计算可能溢出的 $j(j-1)$ 。令 L 为不小于 $n+m-1$ 的最小二次幂，直接做模 x^L-1 的循环卷积。普通乘积最高次数为 $2n+m-3$ ，而所需区间任意下标 $t \geq n-1$ 满足 $t+L \geq 2n+m-2$ ，因此不会收到高次项回卷，所取系数仍然精确。要求 $n+m-1$ 不超过 NTT 容量，总时间 $O((n+m) \log(n+m))$ 、空间 $O(n+m)$ 。

约定 $r^0 = 1$ ，故 $r=0$ 时第一项为 $f(a)$ ，余项均为 $f(0)$ ； $a=0$ 时所有点为零， $r=1$ 时所有点相同。这些情况单独计算，避免对零求逆。 $m=0$ 返回空数组，空输入返回 m 个零。同一份算法支持小阶公比、重复点与有限域中多次绕回，输出保持原顺序。

独立验证采用 Pascal 二项式展开和 Horner 求值，涵盖 7 种 NTT 素数、零公比与单位公比、小素数完整容量边界。大规模平移用 $(x+b)^d$ 检查每个系数；等比点求值用几何级数闭式逐点检查。普通与 ASan/UBSan 均通过。Library Checker 的 Polynomial Taylor Shift 和 等比点求值 已准备两套完整驱动；本地检查各题 524288 的最大规模，尚无在线 AC。参考OI Wiki 平移及 Chirp Z，更广泛的点值平移和应用另行覆盖。

181.45 可持久化区间拼接的懒标记

代码见第 173 页。对象从全零版本开始，同一对象内版本共享坐标域。区间加仅复制被访问节点，不向旧子节点下传标记。节点的和满足 $\text{sum} = \text{left.sum} + \text{right.sum} + \text{lazy} \cdot \text{len}$ 。读取时把祖先累积增量作为参数传下去，因此查询完全只读。

拼接含义为将目标版本某个半开区间替换成源版本同一坐标区间。由于两棵树祖先的 lazy 可能不同，不能直接将源子树挂到目标祖先下面。递归参数 offset 保持“源祖先增量减目标祖先增量”；下降时更新为 $\text{offset} + \text{source.lazy} - \text{destination.lazy}$ 。当区间完整覆盖，复制源节点并将这个 offset 加到其 lazy 和 sum，此后挂到目标祖先下面时，逻辑值才与源版本相同。offset 为零时直接共享源节点；没有覆盖时保留目标节点。空区间、零增量与全区间拼接都可仅增加版本别名。

每一层至多有两个边界节点，其他分支直接返回或复制一个整段节点，所以单次修改、拼接均为 $O(\log n)$ 新节点，查询为 $O(\log n)$ 时间、零新节点。不支持不同坐标偏移的平移粘贴，也不回收仍被旧版本引用的节点。所有 long long 的和、差、懒标记与长度乘积都应在范围内；节点数与版本数须小于 INT_MAX。

验证用独立向量保存各分支版本，逐区间检查更新、拼接和历史版本；包含正负增量、不同祖先标记、空区间、全段共享，以及查询前后的节点数量检查。规模验证包含长度 200000 的 10000 次区间版本组合，并在长度 INT_MAX 的隐式坐标域验证端点操作与共享根。普通与 ASan/UBSan 均通过；区间加 1、后缀拼接和单点读取的应用已获 QOJ8240 双 AC，其余通用接口范围仍由本地验证覆盖。来源需求是 2023 杭州站题解 第 24-26 页的 K 题，完整驱动及验证见下节。

181.46 杭州 2023 K: 跨版本拼接应用

[Card Game](#) 的完整驱动在 `verify/qoj/8240/*.cpp`, 使用前节的可持久化区间模板。将下标转为从 0 开始, 设 $F_i(r)$ 为区间 $[i, r]$ 的剩余牌数, p 为 i 之后第一张与 a_i 相同的牌, 若不存在则 $p = n$ 。当 $r < p$ 时, 第一张牌始终保留, 剩余牌的过程与从 $i + 1$ 开始相同; 当 $r \geq p$ 时, 到达 p 会将此前区间全部清空。因此

$$F_i(r) = \begin{cases} 1 + F_{i+1}(r), & r < p, \\ F_{p+1}(r), & r \geq p. \end{cases}$$

每个版本的坐标是右端点 r , 值是 $F_i(r)$ 。从 $i = n - 1$ 向前处理: 复制版本 $i + 1$ 并在 $[i, p)$ 加 1, 再把 $[p, n)$ 拼接自版本 $p + 1$ 。 $p = n$ 时省去后一步。询问原题的 $[l, r]$ 时, 从版本 $l - 1$ 读取坐标 $r - 1$ 的单点值。端点先与上次答案异或解码, 不能将输入查询预先离线排序。

$n, q \leq 300000$, 牌号位于 $[1, n]$ 。构建及查询时间分别为 $O(n \log n)$ 、 $O(q \log n)$ 。驱动另计算各次操作可能访问的相交节点数作为分配上界, 一次 reserve 足够容纳整个构建过程, 避免复制扩容的临时峰值。该计数仅依据区间结构, 不影响牌局答案, 且同样为 $O(n \log n)$ 。

本地参照按题意逐张放牌, 遇重复就实际删除栈中相应后缀。穷举长度不超过 7 的二元牌序列与全部区间, 并打乱询问顺序、真实编码异或输入。另有随机牌局、 $n = q = 300000$ 的全异、全同、周期序列闭式检查, 以及稀疏重复大数组的短区间逐张模拟。两套驱动均在线 AC, 各 41 个正式测试和 3 个额外测试全过: [动态版](#) 最慢 789 ms、452688 KB, [传统版](#) 最慢 752 ms、452492 KB。此应用仅调用区间加 1、后缀拼接、单点读取; 一般有符号增量与区间求和仍由前节本地测试支持。

181.47 非负权树的点集直径摘要

代码见第 64 页。一个非空点集只需保存任意一对直径端点 a, b ，即可回答任意树上点 x 到该点集的最远距离： $\max(d(x, a), d(x, b))$ 。边权非负，包括零权；该结论不能推广到一般图或负权树。

证明可将点 u 投影到路径 $a-b$ ，记投影距 a 为 t ，离路径的距离为 h ，直径长为 D 。由于 $d(a, u), d(b, u) \leq D$ ，有 $h \leq \min(t, D - t)$ 。再将查询点投影到同一路径，投影距 a 为 s ，离路径的距离为 g 。经两个投影点连接得到 $d(x, u) \leq g + h + |s - t| \leq g + \max(s, D - s)$ ，右侧恰为 $\max(d(x, a), d(x, b))$ 。因此插入新点只需比较到两个旧端点的距离；合并点集时，只插入另一个摘要的两个端点即可。并列直径可任选，不应假设端点编号固定或合并后的端点与合并顺序无关。

Euler LCA 现在同时保存根距离，但其原有 `distance` 接口仍返回边数。带权距离请显式调用 `weighted_distance / Weighted_Distance`；采用 $(D_u - D_z) + (D_v - D_z)$ 避免不必要的两倍根距离中间值。所有路径权和须能用 `long long` 表示。独立验证穷举小点集及合并，逐点检查最远距离与返回顶点；带权路径用独立树遍历参照，并覆盖全部根、零边权与 `LLONG_MAX` 单边。原有 Euler LCA 全小树回归，以及 50 万点、边权 10^9 的递归链也通过普通与 ASan/UBSan 检查。

杭州 2023 F 中，点权互异。令 g 为全树点权的 mex，只需按点权 $0, 1, \dots, g - 1$ 建立前缀摘要。某个前缀全部位于以 x 为中心、半径 k 的范围内，当且仅当该摘要的最远距离不超过 k 。二分最长满足条件的前缀长度即为答案； $g = 0$ 时直接为零。题面树边带权，不能用边数距离。完整驱动在 `verify/qoj/8235.*.cpp`，本地使用实际权距与 mex 集合参照， $n = q = 500000$ 的链和星形树保持递归 DFS。两套驱动各 50 个正式测试和 1 个额外测试全部在线 AC：动态版 666 ms、198076 KB，传统版 588 ms、198196 KB。OJ 验证题面正权边、前缀插入和最远距离判定；零权边、完整 64 位距离与任意摘要合并仍由独立本地验证支持。

181.48 函数图的人环链与到达查询

代码见第 4 页。每个点有且仅有一个后继，故每个弱连通分量由一个有向环和若干指向环的树组成。先不断移除入度为零的点，剩下的点恰在环上；沿后继枚举各环。反向处理移除序列时，后继的环编号、入环点和入环距离均已确定，可直接传给当前点。这里使用的是入度剥离队列，不是用显式栈模拟 DFS。

跳转 k 步时，若 k 小于入环距离，用普通倍增；否则先到入环点，减去入环步数，再对环长取模。因此即使 k 达到无符号 64 位上界，也只需 $O(\log n)$ 层倍增表。环上位置相加使用无符号宽整数，避免两个大下标的 `int` 中间值溢出。

判断从 u 到 v 的最少步数时：不同环编号必不可达。若 v 在入环树上，先检查 u 的深度至少为 v 的深度，再跳过深度差，看是否恰好到 v ，从而排除同分量的不同分支。若 v 在环上，答案为 u 的入环步数加上沿后继方向的环上距离。不可达返回 -1 ，从自己到自己为零步。字段 `position` 只对环点有意义，不要将树点的 -1 当作环坐标。

独立参照逐步行走直至首次重复，同时记录每个点的第一次出现位置。穷举不超过 5 个点的全部后继映射，并检查跳转、最少步数、入环点与环长；另验证同对象重建、空图、50 万点长链连环、50 万个自环和完整无符号 64 位步数。普通与 ASan/UBSan 均通过。完整驱动对应 [CSES 1750](#)、[CSES 1160](#) 和 [P2921](#)，均准备两套码风。前两题检查跳转与可达步数，后一题仅使用“入环步数 + 环长”，不能替代查询接口的验证。线上评测尚待完成。

181.49 杭州 2023 H: 随机事件顺序与函数图

[Sugar Sweet II](#) 的完整驱动为 `verify/qoj/8237/*.cpp`，复用函数图分解与逆阶乘表。事件 u 发生前，自己的数量仍为 a_u ；目标 $v = b_u$ 只可能为 a_v 或 $a_v + w_v$ 。因此 $a_u < a_v$ 时必定发生， $a_u \geq a_v + w_v$ 时必定不发生。其余情况下，事件 u 成功当且仅当事件 v 更早执行且成功。

将两类确定事件的后继改为自己，另保存哪些是确定成功的终点。未修改的依赖环没有成功起点：最早执行的环上事件必失败，后续也无法形成成功链。自环同理，自己不可能严格少于自己。沿函数图到达确定失败终点或纯依赖环的节点，成功概率都是零。若到达确定成功终点，且入环距离为 d ，沿途 $d + 1$ 个事件必须按唯一的顺序出现。均匀随机全排列对这 $d + 1$ 个事件诱导均匀相对顺序，因此成功概率为 $1/(d + 1)!$ 。这些顺序约束有关联，不能将每条依赖当作独立的二分之一概率。期望为 $a_u + w_u/(d + 1)!$ ，在模 $10^9 + 7$ 下计算。

题面所有增量为正， $\sum n \leq 500000$ ；阶乘表跨测试组只扩充不缩小，避免大量单点测试时反复求逆。当前驱动复用带倍增表的通用函数图，时间、空间为 $O(n \log n)$ ；概率传播本身只读取 `entry` 和 `depth`，不调用跳转与最少步数接口。

验证穷举小输入及事件全排列，逐步修改真实糖果数量，最后汇总精确总和并除以排列数。另外检查 50 万点的依赖链、纯环，以及 50 万组单点测试，普通与 ASan/UBSan 均通过。两套驱动各 31 个正式测试与 2 个额外测试全过：[动态版](#) 162 ms、84608 KB，[传统版](#) 158 ms、84612 KB。本应用不能验证函数图的 `advance` 或 `steps` 接口。

第一百八十二章 计算几何：精确判定与适用范围

182.1 整数系数的有界半平面交

实现见第 194 页。输入是不等式 $ax + by \leq c$ ，三个系数均为绝对值不超过 10^9 的整数。本接口要求交集有界，只输出正面积多边形；无解、点交集和线段交集都返回空向量。它不是一般的可行性分类接口，不可将其空返回直接解读为闭半平面交集必为空。无界输入不在约定范围内，也不偷偷添加一个人工边框。

两条非平行边界交点用齐次坐标 (X, Y, D) 表示：

$$D = a_1b_2 - b_1a_2, \quad X = c_1b_2 - b_1c_2, \quad Y = a_1c_2 - c_1a_2.$$

若 $D < 0$ ，三者同时取负，使 $D > 0$ 。测试 $aX + bY \leq cD$ 即可判断交点位于半平面内，无需除法。系数界保证 $|X|, |Y|, D \leq 2 \times 10^{18}$ ，可用 64 位保存；相等判断的交叉乘积至多为 4×10^{36} ，使用 128 位。因此本接口内的构造、排序和位置判定都是精确整数运算。

按法向量的半圆与叉积排序，比较器不使用 epsilon。同方向平行约束按正规化后的常数项保留更紧者，避免除以法向量长度。扫描时若新约束排除队首或队尾交点，就删除相应无效边界；每条边界最多入队、出队一次，排序后为线性时间。最后闭合队列，清除重复顶点；有界的非正面积结果不会输出多边形。零法向量表示常真约束或直接矛盾，先单独处理。

测试枚举所有边界对的交点，用任意精度整数逐条验证可行性，将可行顶点集合与返回集合比较，并检查循环顺序的严格凸性。两套版本通过 5000 组有界随机输入、平行和退化案例、近乎平行的十亿系数，以及十万条冗余约束的规模测试；普通与 ASan/UBSan 均通过。将有理数坐标转换为浮点、计算面积或处理浮点输入属于另一个数值问题，尚未由此验证。WIDA 的一般半平面交条目保留部分完成状态，浮点版本与无界分类仍需补齐。

182.2 最近点对、Minkowski 和与三维整数判定

最近点对与 Minkowski 和见第 205 页。最近点对返回距离平方，不开方；少于两个点时返回空 optional，重复点返回零。分治使用递归，每层按纵坐标归并，总时间 $O(n \log n)$ ，额外空间 $O(n)$ 。坐标绝对值上限为 10^{12} ，作差后平方使用 128 位。

Minkowski 和输入必须是严格逆时针凸包，不重复首点，不含边上的冗余共线点；允许循环移动起点，也允许空集、单点和线段。输入坐标以及任意两个输入点的坐标和都须在 $[-10^{12}, 10^{12}]$ 内。非退化凸包按边方向归并，时间 $O(n + m)$ ；退化分支枚举点和后求凸包，时间 $O((n + m) \log(n + m))$ 。返回同样不重复首点的严格凸包。

两套代码通过任意精度整数距离枚举，以及对所有点和独立执行礼品包装法的对拍；覆盖退化、循环起点和大坐标。规模测试包含二十万点最近点对，以及 40001 顶点的抛物线凸包与自身求和，结果应为原凸包各坐标的两倍。这些属于本地验证，不表示新增在线 AC。

三维判定见第 208 页。点的三个坐标为 x, y, z ，绝对值均不超过 10^9 。`orient(a,b,c,d)` 返回有向六倍四面体体积，零表示四点共面，交换任意两个点会改变符号。`collinear` 判断三点共线，`on_segment` 包含端点，且支持零长度线段。传统版线段接口名为 `On_Segment`。

点差每个分量至多 2×10^9 ，叉积分量至多 8×10^{18} ，三重积绝对值至多 4.8×10^{28} ，在有符号 128 位范围内。公开的叉积、点积辅助函数不对任意 128 位向量保证不溢出；上述保证针对点差及其叉积。任意精度整数按 4×4 行列式的 24 个排列独立计算方向，并用二阶子式及坐标区间验证共线与在线段上。已通过立方体八个顶点的全部有序四元组、随机十亿级坐标、换序及退化测试，普通与 ASan/UBSan 均通过。这里不包含三维凸包构造。

182.3 浮点最近点对

实现见第 203 页。`ClosestPair::solve` 与传统版 `Closest_Pair::Solve` 接收 `Point{x,y}` 数组，返回最短距离，不是距离平方。少于两点返回空 optional；重复点返回零。输入为有限 `long double`，坐标绝对值不超过 10^{100} 。

递归前按横坐标排序，每次在递归改变数组前保存分界横坐标。左右区间分别递归后按纵坐标归并，再检查分界线附近的点对。时间 $O(n \log n)$ ，额外空间 $O(n)$ 。直接使用 `hypotl`，避免先平方造成上溢或下溢。排序比较器不使用 `epsilon`；不能把相近坐标随意视为相同点。

浮点结果仍有舍入误差，也无法恢复读入之前已经丢失的精度。整数坐标要求精确判定时，优先使用整数最近点对，最后按题目要求开方输出。一百位十进制浮点枚举所有点对，验证两套实现，覆盖小数、相邻可表示浮点数、极小非正规数、 10^{-100} 至 10^{100} 的尺度，以及二十万点规模；普通与 ASan/UBSan 均通过。本地完整 P1429 驱动测试包含 103 组输入和四位小数输出检查。在线评测的整数输入范围不能替代这里对小数与极端尺度的检查。

两套实现已在 P1429 通过全部 11 个测试点：[动态版记录 297602650](#)、[传统版记录 297603238](#)。单点最大用时均为 75 ms，内存峰值均为 12.32 MB。

182.4 公切线与过点切线

实现见第 201 页，依赖浮点几何的点、圆和向量运算。令两圆心差为 v , $d = |v|$, $u = v/d$ 。外公切线取 $s = 1$, 内公切线取 $s = -1$, 单位法向量满足

$$n \cdot u = \frac{r_1 - sr_2}{d} = c, \quad n = cu \pm \sqrt{(1-c)(1+c)} u^\perp.$$

切点为 $p = o_1 + r_1 n$ 、 $q = o_2 + sr_2 n$ 。直线以 $n \cdot (x - p) = 0$ 表示，避免在两圆相切、两个切点相同时丢失方向。结果中的 **side** 为 1 或 -1，表示外公切线或内公切线。每个非相切分支有两条线，相切分支只输出一次。

半径零解释为点。有一个点圆时，只枚举外公切线分支，避免内外分支重复；两个不同点直接返回连线，**side**=0。严格同心且等半径返回 **infinite=true**，包括重合的两个点圆；严格同心但半径不同没有公切线。**from_point(p,c)** 把第一圆设为点圆，圆上的切点读返回的 **b**。点在圆上时返回一条，点在圆内时空；点与零半径圆重合时返回无限条过点直线。

输入须有限，坐标和半径绝对值不超过 10^{100} ，半径非负。容差 $0 < \varepsilon < 1$ 默认 10^{-12} ：当 $|r_1 - sr_2|$ 与 d 的相对差在容差内时，合并为相切。因此极接近相切的输入可能返回一条近似切线，不承诺恢复舍入前的几何分类；使用方程时宜计算 $n \cdot (x - p)$ ，避免先计算大常数项造成额外消减。先比较半径差和圆心距，再做除法，避免极小圆心距造成比值溢出。

计数对拍使用任意精度整数比较圆心距平方与半径和、差的平方；一百位十进制运算验证切点到圆心的距离、圆心到切线的距离、法向量和切线一致性。两套实现通过 3000 组随机圆，在三个尺度下交换顺序测试，并覆盖重合、内外相切、点圆、极近同心与容差边界。普通与 ASan/UBSan 均通过；尚无此接口的在线 AC 记录。

182.5 随机增量最小覆盖圆

实现见第 199 页。随机打乱点集后依次加入。新点在当前圆外时，它必须成为新覆盖圆的边界点；以该点为固定边界重扫前缀，再遇到圆外点就固定两个边界点，最后由第三个圆外点确定外接圆。随机化使期望时间为 $O(n)$ ，最坏时间仍为 $O(n^3)$ 。传入初始化好的 `mt19937_64`，不可省略打乱。本实现使用三层循环，没有递归栈需求。

空集返回空 optional，单点及重复同一点的半径为零。直径圆用两端点到计算圆心的较大距离作为半径。三点外接圆先把第一点平移到原点，再按最大坐标差缩放后解两条垂直平分线方程；共线时使用最远点对的直径圆。`through` 对非共线三点求外接圆，不是任意三点的最小覆盖圆；例如钝角三角形的最小覆盖圆可能仅由最长边支撑，须调用主过程。

包含判断使用相对半径容差，默认 32 倍机器精度。输入与中间运算须有限、尺度适当；近共线三点的外接圆仍可能数值敏感，浮点比较不能恢复读入之前的精度。[P1742](#) 的驱动已准备，输出半径与圆心，该题坐标绝对值不超过 10^4 ，最多两位小数。

本地参考答案枚举所有单点、点对直径圆和三点外接圆，用一百位十进制运算筛出覆盖全部点且半径最小者，检查圆心、半径及覆盖性。两套版本通过 700 组随机小数点集，每组使用 12 个随机种子，另测共线、近共线、钝角/锐角三角形、平移后的小三角形、十万点直线与重复点。普通与 ASan/UBSan 均通过。两套代码已在 P1742 通过全部 10 个测试点：[动态版记录 297608403](#)、[传统版记录 297608801](#)。单点最大用时分别为 52 ms、49 ms，内存峰值为 6.50 MB、6.49 MB。该评测验证半径与圆心，不覆盖一并展开但未调用的其他几何接口。随机增量算法可参考 [Project Nayuki 的算法说明](#)。

182.6 圆与简单多边形相交面积

实现见第 197 页。先把圆心移到原点，再沿多边形有向边累加贡献，最后取绝对值。边与圆的交点把边分成至多三段。若小段中点在圆内，贡献是 $\text{cross}(p, q)/2$ ；否则贡献是有向扇形面积

$$\frac{r^2}{2} \text{atan2}(\text{cross}(p, q), p \cdot q).$$

必须保留符号，不能先对每条边取绝对值，否则凹多边形和圆外的边无法抵消。使用补偿求和减小大量边累加时的误差，总时间 $O(n)$ 、额外空间 $O(1)$ 。

求交点时令边的单位方向为 u ，有向距离 $h = \text{cross}(u, a)$ ，垂足直接构造为 $hu^\perp$ ，交点为

$$hu^\perp \pm u\sqrt{(r - |h|)(r + |h|)}.$$

这避免用远端点加很长的位移来重建圆附近的交点。沿边的投影参数用于筛选位于线段内部的交点；端点本身已在分段列表内。相切交点重复不改变面积，零长度边返回零。

多边形须简单，支持凹形、两种方向和相邻重复顶点；半径非负，零半径或少于三个顶点时面积为零。输入与中间运算须有限且尺度适当，特别是平移后的乘积和面积要可表示。极小相交面积可能受到扇形贡献相消和近相切计算的影响，不作相对误差的统一保证。

测试包括完整圆、半圆、四分之一圆、内含多边形、相离、相切和高精度圆缺面积。独立参考实现沿竖直方向求多边形截线区间，再与圆的截线相交并数值积分，检查 150 组含凹形的随机多边形，另检查方向反转、重复点和平移。规模与数值测试含十万条边、极长的过圆心边和不同尺度。两套实现普通与 ASan/UBSan 均通过；尚无此模块的在线 AC。对应 kuangbin 二维几何中的圆与三角形、多边形相交面积部分。

第一百八十三章 来源与覆盖维护

183.1 参考基线

- kuangbin, ACM 常用算法模板, 2018 年版本: <https://kuangbin.github.io/2018/08/01/ACM-template/>。
- WIDA XCPC: <https://github.com/hh2048/XCPC>, 参考快照 484e9e0。
- OI Wiki 数学: <https://oi-wiki.org/math/>。

本项目重新实现的代码与公式速查不能冒用原作者的验证记录。完整覆盖审计应为每个来源条目记录实现文件、替代关系与验证证据, 保留遗漏项直至完成。

183.2 后续覆盖项

尚需完成的范围以覆盖表为准, 包括动态树、平衡树、一般图匹配、最小树形图、上下界流、高阶几何构造、半平面交、三维凸包、数值积分、复杂数论及多项式、搜索与动态规划等。不能用通用模块的存在推断其覆盖了所有原书例题。

算法索引

AC_Automaton, 320
Affine_Segment_Tree, 266
AffineSegTree, 28
AhoCorasick, 81
Arborescence, 186

Basis_Intersection, 264
basis_intersection, 26
Basis_Sum_Intersection, 265
basis_sum_intersection, 27
Batch_Inverse, 342
batch_inverse, 103
Berlekamp-Massey, 482
Berlekamp_Massey, 468
berlekamp_massey, 234
Bezout identity, 478
Biconnected, 7
Biconnected_Graph, 246
Binary_Indexed_Tree, 257
Binomial, 99
Bipartite_Matching, 279
BipartiteMatching, 41
Blossom, 231
Bostan_Mori, 470
BostanMori, 238
Bounded_Circulation, 274
BoundedCirculation, 35
Bridge_Augmentation, 254
bridge_augmentation, 15
Burnside, 482

Carmichael, 352, 480
carmichael, 115
Catalan, 482
Centroid_Pairs, 309
CentroidPairs, 70
Chirp_Z, 372
chirp_z, 136
Cipolla, 480
Circle_Polygon, 431
Circle_Tangents, 435
CirclePolygon, 197
CircleTangents, 201
Closest_Pair, 437

ClosestPair, [203](#)
Combination, [338](#)
Composite_Roots, [365](#)
CompositeRoots, [129](#)
coprime pairs, [475](#)
Coprime_Pairs, [348](#)
CoprimePairs, [110](#)
CRT, [477](#)
Cut_Tree_Values, [425](#)
cut_tree_values, [191](#)

Determinant_Mod, [389](#)
determinant_mod, [155](#)
Dijkstra, [38](#)
Dinic, [31](#)
Directed_Euler, [285](#)
Directed_MST, [420](#)
DirectedEuler, [47](#)
Dirichlet convolution, [473](#)
Discrete_Log, [388](#)
DiscreteLog, [153](#)
Disjoint_Set, [255](#)
divisor sum, [476](#)
Divisor_Sum_Power, [350](#)
divisor_sum_power, [113](#)
DSU, [16](#)
Du Jiao sieve, [473](#)
Du_Jiao, [387](#)
DuJiao, [151](#)
Dynamic_Kth, [413](#)
DynamicKth, [179](#)

Edge_Compression, [252](#)
EdgeCompression, [13](#)
Enclosing_Circle, [433](#)
EnclosingCircle, [199](#)
Euler theorem, [477](#)
Euler totient, [474](#)
Euler_LCA, [301](#)
Euler_Phi, [351](#)
euler_phi, [114](#)
EulerLCA, [61](#)
Ex_Lucas, [355](#)
exCRT, [479](#)
ExLucas, [119](#)
exLucas, [481](#)
expectation, [483](#)

Fenwick, [18](#)

floor moments, [477](#)
floor sum, [476](#)
Floor__Moments, [349](#)
floor__moments, [111](#)
Functional__Graph, [243](#)
FunctionalGraph, [4](#)

Garner, [344](#), [479](#)
garner, [105](#)
Gcd__Sequence__Treap, [461](#)
GcdSequenceTreap, [227](#)
General__Matching, [465](#)
generating functions, [482](#)
geometric series, [476](#)
Geometry__Extra, [439](#)
GeometryExtra, [205](#)
Global__Min__Cut, [422](#)
Gomory__Hu, [424](#)
gomory__hu, [190](#)

Heavy__Light__Decomposition, [297](#)
HLD, [57](#)
Hungarian, [184](#)

inclusion-exclusion, [482](#)
integer partitions, [475](#)
Integer__Geometry, [392](#)
Integer__Geometry__3D, [442](#)
Integer__Halfplanes, [428](#)
Integer__Partitions, [353](#)
IntegerGeometry, [158](#)
IntegerGeometry3D, [208](#)
IntegerHalfplanes, [194](#)
Inverse__Table, [343](#)
inverse__table, [104](#)

Kmp__Match, [318](#)
kmp__match, [79](#)
Korselt criterion, [480](#)
Kth__Residue, [360](#)
KthResidue, [124](#)
Kuhn__Munkres, [418](#)

Lagrange, [131](#)
Lagrange interpolation, [481](#)
Lagrange__Interpolation, [367](#)
LazySeg, [18](#)
Lex__Two__SAT, [283](#)
LexTwoSAT, [45](#)
Li__Chao__Tree, [399](#)

LiChao, [165](#)
Lifting_LCA, [305](#)
LiftingLCA, [66](#)
linear congruence, [479](#)
linear sieve, [474](#)
Linear_Algebra, [384](#)
Linear_Congruence, [340](#)
linear_congruence, [101](#)
Linear_Equation, [339](#)
linear_equation, [100](#)
Linear_Sieve, [336](#)
LinearAlgebra, [148](#)
LinearSieve, [97](#)
Link_Cut_Tree, [445](#)
LinkCutTree, [211](#)
Low_Link, [281](#)
Lowlink, [43](#)
Lucas, [117](#), [481](#)
Lucas theorem, [477](#)
Lucas_Theorem, [354](#)

Manacher, [319](#)
manacher, [80](#)
Matrix Tree theorem, [482](#)
Matrix_Tree, [390](#)
Matrix_Tree_Mod, [391](#)
MatrixTree, [156](#)
MatrixTreeMod, [157](#)
Max_Plus_Matrix, [399](#)
Maximum_Closure, [275](#)
maximum_closure, [37](#)
MaxPlusMatrix, [165](#)
Miller–Rabin, [473](#)
Min_Cost_Flow, [271](#)
MinCostFlow, [33](#)
Minimum_Rotation, [320](#)
minimum_rotation, [81](#)
Mixed_Euler_Orientation, [292](#)
mixed_euler_orientation, [52](#)
Mixed_Euler_Trail, [294](#)
mixed_euler_trail, [54](#)
Mobius inversion, [473](#)
Mod_Int, [337](#)
Mod_Sqrt, [358](#)
mod_sqrt, [122](#)
ModInt, [98](#)
multiplicative order, [480](#)

Network_Flow, [269](#)

Ntt_Convolution, [369](#)
NttConvolution, [133](#)
Number_Theory, [332](#)
NumberTheory, [93](#)
numerical stability, [483](#)

Odd_Cycle_Vertices, [249](#)
odd_cycle_vertices, [10](#)
Offline_LCA, [299](#)
OfflineLCA, [59](#)
Ordered_Splay, [456](#)
Ordered_Treap, [449](#)
OrderedSplay, [222](#)
OrderedTreap, [215](#)
Orient_Edges, [253](#)
orient_edges, [14](#)

Palindromic_Tree, [330](#)
PalindromicTree, [91](#)
Partitions, [116](#)
Path_Intersection, [308](#)
path_intersection, [69](#)
pentagonal number theorem, [475](#)
Persistent_Array, [404](#)
Persistent_Distinct, [416](#)
Persistent_Kth, [401](#)
Persistent_Range, [407](#)
PersistentArray, [170](#)
PersistentDistinct, [182](#)
PersistentKth, [167](#)
PersistentRange, [173](#)
Pick theorem, [483](#)
Pollard–Rho, [473](#)
Pollard_Rho, [334](#)
PollardRho, [95](#)
Polya, [482](#)
Polynomial, [143](#), [379](#)
Polynomial_Shift, [371](#)
polynomial_shift, [135](#)
Position_Basis, [262](#)
PositionBasis, [24](#)
Power_Sum, [350](#)
power_sum, [112](#)
Prefix_Function, [318](#)
prefix_function, [79](#)
Prefix_Lcs, [328](#)
prefix_lcs, [89](#)
Prime_Power_Roots, [361](#)

PrimePowerRoots, [125](#)
Primitive_Root, [345](#)
PrimitiveRoot, [107](#)

quadratic residue, [480](#)

rank, [482](#)
Real_Geometry, [395](#)
RealGeometry, [161](#)
Recurrence_Nth, [469](#)
recurrence_nth, [236](#)
Removal_Components, [251](#)
removal_components, [12](#)
Rollback_DSU, [256](#)
RollbackDSU, [17](#)
Root_Factors, [364](#)
root_factors, [128](#)

SCC, [39](#)
Segment_Tree, [258](#)
segmented sieve, [475](#)
Segmented_Primes, [341](#)
segmented_primes, [102](#)
Sequence_Treap, [452](#)
SequenceTreap, [217](#)
Set_Convolution, [376](#)
SetConvolution, [140](#)
Shortest_Path, [276](#)
Sprague-Grundy, [483](#)
Square_Counts, [329](#)
square_counts, [90](#)
Stirling numbers, [482](#)
Stirling_First_Row, [375](#)
stirling_first_row, [139](#)
Stirling_Second_Row, [374](#)
stirling_second_row, [138](#)
StoerWagner, [188](#)
Strong_Component, [277](#)
Subset_Convolution, [378](#)
subset_convolution, [142](#)
Subtree_Colors, [312](#)
SubtreeColors, [73](#)
Suffix_Array, [323](#)
Suffix_Automaton, [325](#)
Suffix_LCP, [327](#)
SuffixArray, [84](#)
SuffixAutomaton, [86](#)
SuffixLCP, [88](#)
Support_Hull, [443](#)

SupportHull, [209](#)

Tarjan_SCC, [241](#)

TarjanSCC, [2](#)

Tree_Diameter, [304](#)

Tree_Path_Kth, [410](#)

TreeDiameter, [64](#)

TreePathKth, [176](#)

Two_SAT, [278](#)

TwoSAT, [40](#)

Undirected_Euler, [289](#)

UndirectedEuler, [50](#)

Virtual_Tree, [315](#)

VirtualTree, [76](#)

Weighted_Matching, [426](#)

WeightedMatching, [192](#)

Widget Factory, [483](#)

Word_Chain, [288](#)

word_chain, [49](#)

Xor_Basis, [260](#)

Xor_Walk, [295](#)

XorBasis, [21](#)

XorWalk, [55](#)

Z_Function, [319](#)

z_function, [80](#)

二元一次不定方程, [478](#)

区间筛素数, [475](#)

原根, [480](#)

扩展中国剩余定理, [479](#)

批量逆元, [477](#)

排列数, [481](#)

模高斯消元, [483](#)

混合进制, [479](#)

矩形 GCD 计数, [475](#)

类欧几里德, [476](#)

线性同余方程, [479](#)

线性逆元表, [478](#)

线性递推, [483](#)

阶乘表, [481](#)